# UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

# FACULTAD DE INGENIERÍA Y CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

# Sistema de Información Web y Móvil para la Postulación y Preselección Automatizada de la Beca Comedor mediante Inteligencia Artificial, con Reportes de Business Intelligence para la Dirección Universitaria de Bienestar Social y Salud (DUBSS) de la Universidad Autónoma Gabriel René Moreno

**INTEGRANTES:**

- Barrios Mayorga Carlos Leonel - 222007591
- Gutierrez Primintela Mauro Boris - 222054972

**Materia:** Taller de Grado (INF511 - SS)

**Docente:** Msc. Ing. José Junior Villagómez Melgar

**Gestión:** Semestre 2 – 2026

# Índice General

Índice General . . . . . 2
Índice de Tablas . . . . . 12
Índice de Figuras . . . . . 18
Resumen . . . . . 25
Abstract . . . . . 26
Introducción . . . . . 27
Capítulo 1: Perfil del Proyecto . . . . . 30
1. Antecedentes . . . . . 30
2. Definición del problema . . . . . 30
2.1. Situación problemática . . . . . 30
2.2. Situación deseada . . . . . 32
2.3. Objeto de la investigación . . . . . 33
3. Objetivos . . . . . 35
3.1. Objetivo General . . . . . 35
3.2. Objetivos Específicos . . . . . 36
4. Justificación . . . . . 36
4.1. Justificación operativa y técnica . . . . . 36
4.2. Justificación social . . . . . 37
4.3. Justificación económica y operativa: optimización de recursos . . . . . 38
5. Metodología . . . . . 39
5.1. El Proceso Unificado de Desarrollo de Software . . . . . 40
5.1.1. Fases. . . . . 40
5.1.1.1. Inicio. . . . . 41
5.1.1.2. Elaboración. . . . . 41
5.1.1.3. Construcción. . . . . 42
5.1.1.4. Transición. . . . . 42

5.1.2. Flujos de trabajo. . . . . 42
5.1.2.1. Captura de requisitos. . . . . 43
5.1.2.2. Análisis. . . . . 43
5.1.2.3. Diseño. . . . . 43
5.1.2.4. Implementación. . . . . 43
5.1.2.5. Pruebas. . . . . 43
5.2. Unified Modeling Language . . . . . 44
5.2.1. Estructura general de UML. . . . . 44
5.3. Especificación de Requisitos de Software según la IEEE 830 . . . . . 45
5.3.1. Estructura y Propósito del Estándar IEEE 830 (IEEE 830-1998). 45
5.3.2. Relación del IEEE 830 con UML y PUDS. . . . . 46
5.3.3. Reemplazo por IEEE 29148-2011 y Vigencia Actual. . . . . 46
6. Alcances y limitaciones . . . . . 47
6.1. Funciones principales . . . . . 47
6.2. Usuarios o beneficiarios . . . . . 49
6.3. Limitaciones y Restricciones . . . . . 51
7. Recursos necesarios . . . . . 52
7.1. Software y herramientas . . . . . 52
7.2. Equipos y hardware . . . . . 53
7.3. Presupuesto estimado . . . . . 53
8. Cronograma . . . . . 56
8.1. Plan de ejecución . . . . . 56
8.2. Hitos importantes . . . . . 57
Capítulo 2: Inteligencia Artificial y Machine Learning . . . . . 60
2.1. Fundamentos de Inteligencia Artificial y Aprendizaje Automático . . . . . 60
2.1.1. Conceptos Generales de IA Aplicados a la Gestión Social Universitaria 60
2.1.2. Modelos y Algoritmos de Clasificación para Evaluación Socioeconómica 60

2.1.3. Scoring Automático de Vulnerabilidad y Selección Priorizada de Postulantes 61
2.2. Business Intelligence y Analítica de Datos . . . . . 62
2.2.1. Fundamentos de Business Intelligence para Auditoría Institucional . . 62
2.2.2. Procesos ETL (Extract, Transform, Load) para la Integración de Expedientes . . . . . 62
2.2.3. Dashboards y Visualización Dinámica de Indicadores Socioeconómicos 63
2.3. Arquitectura Backend con Spring Boot . . . . . 63
2.3.1. Framework Spring Boot y Lenguaje Java . . . . . 63
2.3.2. Diseño de Servicios Web y Arquitectura RESTful . . . . . 64
2.3.3. Seguridad, Autenticación y Cifrado de Datos (Spring Security y JWT) 64
2.4. Desarrollo Web Frontend con React y TypeScript . . . . . 65
2.4.1. Biblioteca React con TypeScript para Aplicaciones Web Escalables . . 65
2.4.2. Gestión de Estado Asíncrono y Caché con TanStack Query . . . . . 65
2.4.3. Diseño de Interfaces de Usuario para Paneles de Auditoría y Control . 65
2.5. Desarrollo Móvil Multiplataforma con Flutter . . . . . 66
2.5.1. Framework Flutter y Lenguaje Dart . . . . . 66
2.5.2. Arquitectura Móvil para Carga Remota de Expedientes Digitales . . . 66
2.5.3. Consumo de APIs y Manejo de Almacenamiento Local . . . . . 66
2.6. Sistema de Gestión de Bases de Datos . . . . . 66
2.6.1. Base de Datos Relacional PostgreSQL . . . . . 66
2.6.2. Modelado de Datos, Integridad Referencial y Consultas Masivas . . . . 67
2.7. Ecosistema de Desarrollo en Python para Modelos de IA . . . . . 67
Capítulo 3: Dirección Universitaria de Bienestar Social y Salud (DUBSS) . . . . . 70
3.1. Dirección Universitaria de Bienestar Social y Salud (DUBSS) . . . . . 70
3.1.1. Nombre . . . . . 70
3.1.2. Misión . . . . . 70
3.1.3. Visión . . . . . 71

3.1.4. Imagen Corporativa / Marca . . . . . 71
3.1.5. Web, Contacto, Redes Sociales . . . . . 72
3.2. Estructura Organizacional . . . . . 75
3.3. Ubicación e Infraestructura . . . . . 76
3.4. Modelo de Negocio . . . . . 78
3.5. Flujo Operativo . . . . . 79
Capítulo 4: Especificación de Requisitos de Software según la IEEE 830 . . . . . 82
4.1. Introducción . . . . . 82
4.1.1. Propósito . . . . . 82
4.1.2. Ámbito del Sistema . . . . . 82
4.1.3. Definiciones, Acrónimos y Abreviaturas . . . . . 83
4.1.4. Referencias . . . . . 84
4.1.5. Visión General del Documento . . . . . 85
4.2. Descripción General . . . . . 85
4.2.1. Perspectiva del Producto . . . . . 85
4.2.2. Funciones del Producto . . . . . 85
4.2.3. Características de los Usuarios . . . . . 87
4.2.4. Restricciones Generales . . . . . 87
4.2.5. Suposiciones y Dependencias . . . . . 88
4.2.6. Modelo de Dominio . . . . . 89
4.3. Requisitos Específicos . . . . . 91
4.3.1. Requisitos Funcionales . . . . . 91
4.3.2. Requisitos No Funcionales . . . . . 112
4.3.3. Requisitos de Interfaz . . . . . 117
4.3.4. Requisitos de Base de Datos . . . . . 118
4.3.5. Restricciones de Diseño . . . . . 119
4.3.6. Requisitos de Rendimiento . . . . . 119

4.3.6.1. Tiempo de respuesta. . . . . 119
4.3.6.2. Capacidad y carga máxima. . . . . 120
4.3.6.3. Uso de recursos (CPU, memoria y ancho de banda). . . . . 120
4.3.6.4. Disponibilidad y tiempo de actividad (*uptime*). . . . . 120
4.3.6.5. Escalabilidad. . . . . 121
4.3.6.6. Procesamiento en lote o en tiempo real. . . . . 121
Capítulo 5: Captura de Requisitos . . . . . 123
5.1. Identificación de Actores . . . . . 123
5.2. Identificación de Casos de Uso . . . . . 126
5.3. Priorización de Casos de Uso . . . . . 128
5.4. Detalle de Casos de Uso . . . . . 133
5.4.1. CU-01 Registrarse y autenticarse . . . . . 133
5.4.2. CU-02 Enviar postulación a Beca Comedor . . . . . 137
5.4.3. CU-03 Precalificar postulación con Inteligencia Artificial . . . . . 140
5.4.4. CU-04 Consultar estado de postulación . . . . . 142
5.4.5. CU-05 Auditar y adjudicar becas . . . . . 144
5.4.6. CU-06 Visualizar lista priorizada de postulantes . . . . . 147
5.4.7. CU-07 Auditar expediente finalista . . . . . 150
5.4.8. CU-08 Generar acta de adjudicación . . . . . 154
5.4.9. CU-09 Generar planilla de asistencia . . . . . 156
5.4.10. CU-10 Registrar asistencia digital . . . . . 158
5.4.11. CU-11 Cotejar registro contra respaldo físico . . . . . 160
5.4.12. CU-12 Visualizar dashboard de indicadores BI . . . . . 162
5.4.13. CU-13 Gestionar convocatoria y gestión académica . . . . . 164
5.4.14. CU-14 Gestionar usuarios y roles . . . . . 166
5.4.15. CU-15 Recuperar contraseña / gestionar perfil . . . . . 169
5.4.16. CU-16 Digitalizar y adjuntar documentos . . . . . 172

5.4.17. CU-17 Guardar borrador de postulación . . . . . 174
5.4.18. CU-18 Validar elegibilidad y duplicados . . . . . 176
5.4.19. CU-19 Devolver postulación para corrección . . . . . 178
5.4.20. CU-20 Archivar o rechazar postulación . . . . . 181
5.4.21. CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos . . . . . 183
5.4.22. CU-22 Publicar resultados de adjudicación . . . . . 186
5.4.23. CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo) . . 189
5.4.24. CU-24 Consultar historial de asistencia y estado de beca . . . . . 191
5.4.25. CU-25 Consultar bitácora de acciones . . . . . 193
5.4.26. CU-26 Parametrizar modelo de preselección y reprocesar puntajes . . 195
5.5. Diagrama General de Casos de Uso . . . . . 197
Capítulo 6: Análisis . . . . . 200
6.1. Análisis de la Arquitectura . . . . . 200
6.1.1. Identificación de Paquetes . . . . . 200
6.1.2. Relación de Paquetes . . . . . 201
6.1.3. Encapsulamiento de Casos de Uso por Paquetes . . . . . 207
6.2. Análisis de Casos de Uso . . . . . 213
6.2.1. Analizar CU-01 Registrarse y autenticarse . . . . . 214
6.2.2. Analizar CU-02 Enviar postulación a Beca Comedor . . . . . 216
6.2.3. Analizar CU-03 Precalificar postulación con Inteligencia Artificial . . . 216
6.2.4. Analizar CU-04 Consultar estado de postulación . . . . . 217
6.2.5. Analizar CU-05 Auditar y adjudicar becas . . . . . 218
6.2.6. Analizar CU-06 Visualizar lista priorizada de postulantes . . . . . 219
6.2.7. Analizar CU-07 Auditar expediente finalista . . . . . 220
6.2.8. Analizar CU-08 Generar acta de adjudicación . . . . . 221
6.2.9. Analizar CU-09 Generar planilla de asistencia . . . . . 221

6.2.10. Analizar CU-10 Registrar asistencia digital . . . . . 222
6.2.11. Analizar CU-11 Cotejar registro contra respaldo físico . . . . . 223
6.2.12. Analizar CU-12 Visualizar dashboard de indicadores BI . . . . . 224
6.2.13. Analizar CU-13 Gestionar convocatoria y gestión académica . . . . . 225
6.2.14. Analizar CU-14 Gestionar usuarios y roles . . . . . 226
6.2.15. Analizar CU-15 Recuperar contraseña / gestionar perfil . . . . . 226
6.2.16. Analizar CU-16 Digitalizar y adjuntar documentos . . . . . 228
6.2.17. Analizar CU-17 Guardar borrador de postulación . . . . . 228
6.2.18. Analizar CU-18 Validar elegibilidad y duplicados . . . . . 229
6.2.19. Analizar CU-19 Devolver postulación para corrección . . . . . 230
6.2.20. Analizar CU-20 Archivar o rechazar postulación . . . . . 231
6.2.21. Analizar CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos . . . . . 232
6.2.22. Analizar CU-22 Publicar resultados de adjudicación . . . . . 233
6.2.23. Analizar CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo) . . . . . 233
6.2.24. Analizar CU-24 Consultar historial de asistencia y estado de beca . . 234
6.2.25. Analizar CU-25 Consultar bitácora de acciones . . . . . 235
6.2.26. Analizar CU-26 Parametrizar modelo de preselección y reprocesar puntajes 236
6.3. Análisis de Paquetes . . . . . 237
Capítulo 7: Diseño . . . . . 239
7.1. Diseño de la Arquitectura . . . . . 239
7.1.1. Diseño Físico . . . . . 239
7.1.2. Diseño Lógico . . . . . 241
Arquitectura lógica general. . . . . 241
Arquitectura lógica del *backend*. . . . . 243
7.2. Diagrama de Clase . . . . . 245

7.3. Diseño de Datos . . . . . 247
7.3.1. Diseño de Datos Lógico . . . . . 247
7.3.1.1. Diseño lógico de la Base de Datos. . . . . 247
7.3.1.2. Mapeo. . . . . 248
7.3.2. Diseño de Datos Físico . . . . . 263
7.3.2.1. Volumen. . . . . 263
7.3.2.2. Script. . . . . 286
7.4. Diseñar Caso de Uso . . . . . 304
7.4.1. Diseñar CU-01 Registrarse y autenticarse . . . . . 306
7.4.2. Diseñar CU-02 Enviar postulación a Beca Comedor . . . . . 308
7.4.3. Diseñar CU-03 Precalificar postulación con Inteligencia Artificial . . . 309
7.4.4. Diseñar CU-04 Consultar estado de postulación . . . . . 310
7.4.5. Diseñar CU-05 Auditar y adjudicar becas . . . . . 311
7.4.6. Diseñar CU-06 Visualizar lista priorizada de postulantes . . . . . 312
7.4.7. Diseñar CU-07 Auditar expediente finalista . . . . . 313
7.4.8. Diseñar CU-08 Generar acta de adjudicación . . . . . 314
7.4.9. Diseñar CU-09 Generar planilla de asistencia . . . . . 315
7.4.10. Diseñar CU-10 Registrar asistencia digital . . . . . 316
7.4.11. Diseñar CU-11 Cotejar registro contra respaldo físico . . . . . 317
7.4.12. Diseñar CU-12 Visualizar dashboard de indicadores BI . . . . . 318
7.4.13. Diseñar CU-13 Gestionar convocatoria y gestión académica . . . . . 319
7.4.14. Diseñar CU-14 Gestionar usuarios y roles . . . . . 320
7.4.15. Diseñar CU-15 Recuperar contraseña / gestionar perfil . . . . . 321
7.4.16. Diseñar CU-16 Digitalizar y adjuntar documentos . . . . . 322
7.4.17. Diseñar CU-17 Guardar borrador de postulación . . . . . 323
7.4.18. Diseñar CU-18 Validar elegibilidad y duplicados . . . . . 324
7.4.19. Diseñar CU-19 Devolver postulación para corrección . . . . . 325

7.4.20. Diseñar CU-20 Archivar o rechazar postulación . . . . . 326
7.4.21. Diseñar CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos . . . . . 327
7.4.22. Diseñar CU-22 Publicar resultados de adjudicación . . . . . 328
7.4.23. Diseñar CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo) 329
7.4.24. Diseñar CU-24 Consultar historial de asistencia y estado de beca . . . 330
7.4.25. Diseñar CU-25 Consultar bitácora de acciones . . . . . 331
7.4.26. Diseñar CU-26 Parametrizar modelo de preselección y reprocesar puntajes 332
7.5. Diseño de Interfaces . . . . . 332
7.5.1. Interfaz CU-01 Registrarse y autenticarse . . . . . 333
7.5.2. Interfaz CU-02 Enviar postulación a Beca Comedor . . . . . 334
7.5.3. Interfaz CU-03 Precalificar postulación con Inteligencia Artificial . . . 335
7.5.4. Interfaz CU-04 Consultar estado de postulación . . . . . 336
7.5.5. Interfaz CU-05 Auditar y adjudicar becas . . . . . 337
7.5.6. Interfaz CU-06 Visualizar lista priorizada de postulantes . . . . . 338
7.5.7. Interfaz CU-07 Auditar expediente finalista . . . . . 339
7.5.8. Interfaz CU-08 Generar acta de adjudicación . . . . . 340
7.5.9. Interfaz CU-09 Generar planilla de asistencia . . . . . 341
7.5.10. Interfaz CU-10 Registrar asistencia digital . . . . . 342
7.5.11. Interfaz CU-11 Cotejar registro contra respaldo físico . . . . . 343
7.5.12. Interfaz CU-12 Visualizar dashboard de indicadores BI . . . . . 344
7.5.13. Interfaz CU-13 Gestionar convocatoria y gestión académica . . . . . 345
7.5.14. Interfaz CU-14 Gestionar usuarios y roles . . . . . 346
7.5.15. Interfaz CU-15 Recuperar contraseña / gestionar perfil . . . . . 347
7.5.16. Interfaz CU-16 Digitalizar y adjuntar documentos . . . . . 348
7.5.17. Interfaz CU-17 Guardar borrador de postulación . . . . . 349
7.5.18. Interfaz CU-18 Validar elegibilidad y duplicados . . . . . 350

7.5.19. Interfaz CU-19 Devolver postulación para corrección . . . . . 351
7.5.20. Interfaz CU-20 Archivar o rechazar postulación . . . . . 352
7.5.21. Interfaz CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos . . . . . 353
7.5.22. Interfaz CU-22 Publicar resultados de adjudicación . . . . . 354
7.5.23. Interfaz CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo) 355
7.5.24. Interfaz CU-24 Consultar historial de asistencia y estado de beca . . . 356
7.5.25. Interfaz CU-25 Consultar bitácora de acciones . . . . . 357
7.5.26. Interfaz CU-26 Parametrizar modelo de preselección y reprocesar puntajes 358
Conclusiones y Recomendaciones . . . . . 359
Conclusiones . . . . . 359
Recomendaciones . . . . . 359
Bibliografía . . . . . 362
Anexo: Carta de Solicitud de Desarrollo del Proyecto . . . . . 363

## Índice de Tablas

**Tabla 1.** Comparativa del proceso de asignación de la beca comedor: situación actual frente a la esperada con el sistema propuesto . . . . . 38

**Tabla 2.** Proyección de la recepción y gestión documental: proceso actual frente al esquema de preselección automática . . . . . 39

**Tabla 3.** Usuarios y beneficiarios del sistema . . . . . 50

**Tabla 4.** Software y herramientas utilizados para el desarrollo del proyecto . . . 52

**Tabla 5.** Equipos y hardware requeridos para el desarrollo del proyecto . . . . . 53

**Tabla 6.** Presupuesto estimado del proyecto: hardware y software . . . . . 54

**Tabla 7.** Presupuesto estimado del proyecto: recursos humanos, infraestructura y logística . . . . . 55

**Tabla 8.** Hitos del proyecto según las fases de la materia . . . . . 58

**Tabla 9.** Definiciones, acrónimos y abreviaturas empleados en el documento . . 84

**Tabla 10.** Tipos de usuario del sistema y sus características . . . . . 87

**Tabla 11.** Requerimiento funcional RF-01: Registro y autenticación de usuarios . 91

**Tabla 12.** Requerimiento funcional RF-02: Envío de postulación a la Beca Comedor 93

**Tabla 13.** Requerimiento funcional RF-03: Preselección automatizada con Inteligencia Artificial . . . . . 94

**Tabla 14.** Requerimiento funcional RF-04: Panel de monitoreo y toma de decisiones (auditoría y adjudicación) . . . . . 95

**Tabla 15.** Requerimiento funcional RF-05: Registro e impresión de planillas de asistencia . . . . . 96

**Tabla 16.** Requerimiento funcional RF-06: Consultas y notificaciones al postulante 97

**Tabla 17.** Requerimiento funcional RF-07: Dashboard de indicadores de Business Intelligence . . . . . 98

**Tabla 18.** Requerimiento funcional RF-08: Gestión de convocatorias y gestiones académicas . . . . . 99

**Tabla 19.** Requerimiento funcional RF-09: Gestión de usuarios y roles . . . . . 100
**Tabla 20.** Requerimiento funcional RF-10: Recuperación de contraseña y gestión de perfil . . . . . 101
**Tabla 21.** Requerimiento funcional RF-11: Digitalización y carga de documentos de respaldo . . . . . 102
**Tabla 22.** Requerimiento funcional RF-12: Guardado de borrador de postulación 103
**Tabla 23.** Requerimiento funcional RF-13: Validación de elegibilidad y unicidad de postulaciones . . . . . 104
**Tabla 24.** Requerimiento funcional RF-14: Devolución de postulación para corrección 105
**Tabla 25.** Requerimiento funcional RF-15: Archivo o rechazo de postulación . . . 106
**Tabla 26.** Requerimiento funcional RF-16: Notificación a finalistas, entrega de documentación física y comunicados . . . . . 107
**Tabla 27.** Requerimiento funcional RF-17: Publicación de resultados de adjudicación 108
**Tabla 28.** Requerimiento funcional RF-18: Gestión del padrón de becados . . . . 109
**Tabla 29.** Requerimiento funcional RF-19: Historial de asistencia y estado de la beca 110
**Tabla 30.** Requerimiento funcional RF-20: Bitácora de acciones . . . . . 111
**Tabla 31.** Requerimiento funcional RF-21: Parametrización del modelo de preselección y reprocesamiento . . . . . 112
**Tabla 32.** Requerimiento no funcional RNF-01: Seguridad y protección de datos personales . . . . . 113
**Tabla 33.** Requerimiento no funcional RNF-02: Disponibilidad del servicio . . . . 114
**Tabla 34.** Requerimiento no funcional RNF-03: Tiempo de respuesta y concurrencia 115
**Tabla 35.** Requerimiento no funcional RNF-04: Usabilidad . . . . . 116
**Tabla 36.** Requerimiento no funcional RNF-05: Compatibilidad y restricciones tecnológicas . . . . . 117
**Tabla 37.** Identificación de casos de uso del sistema . . . . . 126
**Tabla 38.** Casos de uso de prioridad alta . . . . . 128

**Tabla 39.** Casos de uso de prioridad media . . . . . 130
**Tabla 40.** Casos de uso de prioridad baja . . . . . 131
**Tabla 41.** Detalle del CU-01: Registrarse y autenticarse . . . . . 133
**Tabla 42.** Detalle del CU-02: Enviar postulación a Beca Comedor . . . . . 137
**Tabla 43.** Detalle del CU-03: Precalificar postulación con Inteligencia Artificial . 140
**Tabla 44.** Detalle del CU-04: Consultar estado de postulación . . . . . 142
**Tabla 45.** Detalle del CU-05: Auditar y adjudicar becas . . . . . 144
**Tabla 46.** Detalle del CU-06: Visualizar lista priorizada de postulantes . . . . . 147
**Tabla 47.** Detalle del CU-07: Auditar expediente finalista . . . . . 150
**Tabla 48.** Detalle del CU-08: Generar acta de adjudicación . . . . . 154
**Tabla 49.** Detalle del CU-09: Generar planilla de asistencia . . . . . 156
**Tabla 50.** Detalle del CU-10: Registrar asistencia digital . . . . . 159
**Tabla 51.** Detalle del CU-11: Cotejar registro contra respaldo físico . . . . . 160
**Tabla 52.** Detalle del CU-12: Visualizar dashboard de indicadores BI . . . . . 162
**Tabla 53.** Detalle del CU-13: Gestionar convocatoria y gestión académica . . . . 164
**Tabla 54.** Detalle del CU-14: Gestionar usuarios y roles . . . . . 166
**Tabla 55.** Detalle del CU-15: Recuperar contraseña / gestionar perfil . . . . . 169
**Tabla 56.** Detalle del CU-16: Digitalizar y adjuntar documentos . . . . . 172
**Tabla 57.** Detalle del CU-17: Guardar borrador de postulación . . . . . 174
**Tabla 58.** Detalle del CU-18: Validar elegibilidad y duplicados . . . . . 176
**Tabla 59.** Detalle del CU-19: Devolver postulación para corrección . . . . . 178
**Tabla 60.** Detalle del CU-20: Archivar o rechazar postulación . . . . . 181
**Tabla 61.** Detalle del CU-21: Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos . . . . . 183
**Tabla 62.** Detalle del CU-22: Publicar resultados de adjudicación . . . . . 186
**Tabla 63.** Detalle del CU-23: Gestionar padrón de becados (baja, suspensión, reemplazo) . . . . . 189

**Tabla 64.** Detalle del CU-24: Consultar historial de asistencia y estado de beca . 191
**Tabla 65.** Detalle del CU-25: Consultar bitácora de acciones . . . . . 193
**Tabla 66.** Detalle del CU-26: Parametrizar modelo de preselección y reprocesar puntajes . . . . . 195
**Tabla 67.** Mapeo de la tabla rol . . . . . 249
**Tabla 68.** Mapeo de la tabla usuario . . . . . 249
**Tabla 69.** Mapeo de la tabla carrera . . . . . 250
**Tabla 70.** Mapeo de la tabla sesion . . . . . 250
**Tabla 71.** Mapeo de la tabla postulante . . . . . 250
**Tabla 72.** Mapeo de la tabla gestion_academica . . . . . 251
**Tabla 73.** Mapeo de la tabla convocatoria . . . . . 251
**Tabla 74.** Mapeo de la tabla tipo_documento . . . . . 252
**Tabla 75.** Mapeo de la tabla convocatoria_documento . . . . . 252
**Tabla 76.** Mapeo de la tabla postulacion . . . . . 252
**Tabla 77.** Mapeo de la tabla respuesta_socioeconomica . . . . . 253
**Tabla 78.** Mapeo de la tabla nota_academica . . . . . 253
**Tabla 79.** Mapeo de la tabla documento_adjunto . . . . . 254
**Tabla 80.** Mapeo de la tabla postulacion_historial . . . . . 254
**Tabla 81.** Mapeo de la tabla modelo_preseleccion . . . . . 255
**Tabla 82.** Mapeo de la tabla parametro_modelo . . . . . 255
**Tabla 83.** Mapeo de la tabla puntaje_postulacion . . . . . 256
**Tabla 84.** Mapeo de la tabla lote_reprocesamiento . . . . . 256
**Tabla 85.** Mapeo de la tabla finalista . . . . . 257
**Tabla 86.** Mapeo de la tabla auditoria_expediente . . . . . 257
**Tabla 87.** Mapeo de la tabla adjudicacion . . . . . 258
**Tabla 88.** Mapeo de la tabla acta_adjudicacion . . . . . 258
**Tabla 89.** Mapeo de la tabla publicacion_resultado . . . . . 258

**Tabla 90.** Mapeo de la tabla comunicado . . . . . 259
**Tabla 91.** Mapeo de la tabla notificacion . . . . . 259
**Tabla 92.** Mapeo de la tabla dispositivo_fcm . . . . . 260
**Tabla 93.** Mapeo de la tabla beca . . . . . 260
**Tabla 94.** Mapeo de la tabla movimiento_beca . . . . . 261
**Tabla 95.** Mapeo de la tabla planilla_asistencia . . . . . 261
**Tabla 96.** Mapeo de la tabla asistencia . . . . . 262
**Tabla 97.** Mapeo de la tabla cotejo_asistencia . . . . . 262
**Tabla 98.** Mapeo de la tabla bitacora . . . . . 263
**Tabla 99.** Volumen de la tabla rol . . . . . 263
**Tabla 100.** Volumen de la tabla usuario . . . . . 264
**Tabla 101.** Volumen de la tabla carrera . . . . . 265
**Tabla 102.** Volumen de la tabla sesion . . . . . 265
**Tabla 103.** Volumen de la tabla postulante . . . . . 266
**Tabla 104.** Volumen de la tabla gestion_academica . . . . . 267
**Tabla 105.** Volumen de la tabla convocatoria . . . . . 267
**Tabla 106.** Volumen de la tabla tipo_documento . . . . . 268
**Tabla 107.** Volumen de la tabla convocatoria_documento . . . . . 269
**Tabla 108.** Volumen de la tabla postulacion . . . . . 269
**Tabla 109.** Volumen de la tabla respuesta_socioeconomica . . . . . 270
**Tabla 110.** Volumen de la tabla nota_academica . . . . . 271
**Tabla 111.** Volumen de la tabla documento_adjunto . . . . . 271
**Tabla 112.** Volumen de la tabla postulacion_historial . . . . . 272
**Tabla 113.** Volumen de la tabla modelo_preseleccion . . . . . 273
**Tabla 114.** Volumen de la tabla parametro_modelo . . . . . 273
**Tabla 115.** Volumen de la tabla puntaje_postulacion . . . . . 274
**Tabla 116.** Volumen de la tabla lote_reprocesamiento . . . . . 275

**Tabla 117.** Volumen de la tabla finalista . . . . . 276
**Tabla 118.** Volumen de la tabla auditoria_expediente . . . . . 276
**Tabla 119.** Volumen de la tabla adjudicacion . . . . . 277
**Tabla 120.** Volumen de la tabla acta_adjudicacion . . . . . 278
**Tabla 121.** Volumen de la tabla publicacion_resultado . . . . . 279
**Tabla 122.** Volumen de la tabla comunicado . . . . . 279
**Tabla 123.** Volumen de la tabla notificacion . . . . . 280
**Tabla 124.** Volumen de la tabla dispositivo_fcm . . . . . 281
**Tabla 125.** Volumen de la tabla beca . . . . . 281
**Tabla 126.** Volumen de la tabla movimiento_beca . . . . . 282
**Tabla 127.** Volumen de la tabla planilla_asistencia . . . . . 283
**Tabla 128.** Volumen de la tabla asistencia . . . . . 283
**Tabla 129.** Volumen de la tabla cotejo_asistencia . . . . . 284
**Tabla 130.** Volumen de la tabla bitacora . . . . . 285

## Índice de Figuras

**Figura 1.** Distribución porcentual de las solicitudes de beca comedor por destino en el proceso manual de revisión . . . . . 32

**Figura 2.** Objeto de la investigación: proceso de postulación y preselección de la Beca Comedor apoyado por el sistema de información . . . . . 34

**Figura 3.** Evolución de la selección de la Beca Comedor: de la revisión manual a la precalificación automática . . . . . 35

**Figura 4.** Embudo del proceso manual de gestión de la beca comedor: reducción de las 20,000 solicitudes recibidas a los 900 cupos adjudicados . . . . . 37

**Figura 5.** Las cuatro fases del Proceso Unificado de Desarrollo de Software y su relación con las disciplinas de ingeniería del proceso . . . . . 41

**Figura 6.** Representación gráfica del Lenguaje Unificado de Modelado (UML) . . 44

**Figura 7.** Diagrama de Gantt del plan de ejecución del proyecto: proceso unificado aplicado al Sistema de Gestión Integral del Comedor Universitario (24 de agosto al 30 de noviembre de 2026) . . . . . 56

**Figura 8.** Logotipo del framework Spring Boot . . . . . 64

**Figura 9.** Logotipo de la biblioteca React . . . . . 65

**Figura 10.** Logotipo del sistema de gestión de bases de datos PostgreSQL . . . . . 67

**Figura 11.** Logotipo del lenguaje de programación Python . . . . . 68

**Figura 12.** Logotipo institucional de la Dirección Universitaria de Bienestar Social y Salud (D.U.B.S.S.) de la UAGRM . . . . . 71

**Figura 13.** Sitio web oficial de la Dirección Universitaria de Bienestar Social y Salud (DUBSS) . . . . . 72

**Figura 14.** Página oficial de Facebook de la Dirección Universitaria de Bienestar Social y Salud (DUBSS) . . . . . 73

**Figura 15.** Perfil oficial de Instagram de la Dirección Universitaria de Bienestar Social y Salud (DUBSS) . . . . . 74

**Figura 16.** Organigrama de la Dirección Universitaria de Bienestar Social y Salud (DUBSS) . . . . . 75
**Figura 17.** Comedor Universitario de la Universidad Autónoma Gabriel René Moreno 76
**Figura 18.** Oficinas de la Dirección Universitaria de Bienestar Social y Salud (DUBSS) 76
**Figura 19.** Ubicación de las instalaciones de la DUBSS en el campus universitario 77
**Figura 20.** Modelo de negocio propuesto para el Sistema de Información de la Beca Comedor . . . . . 78
**Figura 21.** Diagrama de actividades del proceso de postulación a la Beca Comedor 80
**Figura 22.** Diagrama de casos de uso del Sistema Web y Móvil de Postulación y Preselección de la Beca Comedor . . . . . 86
**Figura 23.** Modelo de dominio del Sistema Web y Móvil de Postulación y Preselección de la Beca Comedor . . . . . 89
**Figura 24.** Identificación de actores . . . . . 123
**Figura 25.** Descripción de actores . . . . . 125
**Figura 26.** Diagrama de contexto del CU-01 . . . . . 133
**Figura 27.** Diagrama de contexto del CU-02 . . . . . 137
**Figura 28.** Diagrama de contexto del CU-03 . . . . . 140
**Figura 29.** Diagrama de contexto del CU-04 . . . . . 142
**Figura 30.** Diagrama de contexto del CU-05 . . . . . 144
**Figura 31.** Diagrama de contexto del CU-06 . . . . . 147
**Figura 32.** Diagrama de contexto del CU-07 . . . . . 150
**Figura 33.** Diagrama de contexto del CU-08 . . . . . 154
**Figura 34.** Diagrama de contexto del CU-09 . . . . . 156
**Figura 35.** Diagrama de contexto del CU-10 . . . . . 158
**Figura 36.** Diagrama de contexto del CU-11 . . . . . 160
**Figura 37.** Diagrama de contexto del CU-12 . . . . . 162
**Figura 38.** Diagrama de contexto del CU-13 . . . . . 164

**Figura 39.** Diagrama de contexto del CU-14 . . . . . 166
**Figura 40.** Diagrama de contexto del CU-15 . . . . . 169
**Figura 41.** Diagrama de contexto del CU-16 . . . . . 172
**Figura 42.** Diagrama de contexto del CU-17 . . . . . 174
**Figura 43.** Diagrama de contexto del CU-18 . . . . . 176
**Figura 44.** Diagrama de contexto del CU-19 . . . . . 178
**Figura 45.** Diagrama de contexto del CU-20 . . . . . 181
**Figura 46.** Diagrama de contexto del CU-21 . . . . . 183
**Figura 47.** Diagrama de contexto del CU-22 . . . . . 186
**Figura 48.** Diagrama de contexto del CU-23 . . . . . 189
**Figura 49.** Diagrama de contexto del CU-24 . . . . . 191
**Figura 50.** Diagrama de contexto del CU-25 . . . . . 193
**Figura 51.** Diagrama de contexto del CU-26 . . . . . 195
**Figura 52.** Diagrama general de casos de uso del Sistema Web y Móvil de Postulación y Preselección de la Beca Comedor . . . . . 198
**Figura 53.** Identificación de paquetes . . . . . 200
**Figura 54.** Relación del paquete Cuentas y Acceso . . . . . 202
**Figura 55.** Relación del paquete Convocatoria y Postulación . . . . . 203
**Figura 56.** Relación del paquete Preselección con IA . . . . . 204
**Figura 57.** Relación del paquete Auditoría y Adjudicación . . . . . 205
**Figura 58.** Relación del paquete Becados y Asistencia . . . . . 206
**Figura 59.** Relación del paquete Reportes y Trazabilidad . . . . . 207
**Figura 60.** Encapsulamiento de casos de uso del paquete Cuentas y Acceso . . . . 208
**Figura 61.** Encapsulamiento de casos de uso del paquete Convocatoria y Postulación 209
**Figura 62.** Encapsulamiento de casos de uso del paquete Preselección con IA . . . 210
**Figura 63.** Encapsulamiento de casos de uso del paquete Auditoría y Adjudicación 211
**Figura 64.** Encapsulamiento de casos de uso del paquete Becados y Asistencia . . 212

**Figura 65.** Encapsulamiento de casos de uso del paquete Reportes y Trazabilidad 213
**Figura 66.** Análisis del CU-01 Registrarse y autenticarse . . . . . 215
**Figura 67.** Análisis del CU-02 Enviar postulación a Beca Comedor . . . . . 216
**Figura 68.** Análisis del CU-03 Precalificar postulación con Inteligencia Artificial . 217
**Figura 69.** Análisis del CU-04 Consultar estado de postulación . . . . . 217
**Figura 70.** Análisis del CU-05 Auditar y adjudicar becas . . . . . 218
**Figura 71.** Análisis del CU-06 Visualizar lista priorizada de postulantes . . . . . 219
**Figura 72.** Análisis del CU-07 Auditar expediente finalista . . . . . 220
**Figura 73.** Análisis del CU-08 Generar acta de adjudicación . . . . . 221
**Figura 74.** Análisis del CU-09 Generar planilla de asistencia . . . . . 222
**Figura 75.** Análisis del CU-10 Registrar asistencia digital . . . . . 223
**Figura 76.** Análisis del CU-11 Cotejar registro contra respaldo físico . . . . . 224
**Figura 77.** Análisis del CU-12 Visualizar dashboard de indicadores BI . . . . . 225
**Figura 78.** Análisis del CU-13 Gestionar convocatoria y gestión académica . . . . 225
**Figura 79.** Análisis del CU-14 Gestionar usuarios y roles . . . . . 226
**Figura 80.** Análisis del CU-15 Recuperar contraseña / gestionar perfil . . . . . 227
**Figura 81.** Análisis del CU-16 Digitalizar y adjuntar documentos . . . . . 228
**Figura 82.** Análisis del CU-17 Guardar borrador de postulación . . . . . 229
**Figura 83.** Análisis del CU-18 Validar elegibilidad y duplicados . . . . . 229
**Figura 84.** Análisis del CU-19 Devolver postulación para corrección . . . . . 230
**Figura 85.** Análisis del CU-20 Archivar o rechazar postulación . . . . . 231
**Figura 86.** Análisis del CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos . . . . . 232
**Figura 87.** Análisis del CU-22 Publicar resultados de adjudicación . . . . . 233
**Figura 88.** Análisis del CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo) . . . . . 234
**Figura 89.** Análisis del CU-24 Consultar historial de asistencia y estado de beca . 235

**Figura 90.** Análisis del CU-25 Consultar bitácora de acciones . . . . . 235
**Figura 91.** Análisis del CU-26 Parametrizar modelo de preselección y reprocesar puntajes . . . . . 236
**Figura 92.** Diagrama de análisis de paquetes del Sistema Web y Móvil de Postulación y Preselección de la Beca Comedor . . . . . 237
**Figura 93.** Diagrama de despliegue general del sistema . . . . . 240
**Figura 94.** Arquitectura lógica general del sistema en capas . . . . . 242
**Figura 95.** Arquitectura lógica del *backend* (Spring Boot) . . . . . 244
**Figura 96.** Diagrama de clases del sistema . . . . . 246
**Figura 97.** Diseño lógico de la base de datos . . . . . 248
**Figura 98.** Diagrama de secuencia del CU-01 Registrarse y autenticarse . . . . . 306
**Figura 99.** Diagrama de secuencia del CU-02 Enviar postulación a Beca Comedor 308
**Figura 100.** Diagrama de secuencia del CU-03 Precalificar postulación con Inteligencia Artificial . . . . . 309
**Figura 101.** Diagrama de secuencia del CU-04 Consultar estado de postulación . . 310
**Figura 102.** Diagrama de secuencia del CU-05 Auditar y adjudicar becas . . . . . 311
**Figura 103.** Diagrama de secuencia del CU-06 Visualizar lista priorizada de postulantes 312
**Figura 104.** Diagrama de secuencia del CU-07 Auditar expediente finalista . . . . . 313
**Figura 105.** Diagrama de secuencia del CU-08 Generar acta de adjudicación . . . . 314
**Figura 106.** Diagrama de secuencia del CU-09 Generar planilla de asistencia . . . . 315
**Figura 107.** Diagrama de secuencia del CU-10 Registrar asistencia digital . . . . . 316
**Figura 108.** Diagrama de secuencia del CU-11 Cotejar registro contra respaldo físico 317
**Figura 109.** Diagrama de secuencia del CU-12 Visualizar dashboard de indicadores BI 318
**Figura 110.** Diagrama de secuencia del CU-13 Gestionar convocatoria y gestión académica . . . . . 319
**Figura 111.** Diagrama de secuencia del CU-14 Gestionar usuarios y roles . . . . . 320
**Figura 112.** Diagrama de secuencia del CU-15 Recuperar contraseña / gestionar perfil 321

**Figura 113.** Diagrama de secuencia del CU-16 Digitalizar y adjuntar documentos . 322
**Figura 114.** Diagrama de secuencia del CU-17 Guardar borrador de postulación . . 323
**Figura 115.** Diagrama de secuencia del CU-18 Validar elegibilidad y duplicados . . 324
**Figura 116.** Diagrama de secuencia del CU-19 Devolver postulación para corrección 325
**Figura 117.** Diagrama de secuencia del CU-20 Archivar o rechazar postulación . . . 326
**Figura 118.** Diagrama de secuencia del CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos . . . . . 327
**Figura 119.** Diagrama de secuencia del CU-22 Publicar resultados de adjudicación . 328
**Figura 120.** Diagrama de secuencia del CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo) . . . . . 329
**Figura 121.** Diagrama de secuencia del CU-24 Consultar historial de asistencia y estado de beca . . . . . 330
**Figura 122.** Diagrama de secuencia del CU-25 Consultar bitácora de acciones . . . 331
**Figura 123.** Diagrama de secuencia del CU-26 Parametrizar modelo de preselección y reprocesar puntajes . . . . . 332
**Figura 124.** Maqueta de interfaz del CU-01 Registrarse y autenticarse . . . . . 333
**Figura 125.** Maqueta de interfaz del CU-02 Enviar postulación a Beca Comedor . . 334
**Figura 126.** Maqueta de interfaz del CU-03 Precalificar postulación con Inteligencia Artificial . . . . . 335
**Figura 127.** Maqueta de interfaz del CU-04 Consultar estado de postulación . . . . 336
**Figura 128.** Maqueta de interfaz del CU-05 Auditar y adjudicar becas . . . . . 337
**Figura 129.** Maqueta de interfaz del CU-06 Visualizar lista priorizada de postulantes 338
**Figura 130.** Maqueta de interfaz del CU-07 Auditar expediente finalista . . . . . 339
**Figura 131.** Maqueta de interfaz del CU-08 Generar acta de adjudicación . . . . . 340
**Figura 132.** Maqueta de interfaz del CU-09 Generar planilla de asistencia . . . . . 341
**Figura 133.** Maqueta de interfaz del CU-10 Registrar asistencia digital . . . . . 342
**Figura 134.** Maqueta de interfaz del CU-11 Cotejar registro contra respaldo físico . 343

**Figura 135.** Maqueta de interfaz del CU-12 Visualizar dashboard de indicadores BI 344

**Figura 136.** Maqueta de interfaz del CU-13 Gestionar convocatoria y gestión académica 345

**Figura 137.** Maqueta de interfaz del CU-14 Gestionar usuarios y roles . . . . . 346

**Figura 138.** Maqueta de interfaz del CU-15 Recuperar contraseña / gestionar perfil 347

**Figura 139.** Maqueta de interfaz del CU-16 Digitalizar y adjuntar documentos . . . 348

**Figura 140.** Maqueta de interfaz del CU-17 Guardar borrador de postulación . . . 349

**Figura 141.** Maqueta de interfaz del CU-18 Validar elegibilidad y duplicados . . . . 350

**Figura 142.** Maqueta de interfaz del CU-19 Devolver postulación para corrección . 351

**Figura 143.** Maqueta de interfaz del CU-20 Archivar o rechazar postulación . . . . 352

**Figura 144.** Maqueta de interfaz del CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos . . . . . 353

**Figura 145.** Maqueta de interfaz del CU-22 Publicar resultados de adjudicación . . 354

**Figura 146.** Maqueta de interfaz del CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo) . . . . . 355

**Figura 147.** Maqueta de interfaz del CU-24 Consultar historial de asistencia y estado de beca . . . . . 356

**Figura 148.** Maqueta de interfaz del CU-25 Consultar bitácora de acciones . . . . . 357

**Figura 149.** Maqueta de interfaz del CU-26 Parametrizar modelo de preselección y reprocesar puntajes . . . . . 358

**Figura A1.** Carta de solicitud de desarrollo de proyecto dirigida a la DUBSS . . . 364

**Resumen**

El presente proyecto desarrolla un sistema de información web y móvil para optimizar la postulación y selección de estudiantes a la «Beca Comedor» de la Universidad Autónoma Gabriel René Moreno (UAGRM), gestionada por la Dirección Universitaria de Bienestar Social y Salud (DUBSS). Actualmente, la revisión de cerca de 20,000 carpetas por gestión se realiza de forma manual, lo que genera cuellos de botella, tiempos de respuesta prolongados y solicitudes que nunca llegan a ser revisadas. Además, los estudiantes carecen de un canal digital unificado para postular y consultar el estado de su solicitud.

La solución consiste en un portal web para la gestión administrativa y una aplicación móvil para los estudiantes. Su núcleo es un modelo de Inteligencia Artificial que preselecciona las solicitudes, calcula un puntaje de vulnerabilidad socioeconómica y prioriza las carpetas que el personal debe auditar. Complementariamente, un módulo de Business Intelligence ofrece tableros interactivos con indicadores sobre la preselección y el uso del comedor, que apoyan la toma de decisiones de las autoridades.

El desarrollo se conduce con el Proceso Unificado de Desarrollo de Software (PUDS), modelado con UML, y los requisitos se especifican según el estándar IEEE 830. El resultado esperado es un proceso de becas más ágil, transparente y auditable.

*Palabras clave: Sistema de Información, Inteligencia Artificial, Business Intelligence, PUDS, IEEE 830.*

**Abstract**

This project develops a web and mobile information system to optimize the application and selection process for the «Beca Comedor» (Dining Hall Scholarship) at the Universidad Autónoma Gabriel René Moreno (UAGRM), managed by the University Directorate of Social Welfare and Health (DUBSS). Currently, the review of about 20,000 application folders per academic term is done manually, which causes bottlenecks, long response times, and applications that are never reviewed. Students also lack a unified digital channel to apply and to check the status of their application.

The solution consists of a web portal for administrative management and a mobile application for students. Its core is an Artificial Intelligence model that pre-selects applications, computes a socioeconomic vulnerability score, and prioritizes the folders that staff must audit. Additionally, a Business Intelligence module provides interactive dashboards with indicators on pre-selection and dining hall usage, supporting decision-making by the authorities.

Development follows the Unified Software Development Process (PUDS), modeled with UML, and requirements are specified according to the IEEE 830 standard. The expected result is a faster, more transparent, and auditable scholarship process.

*Keywords: Information System, Artificial Intelligence, Business Intelligence, PUDS, IEEE 830.*

## Introducción

La solicitud de becas es un elemento esencial para garantizar la estabilidad académica y económica de los estudiantes durante su formación universitaria. Sin embargo, el proceso actual de calificación y selección de los beneficiarios de la «Beca Comedor» en la UAGRM es manual, tedioso y susceptible a demoras. Por ello, este proyecto propone el desarrollo de un sistema de información integral impulsado por Inteligencia Artificial (IA) para la preselección automatizada.

Esta tecnología permitirá optimizar los tiempos de revisión y filtrar de manera eficiente aquellas postulaciones que presenten inconsistencias o que no cumplan con los requisitos mínimos establecidos para obtener el beneficio. Adicionalmente, la propuesta aborda la necesidad de mejorar la comunicación y el análisis de datos mediante un ecosistema digital conectado. Una aplicación móvil facilitará a los universitarios la postulación remota y la consulta transparente del menú diario. Simultáneamente, un portal web dotado de herramientas de Business Intelligence (BI) proveerá a la administración de tableros interactivos para monitorear indicadores clave del servicio, apoyando una toma de decisiones estratégica e informada.

El estudio se justifica por la necesidad imperante de modernizar y agilizar este proceso administrativo. Para estructurar este desarrollo, se aplica el Proceso Unificado de Desarrollo de Software (PUDS) como marco metodológico, junto con el estándar IEEE 830 para la especificación de requisitos de software.

El presente documento detalla todo el ciclo de vida del proyecto, desde el análisis de requerimientos hasta su implementación técnica. Asimismo, detalla el diseño arquitectónico, entrenamiento de IA, escalabilidad y los estándares de calidad.

Esta investigación no solo contribuye a mejorar la eficiencia operativa en la administración de la DUBSS, sino que establece un marco metodológico y tecnológico de referencia para futuros desarrollos que integren modelos de IA para la automatización de procesos administrativos y la toma de decisiones basada en datos dentro de la universidad.

El desarrollo del presente proyecto fue formalmente autorizado mediante solicitud cursada por la Carrera de Ingeniería en Sistemas a la DUBSS (ver Anexo).

Capítulo 1

# Perfil del Proyecto

# Capítulo 1: Perfil del Proyecto

## 1. Antecedentes

El comedor universitario constituye un servicio esencial dentro del bienestar estudiantil en las instituciones de educación superior, pues garantiza que los estudiantes —especialmente aquellos con severas limitaciones socioeconómicas— accedan a una alimentación adecuada, equilibrada y asequible (cubriendo prestaciones de desayuno, almuerzo y cena). Históricamente, estos programas de asistencia social han cumplido un rol determinante al contribuir a la permanencia estudiantil, disminuir la deserción por vulnerabilidad económica y favorecer un rendimiento académico óptimo.

En el modelo institucional actual, la administración del servicio de alimentación se encuentra tercerizada a una empresa privada. Bajo este esquema de convenio corporativo, la empresa concesionaria asume la operación del comedor a cambio de la libertad de comercializar platos a la comunidad universitaria regular (estudiantes no becados, docentes y administrativos), percibiendo ingresos directos por dichas ventas. A su vez, como parte del acuerdo institucional, la empresa privada asume compromisos de retribución como la atención alimentaria de comitivas, eventos oficiales e invitados especiales de la universidad.

Sin embargo, mientras que la logística operativa de la cocina y el expendio directo de alimentos recae en el sector privado, la responsabilidad de evaluar, seleccionar y adjudicar el beneficio de la beca comedor sigue siendo exclusivamente institucional. Tradicionalmente, este proceso de postulación se realiza de forma manual mediante la recepción masiva de documentación en físico (folders), lo cual genera una brecha crítica entre la alta demanda estudiantil y la capacidad operativa de revisión por parte de la administración universitaria.

## 2. Definición del problema

### *2.1. Situación problemática*

Actualmente, el proceso de postulación y asignación de la beca comedor se enfrenta a una ineficiencia estructural severa debido a la falta de herramientas tecnológicas automatizadas de filtrado y evaluación.

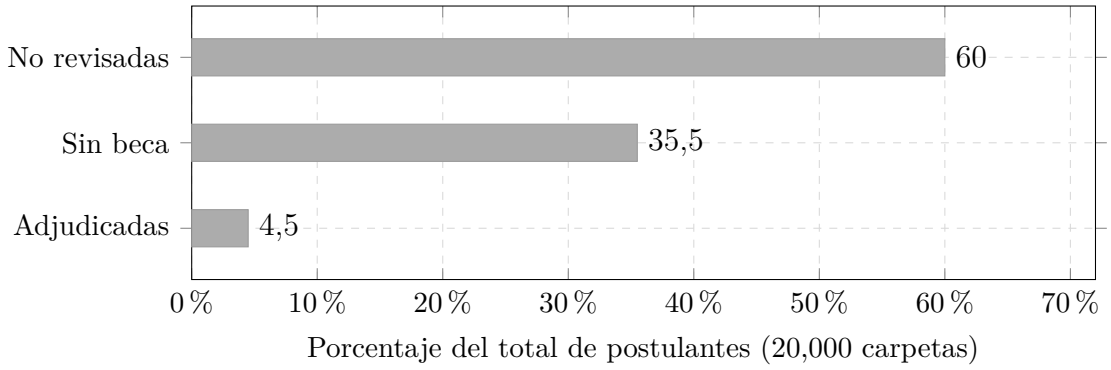
En cada gestión académica, la universidad recibe una cifra masiva que ronda las 20,000 carpetas (folders) de postulantes. Dada la escala de esta demanda y los limitados recursos humanos administrativos para la revisión física de documentos, no se logra auditar ni el 50 % de las solicitudes presentadas, alcanzando a revisar únicamente un aproximado de 8,000 carpetas. Considerando que la disponibilidad de becas es altamente restrictiva (cubriendo únicamente a cerca de 900 beneficiarios, cifra que fluctúa según el presupuesto de cada gestión), el método actual de revisión manual introduce serios sesgos e injusticias operativas:

- **Revisión por orden de llegada o al azar:** Al no poder revisar la totalidad de las 20,000 carpetas, miles de postulantes en situación de extrema vulnerabilidad quedan automáticamente descalificados o ignorados simplemente porque su expediente no llegó a ser abierto.
- **Carga burocrática y desperdicio de recursos:** La manipulación, archivo y almacenamiento de miles de folders físicos genera un costo logístico elevado, consumo masivo de papel y el riesgo constante de extravío de documentos socioeconómicos sensibles.
- **Falta de transparencia y lentitud:** La verificación manual de requisitos (ingresos familiares, patrimonio, procedencia, rendimiento académico) es propensa a errores humanos, duplicidades y posibles manipulaciones, ralentizando la publicación de las listas definitivas de beneficiarios.

Por lo tanto, la problemática central reside en la incapacidad del sistema manual actual para garantizar que las limitadas becas disponibles se otorguen objetivamente a los estudiantes que presentan el mayor grado de necesidad socioeconómica, derivado de un cuello de botella en la fase de revisión de documentos. La Figura 1 ilustra la distribución porcentual de las solicitudes según su destino en el proceso actual.

**Figura 1**

*Distribución porcentual de las solicitudes de beca comedor por destino en el proceso manual de revisión*




*Nota.* Elaboración propia en base a datos institucionales de la gestión académica vigente. De las 20,000 carpetas recibidas, aproximadamente 12,000 (60 %) no llegan a ser revisadas; 7,100 (35,5 %) son revisadas pero no obtienen el beneficio; y solo 900 (4,5 %) corresponden a becas efectivamente adjudicadas.

### *2.2. Situación deseada*

La situación deseada contempla la transformación digital del proceso de selección mediante la implementación de un Sistema Web y Móvil Inteligente para la Postulación y Precalificación Automática de Becas Alimentarias. Bajo este nuevo esquema:

- **Digitalización y Autogestión:** Los estudiantes realizarán su postulación 100 % en línea, adjuntando la documentación requerida en formato digital.
- **Precalificación y Puntuación Autómata (Algoritmo de Priorización):** El sistema implementará un algoritmo de evaluación socioeconómica ponderada que analizará automáticamente los datos ingresados por el postulante (ingresos, dependientes, procedencia, estado de vulnerabilidad, promedio académico, etc.), asignando una puntuación de riesgo o necesidad en tiempo real.
- **Optimización de la Revisión Administrativa:** En lugar de revisar 8,000 carpetas

al azar de un universo de 20,000, el personal administrativo enfocará su trabajo de validación física y auditoría exclusivamente en las carpetas con los puntajes de precalificación más altos. Esto asegura matemáticamente que los expedientes revisados correspondan a los mejores y más necesitados candidatos para adjudicarse los 900 cupos disponibles.

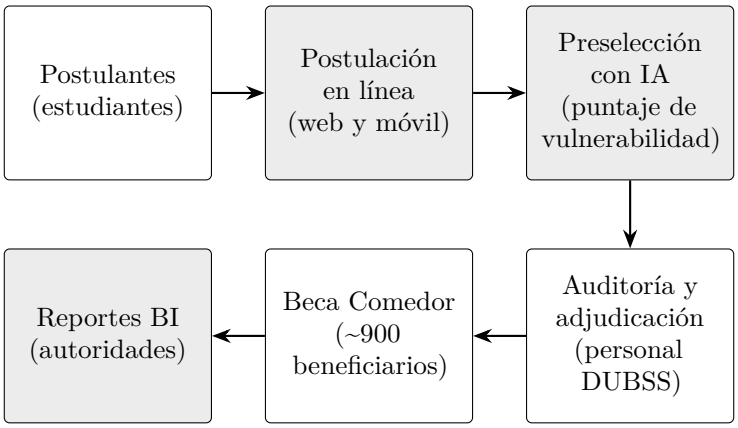
- **Transparencia y Trazabilidad:** Los estudiantes podrán consultar en tiempo real el estado de su solicitud y la puntuación obtenida, reduciendo la incertidumbre y garantizando la equidad del proceso.

### *2.3. Objeto de la investigación*

El objeto de la investigación es diseñar e implementar un sistema de información para la gestión de postulaciones y precalificación automatizada de becas alimentarias, orientado a la digitalización del expediente socioeconómico estudiantil y a la optimización del proceso de selección de beneficiarios en el comedor universitario. La Figura 2 muestra el objeto de estudio: el proceso mediante el cual las solicitudes de los postulantes son recibidas, preseleccionadas con apoyo de Inteligencia Artificial, auditadas por la DUBSS y convertidas en becas adjudicadas, junto con la información que el sistema entrega a las autoridades.

**Figura 2**

*Objeto de la investigación: proceso de postulación y preselección de la Beca Comedor apoyado por el sistema de información*




*Nota.* Elaboración propia. Los bloques sombreados corresponden a los componentes que se desarrollan en el proyecto.

**Figura 3**

*Evolución de la selección de la Beca Comedor: de la revisión manual a la precalificación automática*

*Nota.* Elaboración propia.

## 3. Objetivos

### *3.1. Objetivo General*

Desarrollar un Sistema de Información Web y Móvil para la Postulación y Preselección Automatizada de la Beca Comedor mediante Inteligencia Artificial, con Reportes de Business Intelligence para la Dirección Universitaria de Bienestar Social y Salud (DUBSS) de la Universidad Autónoma Gabriel René Moreno, que permita automatizar la postulación en línea, precalificar algorítmicamente la vulnerabilidad socioeconómica de los estudiantes, gestionar la asistencia física/digital y proveer dashboards analíticos y proyecciones predictivas que apoyen la toma de decisiones institucionales y auditorías.

### *3.2. Objetivos Específicos*

- Capturar los requisitos funcionales y no funcionales del proceso de postulación y adjudicación de la Beca Comedor mediante entrevistas con el personal de la DUBSS, y especificarlos según el estándar IEEE 830.
- Analizar los procesos del sistema modelando los casos de uso y el dominio mediante UML, y definir las reglas de negocio de la preselección.
- Diseñar la arquitectura general del sistema web y móvil, y las interfaces de usuario para cada tipo de usuario.
- Implementar los módulos del sistema web y móvil, integrando la preselección automatizada con inteligencia artificial y los reportes de Business Intelligence.
- Realizar pruebas de caja negra y de caja blanca que verifiquen el cumplimiento de los requisitos especificados.

## 4. Justificación

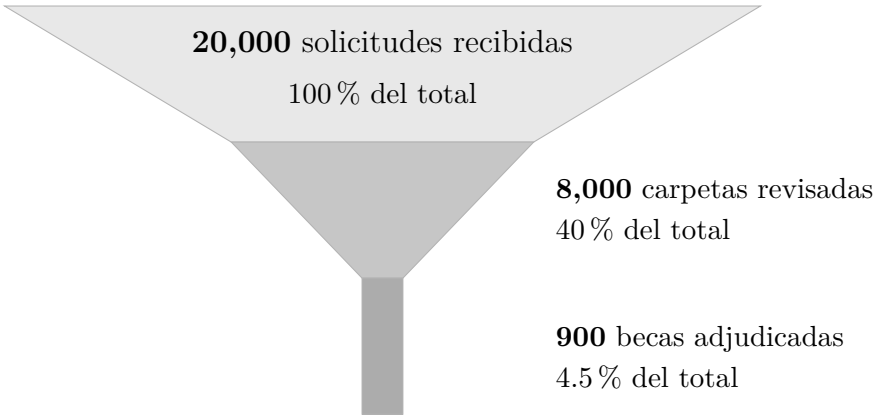
El desarrollo del Sistema de Información Web y Móvil para la Gestión Integral del Comedor Universitario de la UAGRM se justifica desde tres dimensiones fundamentales: la operativa y técnica, la social y la económica.

### *4.1. Justificación operativa y técnica*

La capacidad administrativa actual presenta una severa limitación, ya que únicamente se logra revisar un aproximado de 8,000 carpetas físicas por gestión. La implementación de un modelo de inteligencia artificial para la preselección automatizada permitirá procesar y evaluar el 100 % de las 20,000 solicitudes recibidas anualmente. Con ello se elimina el cuello de botella operativo y se agilizan los tiempos de respuesta de la DUBSS, de modo que el personal administrativo pueda enfocar sus esfuerzos de auditoría exclusivamente en los expedientes que el sistema señale con el mayor puntaje de prioridad (véase la Figura 4).

**Figura 4**

*Embudo del proceso manual de gestión de la beca comedor: reducción de las 20,000 solicitudes recibidas a los 900 cupos adjudicados*




*Nota.* Elaboración propia en base a los datos institucionales de la DUBSS. De las 20,000 solicitudes recibidas en cada gestión, aproximadamente 8,000 carpetas logran revisarse, mientras que únicamente cerca de 900 cupos resultan efectivamente adjudicados.

### *4.2. Justificación social*

El proceso de evaluación manual vigente genera una exclusión involuntaria, pues alrededor del 60 % de los postulantes (cerca de 12,000 estudiantes) no llega a ser evaluado. El algoritmo de priorización socioeconómica resolverá esta brecha de equidad al garantizar que los cerca de 900 cupos disponibles para la beca comedor se asignen de manera transparente, medible y justa a los estudiantes que presenten el mayor nivel de vulnerabilidad, erradicando los sesgos propios del método de revisión al azar o por orden de llegada. La Tabla 1 contrasta la situación actual con la esperada tras la implantación del sistema.

**Tabla 1**

*Comparativa del proceso de asignación de la beca comedor: situación actual frente a la esperada con el sistema propuesto*

| Indicador | Proceso manual actual | Con el sistema propuesto |
|---|---|---|
| Solicitudes evaluadas por gestión | 8,000 (40 %) | 20,000 (100 %) |
| Postulantes sin evaluación | 12,000 (60 %) | 0 |
| Cupos disponibles | 900 | 900 |
| Criterio de selección | Azar u orden de llegada | Priorización socioeconómica |
| Transparencia y trazabilidad | Baja | Alta |

*Nota.* Elaboración propia en base a los datos institucionales de la DUBSS. Las cifras corresponden a una gestión académica ordinaria y pueden variar según el presupuesto asignado.

### *4.3. Justificación económica y operativa: optimización de recursos*

Aunque el proceso final exige la validación física de los respaldos, el sistema actuará como un embudo digital que reduce drásticamente el volumen de recepción documental. En lugar de exigir, recepcionar y almacenar administrativamente 20,000 carpetas físicas al inicio de cada gestión, la DUBSS solicitará la documentación en físico únicamente a las carpetas con los puntajes de precalificación más altos. Esto transforma un proceso logísticamente insostenible en uno altamente eficiente, generando un ahorro significativo en horas-hombre y espacio de archivo para la universidad y evitando gastos de impresión y tramitación en vano para la inmensa mayoría de los estudiantes que no llegarán a la fase final de selección. La Tabla 2 proyecta la recepción documental actual y la estimada con el sistema de preselección.

**Tabla 2**

*Proyección de la recepción y gestión documental: proceso actual frente al esquema de preselección automática*

| Aspecto | Proceso manual actual | Con preselección automática |
|---|---|---|
| Carpetas solicitadas en físico | 20,000 | Aprox. 2,700 (finalistas) |
| Carpetas almacenadas | 20,000 | Aprox. 2,700 |
| Hojas de papel gestionadas | Alto | Mínimo |
| Horas-hombre de revisión | Elevadas | Enfocadas y auditadas |
| Trámite de impresión en vano | Mayoría de postulantes | Solo fase final |

*Nota.* Elaboración propia. La proyección de 2,700 carpetas se basa en un margen de revisión equivalente a tres carpetas por cada uno de los 900 cupos disponibles, a fin de cubrir desistimientos o inconsistencias documentales.

## 5. Metodología

Una metodología de desarrollo de software es un conjunto estructurado de principios, procedimientos, técnicas y herramientas que orientan el proceso de construcción de un sistema, desde la captura de requisitos hasta su entrega y mantenimiento. Su propósito es establecer un marco disciplinado y repetible que reduzca la incertidumbre, gestione los riesgos y garantice la calidad del producto final. La selección de una metodología no es un acto trivial: debe responder a las características del proyecto, tales como su tamaño, complejidad, grado de incertidumbre en los requisitos y el perfil del equipo de desarrollo.

Para el presente proyecto, referente al desarrollo de un Sistema de Información Web y Móvil para la Gestión Integral del Comedor Universitario de la UAGRM, se eligió el Proceso Unificado de Desarrollo de Software (PUDS) como metodología de desarrollo. La decisión se sustenta en tres razones fundamentales. En primer lugar, el sistema involucra múltiples módulos interrelacionados (gestión de postulaciones, evaluación socioeconómica, modelo de aprendizaje automático y reportes de inteligencia de negocios), cuyo alcance se comprende

mejor mediante un proceso iterativo e incremental que permita refinarlos progresivamente. En segundo lugar, la captura de requisitos requiere una participación activa de los interesados (DUBSS, estudiantes y personal administrativo), lo cual se alinea con un proceso dirigido por casos de uso. Finalmente, la arquitectura del sistema, centrada en servicios web, aplicación móvil y una base de datos centralizada, exige una modelación previa rigurosa, un aspecto central del PUDS.

### *5.1. El Proceso Unificado de Desarrollo de Software*

El Proceso Unificado de Desarrollo de Software (PUDS) es un marco de trabajo iterativo e incremental, dirigido por casos de uso y centrado en la arquitectura, desarrollado por Ivar Jacobson, Grady Booch y James Rumbaugh (Jacobson et al., 1999). A diferencia de los modelos tradicionales de cascada, el PUDS estructura el desarrollo en iteraciones que combinan, en cada una, actividades de captura de requisitos, análisis, diseño, implementación y pruebas, de modo que el riesgo se ataca desde las primeras repeticiones y el software crece de manera controlada.

Tres características definen la esencia del PUDS (Jacobson et al., 1999, p. 45):

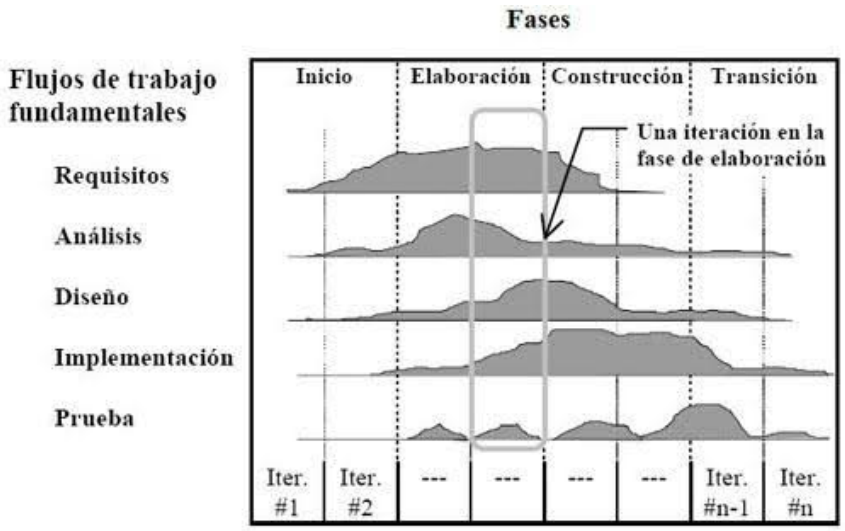
- **Dirigido por casos de uso:** los casos de uso guían todo el proceso de desarrollo, pues representan la funcionalidad que el sistema debe brindar a los usuarios y permiten planificar las iteraciones en función del valor que aporta cada funcionalidad.
- **Centrado en la arquitectura:** la arquitectura del sistema se define de manera temprana y evoluciona a lo largo del proyecto, sirviendo de esqueleto sobre el cual se integran los distintos componentes.
- **Iterativo e incremental:** el sistema se construye mediante incrementos sucesivos, cada uno de los cuales es una versión ejecutable del producto que agrega nueva funcionalidad y reduce el riesgo de manera progresiva (Jacobson et al., 1999, p. 47).

**5.1.1. Fases.** El ciclo de vida del PUDS se estructura a través de cuatro fases consecutivas, cada una de las cuales culmina con un hito o *milestone* que permite evaluar

el avance del proyecto (Jacobson et al., 1999). La Figura 5 ilustra la relación existente entre las fases y las disciplinas o flujos de trabajo del proceso.

**Figura 5**

*Las cuatro fases del Proceso Unificado de Desarrollo de Software y su relación con las disciplinas de ingeniería del proceso*




*Nota.* Adaptada de *The Unified Software Development Process*, de I. Jacobson, G. Booch y J. Rumbaugh, 1999, Addison-Wesley Professional. Reproducida con fines académicos.

***5.1.1.1. Inicio.*** La fase de Inicio tiene como propósito definir el alcance del sistema y establecer su viabilidad. En esta etapa se identifican los actores principales, los casos de uso más relevantes, los riesgos críticos y los objetivos del proyecto (Jacobson et al., 1999, p. 214). Un análisis preliminar de negocio y una estimación aproximada de costo y tiempo forman parte de las actividades de este hito. En el contexto del presente proyecto, la fase de inicio corresponde a la elaboración del perfil que aquí se documenta: se delimitan los problemas, los objetivos, la justificación y el alcance del sistema, y se evalúan las alternativas tecnológicas para su implementación.

***5.1.1.2. Elaboración.*** La fase de Elaboración se orienta a la especificación detallada de los casos de uso y a la definición de la arquitectura base del software. El objetivo de esta fase es eliminar los mayores riesgos técnicos mediante la construcción de un prototipo

ejecutable de la arquitectura que sirva de fundamento para las fases posteriores (Jacobson et al., 1999, p. 248). En el proyecto, esta fase comprende el levantamiento detallado de requisitos funcionales y no funcionales, la elaboración de los diagramas del Lenguaje Unificado de Modelado (UML), el diseño de la base de datos y la definición de la arquitectura de los módulos web, móvil e inteligencia artificial.

***5.1.1.3. Construcción.*** La fase de Construcción está orientada al desarrollo e integración de los componentes y funcionalidades del sistema. Durante esta fase se implementan incrementalmente los casos de uso priorizados, se integran los subsistemas en una solución completa y se llevan a cabo las pruebas necesarias para verificar el cumplimiento de los requisitos (Jacobson et al., 1999, p. 332). En el presente proyecto, esta fase corresponde al desarrollo de los módulos de gestión de postulaciones, el algoritmo de priorización socioeconómica, la aplicación móvil, los reportes de inteligencia de negocios y su integración con la base de datos central.

***5.1.1.4. Transición.*** La fase de Transición se dirige a la entrega, despliegue, capacitación de los usuarios y validación del sistema en un entorno de producción. Su finalidad es asegurar que el software esté disponible para los usuarios finales, que funcione de manera confiable y que se disponga del soporte adecuado para su operación (Jacobson et al., 1999, p. 379). En el contexto del sistema del comedor universitario, esta fase comprende el despliegue en los servidores de la DUBSS, la capacitación del personal administrativo, la migración de los datos de los postulantes y el acompañamiento durante la primera gestión académica en que el sistema opere en forma oficial.

**5.1.2. Flujos de trabajo.** Además de las fases, el PUDS organiza las actividades del proyecto en flujos de trabajo o disciplinas (captura de requisitos, análisis, diseño, implementación y pruebas), que se ejecutan en cada iteración con mayor o menor intensidad según la fase en que se encuentre el proyecto (Jacobson et al., 1999). Mientras las fases responden a *cuándo* se avanza, los flujos de trabajo responden a *qué* se hace en cada momento. A continuación se describe cada uno y su aplicación en el presente proyecto.

***5.1.2.1. Captura de requisitos.*** Este flujo busca identificar qué debe hacer el sistema desde la perspectiva de los usuarios y acordarlo con ellos. Sus resultados son el modelo de casos de uso, que agrupa a los actores y las funcionalidades esperadas, y la especificación de los requisitos funcionales y no funcionales (Jacobson et al., 1999). En el proyecto, se realiza mediante entrevistas con el personal de la DUBSS y el estudio del proceso actual de postulación, y se documenta con el estándar IEEE 830.

***5.1.2.2. Análisis.*** El análisis refina y estructura los requisitos capturados, describiéndolos en el lenguaje de los desarrolladores y construyendo un modelo conceptual del sistema que no depende de la tecnología de implementación (Jacobson et al., 1999). En el proyecto, este flujo comprende el modelado de los casos de uso y del dominio con UML, y la definición de las reglas de negocio que gobiernan la calificación y la adjudicación de la beca.

***5.1.2.3. Diseño.*** El diseño adapta el modelo de análisis a la tecnología elegida, definiendo la arquitectura del sistema, sus componentes y las interfaces entre ellos (Jacobson et al., 1999). En el proyecto, se traduce en la arquitectura del sistema web y móvil, sus módulos de inteligencia artificial y de Business Intelligence, y el diseño de las interfaces de usuario.

***5.1.2.4. Implementación.*** En este flujo se construye el sistema a partir del diseño, codificando los componentes y organizándolos en subsistemas que se integran de forma incremental (Jacobson et al., 1999). En el proyecto, se desarrollan los módulos de postulación, preselección automatizada, gestión administrativa, asistencia y reportes, tanto en su versión web como móvil.

***5.1.2.5. Pruebas.*** Las pruebas verifican que el sistema funcione como se especificó y que los requisitos se hayan implementado correctamente, detectando defectos antes de la entrega (Jacobson et al., 1999). En el proyecto, se aplican pruebas de caja negra, que validan el comportamiento del sistema a partir de sus entradas y salidas sin considerar su código, y pruebas de caja blanca, que examinan la estructura interna y la lógica del código.

## *5.2. Unified Modeling Language*

El Lenguaje Unificado de Modelado (*Unified Modeling Language*, UML) es un lenguaje gráfico estandarizado empleado para visualizar, especificar, construir y documentar los artefactos de un sistema de software (Booch et al., 2005). A diferencia de un método o de un proceso, UML se define estrictamente como un lenguaje de modelado: proporciona la notación y la semántica necesarias para representar las decisiones de análisis y diseño de un sistema de manera formal, comprensible y compartida entre los actores involucrados en el proyecto.

El UML surge de la unificación de las metodologías orientadas a objetos más difundidas de la década de 1990, impulsada por Grady Booch, James Rumbaugh e Ivar Jacobson, y fue adoptado posteriormente por el *Object Management Group* (OMG) como estándar internacional (Booch et al., 2005, p. 3). Su propósito no es decir *cómo* desarrollar software, sino ofrecer un vocabulario común que permita expresar los modelos sobre los cuales los equipos toman decisiones técnicas. En la Figura 6 se muestra la representación gráfica del lenguaje.

**Figura 6**

*Representación gráfica del Lenguaje Unificado de Modelado (UML)*

*Nota.* Imagen de dominio público correspondiente al logo oficial del Lenguaje Unificado de Modelado, propiedad del *Object Management Group* (OMG).

**5.2.1. Estructura general de UML..** La estructura general de UML se compone de tres bloques de construcción principales: los elementos, las relaciones y los diagramas

(Booch et al., 2005, p. 15).

Los elementos son las unidades abstractas que modelan el sistema y se clasifican en cuatro tipos: elementos estructurales (como clases, interfaces y componentes), elementos de comportamiento (como interacciones y máquinas de estados), elementos de agrupación (como los paquetes) y elementos de anotación (como las notas explicativas). Las relaciones expresan cómo se vinculan dichos elementos y comprenden la asociación, la dependencia, la generalización y la realización.

## *5.3. Especificación de Requisitos de Software según la IEEE 830*

Para garantizar que el desarrollo del sistema de gestión, postulación y precalificación de becas alimentarias se ejecute de forma ordenada y transparente, se empleará el estándar IEEE 830 (específicamente la recomendación IEEE Std 830-1998) para la elaboración de la Especificación de Requisitos de Software (SRS, por sus siglas en inglés, Software Requirements Specification).

Este estándar constituye una guía normativa esencial en la ingeniería de software para documentar con precisión qué debe hacer el sistema, sin prescribir prematuramente cómo debe ser implementado en términos de arquitectura interna.

### 5.3.1. Estructura y Propósito del Estándar IEEE 830 (IEEE 830-1998).

El objetivo de implementar la norma IEEE 830 en este proyecto es definir un contrato claro y sin ambigüedades entre los usuarios interesados (estudiantes postulantes y personal administrativo de bienestar estudiantil) y el equipo de desarrollo tecnológico. La especificación bajo este estándar se organiza formalmente en tres secciones principales:

- **Introducción:** Define el propósito del software, el alcance del sistema de becas alimentarias, la definición de términos y las referencias.
- **Descripción General:** Describe los factores generales que afectan al producto, tales como las interfaces de usuario (portales web y móviles), las restricciones de la plataforma (almacenamiento digital de expedientes) y las suposiciones operativas.

- **Requisitos Específicos:** Detalla en profundidad las funciones del sistema. En este proyecto se formalizarán:
  - **Requisitos Funcionales:** Procesos de registro de postulantes, carga de carpetas socioeconómicas digitales, cálculo automatizado del puntaje de vulnerabilidad por medio del algoritmo de precalificación, y generación de listas priorizadas para auditoría administrativa.
  - **Requisitos No Funcionales:** Criterios de rendimiento, seguridad de la información socioeconómica confidencial, disponibilidad del portal durante picos de alta postulación y usabilidad.

**5.3.2. Relación del IEEE 830 con UML y PUDS..** Dentro de la metodología adoptada para este proyecto, el estándar IEEE 830 actúa como el puente de formalización entre el Proceso Unificado de Desarrollo de Software (PUDS) y el Lenguaje Unificado de Modelado (UML):

- **Integración con PUDS:** En el flujo de trabajo de Captura de Requisitos durante las fases de Inicio y Elaboración, la especificación IEEE 830 consolida el levantamiento de información operacional para evitar desviaciones en el alcance del proyecto.
- **Integración con UML:** La descripción textual de los requisitos especificados en el estándar IEEE 830 se representa gráficamente a través de diagramas UML (casos de uso, actividades y secuencias). Esto asegura que los requerimientos de los postulantes y administradores se traduzcan de manera transparente a modelos técnicos estructurados.

**5.3.3. Reemplazo por IEEE 29148-2011 y Vigencia Actual.** Aunque el estándar IEEE 830-1998 fue sustituido formalmente por el estándar unificado ISO/IEC/IEEE 29148:2011 (el cual consolida la ingeniería de requisitos dentro del ciclo de vida de los sistemas), la norma IEEE 830 se mantiene como el referente metodológico para este proyecto debido a:

- Su estructura altamente clara, directa y didáctica para proyectos académicos de software.
- Su amplia adopción histórica en el ámbito educativo universitario.
- Su perfecta compatibilidad con lenguajes de modelado visual como UML y metodologías iterativas basadas en el PUDS.
- Su inclusión consolidada en la bibliografía académica de ingeniería de sistemas.

## 6. Alcances y limitaciones

El presente proyecto abarca el diseño y desarrollo de un sistema de información web y móvil para la automatización, gestión y precalificación del proceso de postulación a la beca comedor universitaria, así como el seguimiento del beneficio otorgado. A continuación, se detallan las funciones cubiertas, los actores involucrados y las restricciones del software.

### *6.1. Funciones principales*

**Registro y autenticación:**

a) Permite a los postulantes, al personal administrativo de la DUBSS y a los directivos crear una cuenta e ingresar de forma segura desde el portal web o la aplicación móvil.

b) Identifica el rol de cada usuario y lo dirige al módulo que le corresponde.

c) Rechaza las credenciales inválidas y muestra un mensaje de error.

**Postulación en línea:**

a) Permite al estudiante completar el formulario socioeconómico (ingresos, dependientes, procedencia, carrera, entre otros).

b) Permite adjuntar los documentos de respaldo digitalizados y enviar la solicitud.

c) Valida que los datos y los documentos obligatorios estén completos y registra la fecha de envío.

**Preselección con inteligencia artificial:**

a) Evalúa cada solicitud recibida (20,000 o más por gestión) a partir de las variables socioeconómicas y académicas declaradas.

b) Calcula un puntaje de vulnerabilidad por postulante y registra su valor y la fecha de cálculo para garantizar la trazabilidad.

c) Genera una lista ordenada que prioriza a los estudiantes con mayor necesidad.

**Priorización de carpetas para auditoría:**

a) Ordena las solicitudes según el puntaje obtenido.

b) Presenta al personal de la DUBSS, en primer lugar, los expedientes con mayor requerimiento económico, evitando la revisión al azar.

**Auditoría y adjudicación:**

a) Permite al personal de la DUBSS revisar los expedientes priorizados y contrastar los datos declarados con la documentación física de los finalistas.

b) Permite aprobar, devolver o archivar una postulación.

c) Adjudica la beca a los aproximadamente 900 beneficiarios y emite el acta de adjudicación.

**Asistencia (físico y digital):**

a) Registra la asistencia diaria de los becados al comedor.

b) Genera las planillas y comprobantes impresos, con espacio para la firma y la fotocopia del carnet que exigen las auditorías externas.

c) Sincroniza la información con el registro digital para permitir auditorías cruzadas.

**Consultas y notificaciones:**

a) Permite al estudiante consultar en tiempo real el estado de su postulación y su puntaje.

b) Envía alertas (notificaciones push) sobre la convocatoria, la solicitud de documentación física y el resultado de la adjudicación.

**Reportes y tableros de Business Intelligence:**

a) Ofrece a las autoridades tableros interactivos con indicadores sobre postulaciones, eficiencia de la preselección y asistencia al comedor.

b) Presenta proyecciones predictivas que apoyan la planificación y la rendición de cuentas.

### *6.2. Usuarios o beneficiarios*

La Tabla 3 resume los usuarios del sistema y la forma en que cada uno interactúa con él.

**Tabla 3**

*Usuarios y beneficiarios del sistema*

| Usuario o beneficiario | Descripción | Interacción con el sistema |
|---|---|---|
| Estudiantes postulantes | Alumnos matriculados que solicitan la Beca Comedor. | Registran su postulación y cargan sus documentos desde la aplicación móvil o la plataforma web. |
| Estudiantes adjudicados (becados) | Beneficiarios de la beca. | Consultan el estado de su beca y gestionan sus requisitos de asistencia diaria. |
| Personal administrativo de Bienestar Estudiantil | Encargados de la gestión del proceso dentro de la DUBSS. | Revisan y auditan las carpetas priorizadas por el sistema y gestionan las convocatorias. |
| Auditores institucionales | Personal fiscalizador interno o externo. | Validan la entrega del beneficio cotejando el registro digital con los respaldos físicos (fotocopias y firmas). |
| Personal del comedor / empresa concesionaria | Encargados del servicio de alimentación. | Verifican la identidad del estudiante y participan en el registro de la asistencia diaria. |

*Nota.* Elaboración propia.

### *6.3. Limitaciones y Restricciones*

- **Requisito de Auditoría Física (Restricción Normativa):** El sistema no elimina por completo el uso de papel en la fase de asistencia diaria, debido a la exigencia reglamentaria de los organismos auditores que requieren carpetas físicas con fotocopia de carnet y firma manuscrita para validar la entrega del alimento a cada becado. El software actuará como un medio de control, validación y archivo digitalizador complementario.

- **Operación Interna de la Cocina:** El sistema no gestiona el control de inventarios de insumos, compras de materia prima ni la contabilidad interna de la empresa privada concesionaria del comedor.

- **Comercialización a No Becados:** La venta de almuerzos y platos a estudiantes regulares no becados y personal universitario queda fuera del alcance del sistema, correspondiendo a la gestión propia de la empresa privada.

- **Dependencia de Conectividad:** La carga de expedientes y la sincronización de asistencias requieren conexión a Internet activa.

- **Límites de Verificación Automática:** El sistema realiza una precalificación algorítmica basada en la información declarada y digitalizada; sin embargo, la verificación presencial o visitas domiciliarias mantendrán el criterio operativo del personal de Trabajo Social.

# 7. Recursos necesarios

## *7.1. Software y herramientas*

**Tabla 4**

*Software y herramientas utilizados para el desarrollo del proyecto*

| Software y herramienta | Descripción |
|---|---|
| Windows 11 | Sistema operativo utilizado como plataforma base en los equipos de desarrollo. |
| Visual Studio Code | Editor de código fuente donde se escribe y depura el código del sistema. |
| Bruno / Postman | Herramientas para diseñar, documentar y probar las API del sistema. |
| GitHub | Plataforma para el control de versiones y el trabajo colaborativo del código fuente. |
| Enterprise Architect | Herramienta de modelado UML para elaborar los diagramas del sistema. |
| Jira | Plataforma de gestión de proyectos para organizar y dar seguimiento a las tareas de desarrollo. |
| Docker | Plataforma de contenedores para estandarizar los entornos de desarrollo y despliegue. |

*Nota.* Elaboración propia.

### *7.2. Equipos y hardware*

**Tabla 5**

*Equipos y hardware requeridos para el desarrollo del proyecto*

| Recurso / equipo | Especificación y uso |
|---|---|
| PC de desarrollo 1 | Computadora con procesador AMD Ryzen 7 8700G y 16 GB de RAM, utilizada para el desarrollo del sistema. |
| PC de desarrollo 2 | Computadora con procesador AMD Ryzen 7 5600G y 16 GB de RAM, utilizada para el desarrollo del sistema. |
| Redmi Note 14 Pro | Dispositivo móvil Android utilizado para probar la aplicación móvil. |
| Honor X8 | Dispositivo móvil Android utilizado para probar la aplicación móvil. |
| Servidor AWS | Servidor en la nube de Amazon Web Services utilizado para el despliegue de la aplicación y la base de datos. |

*Nota.* Elaboración propia.

### *7.3. Presupuesto estimado*

El presupuesto estimado del proyecto contempla los costos de hardware, software, recursos humanos, infraestructura y logística durante el periodo comprendido entre el 24 de agosto de 2026 y el 30 de noviembre de 2026 (3 meses). Sobre los equipos de hardware se aplica una depreciación del 10 % por su uso durante el desarrollo. Las Tablas 6 y 7 detallan cada partida y su costo.

**Tabla 6**

*Presupuesto estimado del proyecto: hardware y software*

| Recurso | *Tiempo* (meses) | *Cant.* | *Costo unit.* ($) | *Deprec.* (%) | *Costo total* ($) |
|---|---|---|---|---|---|
| **Hardware** | | | | | |
| PC dev. 1 (Ryzen 7 8700G, 16 GB RAM) | — | 1 | 850.00 | 10 | 765.00 |
| PC dev. 2 (Ryzen 7 5600G) | — | 1 | 600.00 | 10 | 540.00 |
| Redmi Note 14 Pro | — | 1 | 300.00 | 10 | 270.00 |
| Honor X8 | — | 1 | 200.00 | 10 | 180.00 |
| | | | *Subtotal Hardware* | | 1,755.00 |
| **Software** | | | | | |
| Windows 11 | — | 1 | 199.00 | 0 | 199.00 |
| Visual Studio Code | — | 1 | 0.00 | 0 | 0.00 |
| Bruno / Postman | — | 1 | 0.00 | 0 | 0.00 |
| GitHub | — | 1 | 0.00 | 0 | 0.00 |
| Enterprise Architect | — | 1 | 229.00 | 0 | 229.00 |
| Jira | — | 1 | 0.00 | 0 | 0.00 |
| Docker | — | 1 | 0.00 | 0 | 0.00 |
| | | | *Subtotal Software* | | 428.00 |
| **Subtotal (hardware + software)** | | | | | **2,183.00** |

*Nota.* Elaboración propia. Los precios unitarios corresponden a estimaciones de mercado. Los costos de hardware consideran una depreciación del 10 %. Para el software gratuito se indica costo cero.

**Tabla 7**

*Presupuesto estimado del proyecto: recursos humanos, infraestructura y logística*

| Recurso | *Tiempo* (meses) | *Cant.* | *Costo unit.* ($) | *Deprec.* (%) | *Costo total* ($) |
|---|---|---|---|---|---|
| **Gente** | | | | | |
| Desarrolladores fullstack | 3 | 2 | 450.00 | 0 | 2,700.00 |
| | | | | *Subtotal Gente* | 2,700.00 |
| **Infraestructura** | | | | | |
| Espacio de oficina | 3 | 1 | 150.00 | 0 | 450.00 |
| Servicio de luz | 3 | 1 | 40.00 | 0 | 120.00 |
| Agua | 3 | 1 | 36.00 | 0 | 108.00 |
| Internet | 3 | 1 | 40.00 | 0 | 120.00 |
| Servidor AWS | 3 | 1 | 50.00 | 0 | 150.00 |
| | | | | *Subtotal Infraestructura* | 948.00 |
| **Logística** | | | | | |
| Transporte | 3 | 1 | 25.00 | 0 | 75.00 |
| Alimentación | 3 | 1 | 100.00 | 0 | 300.00 |
| | | | | *Subtotal Logística* | 375.00 |
| **Costo total estimado del proyecto** | | | | | **6,206.00** |

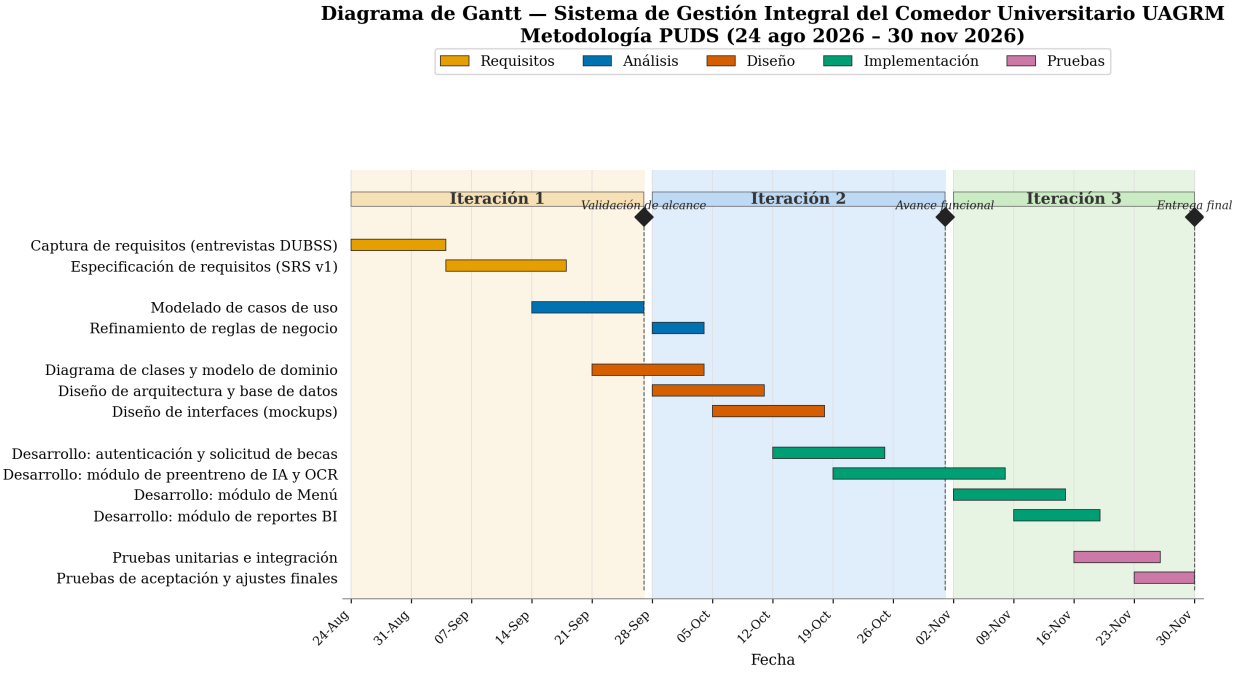
*Nota.* Elaboración propia. Los servicios de oficina, luz, agua, internet y logística consideran el uso conjunto de ambos desarrolladores. El servidor AWS se estima en $50 mensuales según el plan de cómputo básico. La depreciación de los equipos de hardware se detalla en la Tabla 6.

# 8. Cronograma

## *8.1. Plan de ejecución*

**Figura 7**

*Diagrama de Gantt del plan de ejecución del proyecto: proceso unificado aplicado al Sistema de Gestión Integral del Comedor Universitario (24 de agosto al 30 de noviembre de 2026)*




*Nota.* Elaboración propia. El diagrama detalla las fases del proyecto, sus duraciones estimadas en días hábiles y los hitos de validación de alcance, avance funcional y entrega final.

### *8.2. Hitos importantes*

Los hitos del proyecto se organizan según las tres fases de la materia (Fase 1, Fase 2 y Fase final), cada una con su avance o explicación y su presentación, como se detalla en la Tabla 8.

**Tabla 8**

*Hitos del proyecto según las fases de la materia*

| Fase | Actividad | Información adicional |
|---|---|---|
| Fase 1 | Avance/explicación de los Capítulos 1 a 4, formato y bibliografía. | Tema del proyecto definido en la clase previa. |
| | Presentación 1. | Revisión 1 del documento, hasta el Capítulo 4, en formato digital. |
| Fase 2 | Avance/explicación de los Capítulos 5, 6 y 7. | Captura de requisitos, análisis y diseño. |
| | Presentación 2. | Revisión 2 del documento, hasta el Capítulo 7, en formato digital. |
| Fase final | Avance/explicación del Capítulo 8 en adelante. | Implementación y pruebas del sistema. |
| | Presentación final del documento y del software, y defensa final. | Revisión final: documento completo y software desarrollado, en formato digital y físico, con presentación y defensa. |

*Nota.* Elaboración propia con base en las fases de avance definidas por la asignatura.

Capítulo 2

# Inteligencia Artificial y Machine Learning

# Capítulo 2: Inteligencia Artificial y Machine Learning

La evolución de los sistemas de información en la gestión universitaria exige la transición de plataformas transaccionales tradicionales a arquitecturas asistidas por analítica avanzada e Inteligencia Artificial (IA). El presente capítulo desarrolla el marco teórico y conceptual que respalda el diseño del sistema web y móvil para la postulación y preselección automatizada de la Beca Comedor en la Dirección Universitaria de Bienestar Social y Salud (DUBSS - UAGRM).

## 2.1. Fundamentos de Inteligencia Artificial y Aprendizaje Automático

### *2.1.1. Conceptos Generales de IA Aplicados a la Gestión Social Universitaria*

La Inteligencia Artificial (IA) se define como el campo de la ciencia de la computación dedicado al desarrollo de sistemas capaces de efectuar tareas que simulan procesos cognitivos humanos, tales como la resolución de problemas, el aprendizaje adaptativo y la toma de decisiones (Russell & Norvig, 2021). En las instituciones de educación superior, la IA ha ampliado su espectro desde la automatización académica hacia la optimización de la gestión social y administrativa.

La aplicación de la IA en la asignación de incentivos socioeconómicos permite procesar grandes volúmenes de solicitudes de forma imparcial y expedita. En contextos universitarios masivos, la evaluación manual de expedientes suele incurrir en cuellos de botella operativos y variabilidad en la aplicación de criterios. Los sistemas inteligentes analizan variables multifactoriales —ingresos familiares, condición de vivienda, rendimiento académico y carga familiar— para establecer patrones homogéneos de vulnerabilidad, garantizando que los recursos públicos se destinen a los estudiantes con mayor prioridad socioeconómica (Goodfellow et al., 2016).

### *2.1.2. Modelos y Algoritmos de Clasificación para Evaluación Socioeconómica*

El Aprendizaje Automático (*Machine Learning*, ML) es un subcampo de la IA enfocado en la construcción de algoritmos que aprenden patrones a partir de datos sin ser programados

explícitamente (Mitchell, 1997). Para el proceso de preselección de becados, se emplean modelos de aprendizaje supervisado, donde el algoritmo se entrena utilizando un histórico de expedientes previamente dictaminados por el equipo de Trabajadoras Sociales de la DUBSS.

Dentro del aprendizaje supervisado, los algoritmos de clasificación son los más adecuados para catalogar la vulnerabilidad de un postulante en categorías predefinidas (ej. Vulnerabilidad Alta, Media, Baja). Los modelos principales aplicables a este dominio incluyen:

- **Árboles de Decisión (*Decision Trees*):** Modelos no paramétricos que dividen el espacio de datos en regiones mediante reglas lógicas explícitas. Su principal ventaja en la gestión pública es la explicabilidad, permitiendo auditar la razón exacta por la cual un postulante fue asignado a determinado segmento (Breiman et al., 1984).
- **Bosques Aleatorios (*Random Forest*):** Algoritmo de ensamble (*ensemble learning*) que combina múltiples árboles de decisión entrenados sobre subconjuntos aleatorios de datos. Incrementa la precisión predictiva y reduce el sobreajuste (*overfitting*), resultando ideal para lidiar con variables heterogéneas (Breiman, 2001).
- **Regresión Logística:** Modelo estadístico utilizado para predecir la probabilidad de una variable categórica binaria o multinomial, adecuado como línea base (*baseline*) de comparación frente a modelos más complejos (James et al., 2013).

### *2.1.3. Scoring Automático de Vulnerabilidad y Selección Priorizada de Postulantes*

El Scoring Socioeconómico es una técnica cuantitativa que asigna una puntuación numérica continua a cada expediente de postulación. A diferencia de una clasificación rígida, el scoring permite generar un ordenamiento jerárquico (*ranking*) absoluto de los postulantes, ordenados estrictamente desde el menor al mayor nivel de capacidad económica.

La construcción del índice de vulnerabilidad ($V$) mediante un modelo de ponderación algorítmica se expresa formalmente como:

$$V = \sum_{i=1}^{n} w_i \cdot x_i$$

Donde $x_i$ representa el valor estandarizado de la variable socioeconómica $i$ (ej. ingreso per cápita, tipo de vivienda, miembros a cargo), y $w_i$ corresponde al peso o coeficiente determinado por el modelo de ML en función de la correlación histórica con la vulnerabilidad real.

Este enfoque permite a la DUBSS procesar de forma instantánea el universo total de carpetas (con capacidad superior a las 20,000 solicitudes) y presentar al personal administrativo una lista priorizada para la etapa de verificación de respaldos físicos.

## 2.2. Business Intelligence y Analítica de Datos

### *2.2.1. Fundamentos de Business Intelligence para Auditoría Institucional*

La Inteligencia de Negocios (*Business Intelligence*, BI) engloba las estrategias, arquitecturas y tecnologías encargadas de transformar datos crudos en información estructurada y conocimiento de alto valor para el soporte en la toma de decisiones organizacionales (Power, 2002).

En el contexto de la gestión social en instituciones públicas como la UAGRM, BI actúa como una herramienta fundamental de transparencia y auditoría. Permite a las autoridades universitarias y fiscalizadores externos analizar la distribución de las becas mediante paneles interactivos, identificando la dispersión geográfica de los becados, la tasa de cobertura por facultad y la correlación entre el rendimiento académico y el beneficio otorgado (Golfarelli & Rizzi, 2009).

### *2.2.2. Procesos ETL (Extract, Transform, Load) para la Integración de Expedientes*

La arquitectura de BI se fundamenta en los procesos de Extracción, Transformación y Carga (ETL). Estos procesos consolidan la información dispersa de las postulaciones web y móviles en un repositorio analítico o *Data Warehouse* optimizado para consultas

multidimensionales (Kimball & Caserta, 2004):

- **Extracción (*Extract*):** Captura de datos transaccionales de las postulaciones, adjuntos y registros de asistencia alojados en la base de datos operativa.
- **Transformación (*Transform*):** Limpieza de datos (eliminación de duplicados, estandarización de campos de texto), imputación de valores faltantes y homologación de variables socioeconómicas y académicas.
- **Carga (*Load*):** Inserción de los datos transformados en esquemas analíticos (ej. esquema en estrella o en copo de nieve), facilitando el procesamiento analítico en tiempo real (OLAP).

### *2.2.3. Dashboards y Visualización Dinámica de Indicadores Socioeconómicos*

La visualización de datos constituye la capa superior del sistema de BI. A través de cuadros de mando o *dashboards* interactivos, las autoridades pueden monitorear los Indicadores Clave de Desempeño (KPIs) del programa de becas en tiempo real (Few, 2006).

Entre los indicadores principales integrados en la plataforma destacan:

- **Tasa de Vulnerabilidad General:** Proporción de postulantes clasificados por estratos socioeconómicos.
- **Índice de Cobertura y Demanda:** Relación entre el total de cupos disponibles (900 becados) y el volumen total de postulantes por gestión académica.
- **Métricas de Asistencia y Eficiencia:** Control cruzado entre la asistencia registrada en el comedor y los respaldos físicos exigidos por auditoría.

## 2.3. Arquitectura Backend con Spring Boot

### *2.3.1. Framework Spring Boot y Lenguaje Java*

Spring Boot es un framework de código abierto basado en Java diseñado para simplificar el desarrollo de aplicaciones empresariales mediante la configuración automática y

la creación de microservicios independientes (Walls, 2019). Su ecosistema ofrece alta robustez, escalabilidad y un tipado fuerte, garantizando el procesamiento seguro de transacciones en la gestión del comedor universitario.

**Figura 8**

*Logotipo del framework Spring Boot*

*Nota.* Adaptado de *Spring Boot* [Logotipo], por Spring (VMware Tanzu), 2024, https://spring.io/projects/spring-boot.

### *2.3.2. Diseño de Servicios Web y Arquitectura RESTful*

La arquitectura REST (*Representational State Transfer*) es un estilo arquitectónico que utiliza los verbos del protocolo HTTP (GET, POST, PUT, DELETE) para desacoplar el servidor de las aplicaciones clientes (Fielding, 2000). Spring Boot permite construir *endpoints* RESTful eficientes para conectar la base de datos central con el frontend web, la aplicación móvil y el microservicio de IA.

### *2.3.3. Seguridad, Autenticación y Cifrado de Datos (Spring Security y JWT)*

Spring Security proporciona mecanismos avanzados de autenticación y control de acceso basado en roles (RBAC) (Mularien, 2010). En el sistema, el propio backend verifica las credenciales y emite un *JSON Web Token* (JWT) de acceso, firmado y de vida corta, junto con un token de renovación por dispositivo; el backend verifica el JWT en cada petición de forma *stateless* (Rouse et al., 2018). Las contraseñas de estudiantes y personal administrativo se almacenan únicamente como hash (BCrypt) y el token de renovación solo como hash, de modo que los datos sensibles de postulación permanecen protegidos.

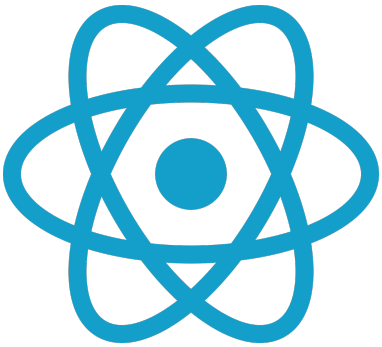
## 2.4. Desarrollo Web Frontend con React y TypeScript

### *2.4.1. Biblioteca React con TypeScript para Aplicaciones Web Escalables*

React es una biblioteca JavaScript orientada a la construcción de interfaces de usuario basadas en componentes reutilizables y un DOM virtual que optimiza el rendimiento del renderizado (Banker, 2014). Al integrar TypeScript, se añade un tipado estático que reduce los errores en tiempo de desarrollo y mejora la mantenibilidad del código del panel web (Cherny, 2019).

**Figura 9**

*Logotipo de la biblioteca React*



*Nota.* Adaptado de *React* [Logotipo], por Meta Platforms, 2024, https://react.dev/.

### *2.4.2. Gestión de Estado Asíncrono y Caché con TanStack Query*

TanStack Query (anteriormente React Query) es una librería especializada en la gestión del estado del servidor en aplicaciones frontend (Linsley, 2021). Optimiza el rendimiento mediante el almacenamiento en caché, la sincronización automática y la eliminación de peticiones HTTP duplicadas durante la revisión masiva de carpetas por los auditores.

### *2.4.3. Diseño de Interfaces de Usuario para Paneles de Auditoría y Control*

El diseño de la interfaz web prioriza la usabilidad y la *scannabilidad* visual, permitiendo a las trabajadoras sociales auditar los expedientes priorizados por la IA, revisar respaldos y gestionar el listado de los 900 becados de manera intuitiva (Tidwell et al., 2020).

## 2.5. Desarrollo Móvil Multiplataforma con Flutter

### *2.5.1. Framework Flutter y Lenguaje Dart*

Flutter es el toolkit de UI de Google para crear aplicaciones compiladas nativamente para móvil, web y escritorio a partir de un único código base escrito en Dart (Windmill, 2020). Permite una tasa de refresco elevada y un rendimiento cercano al código nativo en dispositivos Android e iOS.

### *2.5.2. Arquitectura Móvil para Carga Remota de Expedientes Digitales*

La arquitectura de la app móvil organiza la lógica de negocio y la captura de datos (fotografías de documentos socioeconómicos y formularios) facilitando que los estudiantes postulen remotamente sin saturar las instalaciones físicas de la universidad (Esposito, 2021).

### *2.5.3. Consumo de APIs y Manejo de Almacenamiento Local*

La aplicación utiliza servicios para el consumo asíncrono de las APIs REST de Spring Boot y emplea almacenamiento local en el dispositivo para gestionar sesiones de usuario de forma segura y permitir el guardado borrador de la postulación (Decarli, 2021).

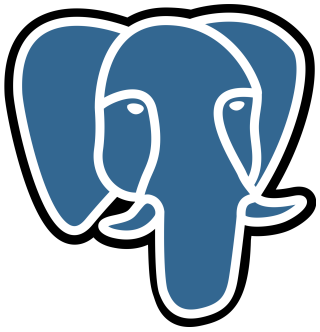
## 2.6. Sistema de Gestión de Bases de Datos

### *2.6.1. Base de Datos Relacional PostgreSQL*

PostgreSQL es un sistema de gestión de bases de datos relacional orientada a objetos (ORDBMS) de código abierto, reconocido por su fiabilidad, integridad de datos y soporte para transacciones complejas compatibles con ACID (Riggs et al., 2015).

**Figura 10**

*Logotipo del sistema de gestión de bases de datos PostgreSQL*



*Nota.* Adaptado de *PostgreSQL* [Logotipo], por The PostgreSQL Global Development Group, 2024, https://www.postgresql.org/.

### *2.6.2. Modelado de Datos, Integridad Referencial y Consultas Masivas*

El diseño relacional mediante esquemas bien estructurados y claves foráneas garantiza la consistencia entre los registros de estudiantes, solicitudes socioeconómicas y asistencias al comedor. Además, sus índices optimizados permiten ejecutar consultas masivas de forma rápida durante la fase de postulación (Chodorow, 2013).

## 2.7. Ecosistema de Desarrollo en Python para Modelos de IA

Para el desarrollo, entrenamiento y despliegue de los modelos de IA descritos en la sección 2.1, se seleccionó el lenguaje Python debido a su madurez en la comunidad científica y la robustez de sus librerías especializadas (McKinney, 2012):

**Figura 11**

*Logotipo del lenguaje de programación Python*

*Nota.* Adaptado de *Python* [Logotipo], por Python Software Foundation, 2024, https://www.python.org/.

- **NumPy:** Biblioteca fundamental para el cálculo numérico multidimensional eficiente, optimizada para la manipulación de vectores y matrices (Harris et al., 2020).
- **Pandas:** Herramienta de análisis y manipulación de datos estructurados que proporciona estructuras de datos como los *DataFrames*, esenciales para la limpieza, transformación y preprocesamiento de los formularios socioeconómicos (McKinney, 2010).
- **Scikit-Learn:** Framework de Machine Learning en Python que incluye implementaciones optimizadas de algoritmos de clasificación, regresión, evaluación de métricas (precisión, *recall*, F1-score) y técnicas de validación cruzada (Pedregosa et al., 2011).
- **FastAPI:** Framework web moderno de alto rendimiento diseñado para construir APIs RESTful en Python. Se utiliza para exponer las funcionalidades predictivas del modelo de ML como un microservicio independiente, consumible por el backend principal en Spring Boot (Ramírez, 2019).

Capítulo 3

# Dirección Universitaria de Bienestar Social y Salud (DUBSS)

# Capítulo 3: Dirección Universitaria de Bienestar Social y Salud (DUBSS)

## 3.1. Dirección Universitaria de Bienestar Social y Salud (DUBSS)

La Dirección Universitaria de Bienestar Social y Salud (DUBSS) es la unidad operativa central dependiente de la Universidad Autónoma Gabriel René Moreno (UAGRM) encargada de planificar, programar, coordinar y ejecutar acciones orientadas a mejorar la calidad de vida de la comunidad universitaria. Como órgano ejecutor del bienestar social, la DUBSS administra los programas de incentivos socioeconómicos y de salud integral, entre los cuales destaca la gestión del Comedor Universitario y la asignación del beneficio de Beca Comedor para los estudiantes en situación de vulnerabilidad.

La Dirección Universitaria de Bienestar Social y Salud (DUBSS) fue creada como una respuesta a la necesidad histórica de brindar asistencia integral a la comunidad universitaria de la UAGRM. A lo largo de los años, se ha consolidado como el pilar fundamental para garantizar la permanencia y graduación de los estudiantes con escasos recursos socioeconómicos, implementando programas emblemáticos como el Comedor Universitario, las Becas Trabajo, la guardería universitaria y el apoyo médico-psicológico. En la actualidad, la DUBSS afronta el reto de modernizar y digitalizar sus procesos de postulación e intensificar la transparencia en la adjudicación de sus beneficios sociales.

### *3.1.1. Nombre*

- **Nombre de la Institución:** Dirección Universitaria de Bienestar Social y Salud (DUBSS).
- **Entidad Superior:** Universidad Autónoma Gabriel René Moreno (UAGRM).

### *3.1.2. Misión*

La Dirección Universitaria de Bienestar Social y Salud (D.U.B.S.S.) es una unidad que contribuye al bienestar social de la población de la UAGRM, siendo el órgano que planifica, programa y coordina acciones orientadas a promover el desarrollo físico, mental, psicoafectivo, cognoscitivo y social de los estudiantes, docentes y administrativos.

### *3.1.3. Visión*

Ser una dirección que brinde servicio de calidez a la comunidad universitaria velando por el bienestar social, académico, cultural y deportivo de la población universitaria, proyectándose como una de las mejores direcciones de la universidad.

### *3.1.4. Imagen Corporativa / Marca*

La identidad visual de la DUBSS se encuentra alineada a la imagen institucional de la Universidad Autónoma Gabriel René Moreno (UAGRM), caracterizada por:

- **Logotipo Institucional:** Emblema circular con el acrónimo DUBSS, integrado en conjunto con el escudo y la tipografía oficial de la UAGRM.
- **Colores Corporativos:** Azul institucional (que transmite confianza, seguridad y respaldo académico) y Rojo (asociado a la identidad y dinamismo de la UAGRM).
- **Posicionamiento:** Identificada por la comunidad estudiantil como el canal oficial de apoyo social, becas e incentivos universitarios.

**Figura 12**

*Logotipo institucional de la Dirección Universitaria de Bienestar Social y Salud (D.U.B.S.S.) de la UAGRM*

*Nota.* Recuperado del sitio web oficial de la DUBSS, de https://dubs.uagrm.edu.bo/

### *3.1.5. Web, Contacto, Redes Sociales*

**Sitio Web Oficial:** https://dubs.uagrm.edu.bo/

**Figura 13**

*Sitio web oficial de la Dirección Universitaria de Bienestar Social y Salud (DUBSS)*

*Nota.* Captura de pantalla del sitio web institucional. Recuperado el 13 de septiembre de 2026, de https://dubs.uagrm.edu.bo/

**Facebook:** https://www.facebook.com/DireccionUniversitariaDeBienestarSocial

**Figura 14**

*Página oficial de Facebook de la Dirección Universitaria de Bienestar Social y Salud (DUBSS)*

*Nota.* Captura de pantalla de la red social Facebook. Recuperado el 13 de septiembre de 2026, de https://www.facebook.com/DireccionUniversitariaDeBienestarSocial

**Instagram:** https://www.instagram.com/dubs_uagrm/

**Figura 15**

*Perfil oficial de Instagram de la Dirección Universitaria de Bienestar Social y Salud (DUBSS)*

*Nota.* Captura de pantalla de la red social Instagram. Recuperado el 13 de septiembre de 2026, de https://www.instagram.com/dubs_uagrm/

**Correo Electrónico:** dubs@uagrm.edu.bo

**Teléfono / WhatsApp de Atención:** +591 61537017

## 3.2. Estructura Organizacional

### Figura 16

*Organigrama de la Dirección Universitaria de Bienestar Social y Salud (DUBSS)*

*Nota.* Elaboración propia en base a los datos institucionales de la DUBSS.

## 3.3. Ubicación e Infraestructura

**Figura 17**

*Comedor Universitario de la Universidad Autónoma Gabriel René Moreno*

*Nota.* Elaboración propia.

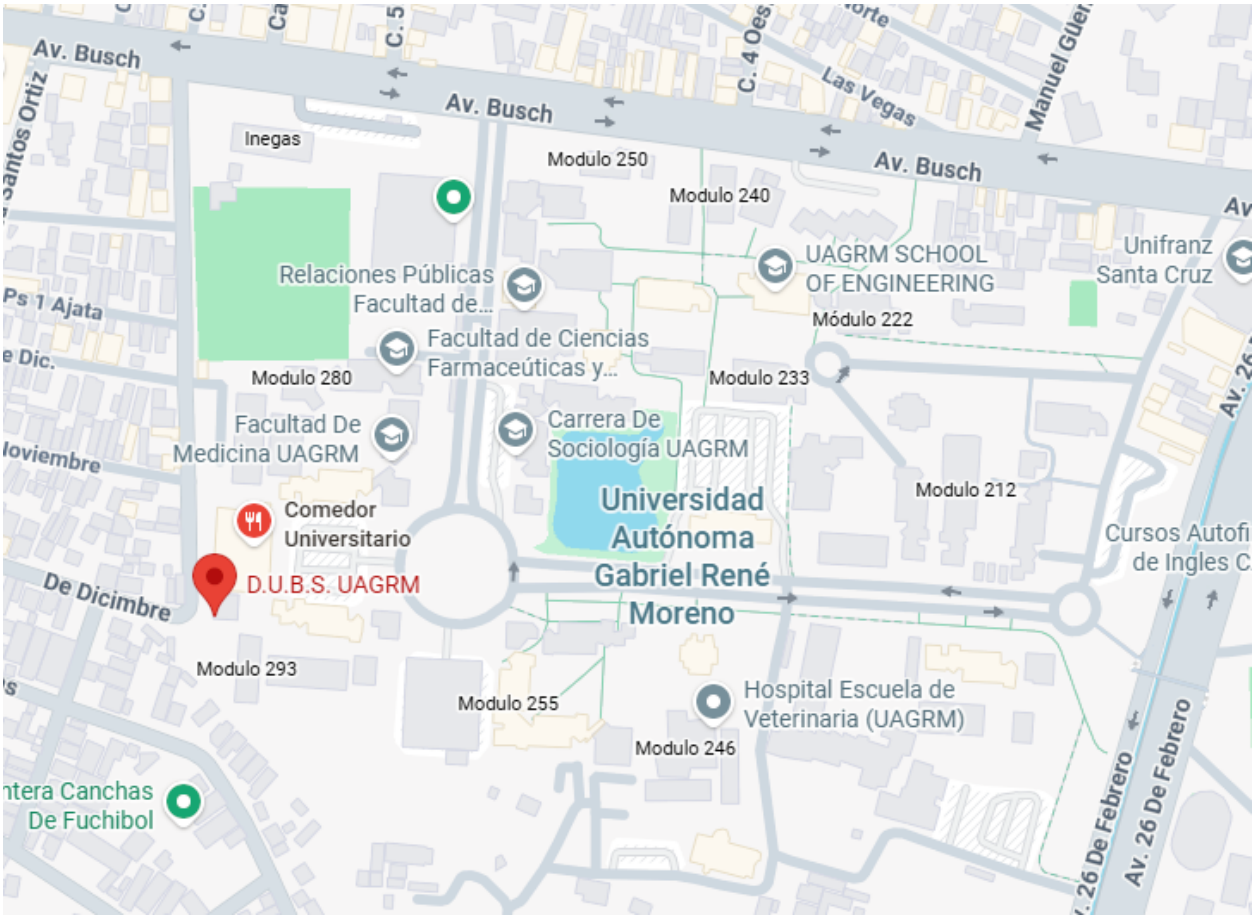
**Figura 18**

*Oficinas de la Dirección Universitaria de Bienestar Social y Salud (DUBSS)*

*Nota.* Elaboración propia.

**Figura 19**

*Ubicación de las instalaciones de la DUBSS en el campus universitario*

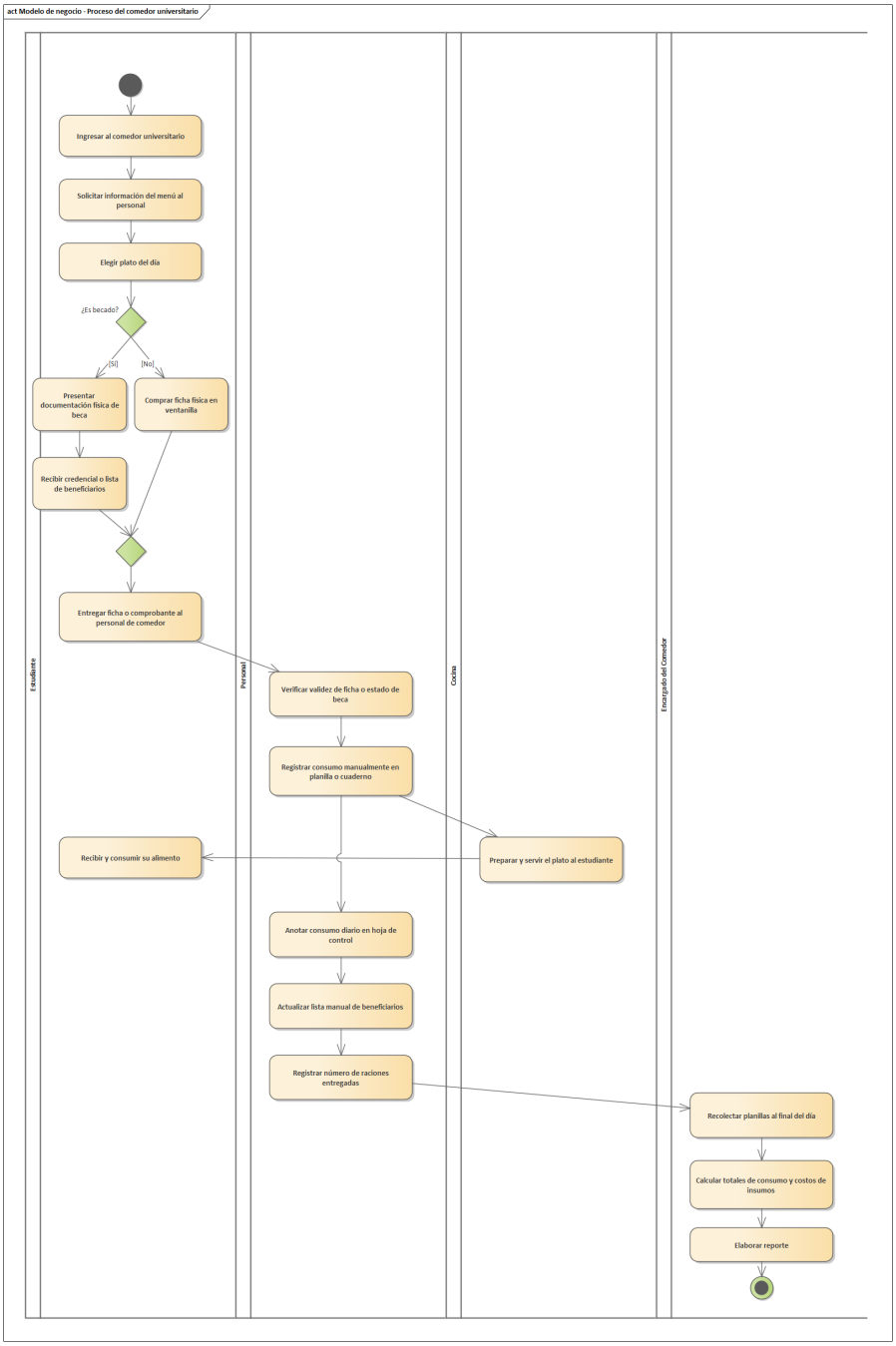



*Nota.* Elaboración propia a partir de una captura de Google Maps.

## 3.4. Modelo de Negocio

### Figura 20

*Modelo de negocio propuesto para el Sistema de Información de la Beca Comedor*



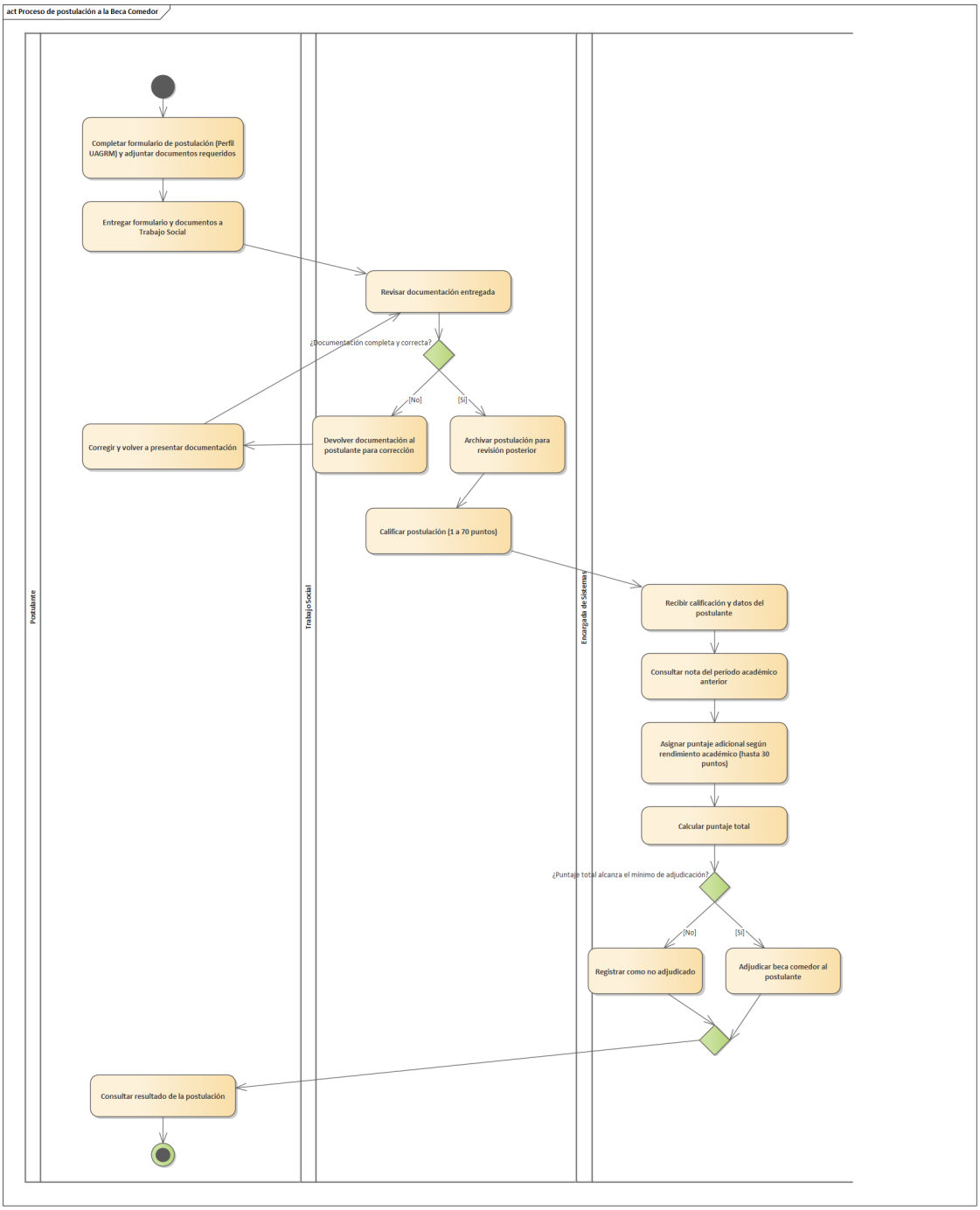

*Nota.* Elaboración propia con información de la DUBSS.

### 3.5. Flujo Operativo

El flujo operativo describe cómo se desarrolla actualmente, en la Dirección Universitaria de Bienestar Social y Salud (DUBSS), el proceso de postulación y adjudicación de la Beca Comedor. En él intervienen el postulante, que presenta su carpeta con la documentación de respaldo; el personal de Trabajo Social, que revisa los documentos y califica la solicitud; y la Encargada de Sistemas, que incorpora el rendimiento académico y calcula el puntaje total que determina la adjudicación. La Figura 21 representa las actividades de este proceso y las responsabilidades de cada actor.

**Figura 21**

*Diagrama de actividades del proceso de postulación a la Beca Comedor*




*Nota.* Elaboración propia basado en la entrevista con el personal de la DUBSS.

Capítulo 4

# Especificación de Requisitos de Software según la IEEE 830

# Capítulo 4: Especificación de Requisitos de Software según la IEEE 830

## 4.1. Introducción

### *4.1.1. Propósito*

El presente documento tiene como propósito definir, de manera clara, completa y sin ambigüedades, las funcionalidades y restricciones requeridas para el desarrollo del Sistema de Información Web y Móvil para la Postulación y Preselección Automatizada de la Beca Comedor mediante Inteligencia Artificial, con Reportes de Business Intelligence para la Dirección Universitaria de Bienestar Social y Salud (DUBSS) de la Universidad Autónoma Gabriel René Moreno. Esta Especificación de Requisitos del Software (ERS) está dirigida al equipo de desarrollo encargado de la construcción del sistema, al personal directivo y administrativo de la Dirección Universitaria de Bienestar Social y Salud (DUBSS) y a los docentes evaluadores del proyecto de grado, con el objetivo de establecer un acuerdo formal sobre el alcance, el comportamiento esperado y las restricciones del software antes de iniciar las etapas de diseño e implementación.

### *4.1.2. Ámbito del Sistema*

**Nombre del proyecto:** Sistema de Información Web y Móvil para la Postulación y Preselección Automatizada de la Beca Comedor mediante Inteligencia Artificial, con Reportes de Business Intelligence para la Dirección Universitaria de Bienestar Social y Salud (DUBSS) de la Universidad Autónoma Gabriel René Moreno.

El sistema comprenderá las siguientes funcionalidades principales:

- Registro y autenticación de postulantes a través de las plataformas web y móvil.
- Postulación en línea a la Beca Comedor mediante el llenado de formularios socioeconómicos digitales.
- Preselección automatizada de solicitudes empleando un modelo de Inteligencia Artificial y Machine Learning que precalifica algorítmicamente el nivel de vulnerabilidad socioeconómica de cada postulante.

- Gestión administrativa de los expedientes finalistas, incluyendo la auditoría física de respaldos y el registro de los estudiantes efectivamente becados.
- Registro y control de la asistencia física y digital de los becarios al Comedor Universitario.
- Generación de reportes y dashboards de Business Intelligence con proyecciones predictivas que apoyen la toma de decisiones institucionales.

Entre los principales beneficios que otorgará el sistema a la DUBSS se encuentran la reducción drástica del tiempo y del personal destinado a la revisión manual de carpetas, la eliminación del cuello de botella administrativo generado por las 20,000 solicitudes recibidas por gestión, la incorporación de un criterio de selección objetivo, transparente y auditable, y la disposición de información oportuna y confiable para la planificación y rendición de cuentas institucional.

### *4.1.3. Definiciones, Acrónimos y Abreviaturas*

La Tabla 9 resume los principales términos, siglas y acrónimos empleados a lo largo del presente documento.

**Tabla 9**

*Definiciones, acrónimos y abreviaturas empleados en el documento*

| Nombre | Descripción |
|---|---|
| DUBSS | Dirección Universitaria de Bienestar Social y Salud, unidad de la UAGRM responsable de la Beca Comedor. |
| Administrador | Personal de la DUBSS que audita, valida y gestiona las postulaciones dentro del sistema. |
| Postulante | Estudiante de la UAGRM que solicita el beneficio de la Beca Comedor a través del sistema. |
| Usuario | Persona que interactúa con el software, ya sea en calidad de postulante o de administrador. |
| RF | Requerimiento funcional. |
| RNF | Requerimiento no funcional. |
| ERS | Especificación de Requisitos del Software. |
| PCN | Prueba de caja negra. |
| IA | Inteligencia Artificial. |
| ML | Machine Learning (aprendizaje automático). |
| PLN | Procesamiento de Lenguaje Natural. |
| API | Interfaz de Programación de Aplicaciones. |
| UI/UX | Interfaz de Usuario y Experiencia de Usuario. |
| JS | Lenguaje de programación JavaScript. |

*Nota.* Elaboración propia.

### *4.1.4. Referencias*

Los principales estándares, libros y artículos empleados para las definiciones y el desarrollo de este capítulo son los siguientes:

- IEEE Computer Society (1998) y ISO/IEC/IEEE (2011).

- Pressman (2010).
- Sommerville (2011).

#### *4.1.5. Visión General del Documento*

El presente documento se compone de tres secciones (Introducción, Descripción General y Requisitos Específicos) que describen detalladamente el producto a desarrollarse. En primer lugar, la sección de Introducción establece el propósito, el ámbito, las definiciones y la visión general del documento. A continuación, la sección de Descripción General sitúa al sistema dentro de su contexto de uso, describiendo sus funciones principales, las características de sus usuarios y las restricciones y suposiciones bajo las cuales operará. Finalmente, la sección de Requisitos Específicos detalla de manera formal cada uno de los requisitos funcionales, no funcionales y de interfaz de usuario que deberá satisfacer el sistema.

### 4.2. Descripción General

#### *4.2.1. Perspectiva del Producto*

El Sistema de Información Web y Móvil para la Postulación y Preselección Automatizada de la Beca Comedor es un producto de software independiente y de nuevo desarrollo; no constituye un módulo dentro de una plataforma empresarial mayor ni reemplaza algún sistema institucional preexistente, dado que la DUBSS gestiona actualmente este proceso de forma manual. No obstante, el sistema sí se relaciona con dos actores externos a su núcleo funcional: por un lado, el punto de control de asistencia del Comedor Universitario, donde los postulantes seleccionados validan físicamente su condición de becados; y por otro, el personal directivo de la DUBSS, que consume los reportes y proyecciones de Business Intelligence generados por el sistema para la toma de decisiones institucionales.

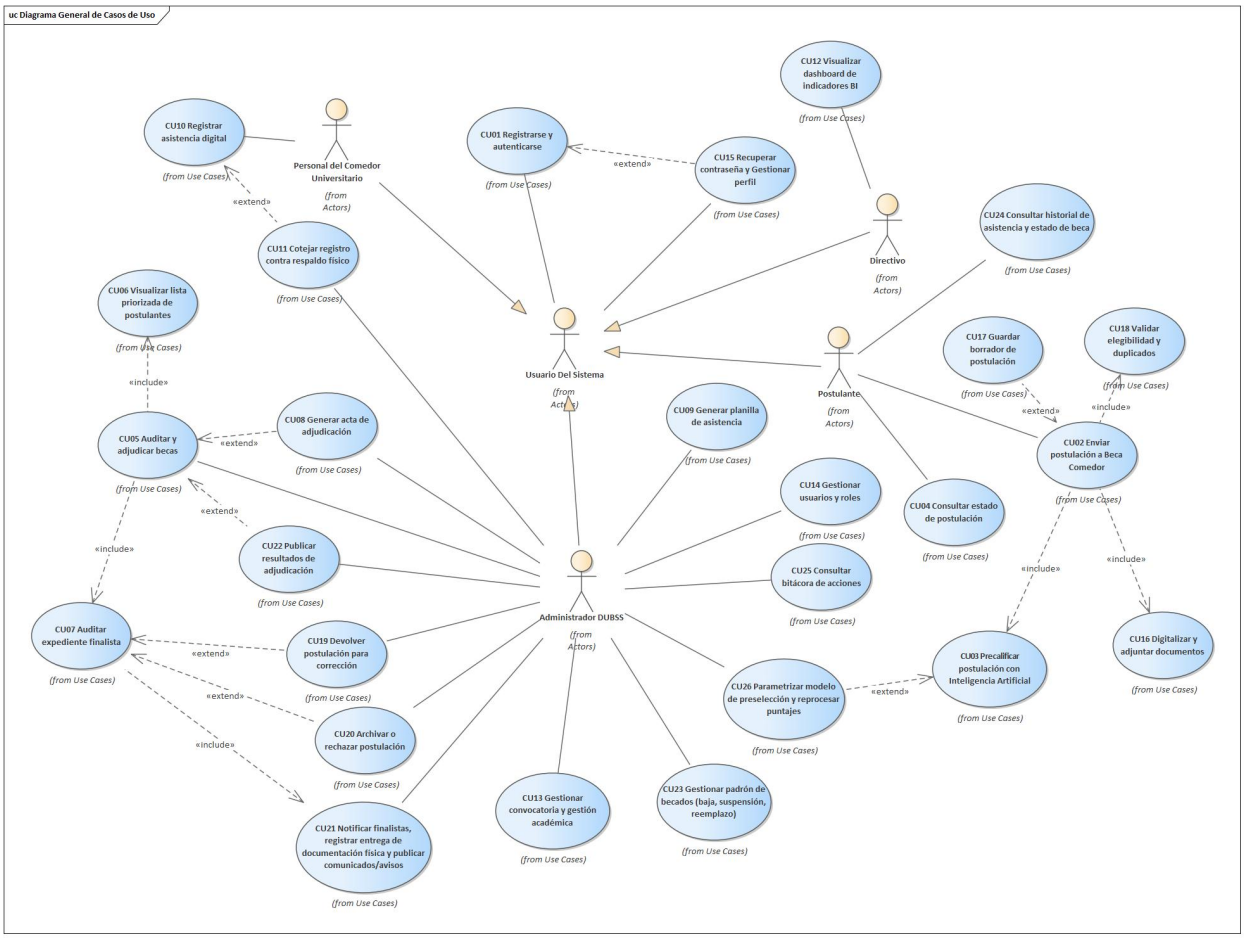
#### *4.2.2. Funciones del Producto*

De manera general, el sistema permitirá a los postulantes registrarse y enviar su postulación a la Beca Comedor completando un formulario socioeconómico digital desde la web o la aplicación móvil. Asimismo, el sistema precalificará automáticamente cada

solicitud mediante un modelo de inteligencia artificial que estima el nivel de vulnerabilidad socioeconómica del postulante, y notificará a los finalistas para que presenten su documentación física de respaldo. Por su parte, el personal de la DUBSS podrá auditar y validar dichos expedientes, administrar el registro de los estudiantes efectivamente becados y controlar su asistencia al Comedor Universitario. Finalmente, el sistema generará reportes y dashboards de Business Intelligence que resuman el comportamiento histórico de las postulaciones y proyecten tendencias futuras para apoyar la planificación institucional. La Figura 22 representa gráficamente los actores del sistema y los casos de uso principales asociados a cada uno.

**Figura 22**

*Diagrama de casos de uso del Sistema Web y Móvil de Postulación y Preselección de la Beca Comedor*




*Nota.* Elaboración propia.

### *4.2.3. Características de los Usuarios*

La Tabla 10 describe los tipos de usuario del sistema, sus responsabilidades y el nivel de conocimiento técnico requerido para su uso.

**Tabla 10**

*Tipos de usuario del sistema y sus características*

| Tipo de usuario | Descripción | Conocimiento técnico requerido |
|---|---|---|
| Postulante | Estudiante de la UAGRM que se registra y postula a la Beca Comedor a través de la web o la aplicación móvil. | Básico: manejo elemental de aplicaciones web y móviles. |
| Administrador DUBSS | Personal administrativo que audita la documentación física de los finalistas y gestiona el proceso de selección dentro del sistema. | Básico: manejo de ofimática y de un panel web administrativo. |
| Directivo DUBSS (Gerente) | Personal directivo que visualiza el dashboard de reportes gráficos y proyecciones de Business Intelligence para la toma de decisiones. | Básico: interpretación de reportes y gráficos, sin necesidad de conocimientos de programación. |

*Nota.* Elaboración propia. Ningún tipo de usuario requiere conocimientos técnicos avanzados de programación; únicamente se requiere acceso a un dispositivo con conexión a internet.

### *4.2.4. Restricciones Generales*

El desarrollo y funcionamiento del sistema está sujeto a las siguientes restricciones:

- El sistema debe desplegarse sobre un servidor en la nube de Amazon Web Services (AWS), empleando PostgreSQL como motor de base de datos relacional.
- Las notificaciones push deben integrarse a través de Firebase Cloud Messaging y el almacenamiento de archivos a través de MinIO; la autenticación la resuelve el propio backend mediante tokens JWT.
- La aplicación móvil debe desarrollarse bajo el framework Flutter, priorizando su compatibilidad con dispositivos Android.
- El desarrollo del proyecto debe ejecutarse dentro del periodo comprendido entre el 24 de agosto y el 30 de noviembre de 2026, conforme al cronograma y al presupuesto estimado del proyecto (véase la Tabla 6).
- El sistema debe garantizar la confidencialidad y el resguardo de los datos socioeconómicos de los postulantes, en cumplimiento de las políticas institucionales de protección de datos personales de la UAGRM.

### *4.2.5. Suposiciones y Dependencias*

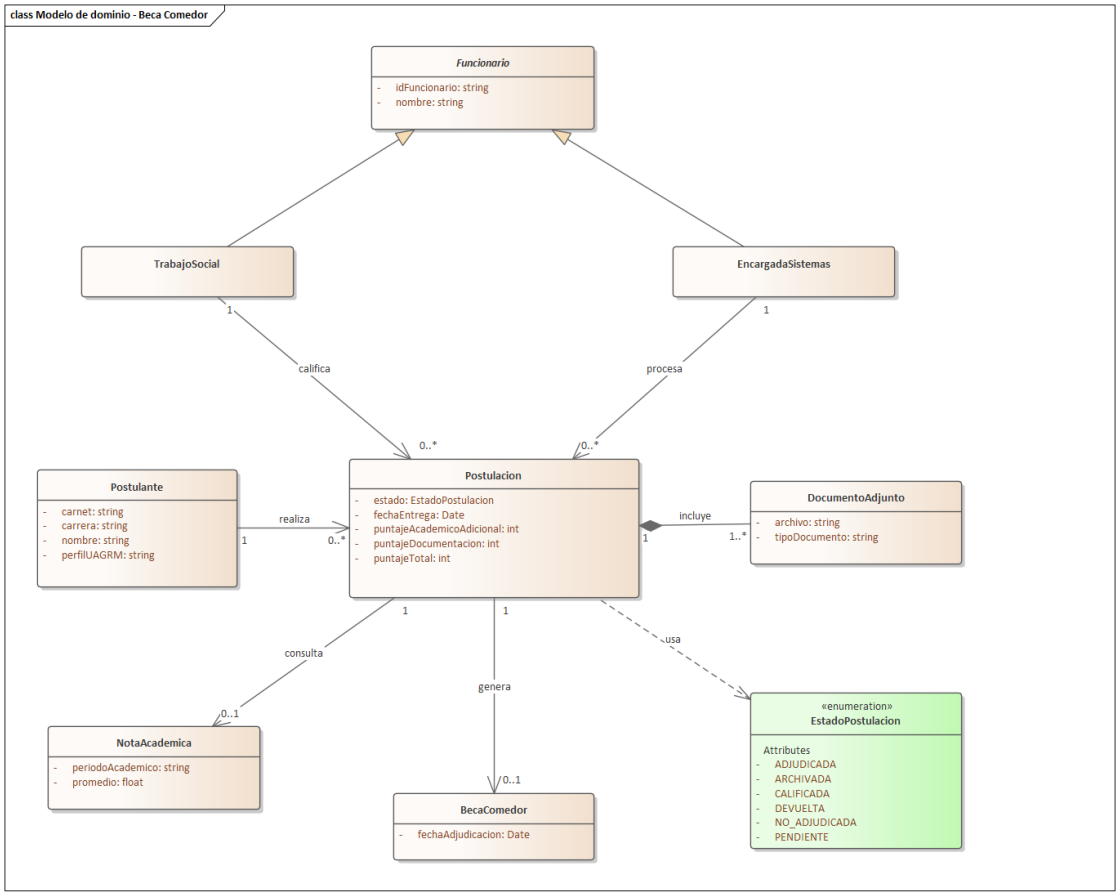
Se asume que los postulantes y el personal de la DUBSS contarán con acceso a un dispositivo (computadora o teléfono móvil) y con una conexión a internet estable durante el periodo de postulación y evaluación. El sistema depende de los servicios de Firebase Cloud Messaging para el envío de notificaciones push y de MinIO para el almacenamiento de archivos, así como de la disponibilidad del servidor AWS para su despliegue y funcionamiento continuo. Asimismo, se asume que la DUBSS proporcionará de manera oportuna los criterios institucionales de vulnerabilidad socioeconómica requeridos para entrenar y calibrar el modelo de preselección con inteligencia artificial. El flujo de actividades y las dependencias entre los actores que intervienen en el proceso de postulación se describen en la Figura 21 de la Sección 3.5.

### *4.2.6. Modelo de Dominio*

El modelo de dominio representa las entidades conceptuales involucradas en el proceso de postulación y adjudicación de la Beca Comedor, así como las relaciones estructurales que existen entre ellas, sin considerar aún su implementación técnica. La Figura 23 presenta el diagrama conceptual correspondiente.

**Figura 23**

*Modelo de dominio del Sistema Web y Móvil de Postulación y Preselección de la Beca Comedor*




*Nota.* Elaboración propia.

El proceso gira en torno a la clase *Postulación*, que agrupa la información generada durante todo el trámite: la fecha de entrega de documentación, el puntaje calculado por la inteligencia artificial (hasta 70 puntos), el puntaje de rendimiento académico que el auditor

registra durante la auditoría del expediente (hasta 30 puntos), el puntaje total resultante (hasta 100 puntos), y su estado actual dentro del flujo. Este estado está formalmente restringido a un conjunto cerrado de valores —pendiente, devuelta, archivada, calificada, adjudicada o no adjudicada—, representados mediante la enumeración *EstadoPostulacion*, lo que evita que el sistema admita estados no contemplados en el proceso.

Cada postulación es iniciada por un *Postulante*, identificado por su nombre, carné de identidad, carrera y perfil institucional UAGRM. Un postulante puede generar más de una postulación a lo largo del tiempo —por ejemplo, si su documentación es devuelta y debe corregirla y volver a presentarla—, por lo que la relación entre ambas clases es de uno a muchos.

Toda postulación incluye, de manera obligatoria, uno o más *DocumentoAdjunto*, cada uno caracterizado por su tipo y el archivo correspondiente. Esta relación se modela como una composición, ya que los documentos no tienen existencia ni sentido fuera de la postulación a la que pertenecen: si la postulación se elimina, sus documentos se eliminan con ella.

En el proceso intervienen dos roles de personal administrativo, representados como especializaciones de una clase abstracta común *Funcionario*, que concentra los atributos compartidos de identificador y nombre. El primer rol, *TrabajoSocial*, es responsable de revisar la documentación entregada y calificar la postulación; el segundo, *EncargadaSistemas*, procesa esa calificación, consulta el rendimiento académico del postulante en *NotaAcademica* —registro identificado por período académico y promedio— y calcula el puntaje total que determina la adjudicación.

Cuando el puntaje total alcanza el mínimo requerido, la postulación genera una *BecaComedor*, entidad que registra la fecha de adjudicación. Esta relación es opcional desde la perspectiva de la postulación, ya que no toda postulación calificada llega a adjudicarse.

En conjunto, el modelo refleja que la postulación es el eje central del dominio: nace de un postulante, se sustenta en documentos que le pertenecen exclusivamente, es intervenida por dos roles funcionales distintos en etapas sucesivas del proceso, se apoya en un registro

académico externo para el cálculo del puntaje, y —de cumplirse las condiciones— culmina en la adjudicación del beneficio.

## 4.3. Requisitos Específicos

### *4.3.1. Requisitos Funcionales*

A continuación se detallan los requisitos funcionales identificados para el sistema, formulados bajo el esquema de historia de usuario (¿quién?, ¿qué? y ¿por qué?) y acompañados de sus respectivos criterios de aceptación. Las Tablas 11 a 31 presentan cada uno de estos requisitos.

**Tabla 11**

*Requerimiento funcional RF-01: Registro y autenticación de usuarios*

| | |
|---|---|
| ¿Quién? | Postulante, Administrador DUBSS, Directivo DUBSS y Personal del Comedor Universitario. |
| ¿Qué? | Quiere registrarse (el Postulante), autenticarse y cerrar su sesión en el sistema desde el portal web o la aplicación móvil. |
| ¿Por qué? | Para acceder de forma segura a las funcionalidades correspondientes a su rol, verificado por el propio sistema mediante un token de acceso firmado. |

**Criterios de aceptación**

- El sistema permite al Postulante crear una cuenta, y a todo usuario autenticarse y cerrar sesión, desde la web y desde la aplicación móvil. La cuenta creada por autorregistro siempre tiene el rol Postulante; las cuentas del personal las crea el Administrador DUBSS.

- El sistema exige que el CI, el registro universitario (9 dígitos numéricos, sin comenzar por 0) y el correo electrónico sean únicos, y que la contraseña tenga al menos 6 caracteres y coincida con su confirmación.

- El sistema identifica el rol del usuario autenticado (Postulante, Administrador DUBSS, Directivo o Personal del Comedor) y lo redirige al módulo correspondiente.

- El sistema rechaza credenciales inválidas con un mismo mensaje de error, sin revelar si el correo existe, y niega el acceso a las cuentas desactivadas.

- El sistema bloquea temporalmente el acceso tras varios intentos fallidos consecutivos (5 intentos y un primer bloqueo de 5 minutos, mayor si se continúa fallando); el bloqueo lo aplica el propio sistema y su escala exacta está por definir con la DUBSS.

- El sistema cierra la sesión únicamente en el dispositivo desde el que se solicita, sin afectar las sesiones abiertas en otros dispositivos, y no permite ver pantallas privadas hasta un nuevo inicio de sesión.

- El sistema almacena las contraseñas únicamente como hash (nunca en texto plano), no las escribe, junto con los tokens de sesión, en sus registros, y anota en la bitácora cada inicio de sesión exitoso.

- El sistema mantiene la sesión mediante un token de acceso de vida corta y un token de renovación por dispositivo, que se guarda solo como hash y se revoca al cerrar sesión.

*Nota.* Elaboración propia.

**Tabla 12**

*Requerimiento funcional RF-02: Envío de postulación a la Beca Comedor*

| | |
|---|---|
| ¿Quién? | Postulante. |
| ¿Qué? | Quiere completar un formulario socioeconómico digital y adjuntar la documentación de respaldo digitalizada, desde el portal web o la aplicación móvil. |
| ¿Por qué? | Para iniciar formalmente su proceso de postulación a la Beca Comedor. |

**Criterios de aceptación**

- El Postulante puede completar la totalidad de los campos del formulario socioeconómico.
- El Postulante puede adjuntar uno o más archivos digitales como respaldo documental.
- El sistema no permite enviar la postulación si falta algún campo o documento obligatorio.
- El sistema confirma la recepción de la postulación y la asocia a la gestión académica vigente.

*Nota.* Elaboración propia.

**Tabla 13**

*Requerimiento funcional RF-03: Preselección automatizada con Inteligencia Artificial*

| | |
|---|---|
| ¿Quién? | Sistema (módulo de preselección con Inteligencia Artificial). |
| ¿Qué? | Quiere precalificar automáticamente cada postulación recibida mediante un modelo de inteligencia artificial que evalúe las variables socioeconómicas y académicas declaradas. |
| ¿Por qué? | Para generar un puntaje de vulnerabilidad que permita ordenar objetivamente a los postulantes según su nivel de necesidad, sin intervención manual sobre el total de solicitudes recibidas (hasta 20,000 o más por gestión). |

**Criterios de aceptación**

- El sistema calcula un puntaje de vulnerabilidad para cada postulación dentro de un tiempo razonable tras su envío.
- El sistema procesa automáticamente la totalidad de las postulaciones recibidas, sin requerir revisión manual previa.
- El sistema registra el puntaje obtenido y la fecha de cálculo, permitiendo su trazabilidad posterior.

*Nota.* Elaboración propia.

**Tabla 14**

*Requerimiento funcional RF-04: Panel de monitoreo y toma de decisiones (auditoría y adjudicación)*

| | |
|---|---|
| ¿Quién? | Administrador DUBSS. |
| ¿Qué? | Quiere visualizar la lista priorizada de postulantes según su puntaje de vulnerabilidad, auditar los expedientes finalistas y aprobar o rechazar la adjudicación de la beca. |
| ¿Por qué? | Para validar los expedientes físicos de los finalistas y generar las actas o comprobantes correspondientes a los beneficiarios aprobados. |

**Criterios de aceptación**

- El Administrador visualiza la lista de postulantes ordenada de mayor a menor puntaje de vulnerabilidad.
- El Administrador puede exportar la lista priorizada a un documento PDF, y la exportación queda registrada en la bitácora.
- El Administrador puede registrar la auditoría de un expediente finalista (resultado, observaciones de hasta 500 caracteres y, cuando el resultado es validado, el puntaje académico de 0 a 30) tras revisar el respaldo físico, y el sistema muestra el puntaje de la IA (hasta 70) y el total (hasta 100).
- El sistema solo permite auditar a un finalista con la entrega física registrada y con la postulación en estado calificada, y avisa al postulante el resultado de la auditoría.
- El Administrador puede aprobar o rechazar la adjudicación de la beca para cada postulante auditado.
- El sistema genera un acta o comprobante digital de adjudicación

**Tabla 15**

*Requerimiento funcional RF-05: Registro e impresión de planillas de asistencia*

| | |
|---|---|
| ¿Quién? | Administrador DUBSS y personal del Comedor Universitario. |
| ¿Qué? | Quiere generar planillas de asistencia física y digital para su firma y fotocopia de carnet, y registrar digitalmente la asistencia diaria ya verificada de los becados. |
| ¿Por qué? | Para mantener un respaldo cruzable frente a las auditorías externas, sin que el sistema dependa de ningún dispositivo de identificación biométrica propio, dado que dicha verificación física es responsabilidad de la empresa concesionaria. |

**Criterios de aceptación**

- El sistema genera planillas de asistencia imprimibles, listas para firma física y fotocopia de carnet.
- El sistema permite registrar digitalmente la asistencia diaria de cada becario, una vez verificada su identidad por el personal del comedor.
- El sistema permite cotejar el registro digital de asistencia contra el respaldo físico (planilla firmada) durante una auditoría.
- El sistema no depende de ningún dispositivo de identificación biométrica propio; esa verificación es externa al sistema.

*Nota.* Elaboración propia.

**Tabla 16**

*Requerimiento funcional RF-06: Consultas y notificaciones al postulante*

| | |
|---|---|
| ¿Quién? | Postulante. |
| ¿Qué? | Quiere realizar seguimiento en tiempo real del estado de su postulación (recibida, en evaluación, finalista, adjudicada, rechazada) y recibir notificaciones push sobre eventos relevantes. |
| ¿Por qué? | Para mantenerse informado del avance y del resultado de su beneficio sin necesidad de acudir presencialmente a la DUBSS. |

**Criterios de aceptación**

- El Postulante puede consultar en todo momento el estado actual de su postulación.
- El sistema notifica automáticamente al Postulante cuando su estado cambia (por ejemplo, al ser seleccionado como finalista o al adjudicársele la beca).

*Nota.* Elaboración propia.

**Tabla 17**

*Requerimiento funcional RF-07: Dashboard de indicadores de Business Intelligence*

| | |
|---|---|
| ¿Quién? | Directivo DUBSS. |
| ¿Qué? | Quiere visualizar un dashboard interactivo de Business Intelligence con indicadores clave del comportamiento histórico de las postulaciones y del uso del beneficio. |
| ¿Por qué? | Para apoyar la planificación institucional y la rendición de cuentas con información consolidada de cada gestión. |

**Criterios de aceptación**

- El Directivo puede visualizar indicadores clave (postulaciones recibidas, becas adjudicadas, asistencia) en un dashboard interactivo.
- El sistema permite filtrar los indicadores por gestión académica.

*Nota.* Elaboración propia.

**Tabla 18**

*Requerimiento funcional RF-08: Gestión de convocatorias y gestiones académicas*

| | |
|---|---|
| ¿Quién? | Administrador DUBSS. |
| ¿Qué? | Quiere crear, editar, abrir y cerrar convocatorias de la Beca Comedor, definiendo la gestión académica, las fechas, el número de cupos y los documentos requeridos. |
| ¿Por qué? | Para controlar el periodo en que se reciben postulaciones y asociar cada postulación a la gestión vigente. |

**Criterios de aceptación**

- El Administrador puede registrar una convocatoria con gestión académica, fecha de apertura, fecha de cierre y cupos (por definir con la DUBSS).
- El Administrador puede definir la lista de documentos obligatorios de cada convocatoria.
- El sistema solo permite enviar postulaciones mientras la convocatoria esté abierta.
- Solo puede existir una convocatoria vigente por gestión académica.

*Nota.* Elaboración propia.

**Tabla 19**

*Requerimiento funcional RF-09: Gestión de usuarios y roles*

| | |
|---|---|
| ¿Quién? | Administrador DUBSS. |
| ¿Qué? | Quiere crear, editar, desactivar y asignar roles (Administrador, Directivo y Personal del Comedor) a las cuentas del personal. |
| ¿Por qué? | Para garantizar que cada persona acceda únicamente a las funciones de su rol, conforme al control de acceso exigido por el sistema. |

**Criterios de aceptación**

- El Administrador puede crear y desactivar cuentas de personal y asignarles un rol.
- El sistema impide que un usuario acceda a funciones de un rol distinto al asignado.
- El sistema conserva el registro de los cambios de rol realizados.
- El sistema rechaza el alta de una cuenta con CI o correo electrónico ya registrados.

*Nota.* Elaboración propia.

**Tabla 20**

*Requerimiento funcional RF-10: Recuperación de contraseña y gestión de perfil*

| | |
|---|---|
| ¿Quién? | Postulante, Administrador DUBSS, Directivo DUBSS y Personal del Comedor Universitario. |
| ¿Qué? | Quiere recuperar su contraseña en caso de olvido y actualizar sus datos de contacto desde su perfil. |
| ¿Por qué? | Para recuperar el acceso a su cuenta sin intervención del personal y mantener actualizados los datos con que se le notifica. |

**Criterios de aceptación**

- El usuario puede solicitar el restablecimiento de contraseña mediante un enlace enviado a su correo registrado, gestionado por el propio sistema.
- El usuario puede modificar sus datos de contacto y su contraseña desde el perfil.
- El sistema no permite modificar el CI desde el perfil.

*Nota.* Elaboración propia.

**Tabla 21**

*Requerimiento funcional RF-11: Digitalización y carga de documentos de respaldo*

| | |
|---|---|
| ¿Quién? | Postulante. |
| ¿Qué? | Quiere digitalizar sus documentos con la cámara del teléfono, o cargarlos desde un archivo en la web, y asociarlos a su postulación. |
| ¿Por qué? | Para respaldar los datos declarados en el formulario socioeconómico sin presentar carpetas físicas en la etapa inicial. |

**Criterios de aceptación**

- El Postulante puede capturar un documento con la cámara del dispositivo móvil o adjuntar un archivo desde la web.
- El sistema valida el formato y el tamaño del archivo (formatos y tamaño máximo por definir con la DUBSS) y rechaza los no permitidos con un mensaje de error.
- El sistema muestra una vista previa y permite reemplazar o eliminar un documento antes de enviar la postulación.
- El sistema almacena los archivos en MinIO asociados a la postulación.

*Nota.* Elaboración propia.

**Tabla 22**

*Requerimiento funcional RF-12: Guardado de borrador de postulación*

| | |
|---|---|
| ¿Quién? | Postulante. |
| ¿Qué? | Quiere guardar su postulación incompleta como borrador y retomarla después. |
| ¿Por qué? | Para completar el formulario socioeconómico y reunir los documentos sin perder lo ya ingresado. |

**Criterios de aceptación**

- El Postulante puede guardar el formulario con campos obligatorios sin completar.
- El Postulante puede retomar y editar el borrador mientras la convocatoria esté abierta.
- Un borrador no se considera enviado ni es procesado por el módulo de preselección.
- El sistema descarta o bloquea los borradores no enviados al cerrar la convocatoria.

*Nota.* Elaboración propia.

**Tabla 23**

*Requerimiento funcional RF-13: Validación de elegibilidad y unicidad de postulaciones*

| | |
|---|---|
| ¿Quién? | Sistema. |
| ¿Qué? | Quiere verificar, al recibir una postulación, que el postulante cumple las condiciones de elegibilidad y que no existe otra postulación suya en la misma gestión. |
| ¿Por qué? | Para evitar registros duplicados o inválidos que distorsionen la priorización y la adjudicación. |

**Criterios de aceptación**

- El sistema impide enviar más de una postulación vigente por postulante y por convocatoria.
- El sistema verifica las condiciones de elegibilidad (por definir con la DUBSS) antes de aceptar el envío.
- El sistema informa al Postulante el motivo cuando su postulación es rechazada por esta validación.
- Una postulación devuelta (RF-14) y reenviada no se considera duplicada.

*Nota.* Elaboración propia.

**Tabla 24**

*Requerimiento funcional RF-14: Devolución de postulación para corrección*

| | |
|---|---|
| ¿Quién? | Administrador DUBSS (y Postulante, como receptor). |
| ¿Qué? | Quiere devolver una postulación con observaciones para que el Postulante corrija o complete su documentación y la reenvíe. |
| ¿Por qué? | Para evitar descartar postulaciones por defectos subsanables y mantener la trazabilidad de las correcciones. |

**Criterios de aceptación**

- El Administrador devuelve la postulación desde el expediente del finalista, registrando un motivo obligatorio de hasta 500 caracteres.
- Solo se devuelve una postulación en estado calificada y con al menos una auditoría registrada después de su última devolución.
- La postulación cambia al estado devuelta y el Postulante recibe siempre el aviso, con las observaciones, por bandeja, push y correo.
- El Postulante puede corregir y reenviar cuando quiera: no hay plazo de reenvío ni límite de devoluciones.
- La devolución queda en el historial de la postulación y en la bitácora, y no altera los documentos, el puntaje ni las auditorías anteriores.

*Nota.* Elaboración propia.

**Tabla 25**

*Requerimiento funcional RF-15: Archivo o rechazo de postulación*

| | |
|---|---|
| ¿Quién? | Administrador DUBSS. |
| ¿Qué? | Quiere archivar o rechazar una postulación indicando el motivo. |
| ¿Por qué? | Para cerrar formalmente los expedientes que no proceden y dejar constancia de la decisión. |

**Criterios de aceptación**

- El Administrador debe registrar un motivo para archivar o rechazar.
- La postulación pasa al estado correspondiente (archivada o no adjudicada) y deja de aparecer en la lista priorizada activa.
- El Postulante puede ver el estado y el motivo registrado.
- El sistema conserva la postulación archivada para consulta histórica y reportes.

*Nota.* Elaboración propia.

**Tabla 26**

*Requerimiento funcional RF-16: Notificación a finalistas, entrega de documentación física y comunicados*

| | |
|---|---|
| ¿Quién? | Administrador DUBSS (y Postulante, como receptor). |
| ¿Qué? | Quiere notificar a los finalistas para que presenten su documentación física, registrar la fecha en que se recibe, y publicar comunicados y avisos generales dirigidos a postulantes. |
| ¿Por qué? | Para coordinar la auditoría física de los finalistas, dejar constancia de su entrega y mantener informada a la población postulante. |

**Criterios de aceptación**

- El sistema envía una notificación push a cada finalista indicando el plazo y los documentos que debe presentar.
- El Administrador puede registrar la recepción de la documentación física de un finalista con su fecha.
- El Administrador puede publicar un comunicado dirigido a todos los postulantes o a un grupo (por ejemplo, los finalistas).
- Los comunicados se notifican mediante push y quedan consultables desde la aplicación.

*Nota.* Elaboración propia.

**Tabla 27**

*Requerimiento funcional RF-17: Publicación de resultados de adjudicación*

| | |
|---|---|
| ¿Quién? | Administrador DUBSS. |
| ¿Qué? | Quiere publicar la lista oficial de adjudicados de la gestión. |
| ¿Por qué? | Para dar a conocer los resultados de forma transparente y oficial y poner fin al proceso de selección. |

**Criterios de aceptación**

- El Administrador puede publicar los resultados solo cuando todas las adjudicaciones estén decididas.
- La lista publicada incluye a los adjudicados de la gestión y es consultable por los postulantes.
- Al publicar, el sistema notifica a cada postulante su resultado.
- Una vez publicada, la lista no se modifica sin dejar registro en la bitácora (RF-20).

*Nota.* Elaboración propia.

**Tabla 28**

*Requerimiento funcional RF-18: Gestión del padrón de becados*

| | |
|---|---|
| ¿Quién? | Administrador DUBSS. |
| ¿Qué? | Quiere mantener el padrón de becados vigentes, registrando bajas, suspensiones y reemplazos durante la gestión. |
| ¿Por qué? | Para mantener actualizada la lista de beneficiarios activos, de la cual dependen las planillas de asistencia, y reasignar los cupos liberados. |

**Criterios de aceptación**

- El Administrador puede consultar el padrón de becados vigentes de la gestión.
- El Administrador puede dar de baja o suspender a un becado registrando el motivo.
- El Administrador puede reemplazar un cupo liberado con el siguiente postulante priorizado.
- Los cambios en el padrón se reflejan en las planillas de asistencia generadas posteriormente.

*Nota.* Elaboración propia.

**Tabla 29**

*Requerimiento funcional RF-19: Historial de asistencia y estado de la beca*

| | |
|---|---|
| ¿Quién? | Becado (Postulante adjudicado). |
| ¿Qué? | Quiere consultar su historial de asistencia al comedor y el estado vigente de su beca. |
| ¿Por qué? | Para verificar su cumplimiento de los requisitos de asistencia y conservar su beneficio. |

**Criterios de aceptación**

- El Becado visualiza las fechas de asistencia registradas en la gestión vigente.
- El Becado visualiza el estado de su beca (vigente, suspendida o dada de baja) y el motivo, si corresponde.
- El Becado solo accede a su propia información.

*Nota.* Elaboración propia.

**Tabla 30**

*Requerimiento funcional RF-20: Bitácora de acciones*

| | |
|---|---|
| ¿Quién? | Administrador DUBSS. |
| ¿Qué? | Quiere consultar un registro de las acciones relevantes realizadas en el sistema (auditoría, devolución, adjudicación, cambios en el padrón y cambios de roles). |
| ¿Por qué? | Para garantizar la trazabilidad de las decisiones y facilitar las auditorías sobre el proceso. |

**Criterios de aceptación**

- El sistema registra automáticamente usuario, acción, expediente afectado, fecha y hora.
- El Administrador puede filtrar la bitácora por usuario, tipo de acción y rango de fechas.
- Los registros de la bitácora no pueden ser editados ni eliminados desde la interfaz.

*Nota.* Elaboración propia.

**Tabla 31**

*Requerimiento funcional RF-21: Parametrización del modelo de preselección y reprocesamiento*

| | |
|---|---|
| ¿Quién? | Administrador DUBSS. |
| ¿Qué? | Quiere ajustar los criterios y ponderaciones que usa el modelo de preselección y reprocesar los puntajes de las postulaciones de una convocatoria. |
| ¿Por qué? | Para calibrar la priorización según los criterios institucionales de vulnerabilidad y corregir puntajes cuando estos cambien. |

**Criterios de aceptación**

- El Administrador puede consultar y modificar los criterios y ponderaciones del modelo (por definir con la DUBSS).
- El Administrador puede solicitar el reprocesamiento de los puntajes de una convocatoria.
- El sistema registra la fecha de cada recálculo y conserva el puntaje anterior para trazabilidad (ver RF-03).
- El sistema muestra la lista priorizada actualizada tras el reprocesamiento.

*Nota.* Elaboración propia.

### *4.3.2. Requisitos No Funcionales*

A continuación se detallan los requisitos no funcionales identificados para el sistema, describiendo los atributos de calidad exigidos y sus respectivos criterios de aceptación. Las Tablas 32 a 36 presentan cada uno de estos requisitos.

**Tabla 32**

*Requerimiento no funcional RNF-01: Seguridad y protección de datos personales*

| **Descripción** |
| --- |
| El sistema debe garantizar la confidencialidad, integridad y protección de los datos socioeconómicos y personales de los postulantes, aplicando control de acceso basado en roles y cifrado de la información sensible almacenada y transmitida, en cumplimiento de las políticas institucionales de protección de datos personales de la UAGRM. |
| **Criterios de aceptación** |
| ▪ El acceso a los expedientes socioeconómicos está restringido según el rol del usuario autenticado.<br>▪ La información sensible se transmite y almacena de forma cifrada.<br>▪ El sistema mantiene un registro (log) de los accesos realizados a cada expediente. |

*Nota.* Elaboración propia.

**Tabla 33**

*Requerimiento no funcional RNF-02: Disponibilidad del servicio*

| **Descripción** |
| --- |
| El sistema debe mantenerse operativo y accesible durante los periodos de alta demanda, particularmente durante las fechas de apertura y cierre de la convocatoria de postulación, minimizando el riesgo de interrupciones del servicio desplegado en AWS. |
| **Criterios de aceptación** |
| ▪ El portal web y la aplicación móvil permanecen disponibles durante los picos de postulación.<br>▪ El sistema cuenta con un mecanismo de recuperación ante caídas del servidor. |

*Nota.* Elaboración propia.

**Tabla 34**

*Requerimiento no funcional RNF-03: Tiempo de respuesta y concurrencia*

| **Descripción** |
| --- |
| El sistema debe procesar la precalificación automática y las operaciones de consulta dentro de tiempos de respuesta aceptables, incluso bajo condiciones de alta concurrencia, considerando el volumen de hasta 20,000 postulaciones por gestión y la revisión masiva simultánea por parte de los auditores. |
| **Criterios de aceptación** |
| ▪ El algoritmo de precalificación procesa el total de postulaciones dentro de un tiempo máximo a definir mediante pruebas de carga.<br>▪ El sistema atiende solicitudes concurrentes de múltiples usuarios sin degradar significativamente su rendimiento.<br>▪ El sistema maneja correctamente peticiones HTTP duplicadas durante la revisión masiva de carpetas, evitando registros o acciones repetidas (idempotencia). |

*Nota.* Elaboración propia.

**Tabla 35**

*Requerimiento no funcional RNF-04: Usabilidad*

| **Descripción** |
| --- |
| El sistema debe ofrecer interfaces simples e intuitivas, acordes al nivel de conocimiento técnico básico de sus usuarios (Postulante, Administrador y Directivo), sin requerir conocimientos de programación para su uso. |
| **Criterios de aceptación** |
| ▪ Un usuario sin experiencia previa en el sistema puede completar el flujo de postulación sin asistencia externa.<br>▪ El panel de reportes del Directivo se interpreta sin necesidad de conocimientos técnicos. |

*Nota.* Elaboración propia.

**Tabla 36**

*Requerimiento no funcional RNF-05: Compatibilidad y restricciones tecnológicas*

| **Descripción** |
| --- |
| El sistema debe cumplir con las restricciones tecnológicas ya definidas para el proyecto: despliegue en un servidor en la nube de AWS, motor de base de datos PostgreSQL, autenticación propia mediante tokens JWT, notificaciones push a través de Firebase Cloud Messaging, almacenamiento de archivos en MinIO, y aplicación móvil desarrollada en Flutter con prioridad de compatibilidad para dispositivos Android. |
| **Criterios de aceptación** |
| ▪ El backend del sistema se despliega correctamente sobre AWS utilizando PostgreSQL.<br>▪ La autenticación la resuelve el propio backend mediante tokens JWT, las notificaciones push operan a través de Firebase Cloud Messaging y el almacenamiento de archivos en MinIO.<br>▪ La aplicación móvil funciona correctamente en dispositivos Android. |

*Nota.* Elaboración propia.

### *4.3.3. Requisitos de Interfaz*

Los requisitos de interfaz definen cómo el sistema interactúa con sus usuarios, con el hardware disponible y con otros componentes de software, y se organizan en cuatro categorías.

- **Interfaz de usuario:** el portal web debe desarrollarse con React y TypeScript, y la aplicación móvil con Flutter, ambos bajo un diseño responsivo que se adapte a distintos tamaños de pantalla. La interfaz gráfica de la aplicación móvil debe seguir las directrices

de *Material Design*, propias del framework Flutter, garantizando coherencia visual y accesibilidad básica (contraste adecuado, tamaños de fuente legibles) para usuarios sin experiencia técnica previa.

- **Interfaz de hardware:** la aplicación móvil debe ser compatible con dispositivos Android de gama media, como los equipos utilizados para las pruebas del proyecto (Redmi Note 14 Pro y Honor X8), y debe permitir el uso de la cámara del dispositivo para la digitalización de documentos de respaldo.

- **Interfaz de software:** el sistema debe exponer e integrar servicios mediante una API REST desarrollada en Spring Boot, que comunique al frontend web y a la aplicación móvil con la base de datos PostgreSQL y con el módulo de preselección desarrollado en Python. Asimismo, debe integrarse con MinIO para el almacenamiento de archivos y con Firebase Cloud Messaging para el envío de notificaciones push.

- **Interfaz de comunicación:** toda transmisión de datos entre los clientes (web y móvil) y el servidor debe realizarse mediante el protocolo HTTPS, y la autenticación de las peticiones debe validarse mediante el token de acceso (JWT) que emite el propio servidor y que este verifica en cada petición, a fin de proteger la información socioeconómica sensible de los postulantes durante su transmisión.

### *4.3.4. Requisitos de Base de Datos*

Los requisitos de base de datos definen la estructura y las reglas de gestión de los datos del sistema, sin entrar en el diseño detallado de las tablas, el cual corresponde a una etapa posterior del proyecto.

- El sistema debe almacenar los registros de usuarios, postulaciones, expedientes, calificaciones de vulnerabilidad y asistencia en una base de datos relacional PostgreSQL.

- El campo correspondiente al carnet de identidad (CI) y al correo electrónico debe ser único dentro de la tabla de usuarios, impidiendo el registro de cuentas duplicadas.

- Las tablas de postulaciones, expedientes y asistencia deben mantener integridad referencial mediante claves foráneas hacia la tabla de usuarios, evitando registros huérfanos.

- La base de datos debe tener capacidad para almacenar, como mínimo, los registros correspondientes a las 20,000 postulaciones recibidas por gestión académica, conservando el histórico de gestiones anteriores para su explotación en los reportes de Business Intelligence.

### *4.3.5. Restricciones de Diseño*

El diseño e implementación del sistema está sujeto a las siguientes restricciones técnicas y normativas:

- El backend debe desarrollarse en el lenguaje Java utilizando el framework Spring Boot, bajo una arquitectura de servicios RESTful.

- El frontend web debe desarrollarse con React y TypeScript, y la aplicación móvil con Flutter y el lenguaje Dart.

- El módulo de preselección con Inteligencia Artificial debe desarrollarse en Python, como un componente independiente del backend principal, consumido a través de una API.

- El control de versiones del código fuente debe llevarse a cabo en GitHub, y los entornos de desarrollo deben estandarizarse mediante contenedores Docker.

- El diseño de la Especificación de Requisitos del Software debe ajustarse al estándar IEEE 830, y el manejo de los datos socioeconómicos de los postulantes debe cumplir con las políticas institucionales de protección de datos personales de la UAGRM.

### *4.3.6. Requisitos de Rendimiento*

#### 4.3.6.1. Tiempo de respuesta.

- La interfaz principal del portal web y de la aplicación móvil no debe tardar más de 3 segundos en cargar bajo condiciones normales de operación.
- El sistema debe mostrar la lista priorizada de postulantes al Administrador DUBSS en un tiempo no mayor a 3 segundos.
- El cálculo del puntaje de vulnerabilidad de una postulación individual no debe exceder los 5 segundos desde su envío.

#### 4.3.6.2. Capacidad y carga máxima.

- El sistema debe procesar la totalidad de las postulaciones recibidas por gestión (hasta 20,000) sin degradar su rendimiento.
- El sistema debe soportar al menos 50 usuarios administrativos concurrentes durante la fase de auditoría de expedientes, sin afectar el tiempo de respuesta.

#### 4.3.6.3. Uso de recursos (CPU, memoria y ancho de banda).

- El servidor desplegado en AWS no debe superar el 80 % de uso de CPU ni de memoria durante los picos de carga esperados (apertura y cierre de la convocatoria).
- El consumo de ancho de banda de un usuario concurrente no debe degradar de forma perceptible la experiencia de otros usuarios conectados simultáneamente.

#### 4.3.6.4. Disponibilidad y tiempo de actividad (*uptime*).

- El sistema debe garantizar una disponibilidad mínima del 99 % durante los periodos de postulación y evaluación de cada gestión académica.
- Los mantenimientos o actualizaciones programadas deben realizarse fuera del horario de atención administrativa de la DUBSS.

#### 4.3.6.5. Escalabilidad.

- La arquitectura del sistema debe permitir el incremento del volumen de postulaciones en gestiones futuras sin requerir un rediseño estructural.
- El backend debe poder escalar mediante la incorporación de instancias adicionales en AWS conforme crezca la demanda del servicio.

#### 4.3.6.6. Procesamiento en lote o en tiempo real.

- La autenticación, el envío de postulaciones y la consulta del estado de la beca deben procesarse en tiempo real.
- La generación de los reportes y dashboards de Business Intelligence puede procesarse en lotes programados (por ejemplo, de forma diaria o al cierre de la gestión), sin requerir actualización instantánea.

Capítulo 5

# Captura de Requisitos

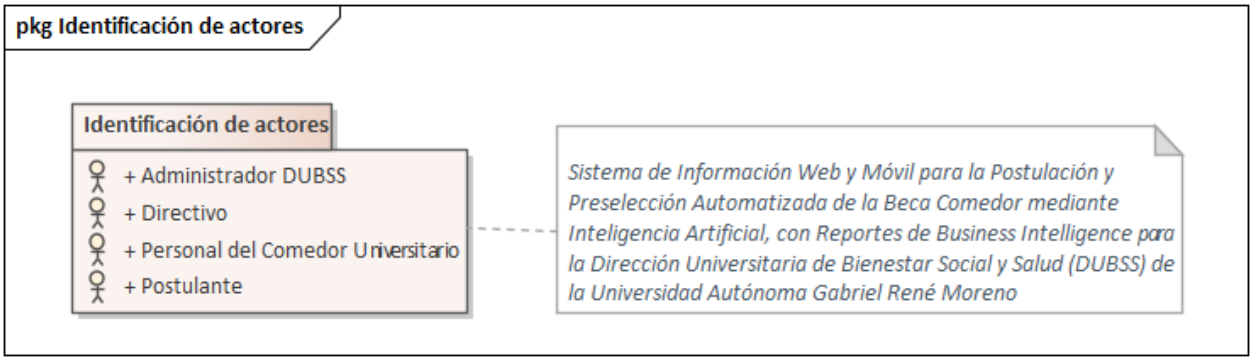
# Capítulo 5: Captura de Requisitos

## 5.1. Identificación de Actores

La Figura 24 presenta los actores que interactúan con el Sistema de Información Web y Móvil para la Postulación y Preselección Automatizada de la Beca Comedor, según el rol que desempeñan dentro del proceso de postulación, adjudicación y control de la Beca Comedor.

**Figura 24**

*Identificación de actores*




*Nota.* Elaboración propia.

A continuación, se describe el rol de cada actor identificado en la Figura 24:

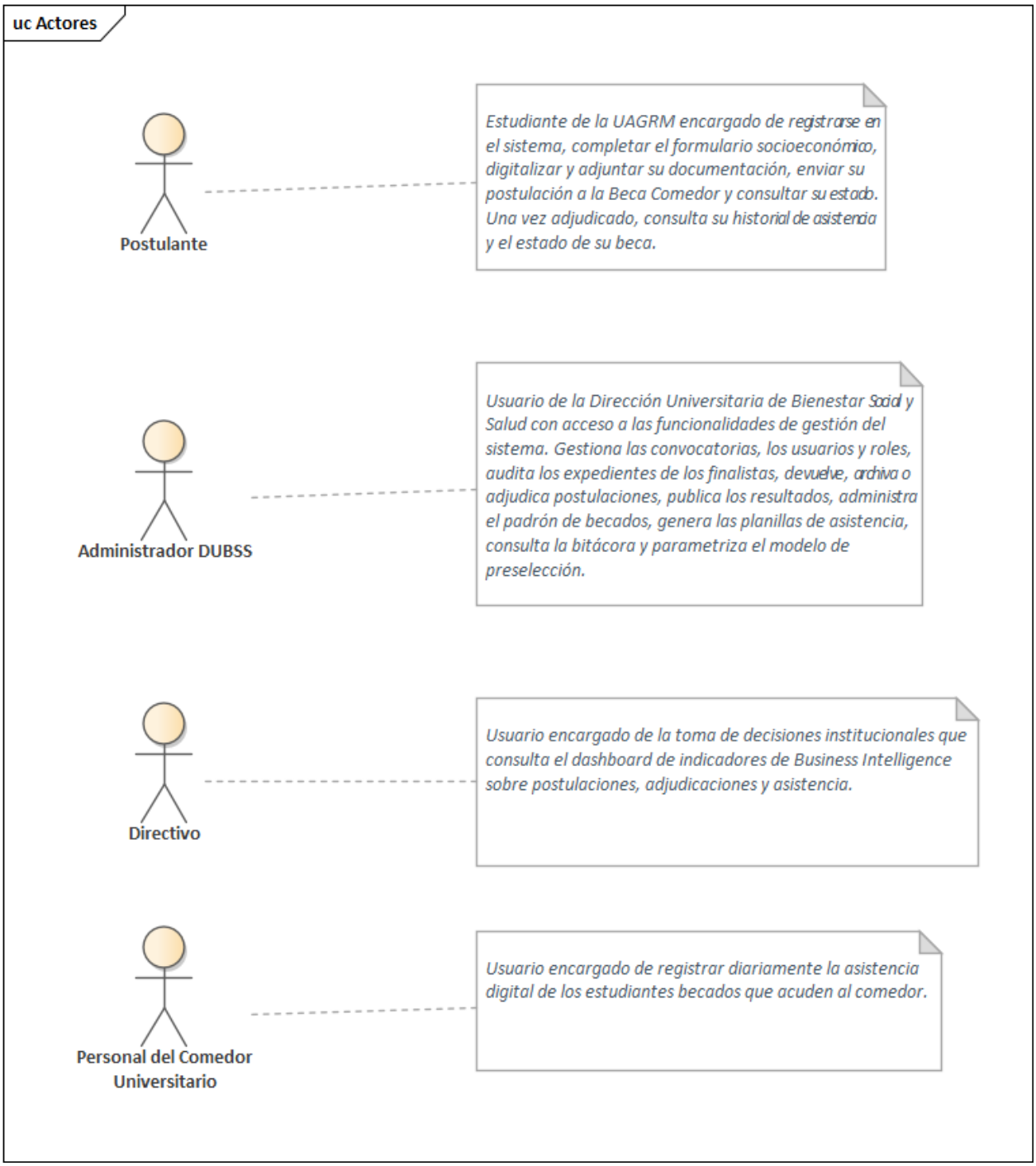
- **Postulante:** estudiante de la UAGRM encargado de registrarse en el sistema, completar el formulario socioeconómico, digitalizar y adjuntar su documentación, enviar su postulación a la Beca Comedor y consultar su estado. Una vez adjudicado, consulta su historial de asistencia y el estado de su beca.
- **Administrador DUBSS:** usuario de la Dirección Universitaria de Bienestar Social y Salud con acceso a las funcionalidades de gestión del sistema. Gestiona las convocatorias, los usuarios y roles, audita los expedientes de los finalistas, devuelve, archiva o adjudica postulaciones, publica los resultados, administra el padrón de becados, genera las planillas de asistencia, consulta la bitácora y parametriza el modelo de preselección.

- **Directivo:** usuario encargado de la toma de decisiones institucionales que consulta el dashboard de indicadores de Business Intelligence sobre postulaciones, adjudicaciones y asistencia.
- **Personal del Comedor Universitario:** usuario encargado de registrar diariamente la asistencia digital de los estudiantes becados que acuden al comedor.

La Figura 25 resume el rol de cada actor mediante un diagrama de actores con su descripción.

**Figura 25**

*Descripción de actores*




*Nota.* Elaboración propia.

## 5.2. Identificación de Casos de Uso

A partir de los actores identificados en la Sección 5.1 y de los requisitos funcionales especificados en el Capítulo 4, se identificaron veintiséis casos de uso que cubren el ciclo completo de la Beca Comedor. La Tabla 37 presenta cada caso de uso con su actor principal y el requisito funcional que lo respalda.

**Tabla 37**

*Identificación de casos de uso del sistema*

| ID | Caso de uso | Actor principal | RF |
|---|---|---|---|
| CU-01 | Registrarse y autenticarse | Usuario del Sistema | RF-01 |
| CU-02 | Enviar postulación a Beca Comedor | Postulante | RF-02 |
| CU-03 | Precalificar postulación con Inteligencia Artificial | Sistema (Módulo IA) | RF-03 |
| CU-04 | Consultar estado de postulación | Postulante | RF-06 |
| CU-05 | Auditar y adjudicar becas | Administrador DUBSS | RF-04 |
| CU-06 | Visualizar lista priorizada de postulantes | Administrador DUBSS | RF-04 |
| CU-07 | Auditar expediente finalista | Administrador DUBSS | RF-04 |
| CU-08 | Generar acta de adjudicación | Administrador DUBSS | RF-04 |
| CU-09 | Generar planilla de asistencia | Administrador DUBSS | RF-05 |

*Continúa en la siguiente página*

| ID | Caso de uso | Actor principal | RF |
|---|---|---|---|
| CU-10 | Registrar asistencia digital | Personal del Comedor | RF-05 |
| CU-11 | Cotejar registro contra respaldo físico | Administrador DUBSS | RF-05 |
| CU-12 | Visualizar dashboard de indicadores BI | Directivo | RF-07 |
| CU-13 | Gestionar convocatoria y gestión académica | Administrador DUBSS | RF-08 |
| CU-14 | Gestionar usuarios y roles | Administrador DUBSS | RF-09 |
| CU-15 | Recuperar contraseña / gestionar perfil | Usuario del Sistema | RF-10 |
| CU-16 | Digitalizar y adjuntar documentos | Postulante | RF-11 |
| CU-17 | Guardar borrador de postulación | Postulante | RF-12 |
| CU-18 | Validar elegibilidad y duplicados | Sistema (Módulo IA) | RF-13 |
| CU-19 | Devolver postulación para corrección | Administrador DUBSS | RF-14 |
| CU-20 | Archivar o rechazar postulación | Administrador DUBSS | RF-15 |
| CU-21 | Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos | Administrador DUBSS | RF-16 |

*Continúa en la siguiente página*

| ID | Caso de uso | Actor principal | RF |
|---|---|---|---|
| CU-22 | Publicar resultados de adjudicación | Administrador DUBSS | RF-17 |
| CU-23 | Gestionar padrón de becados (baja, suspensión, reemplazo) | Administrador DUBSS | RF-18 |
| CU-24 | Consultar historial de asistencia y estado de beca | Postulante (becado) | RF-19 |
| CU-25 | Consultar bitácora de acciones | Administrador DUBSS | RF-20 |
| CU-26 | Parametrizar modelo de preselección y reprocesar puntajes | Administrador DUBSS | RF-21 |

*Nota.* Elaboración propia.

### 5.3. Priorización de Casos de Uso

La priorización de los casos de uso se realizó según su aporte al objetivo del proyecto y sus dependencias, y se clasificó en tres niveles:

- **Alta:** caso de uso indispensable; sin él el sistema no cumple su función.
- **Media:** caso de uso necesario para que el sistema opere de forma completa.
- **Baja:** caso de uso complementario, cuya ausencia no bloquea el flujo principal.

Además, dentro de cada nivel se definió un orden de implementación sugerido que respeta las dependencias entre casos de uso, indicadas en la columna «Depende de». Del total de veintiséis casos de uso, dieciséis tienen prioridad alta, siete media y tres baja. Las Tablas 38, 39 y 40 presentan los casos de uso de cada nivel.

**Tabla 38**

*Casos de uso de prioridad alta*

| Orden | ID | Caso de uso | Depende de | Justificación |
|---|---|---|---|---|
| 1 | CU-01 | Registrarse y autenticarse | — | Base de seguridad y roles de todo el sistema. |
| 2 | CU-14 | Gestionar usuarios y roles | CU-01 | Habilita las cuentas del personal de la DUBSS. |
| 3 | CU-13 | Gestionar convocatoria y gestión académica | CU-14 | Sin convocatoria abierta no hay postulaciones. |
| 4 | CU-16 | Digitalizar y adjuntar documentos | CU-01 | Parte esencial de la postulación. |
| 5 | CU-18 | Validar elegibilidad y duplicados | CU-13 | Garantiza la integridad de las postulaciones. |
| 6 | CU-02 | Enviar postulación a Beca Comedor | CU-13, CU-16, CU-18 | Entrada principal de datos del sistema. |
| 7 | CU-03 | Precalificar postulación con Inteligencia Artificial | CU-02 | Núcleo diferenciador del proyecto. |
| 8 | CU-04 | Consultar estado de postulación | CU-02 | Transparencia para el postulante. |

*Continúa en la siguiente página*

| Orden | ID | Caso de uso | Depende de | Justificación |
|---|---|---|---|---|
| 9 | CU-06 | Visualizar lista priorizada de postulantes | CU-03 | Resuelve la revisión al azar de expedientes. |
| 10 | CU-05 | Auditar y adjudicar becas | CU-06 | Decisión final del proceso. |
| 11 | CU-07 | Auditar expediente finalista | CU-06 | Validación física de los finalistas. |
| 12 | CU-19 | Devolver postulación para corrección | CU-07 | Evita descartar expedientes subsanables. |
| 13 | CU-20 | Archivar o rechazar postulación | CU-07 | Cierra formalmente los expedientes. |
| 14 | CU-21 | Notificar finalistas y registrar entrega de documentación física | CU-07 | Coordina la auditoría física. |
| 15 | CU-22 | Publicar resultados de adjudicación | CU-05 | Cierra el proceso de selección. |
| 16 | CU-12 | Visualizar dashboard de indicadores BI | CU-02, CU-05, CU-10 | Objetivo del proyecto; requiere datos acumulados. |

*Nota.* Elaboración propia.

**Tabla 39**

*Casos de uso de prioridad media*

| Orden | ID | Caso de uso | Depende de | Justificación |
|---|---|---|---|---|
| 1 | CU-23 | Gestionar padrón de becados | CU-05 | Base del control de asistencia. |
| 2 | CU-09 | Generar planilla de asistencia | CU-23 | Exigida por las auditorías externas. |
| 3 | CU-10 | Registrar asistencia digital | CU-23 | Registra la asistencia diaria de los becados. |
| 4 | CU-24 | Consultar historial de asistencia y estado de beca | CU-10 | Vista de seguimiento para el becado. |
| 5 | CU-08 | Generar acta de adjudicación | CU-05 | Formaliza el resultado de la adjudicación. |
| 6 | CU-11 | Cotejar registro contra respaldo físico | CU-10 | Contrasta el registro digital con las planillas físicas. |
| 7 | CU-25 | Consultar bitácora de acciones | CU-01 | Trazabilidad transversal del sistema. |

*Nota.* Elaboración propia.

**Tabla 40**

*Casos de uso de prioridad baja*

| Orden | ID | Caso de uso | Depende de | Justificación |
|---|---|---|---|---|
| 1 | CU-15 | Recuperar contraseña / gestionar perfil | CU-01 | Comodidad; no bloquea el flujo principal. |
| 2 | CU-17 | Guardar borrador de postulación | CU-02 | Facilita completar la postulación en varias sesiones. |
| 3 | CU-26 | Parametrizar modelo de preselección y reprocesar puntajes | CU-03 | Ajuste fino posterior al modelo. |

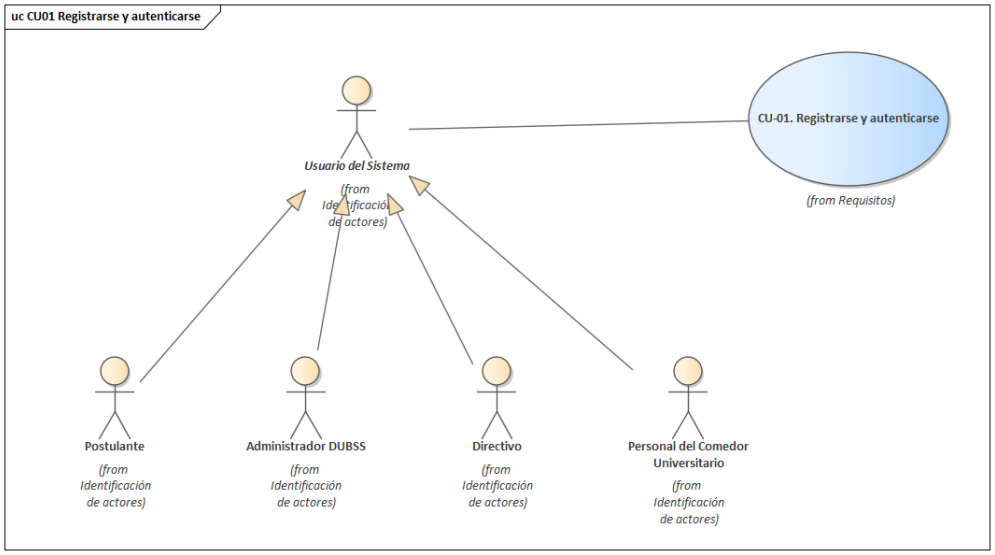
*Nota.* Elaboración propia.

## 5.4. Detalle de Casos de Uso

### *5.4.1. CU-01 Registrarse y autenticarse*

**Figura 26**

*Diagrama de contexto del CU-01*




*Nota.* Elaboración propia.

**Tabla 41**

*Detalle del CU-01: Registrarse y autenticarse*

| **Caso de Uso** | **CU-01. Registrarse y autenticarse** |
|---|---|
| **Propósito** | Permitir que el Postulante cree su cuenta y que cualquier usuario del sistema inicie y cierre sesión para acceder, de forma segura, solo a las funcionalidades correspondientes a su rol, desde el portal web o la aplicación móvil. |

| **Caso de Uso** | **CU-01. Registrarse y autenticarse** |
|---|---|
| **Descripción** | Este caso de uso describe tres flujos: el registro de un Postulante, el inicio de sesión de cualquier usuario del sistema (Postulante, Administrador DUBSS, Directivo o Personal del Comedor Universitario) con su correo electrónico y contraseña, y el cierre de sesión. La autenticación la resuelve el propio sistema (RF-01): el backend guarda la contraseña únicamente como hash, verifica las credenciales, emite un token de acceso de vida corta y un token de renovación por dispositivo, y comprueba el token de acceso en cada petición. Una vez autenticado, el sistema identifica el rol del usuario y lo redirige al módulo que le corresponde. Las cuentas del personal (Administrador DUBSS, Directivo y Personal del Comedor) no se autorregistran: son creadas por el Administrador DUBSS en el CU-14. El cierre de sesión se documenta dentro de este caso de uso y no constituye un caso de uso independiente. El CU-15 extiende este caso de uso para la recuperación de contraseña. Respalda el RF-01. |
| **Actores** | Usuario del Sistema (actor abstracto generalizado por Postulante, Administrador DUBSS, Directivo y Personal del Comedor Universitario). |
| **Actor Iniciador** | Usuario del Sistema. |
| **Precondición** | ▪ El usuario dispone de acceso al portal web o a la aplicación móvil y de conexión a Internet.<br>▪ Para el registro: no existe una cuenta asociada al CI, al registro universitario ni al correo electrónico del postulante.<br>▪ Para el inicio de sesión: el usuario cuenta con una cuenta registrada y activa (creada por autorregistro si es Postulante, o por el Administrador DUBSS en el CU-14 si es personal).<br>▪ Para el cierre de sesión: el usuario tiene una sesión abierta. |

| **Caso de Uso** | **CU-01. Registrarse y autenticarse** |
|---|---|
| **Proceso** | **Flujo principal A: registro de un Postulante.**<br>1. El usuario selecciona la opción «Crear cuenta» en la pantalla de acceso.<br>2. El sistema muestra el formulario de registro (nombres, apellidos, CI, registro universitario, carrera, teléfono, correo electrónico, contraseña y confirmación de contraseña).<br>3. El usuario completa los datos y confirma el registro.<br>4. El sistema valida que los campos obligatorios estén completos, que su formato sea correcto (el registro universitario tiene 9 dígitos numéricos y no empieza por 0), que la contraseña cumpla la política (al menos 6 caracteres, sin obligación de mayúsculas, números ni símbolos) y coincida con su confirmación, y que el CI, el registro universitario y el correo electrónico no existan en el sistema.<br>5. El sistema registra en la base de datos al usuario con el rol Postulante y activo, guardando la contraseña únicamente como hash, junto con su perfil de postulante, y anota el registro en la bitácora.<br>6. El sistema informa que el registro fue exitoso, inicia la sesión del usuario y lo redirige al módulo del Postulante.<br>**Flujo principal B: inicio de sesión.**<br>1. El usuario accede a la pantalla de inicio de sesión del portal web o de la aplicación móvil.<br>2. El usuario ingresa su correo electrónico y su contraseña.<br>3. El sistema verifica las credenciales comparando la contraseña ingresada con el hash almacenado.<br>4. El sistema comprueba que la cuenta esté activa y no bloqueada, y obtiene el rol del usuario.<br>5. El sistema abre la sesión (emite el token de acceso y el token de renovación del dispositivo) y registra el acceso en la bitácora.<br>6. El sistema redirige al usuario según su rol: el Postulante a su módulo de postulación, el Administrador DUBSS al panel de gestión, el Directivo al dashboard de indicadores y el Personal del Comedor al registro de asistencia.<br>**Flujo principal C: cierre de sesión.**<br>1. El usuario selecciona la opción «Cerrar sesión» en el menú de su cuenta.<br>2. El sistema revoca la sesión del usuario en ese dispositivo y deja de reconocerla.<br>3. El sistema redirige al usuario a la pantalla de acceso. |

| Caso de Uso | CU-01. Registrarse y autenticarse |
|---|---|

**Flujos alternativos.**

- A1 (flujo A, paso 4): si el CI, el registro universitario o el correo electrónico ya están registrados, el sistema rechaza el registro, indica el campo en conflicto y no crea la cuenta.
- A2 (flujo A, paso 4): si faltan campos obligatorios, el formato es inválido, la contraseña no cumple la política o no coincide con su confirmación, el sistema marca los campos con error y permite corregirlos, sin crear la cuenta.
- A3 (flujo B, paso 3): si el correo no existe o la contraseña es incorrecta, el sistema muestra el mismo mensaje de error en ambos casos (sin revelar cuál dato falló) y permite reintentar.
- A4 (flujo B, paso 4): si la cuenta está desactivada, el sistema niega el acceso, indica que debe comunicarse con la DUBSS y no abre sesión.
- A5 (flujo B, paso 2): si el usuario selecciona «¿Olvidó su contraseña?», se ejecuta el CU-15.
- A6 (flujo B, paso 3): tras varios intentos fallidos consecutivos, el sistema bloquea temporalmente el acceso de esa cuenta y lo niega aunque la contraseña sea correcta. El sistema bloquea a los 5 intentos, por 5 minutos la primera vez y por un lapso mayor si el usuario continúa fallando (hoy: 5 minutos, 15 minutos, 1 hora y 24 horas); la escala exacta está por definir con la DUBSS. Un inicio de sesión correcto reinicia el contador.
- A7 (flujo A, paso 4): si la petición de registro llega repetida (doble envío o reintento), el sistema la procesa de forma idempotente y no crea dos cuentas.
- A8 (flujo B, pasos 5 y 6): si el token de acceso venció, el sistema lo renueva automáticamente con el token de renovación del dispositivo, que se reemplaza por uno nuevo en cada uso; si la sesión ya no es válida (expiró o fue revocada), el sistema redirige al usuario a la pantalla de inicio de sesión.
- A9 (cualquier paso): si falla la conexión con el servidor o con un servicio del que depende, el sistema informa el error con un mensaje claro, no crea ni modifica datos y permite reintentar.
- A10 (flujo C, paso 2): si la sesión ya había expirado o se había cerrado antes de solicitar el cierre, el sistema igualmente redirige al usuario a la pantalla de acceso, sin mostrar error.

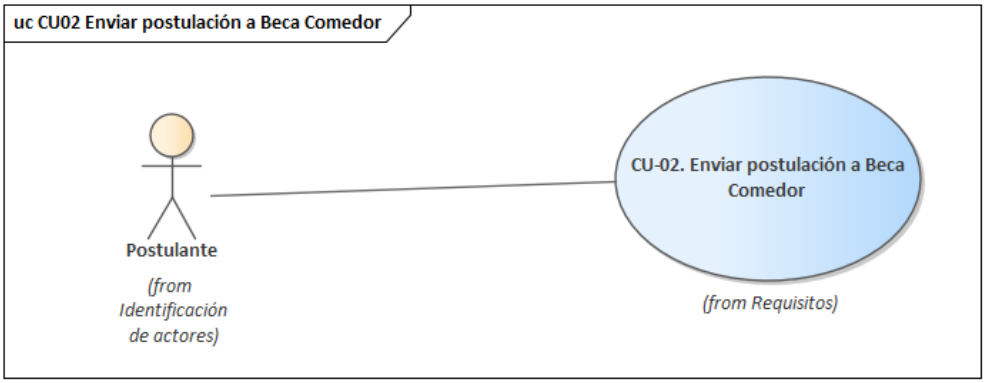
| Caso de Uso | CU-01. Registrarse y autenticarse |
|---|---|
| **Postcondición** | ▪ Registro: la cuenta del Postulante queda creada, activa y con el rol Postulante, y el usuario tiene una sesión abierta.<br>▪ Inicio de sesión: el usuario tiene una sesión activa con los permisos de su rol, se encuentra en el módulo que le corresponde y su acceso queda registrado en la bitácora (consultable en el CU-25).<br>▪ Cierre de sesión: el dispositivo ya no tiene sesión abierta y cualquier pantalla privada lleva a la pantalla de acceso; las sesiones en otros dispositivos no se ven afectadas. |
| **Excepciones** | ▪ Credenciales inválidas o cuenta inexistente.<br>▪ Cuenta desactivada o bloqueada temporalmente.<br>▪ Sesión inválida o expirada.<br>▪ Error de conexión con el servidor o con la base de datos.<br>▪ Petición de registro duplicada (se procesa de forma idempotente, sin crear dos cuentas). |

*Nota.* Elaboración propia.

### *5.4.2. CU-02 Enviar postulación a Beca Comedor*

**Figura 27**

*Diagrama de contexto del CU-02*




*Nota.* Elaboración propia.

**Tabla 42**

*Detalle del CU-02: Enviar postulación a Beca Comedor*

| **Caso de Uso** | **CU-02. Enviar postulación a Beca Comedor** |
|---|---|
| **Propósito** | Permitir al Postulante completar el formulario socioeconómico, adjuntar su documentación digitalizada y enviar formalmente su postulación a la Beca Comedor de la convocatoria vigente. |
| **Descripción** | Este caso de uso describe cómo el Postulante registra su postulación desde el portal web o la aplicación móvil. Incluye la digitalización y carga de documentos (CU-16), la validación de elegibilidad y duplicados (CU-18) y la precalificación automática con Inteligencia Artificial (CU-03). El CU-17 lo extiende para guardar la postulación como borrador. Respalda el RF-02. |
| **Actores** | Postulante. |
| **Actor Iniciador** | Postulante. |
| **Precondición** | ▪ El Postulante está autenticado en el sistema (CU-01).<br>▪ Existe una convocatoria abierta (CU-13) para la gestión académica vigente.<br>▪ El Postulante no tiene otra postulación vigente en la misma gestión, salvo que la suya haya sido devuelta para corrección (CU-19).<br>▪ El Postulante dispone de los documentos obligatorios definidos en la convocatoria. |
| **Proceso** | **Flujo principal.**<br>1. El Postulante selecciona la opción «Nueva postulación».<br>2. El sistema verifica que exista una convocatoria abierta y muestra el formulario socioeconómico con los datos del perfil precargados (nombre, CI y carrera).<br>3. El Postulante completa las variables socioeconómicas: ingresos familiares, número de dependientes, procedencia, patrimonio, estado de vulnerabilidad y promedio académico.<br>4. El Postulante digitaliza o adjunta los documentos de respaldo requeridos (se incluye el CU-16).<br>5. El Postulante selecciona «Enviar postulación».<br>6. El sistema verifica que no falte ningún campo ni documento obligatorio y que los datos tengan un formato válido.<br>7. El sistema valida la elegibilidad del Postulante y la inexistencia de duplicados (se incluye el CU-18).<br>8. El sistema registra la postulación con estado «pendiente», asociándola a la convocatoria y gestión académica vigentes, con su fecha y hora de envío.<br>9. El sistema solicita la precalificación de la postulación (se incluye el CU-03).<br>10. El sistema confirma la recepción de la postulación y muestra al Postulante un comprobante con su código de postulación. |

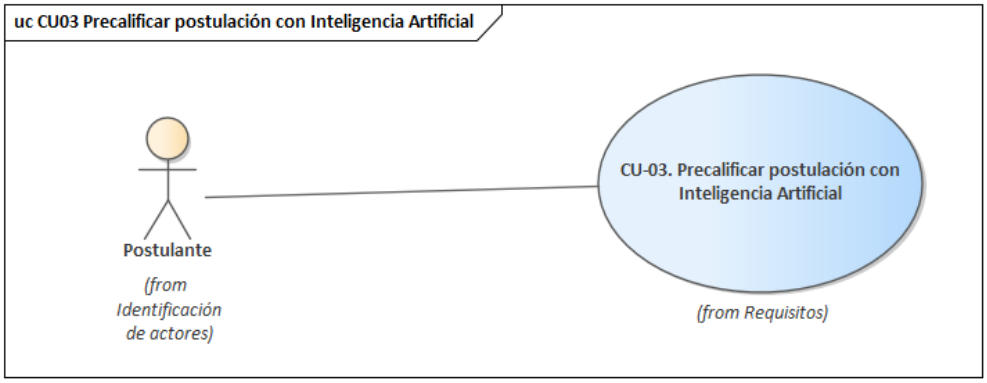
| **Caso de Uso** | **CU-02. Enviar postulación a Beca Comedor** |
|---|---|
| | **Flujos alternativos.**<br>▪ A1 (paso 1): si no existe una convocatoria abierta, el sistema impide iniciar la postulación e informa las fechas de la próxima convocatoria, si están definidas.<br>▪ A2 (pasos 3 y 4): si el Postulante desea continuar más tarde, puede guardar su avance como borrador (CU-17).<br>▪ A3 (paso 6): si faltan campos o documentos obligatorios, el sistema resalta lo faltante y no permite enviar la postulación.<br>▪ A4 (paso 7): si el Postulante no es elegible o ya tiene una postulación vigente en la gestión, el sistema rechaza el envío e informa el motivo.<br>▪ A5 (paso 1): si el Postulante tiene una postulación en estado «devuelta», el sistema le permite abrirla, corregirla y reenviarla; esta no se considera duplicada y se conserva el historial de devoluciones y reenvíos.<br>▪ A6 (paso 5): si el Postulante presiona repetidamente el botón de envío o la petición llega duplicada, el sistema la procesa una sola vez (idempotencia).<br>▪ A7 (paso 9): si el módulo de IA no responde, la postulación queda registrada con estado «pendiente» y la precalificación se reintenta automáticamente; el Postulante no pierde su envío.<br>▪ A8 (pasos 2 a 5): si la convocatoria se cierra mientras el Postulante completa el formulario, el sistema no acepta el envío e informa que el plazo finalizó. |
| **Postcondición** | ▪ La postulación queda almacenada con estado «pendiente» (o «calificada» una vez ejecutado el CU-03), asociada al Postulante, a la convocatoria y a la gestión académica.<br>▪ Los documentos adjuntos quedan almacenados en MinIO y vinculados a la postulación.<br>▪ El Postulante puede consultar el estado de su postulación (CU-04). |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Postulante).<br>▪ Datos ingresados no válidos o campos obligatorios incompletos.<br>▪ Convocatoria cerrada o inexistente.<br>▪ Postulación duplicada o Postulante no elegible.<br>▪ Error al almacenar los archivos en MinIO.<br>▪ Error de conexión con el servidor o la base de datos durante el envío. |

*Nota.* Elaboración propia.

### *5.4.3. CU-03 Precalificar postulación con Inteligencia Artificial*

**Figura 28**

*Diagrama de contexto del CU-03*




*Nota.* Elaboración propia.

**Tabla 43**

*Detalle del CU-03: Precalificar postulación con Inteligencia Artificial*

| **Caso de Uso** | **CU-03. Precalificar postulación con Inteligencia Artificial** |
|---|---|
| **Propósito** | Calcular automáticamente, mediante un modelo de Inteligencia Artificial, el puntaje de vulnerabilidad de cada postulación recibida, para ordenar objetivamente a los postulantes según su nivel de necesidad. |
| **Descripción** | Este caso de uso describe cómo el sistema precalifica cada postulación enviada, enviando sus variables socioeconómicas y académicas al módulo de IA (desarrollado en Python y consumido mediante API) y registrando el puntaje obtenido junto con su fecha de cálculo. El puntaje permite construir la lista priorizada que usa el Administrador DUBSS para auditar únicamente a los finalistas (CU-06). Es incluido por el CU-02 y extendido por el CU-26, que permite reprocesar los puntajes. Respalda el RF-03. |
| **Actores** | Postulante. |
| **Actor Iniciador** | Postulante (a través del CU-02, al enviar su postulación; el sistema ejecuta la precalificación mediante el módulo de IA). |

| **Caso de Uso** | **CU-03. Precalificar postulación con Inteligencia Artificial** |
|---|---|
| **Precondición** | ▪ La postulación fue enviada, superó la validación de elegibilidad y duplicados (CU-18) y se encuentra en estado «pendiente».<br>▪ El módulo de IA está disponible y tiene cargados los parámetros vigentes del modelo (criterios y ponderaciones por definir con la DUBSS).<br>▪ La postulación contiene las variables requeridas por el modelo: ingresos familiares, dependientes, procedencia, carrera, patrimonio, estado de vulnerabilidad y promedio académico. |
| **Proceso** | **Flujo principal.**<br>1. El sistema (backend) detecta una postulación pendiente de precalificar y prepara el conjunto de variables socioeconómicas y académicas declaradas.<br>2. El sistema envía la solicitud de cálculo al módulo de IA mediante la API.<br>3. El módulo de IA valida que los datos recibidos estén completos y sean consistentes.<br>4. El módulo de IA aplica el modelo con los parámetros vigentes y calcula el puntaje de vulnerabilidad.<br>5. El módulo de IA devuelve el puntaje al sistema.<br>6. El sistema registra el puntaje junto con la fecha y hora de cálculo y la versión de los parámetros utilizados, conservando los puntajes anteriores si existieran.<br>7. El sistema actualiza el estado de la postulación a «calificada» e incorpora la postulación a la lista priorizada.<br>**Flujos alternativos.**<br>▪ A1 (paso 3): si los datos están incompletos o son inconsistentes, el módulo de IA no calcula el puntaje; el sistema registra el error y deja la postulación marcada para revisión.<br>▪ A2 (paso 2): si el módulo de IA no está disponible o excede el tiempo de espera, el sistema registra el fallo, mantiene la postulación en estado «pendiente» y reintenta el cálculo automáticamente.<br>▪ A3 (paso 1): si la solicitud proviene del reprocesamiento (CU-26), el sistema calcula un nuevo puntaje con los parámetros actualizados y conserva el puntaje anterior para trazabilidad.<br>▪ A4 (paso 1): si se recibe una solicitud duplicada para la misma postulación y la misma versión de parámetros, el sistema no genera un segundo puntaje (idempotencia).<br>▪ A5 (paso 1): ante picos de envíos (por ejemplo, al cierre de la convocatoria), el sistema encola las solicitudes y las procesa en orden, de modo que la totalidad de las postulaciones (hasta 20,000 por gestión) sea procesada sin intervención manual. |

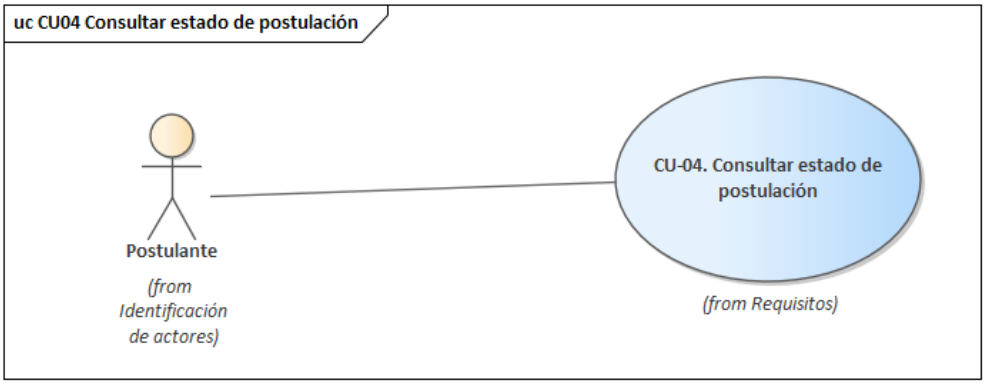
| Caso de Uso | CU-03. Precalificar postulación con Inteligencia Artificial |
|---|---|
| **Postcondición** | ▪ Cada postulación procesada tiene un puntaje de vulnerabilidad registrado con su fecha de cálculo, lo que permite su trazabilidad.<br>▪ La postulación queda en estado «calificada» y disponible en la lista priorizada (CU-06). |
| **Excepciones** | ▪ Módulo de IA no disponible o tiempo de espera agotado (el cálculo individual debe completarse en un máximo de 5 segundos en condiciones normales).<br>▪ Datos de la postulación incompletos o inconsistentes.<br>▪ Error en la comunicación entre el backend y el módulo de IA.<br>▪ Error de conexión con la base de datos al registrar el puntaje. |

*Nota.* Elaboración propia.

### *5.4.4. CU-04 Consultar estado de postulación*

**Figura 29**

*Diagrama de contexto del CU-04*




*Nota.* Elaboración propia.

**Tabla 44**

*Detalle del CU-04: Consultar estado de postulación*

| Caso de Uso | CU-04. Consultar estado de postulación |
|---|---|
| **Propósito** | Permitir al Postulante realizar el seguimiento del estado de su postulación sin necesidad de acudir presencialmente a la DUBSS. |

| **Caso de Uso** | **CU-04. Consultar estado de postulación** |
|---|---|
| **Descripción** | Este caso de uso describe cómo el Postulante consulta, desde el portal web o la aplicación móvil, el estado actual de sus postulaciones, el historial de cambios y, según corresponda, las observaciones de devolución, el motivo de archivo o rechazo y el resultado de la adjudicación. Cuando el estado cambia, el sistema notifica al Postulante mediante notificación push y la notificación lo dirige a esta consulta. Es una consulta de solo lectura. Respalda el RF-06. |
| **Actores** | Postulante. |
| **Actor Iniciador** | Postulante. |
| **Precondición** | ▪ El Postulante está autenticado en el sistema (CU-01).<br>▪ El Postulante ha enviado al menos una postulación. |
| **Proceso** | **Flujo principal.**<br>1. El Postulante accede a la opción «Mis postulaciones».<br>2. El sistema muestra la lista de sus postulaciones con la gestión académica, la fecha de envío y el estado actual.<br>3. El Postulante selecciona una postulación.<br>4. El sistema muestra el detalle: estado actual (pendiente, devuelta, archivada, calificada, adjudicada o no adjudicada), historial de cambios de estado con fecha y hora, y la información asociada al estado.<br>5. El Postulante revisa la información y, si lo desea, regresa a la lista.<br>**Flujos alternativos.**<br>▪ A1 (paso 2): si el Postulante no tiene postulaciones, el sistema muestra un mensaje informativo e invita a postular si existe una convocatoria abierta.<br>▪ A2 (paso 2): si el Postulante tiene postulaciones de varias gestiones, el sistema permite filtrarlas por gestión académica.<br>▪ A3 (paso 4): si la postulación está «devuelta», el sistema muestra las observaciones registradas por el Administrador (el motivo del historial de la devolución, CU-19) y ofrece la opción de corregir y reenviar (CU-02).<br>▪ A4 (paso 4): si la postulación está «archivada» o «no adjudicada», el sistema muestra el motivo registrado (CU-20).<br>▪ A5 (paso 4): si el Postulante fue seleccionado como finalista, el sistema muestra la notificación con el plazo y los documentos físicos que debe presentar (CU-21); si los resultados fueron publicados, muestra su resultado (CU-22).<br>▪ A6 (paso 1): si el Postulante accede desde una notificación push, el sistema abre directamente el detalle de la postulación afectada. |

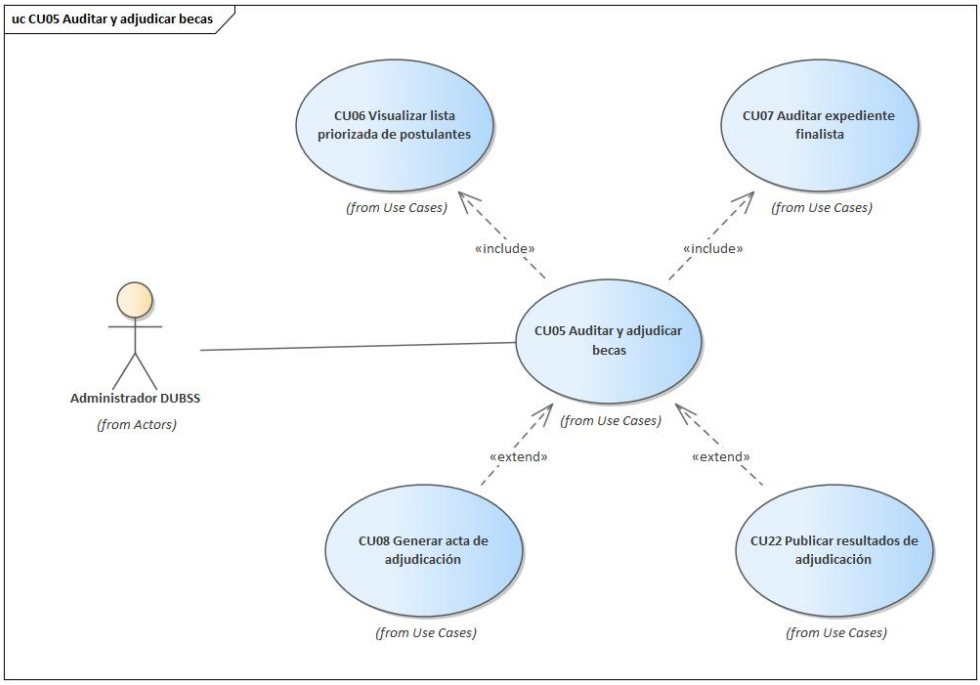
| Caso de Uso | CU-04. Consultar estado de postulación |
|---|---|
| **Postcondición** | ▪ El Postulante visualiza el estado actualizado de su postulación; la consulta no modifica ningún dato del sistema. |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el Postulante solo puede consultar sus propias postulaciones).<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.5. CU-05 Auditar y adjudicar becas*

**Figura 30**

*Diagrama de contexto del CU-05*




*Nota.* Elaboración propia.

**Tabla 45**

*Detalle del CU-05: Auditar y adjudicar becas*

| **Caso de Uso** | **CU-05. Auditar y adjudicar becas** |
|---|---|
| **Propósito** | Permitir al Administrador DUBSS conducir el proceso de selección de los beneficiarios: identificar a los finalistas mejor puntuados, auditar sus expedientes y registrar la decisión de adjudicar o no adjudicar la Beca Comedor a cada uno de ellos. |
| **Descripción** | Este caso de uso describe el proceso completo de auditoría y adjudicación de una convocatoria. A partir de la lista priorizada por el puntaje de vulnerabilidad (se incluye el CU-06), el Administrador DUBSS selecciona a los finalistas, audita el expediente de cada uno (se incluye el CU-07) y registra la decisión de adjudicación, que queda asociada a la postulación junto con el puntaje total (puntaje de la IA más el puntaje académico de la última auditoría), el usuario que decide y la fecha de adjudicación. El CU-08 lo extiende para generar el acta de adjudicación y el CU-22 para publicar los resultados. Respalda el RF-04. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01) con el rol Administrador DUBSS.<br>▪ La convocatoria de la gestión académica vigente se encuentra cerrada a la recepción de postulaciones (CU-13).<br>▪ Existen postulaciones en estado «calificada», con puntaje de vulnerabilidad vigente (CU-03).<br>▪ La convocatoria tiene registrados el número de cupos y el puntaje mínimo requerido (valores por definir con la DUBSS). |

| **Caso de Uso** | **CU-05. Auditar y adjudicar becas** |
|---|---|
| **Proceso** | **Flujo principal.**<br>1. El Administrador DUBSS selecciona la opción «Auditoría y adjudicación» y la convocatoria a procesar.<br>2. El sistema muestra la lista priorizada de postulantes de la convocatoria (se incluye el CU-06).<br>3. El Administrador DUBSS selecciona como finalistas a los postulantes mejor puntuados (cantidad de finalistas por definir con la DUBSS en función de los cupos).<br>4. El sistema registra a cada finalista con la posición que ocupa en la lista priorizada.<br>5. El Administrador DUBSS audita el expediente de cada finalista (se incluye el CU-07).<br>6. El sistema muestra la lista de finalistas con el resultado de su auditoría y el número de cupos disponibles.<br>7. El Administrador DUBSS selecciona un finalista auditado y registra su decisión: adjudicar o no adjudicar la beca.<br>8. El sistema registra la adjudicación con su resultado, el puntaje total de la postulación (puntaje de la IA más el puntaje académico de la última auditoría), el usuario que decide y la fecha de adjudicación.<br>9. El sistema actualiza el estado de la postulación a «adjudicada» o «no adjudicada», registra el cambio en el historial de la postulación y en la bitácora, y actualiza el número de cupos disponibles.<br>10. Los pasos 7 a 9 se repiten hasta decidir sobre todos los finalistas.<br>**Flujos alternativos.**<br>▪ A1 (paso 2): si la convocatoria no tiene postulaciones calificadas, el sistema informa que no existen postulaciones para auditar.<br>▪ A2 (paso 7): si el expediente del finalista no tiene una auditoría con resultado «validado», el sistema no permite adjudicarle la beca.<br>▪ A3 (paso 7): si el puntaje total de la postulación (IA más académico de la última auditoría) no alcanza el puntaje mínimo requerido, el sistema no permite adjudicarle la beca.<br>▪ A4 (paso 7): si los cupos de la convocatoria están agotados, el sistema no permite registrar nuevas adjudicaciones y solo admite la decisión de no adjudicar.<br>▪ A5 (paso 10): una vez decididas todas las adjudicaciones, el Administrador DUBSS puede generar el acta de adjudicación (CU-08) y publicar los resultados (CU-22).<br>▪ A6 (paso 7): si los resultados de la convocatoria ya fueron publicados (CU-22), el sistema no permite modificar las adjudicaciones.<br>▪ A7 (paso 8): si la petición de adjudicación llega duplicada, el sistema la procesa una sola vez (idempotencia). |

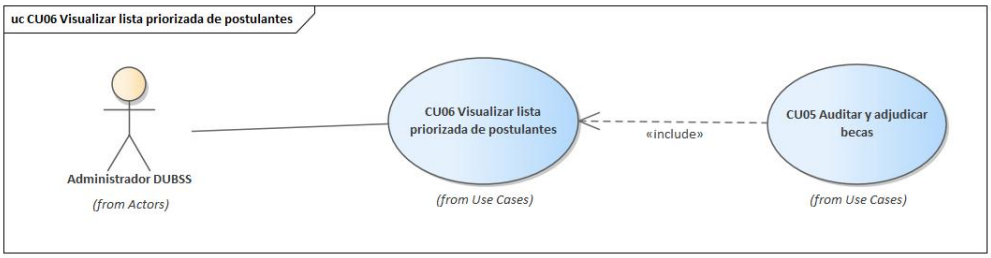
| Caso de Uso | CU-05. Auditar y adjudicar becas |
|---|---|
| **Postcondición** | ▪ Cada finalista decidido tiene una adjudicación registrada con su resultado, puntaje final, responsable y fecha de adjudicación.<br>▪ La postulación queda en estado «adjudicada» o «no adjudicada» y el cambio queda registrado en el historial de la postulación y en la bitácora. |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ Cupos de la convocatoria agotados.<br>▪ Error de conexión con el servidor o la base de datos al registrar la adjudicación. |

*Nota.* Elaboración propia.

### *5.4.6. CU-06 Visualizar lista priorizada de postulantes*

**Figura 31**

*Diagrama de contexto del CU-06*




*Nota.* Elaboración propia.

**Tabla 46**

*Detalle del CU-06: Visualizar lista priorizada de postulantes*

| Caso de Uso | CU-06. Visualizar lista priorizada de postulantes |
|---|---|
| **Propósito** | Presentar al Administrador DUBSS las postulaciones de una convocatoria ordenadas de mayor a menor puntaje de vulnerabilidad, para que la auditoría se concentre únicamente en los postulantes mejor puntuados. |

| **Caso de Uso** | **CU-06. Visualizar lista priorizada de postulantes** |
|---|---|
| **Descripción** | Este caso de uso describe cómo el sistema construye y muestra la lista priorizada de postulantes de una convocatoria a partir de las postulaciones en estado «calificada» y de su puntaje vigente calculado por el módulo de IA (CU-03). La lista no se almacena como tal: se obtiene mediante una consulta ordenada y paginada, por lo que siempre refleja el último puntaje calculado. Es incluido por el CU-05. Respalda el RF-04. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01) con el rol Administrador DUBSS.<br>▪ Existe al menos una convocatoria registrada (CU-13).<br>▪ Las postulaciones de la convocatoria fueron precalificadas por el módulo de IA (CU-03). |
| **Proceso** | **Flujo principal.**<br>1. El Administrador DUBSS selecciona la opción «Lista priorizada» (o accede a ella desde el CU-05).<br>2. El sistema muestra las convocatorias disponibles.<br>3. El Administrador DUBSS selecciona una convocatoria.<br>4. El sistema obtiene las postulaciones de la convocatoria en estado «calificada» y su puntaje de vulnerabilidad vigente.<br>5. El sistema ordena las postulaciones de mayor a menor puntaje y muestra la lista paginada con: posición, código de postulación, nombre, CI y carrera del postulante, puntaje y fecha de cálculo.<br>6. El Administrador DUBSS revisa la lista y, si lo desea, selecciona una postulación para continuar con su auditoría (CU-05). |

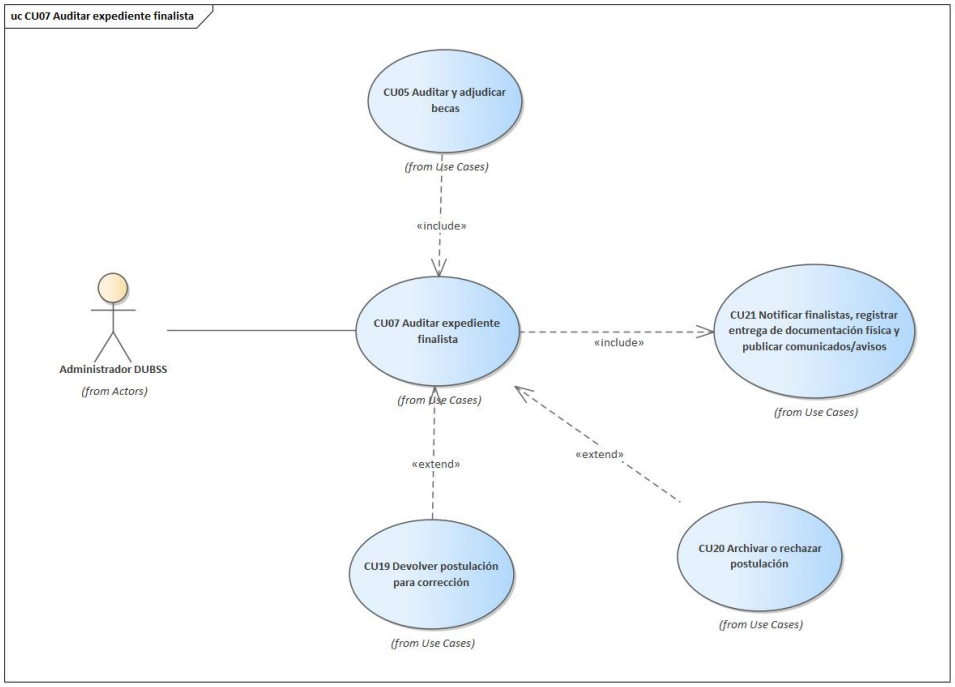
| **Caso de Uso** | **CU-06. Visualizar lista priorizada de postulantes** |
|---|---|
| | **Flujos alternativos.**<br>▪ A1 (paso 4): si la convocatoria no tiene postulaciones calificadas, el sistema muestra un mensaje informativo.<br>▪ A2 (paso 6): si el Administrador DUBSS aplica un filtro o criterio de ordenamiento (por ejemplo, por carrera), el sistema vuelve a consultar las postulaciones y los puntajes y muestra la lista resultante.<br>▪ A3 (paso 5): si dos o más postulaciones tienen el mismo puntaje, el sistema aplica el criterio de desempate vigente (por definir con la DUBSS).<br>▪ A4 (paso 4): las postulaciones en estado «pendiente», «devuelta», «archivada», «adjudicada» o «no adjudicada» no forman parte de la lista priorizada activa; el sistema informa cuántas postulaciones siguen pendientes de precalificación.<br>▪ A5 (paso 6): si el Administrador DUBSS solicita exportar la lista, el sistema genera un documento PDF con la lista priorizada de la convocatoria (respetando el filtro u orden aplicado), registra la exportación en la bitácora (CU-25) y ofrece la descarga del archivo. |
| **Postcondición** | ▪ El Administrador DUBSS visualiza la lista priorizada de la convocatoria; la consulta no modifica ningún dato de las postulaciones ni de los puntajes.<br>▪ Si exportó la lista a PDF (A5), queda un registro de la exportación en la bitácora. |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ Error de conexión con el servidor o la base de datos.<br>▪ Tiempo de respuesta excedido al consultar un volumen alto de postulaciones (hasta 20,000 por gestión). |

*Nota.* Elaboración propia.

### *5.4.7. CU-07 Auditar expediente finalista*

**Figura 32**

*Diagrama de contexto del CU-07*




*Nota.* Elaboración propia.

**Tabla 47**

*Detalle del CU-07: Auditar expediente finalista*

| **Caso de Uso** | **CU-07. Auditar expediente finalista** |
|---|---|
| **Propósito** | Permitir al Administrador DUBSS revisar el expediente de un finalista, cotejar su documentación física con la digital y registrar el resultado de la auditoría junto con su puntaje de rendimiento académico. |

| **Caso de Uso** | **CU-07. Auditar expediente finalista** |
|---|---|
| **Descripción** | Este caso de uso describe cómo el Administrador DUBSS audita el expediente de un postulante seleccionado como finalista. El sistema presenta los datos de la postulación, las respuestas socioeconómicas, los documentos digitales adjuntos y el puntaje de la inteligencia artificial (hasta 70 puntos); el Administrador coteja esa información con la documentación física entregada por el finalista y registra el resultado de la auditoría (validado, observado o no válido) con sus observaciones (hasta 500 caracteres) y, si es validado, el puntaje de rendimiento académico (0 a 30), con lo que el sistema muestra el total (hasta 100). El finalista se elige desde la lista de finalistas del CU-21 (columna «Auditoría») y solo se audita si su entrega física está registrada y su postulación sigue «calificada». Al registrar la auditoría se avisa al postulante. Incluye la notificación al finalista y el registro de la entrega de su documentación física (CU-21). El CU-19 lo extiende para devolver la postulación y el CU-20 para archivarla o rechazarla. Es incluido por el CU-05. Respalda el RF-04. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01) con el rol Administrador DUBSS.<br>▪ La postulación fue seleccionada como finalista en el CU-05 y se encuentra en estado «calificada».<br>▪ Los documentos digitales de la postulación están almacenados en MinIO (CU-16).<br>▪ La entrega de la documentación física del finalista está registrada (CU-21). |

| **Caso de Uso** | **CU-07. Auditar expediente finalista** |
|---|---|
| **Proceso** | **Flujo principal.**<br>1. El Administrador DUBSS selecciona un finalista de la lista de finalistas de la convocatoria.<br>2. El sistema muestra el expediente del finalista: datos del postulante, respuestas socioeconómicas, documentos digitales adjuntos, puntaje de vulnerabilidad de la IA con su desglose y estado de la entrega de la documentación física.<br>3. El sistema notifica al finalista el plazo y los documentos físicos que debe presentar y registra la recepción de dicha documentación (se incluye el CU-21).<br>4. El Administrador DUBSS coteja la documentación física entregada con los documentos digitales y los datos declarados.<br>5. El Administrador DUBSS registra el resultado de la auditoría (validado, observado o no válido), sus observaciones (hasta 500 caracteres) y, si el resultado es «validado», el puntaje de rendimiento académico (0 a 30).<br>6. El sistema valida los datos, registra la auditoría con su resultado, observaciones, puntaje académico, auditor y fecha, y deja constancia en la bitácora.<br>7. El sistema avisa al postulante el resultado de la auditoría y muestra el expediente como auditado con el puntaje de la IA, el puntaje académico y el total; si el resultado es «validado», el finalista queda habilitado para la decisión de adjudicación (CU-05). |

| Caso de Uso | CU-07. Auditar expediente finalista |
|---|---|
| | **Flujos alternativos.**<br>▪ A1 (paso 3): si el finalista no entrega la documentación física dentro del plazo (por definir con la DUBSS), el Administrador DUBSS puede archivar la postulación (CU-20).<br>▪ A2 (paso 5): si el expediente presenta defectos subsanables, el Administrador DUBSS puede devolver la postulación para su corrección (CU-19).<br>▪ A3 (paso 5): si la documentación no es válida o la postulación no procede, el Administrador DUBSS puede archivarla o rechazarla (CU-20).<br>▪ A4 (paso 2): si un documento digital no puede recuperarse del almacenamiento de archivos, el sistema informa el error y permite reintentar la descarga.<br>▪ A5 (paso 6): si el expediente ya tenía una auditoría (por ejemplo, después de una devolución), el sistema registra la nueva auditoría y conserva las anteriores; una postulación devuelta y reenviada recorre de nuevo el flujo finalista, entrega física y auditoría.<br>▪ A6 (paso 1): si la entrega física del finalista no está registrada, el sistema rechaza la auditoría (conflicto) e indica registrar primero la entrega (CU-21).<br>▪ A7 (paso 6): si la postulación ya no está en estado «calificada» (por ejemplo, otro administrador la devolvió o archivó), el sistema rechaza la auditoría (conflicto).<br>▪ A8 (paso 6): si los datos son inválidos (observaciones de más de 500 caracteres, falta el puntaje académico en un resultado «validado» o puntaje fuera del rango de 0 a 30), el sistema informa el error y no registra la auditoría.<br>▪ A9 (paso 6): si la misma petición se repite (doble clic o reintento), el sistema la trata de forma idempotente y no duplica la auditoría. |
| **Postcondición** | ▪ El expediente del finalista tiene registrada una auditoría con su resultado, observaciones, puntaje académico (si es validado), auditor y fecha.<br>▪ El postulante recibe el aviso del resultado de la auditoría.<br>▪ La acción queda registrada en la bitácora. |

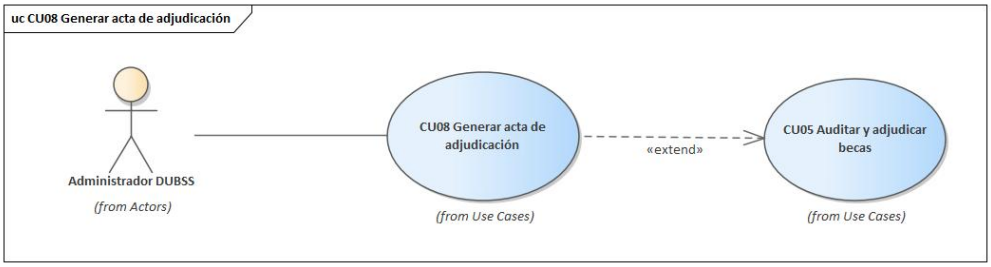
| | |
|---|---|
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ Error al recuperar los documentos desde MinIO.<br>▪ Entrega física no registrada o postulación que ya no está calificada (conflicto de estado).<br>▪ Error de conexión con el servidor o la base de datos al registrar la auditoría. |

*Nota.* Elaboración propia.

### *5.4.8. CU-08 Generar acta de adjudicación*

**Figura 33**

*Diagrama de contexto del CU-08*




*Nota.* Elaboración propia.

**Tabla 48**

*Detalle del CU-08: Generar acta de adjudicación*

| **Caso de Uso** | **CU-08. Generar acta de adjudicación** |
|---|---|
| **Propósito** | Generar el acta oficial que deja constancia de los resultados de la adjudicación de la Beca Comedor de una convocatoria. |
| **Descripción** | Este caso de uso describe cómo el Administrador DUBSS genera, una vez decididas todas las adjudicaciones de una convocatoria, el acta de adjudicación en formato PDF con la lista de beneficiarios aprobados. El acta se almacena en MinIO, se registra con un número único y puede consultarse posteriormente. El formato y el contenido oficial del acta están por definir con la DUBSS. Extiende el CU-05. Respalda el RF-04. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |

| **Caso de Uso** | **CU-08. Generar acta de adjudicación** |
|---|---|
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01) con el rol Administrador DUBSS.<br>▪ Todas las adjudicaciones de los finalistas de la convocatoria están decididas (CU-05). |
| **Proceso** | **Flujo principal A: generar el acta.**<br>1. El Administrador DUBSS selecciona la opción «Generar acta de adjudicación» de la convocatoria.<br>2. El sistema verifica que no existan finalistas con la adjudicación pendiente de decisión.<br>3. El sistema obtiene las adjudicaciones de la convocatoria con el nombre, CI, carrera y puntaje final de cada postulante adjudicado.<br>4. El sistema genera el acta en formato PDF con un número único, la fecha, los datos de la convocatoria y de la gestión académica y la lista de adjudicados.<br>5. El sistema almacena el PDF en MinIO y registra el acta con su número, la ruta del archivo, el usuario que la genera y la fecha de generación, y deja constancia en la bitácora.<br>6. El sistema muestra el acta generada y permite descargarla e imprimirla.<br>**Flujo principal B: consultar el acta.**<br>1. El Administrador DUBSS selecciona la opción «Consultar acta» de la convocatoria.<br>2. El sistema obtiene el acta registrada y la muestra para su descarga e impresión.<br>**Flujos alternativos.**<br>▪ A1 (flujo A, paso 2): si existen adjudicaciones pendientes de decisión, el sistema no genera el acta e indica cuántas faltan.<br>▪ A2 (flujo A, paso 1): si la convocatoria ya tiene un acta generada, el sistema muestra el acta existente en lugar de generar una nueva.<br>▪ A3 (flujo A, paso 5): si ocurre un error al almacenar el PDF, el sistema no registra el acta y permite reintentar.<br>▪ A4 (flujo B, paso 2): si la convocatoria aún no tiene un acta generada, el sistema lo informa. |
| **Postcondición** | ▪ El acta de adjudicación de la convocatoria queda almacenada en MinIO y registrada con su número, responsable y fecha de generación.<br>▪ La generación del acta queda registrada en la bitácora. |

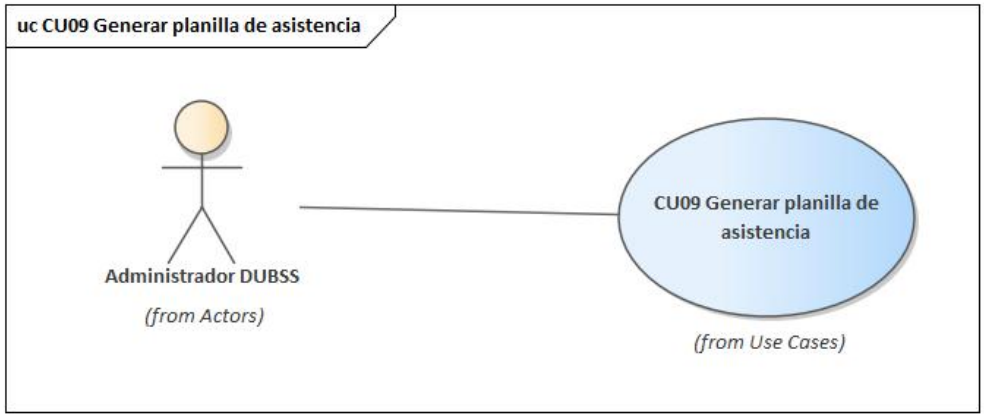
| Caso de Uso | CU-08. Generar acta de adjudicación |
|---|---|
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ Error al generar el documento PDF.<br>▪ Error al almacenar el archivo en MinIO.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.9. CU-09 Generar planilla de asistencia*

**Figura 34**

*Diagrama de contexto del CU-09*




*Nota.* Elaboración propia.

**Tabla 49**

*Detalle del CU-09: Generar planilla de asistencia*

| Caso de Uso | CU-09. Generar planilla de asistencia |
|---|---|
| **Propósito** | Generar planillas de asistencia imprimibles, a partir del padrón de becados vigente, que sirvan como respaldo físico del uso del beneficio ante las auditorías. |

| **Caso de Uso** | **CU-09. Generar planilla de asistencia** |
|---|---|
| **Descripción** | Este caso de uso describe cómo el Administrador DUBSS genera, para un periodo determinado, la planilla de asistencia de los becados vigentes de la convocatoria. La planilla se genera en formato PDF, lista para su impresión, con los espacios para la firma de cada becado y la fotocopia de su carnet, tal como lo exigen las auditorías; se almacena en MinIO y queda disponible para su consulta y para el cotejo con el registro digital (CU-11). Respalda el RF-05. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01) con el rol Administrador DUBSS.<br>▪ Existe un padrón de becados vigentes para la convocatoria (CU-23). |
| **Proceso** | **Flujo principal A: generar una planilla.**<br>1. El Administrador DUBSS selecciona la opción «Planillas de asistencia» y la convocatoria correspondiente.<br>2. El sistema solicita el periodo de la planilla (fecha desde y fecha hasta).<br>3. El Administrador DUBSS ingresa el periodo y confirma la generación.<br>4. El sistema valida que la fecha final no sea anterior a la inicial y que el periodo esté comprendido en la gestión académica.<br>5. El sistema obtiene las becas en estado vigente de la convocatoria.<br>6. El sistema genera la planilla en formato PDF con la lista de becados (nombre, CI y carrera) y los espacios para la firma y la fotocopia del carnet de cada uno (formato por definir con la DUBSS).<br>7. El sistema almacena el PDF en MinIO y registra la planilla con su periodo, la ruta del archivo, el usuario que la genera y la fecha de generación.<br>8. El sistema muestra la planilla generada para su descarga e impresión.<br>**Flujo principal B: consultar planillas.**<br>1. El Administrador DUBSS selecciona la opción «Consultar planillas» de la convocatoria.<br>2. El sistema muestra la lista de planillas generadas con su periodo y fecha de generación, y permite descargar cualquiera de ellas. |

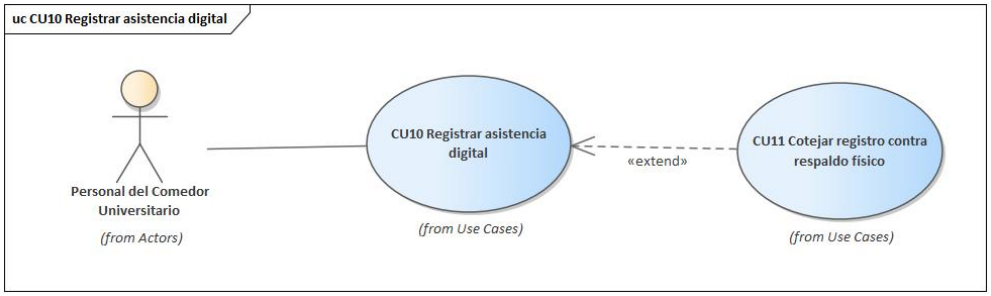
| Caso de Uso | CU-09. Generar planilla de asistencia |
|---|---|
| | **Flujos alternativos.**<br>▪ A1 (flujo A, paso 4): si el periodo es inválido, el sistema lo indica y permite corregirlo.<br>▪ A2 (flujo A, paso 5): si la convocatoria no tiene becas vigentes, el sistema no genera la planilla e informa el motivo.<br>▪ A3 (flujo A, paso 7): si ocurre un error al almacenar el PDF, el sistema no registra la planilla y permite reintentar.<br>▪ A4 (flujo A, paso 5): la planilla refleja el padrón vigente al momento de su generación; los cambios posteriores en el padrón (CU-23) se reflejan en las planillas que se generen después. |
| **Postcondición** | ▪ La planilla de asistencia queda almacenada en MinIO y registrada con su periodo, responsable y fecha de generación.<br>▪ La planilla queda disponible para su impresión y para el cotejo con el registro digital (CU-11). |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ Error al generar el documento PDF.<br>▪ Error al almacenar el archivo en MinIO.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.10. CU-10 Registrar asistencia digital*

**Figura 35**

*Diagrama de contexto del CU-10*




*Nota.* Elaboración propia.

**Tabla 50**

*Detalle del CU-10: Registrar asistencia digital*

| **Caso de Uso** | **CU-10. Registrar asistencia digital** |
|---|---|
| **Propósito** | Registrar digitalmente la asistencia diaria de cada becado al Comedor Universitario, una vez verificada su identidad por el personal del comedor. |
| **Descripción** | Este caso de uso describe cómo el Personal del Comedor Universitario registra, desde la aplicación móvil, la asistencia diaria de un becado. El sistema no cuenta con un mecanismo biométrico propio: la verificación de identidad la realiza el comedor por sus propios medios y el sistema solo registra su resultado. Se admite un único registro por becado y por día, de modo que las peticiones duplicadas no generan registros adicionales. El CU-11 lo extiende para cotejar el registro digital contra el respaldo físico. Respalda el RF-05. |
| **Actores** | Personal del Comedor Universitario. |
| **Actor Iniciador** | Personal del Comedor Universitario. |
| **Precondición** | ▪ El Personal del Comedor Universitario está autenticado en la aplicación móvil (CU-01) con el rol Personal del Comedor Universitario.<br>▪ Existe un padrón de becados vigentes (CU-23).<br>▪ El personal del comedor verificó la identidad del becado por los medios propios del comedor. |
| **Proceso** | **Flujo principal.**<br>1. El Personal del Comedor Universitario selecciona la opción «Registrar asistencia» en la aplicación móvil.<br>2. El sistema muestra la fecha actual y solicita el CI del becado.<br>3. El Personal del Comedor Universitario ingresa el CI del becado.<br>4. El sistema busca la beca asociada al CI y verifica que se encuentre en estado vigente.<br>5. El sistema muestra los datos del becado: nombre, CI, carrera y estado de la beca.<br>6. El Personal del Comedor Universitario confirma el registro de la asistencia.<br>7. El sistema verifica que no exista un registro de asistencia de esa beca en la fecha actual.<br>8. El sistema registra la asistencia con la fecha, la hora de registro y el usuario que la registra.<br>9. El sistema confirma el registro y queda listo para el siguiente becado. |

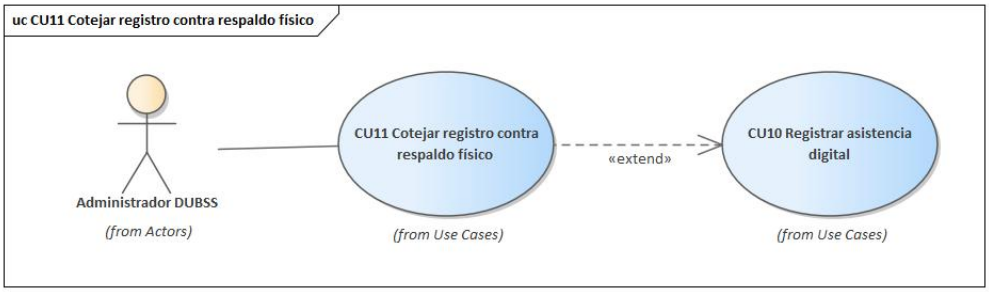
| Caso de Uso | CU-10. Registrar asistencia digital |
|---|---|
| | **Flujos alternativos.**<br>▪ A1 (paso 4): si el CI no corresponde a ningún becado, el sistema informa que el estudiante no figura en el padrón de becados.<br>▪ A2 (paso 4): si la beca se encuentra suspendida o dada de baja, el sistema no permite el registro e informa el estado de la beca.<br>▪ A3 (paso 7): si la asistencia del día ya fue registrada, el sistema no crea un nuevo registro e informa la hora del registro existente (idempotencia).<br>▪ A4 (paso 9): durante una auditoría, el Administrador DUBSS puede cotejar los registros digitales contra la planilla física firmada (CU-11). |
| **Postcondición** | ▪ La asistencia del becado queda registrada una sola vez para la fecha, con la hora y el usuario que la registró.<br>▪ El registro queda disponible para la consulta del becado (CU-24), el cotejo (CU-11) y los indicadores de BI (CU-12). |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Personal del Comedor Universitario).<br>▪ Becado inexistente o beca no vigente.<br>▪ Error de conexión con el servidor o la base de datos (la asistencia no se registra y debe reintentarse). |

*Nota.* Elaboración propia.

### *5.4.11. CU-11 Cotejar registro contra respaldo físico*

**Figura 36**

*Diagrama de contexto del CU-11*




*Nota.* Elaboración propia.

**Tabla 51**

*Detalle del CU-11: Cotejar registro contra respaldo físico*

| **Caso de Uso** | **CU-11. Cotejar registro contra respaldo físico** |
|---|---|
| **Propósito** | Permitir al Administrador DUBSS comparar el registro digital de asistencia con la planilla física firmada, para dejar constancia de su coincidencia ante las auditorías. |
| **Descripción** | Este caso de uso describe cómo el Administrador DUBSS, durante una auditoría, selecciona una planilla de asistencia generada (CU-09), consulta los registros digitales de asistencia del mismo periodo (CU-10) y los compara con la planilla física firmada por los becados. El resultado del cotejo (número de registros, coincidencias, discrepancias, observaciones y resultado) queda registrado. El cotejo no modifica los registros de asistencia. Extiende el CU-10. Respalda el RF-05. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01) con el rol Administrador DUBSS.<br>▪ Existe una planilla de asistencia generada para el periodo a cotejar (CU-09).<br>▪ El Administrador DUBSS dispone de la planilla física firmada correspondiente. |
| **Proceso** | **Flujo principal.**<br>1. El Administrador DUBSS selecciona la opción «Cotejo de asistencia» y la planilla a cotejar.<br>2. El sistema obtiene la planilla y los registros digitales de asistencia de los becados en el periodo de la planilla.<br>3. El sistema muestra, por becado y por día, los registros digitales de asistencia junto con la estructura de la planilla.<br>4. El Administrador DUBSS compara los registros digitales con las firmas de la planilla física.<br>5. El Administrador DUBSS registra el resultado del cotejo: total de registros, coincidencias, discrepancias, observaciones y resultado (conforme o con discrepancias).<br>6. El sistema registra el cotejo con el usuario que lo realiza y la fecha, y deja constancia en la bitácora.<br>7. El sistema confirma el registro del cotejo.<br>**Flujos alternativos.**<br>▪ A1 (paso 2): si no existen registros digitales en el periodo de la planilla, el sistema lo informa.<br>▪ A2 (paso 5): si el resultado es «con discrepancias», el sistema exige registrar observaciones.<br>▪ A3 (paso 6): si la planilla ya fue cotejada, el sistema registra el nuevo cotejo y conserva los anteriores. |

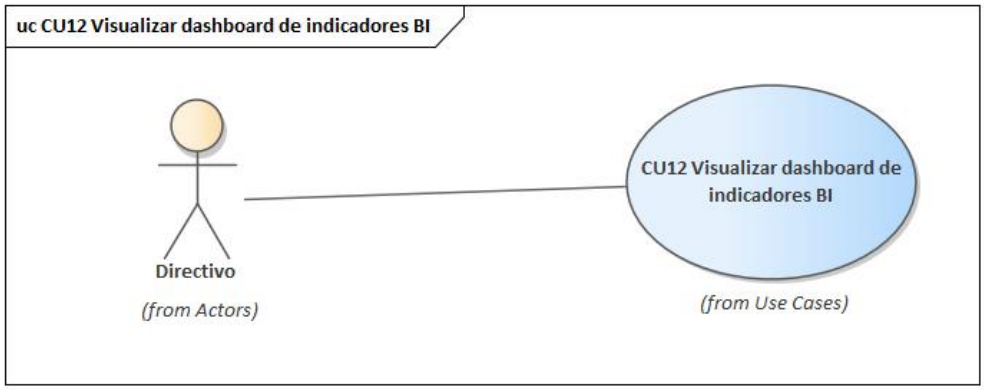
| Caso de Uso | CU-11. Cotejar registro contra respaldo físico |
|---|---|
| **Postcondición** | ▪ El cotejo queda registrado con su resultado, responsable y fecha, y la acción queda en la bitácora.<br>▪ Los registros de asistencia digital no se modifican. |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ Datos del cotejo incompletos o inválidos.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.12. CU-12 Visualizar dashboard de indicadores BI*

**Figura 37**

*Diagrama de contexto del CU-12*




*Nota.* Elaboración propia.

**Tabla 52**

*Detalle del CU-12: Visualizar dashboard de indicadores BI*

| Caso de Uso | CU-12. Visualizar dashboard de indicadores BI |
|---|---|
| **Propósito** | Brindar al Directivo un dashboard interactivo de Business Intelligence con los indicadores clave del proceso de la Beca Comedor, como apoyo a la toma de decisiones. |

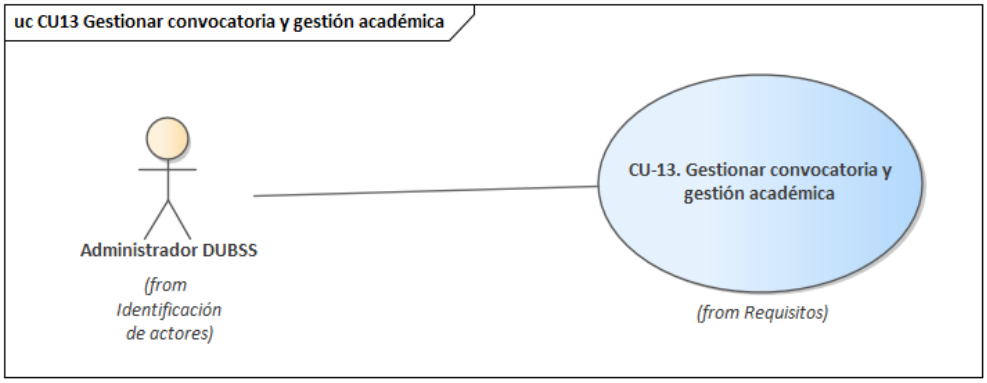
| **Caso de Uso** | **CU-12. Visualizar dashboard de indicadores BI** |
|---|---|
| **Descripción** | Este caso de uso describe cómo el Directivo consulta el dashboard de indicadores: postulaciones recibidas, finalistas, becas adjudicadas y asistencia, por convocatoria y por mes, con la posibilidad de filtrarlos por gestión académica. Los indicadores se obtienen de vistas precalculadas que se actualizan por lotes, de modo que la consulta no afecta el rendimiento de las operaciones transaccionales. Es una consulta de solo lectura. Respalda el RF-07. |
| **Actores** | Directivo. |
| **Actor Iniciador** | Directivo. |
| **Precondición** | ▪ El Directivo está autenticado en el sistema (CU-01) con el rol Directivo.<br>▪ Los indicadores fueron calculados en el último proceso por lotes. |
| **Proceso** | **Flujo principal.**<br>1. El Directivo selecciona la opción «Dashboard de indicadores».<br>2. El sistema consulta los indicadores precalculados de la gestión académica vigente.<br>3. El sistema muestra en gráficos interactivos los indicadores de postulaciones recibidas, finalistas, becas adjudicadas y asistencia, por convocatoria y por mes, junto con la fecha de su última actualización.<br>4. El Directivo selecciona una gestión académica para filtrar los indicadores.<br>5. El sistema consulta los indicadores de la gestión seleccionada y actualiza el dashboard.<br>**Flujos alternativos.**<br>▪ A1 (pasos 2 y 5): si no existen datos para la gestión académica consultada, el sistema muestra un mensaje informativo.<br>▪ A2 (paso 3): si el proceso por lotes aún no se ha ejecutado, el sistema muestra los últimos indicadores disponibles con su fecha de actualización. |
| **Postcondición** | ▪ El Directivo visualiza los indicadores; la consulta no modifica ningún dato del sistema. |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Directivo).<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.13. CU-13 Gestionar convocatoria y gestión académica*

**Figura 38**

*Diagrama de contexto del CU-13*




*Nota.* Elaboración propia.

**Tabla 53**

*Detalle del CU-13: Gestionar convocatoria y gestión académica*

| **Caso de Uso** | **CU-13. Gestionar convocatoria y gestión académica** |
|---|---|
| **Propósito** | Permitir al Administrador DUBSS crear, editar, abrir y cerrar las convocatorias de la Beca Comedor, controlando el periodo en que se reciben postulaciones. |
| **Descripción** | Este caso de uso describe cómo el Administrador DUBSS gestiona las convocatorias y las gestiones académicas, definiendo las fechas de apertura y cierre, el número de cupos y la lista de documentos obligatorios. Solo puede recibirse una postulación (CU-02) mientras la convocatoria esté abierta, y cada postulación queda asociada a la gestión vigente. Respalda el RF-08. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01).<br>▪ El Administrador cuenta con privilegios de administración. |

| Caso de Uso | CU-13. Gestionar convocatoria y gestión académica |
|---|---|
| **Proceso** | **Flujo principal.**<br>1. El Administrador accede a la opción «Convocatorias».<br>2. El sistema muestra la lista de convocatorias con su gestión académica, fechas, cupos y estado (programada, abierta o cerrada).<br>3. El Administrador selecciona la acción deseada: crear, editar, abrir, cerrar o consultar una convocatoria.<br>4. Para crear o editar, el Administrador ingresa la gestión académica, la fecha de apertura, la fecha de cierre, el número de cupos (por definir con la DUBSS) y la lista de documentos obligatorios (nombre, tipo y formatos permitidos).<br>5. El sistema valida que la fecha de cierre sea posterior a la de apertura, que no exista otra convocatoria vigente para la misma gestión académica y que se haya definido al menos un documento obligatorio.<br>6. El Administrador confirma la operación.<br>7. El sistema guarda la convocatoria, actualiza su estado y registra la acción en la bitácora (CU-25).<br>8. El sistema confirma la operación y muestra la lista actualizada.<br>**Flujos alternativos.**<br>▪ A1 (paso 5): si ya existe una convocatoria vigente para la misma gestión académica, el sistema rechaza la operación e informa del conflicto.<br>▪ A2 (paso 5): si las fechas son incoherentes (cierre anterior o igual a la apertura) o hay datos obligatorios vacíos, el sistema indica los errores y permite corregirlos.<br>▪ A3 (paso 3): al abrir una convocatoria, el sistema cambia su estado a «abierta» y habilita el envío de postulaciones desde esa fecha.<br>▪ A4 (paso 3): al cerrar una convocatoria, el sistema cambia su estado a «cerrada», bloquea el envío de nuevas postulaciones y descarta o bloquea los borradores no enviados (CU-17).<br>▪ A5 (paso 3): si la fecha de cierre se cumple, el sistema cierra la convocatoria automáticamente.<br>▪ A6 (paso 4): si la convocatoria ya tiene postulaciones recibidas, el sistema restringe la edición de los documentos obligatorios y de las fechas de apertura (alcance por definir con la DUBSS) para no afectar las postulaciones existentes.<br>▪ A7 (paso 3): el sistema no permite eliminar una convocatoria con postulaciones asociadas; solo permite cerrarla. |
| **Postcondición** | ▪ La convocatoria queda registrada o actualizada con su gestión, fechas, cupos y documentos requeridos.<br>▪ El estado de la convocatoria determina si el sistema acepta o no nuevas postulaciones.<br>▪ La operación queda registrada en la bitácora. |

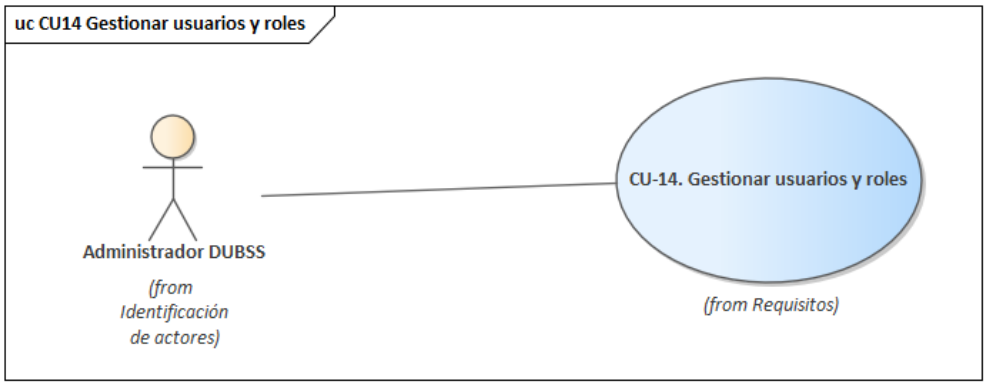
| Caso de Uso | CU-13. Gestionar convocatoria y gestión académica |
|---|---|
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ Datos ingresados no válidos o convocatoria duplicada para la gestión.<br>▪ Intento de eliminar una convocatoria con postulaciones asociadas.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.14. CU-14 Gestionar usuarios y roles*

**Figura 39**

*Diagrama de contexto del CU-14*




*Nota.* Elaboración propia.

**Tabla 54**

*Detalle del CU-14: Gestionar usuarios y roles*

| Caso de Uso | CU-14. Gestionar usuarios y roles |
|---|---|
| **Propósito** | Permitir al Administrador DUBSS gestionar las cuentas del personal y asignarles roles, de modo que cada persona acceda únicamente a las funciones que le corresponden. |

| **Caso de Uso** | **CU-14. Gestionar usuarios y roles** |
|---|---|
| **Descripción** | Este caso de uso describe cómo el Administrador DUBSS crea, edita, desactiva y reactiva las cuentas del personal y les asigna uno de los roles del sistema (Administrador, Directivo o Personal del Comedor). Los Postulantes no se gestionan aquí, pues se registran por sí mismos (CU-01). Las cuentas se crean en la base de datos y los cambios de rol se conservan en la bitácora (CU-25). Respalda el RF-09. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01).<br>▪ El Administrador cuenta con privilegios de administración. |
| **Proceso** | **Flujo principal.**<br>1. El Administrador accede a la opción «Usuarios y roles».<br>2. El sistema muestra la lista de cuentas del personal con su nombre, CI, correo, rol y estado (activa o desactivada), con filtros por rol, estado y búsqueda por CI o nombre.<br>3. El Administrador selecciona la acción deseada: crear, editar, cambiar rol, desactivar o reactivar una cuenta.<br>4. Para crear una cuenta, el Administrador ingresa los datos del funcionario (nombres, apellidos, CI, correo electrónico y rol).<br>5. El sistema valida que los datos sean correctos y que el CI y el correo electrónico no estén registrados.<br>6. El Administrador confirma la operación.<br>7. El sistema crea o actualiza la cuenta en la base de datos y genera una contraseña temporal, que se muestra una sola vez al Administrador para que la entregue al funcionario; este la cambia desde su perfil (CU-15).<br>8. El sistema registra la acción (usuario, cambio realizado, fecha y hora) en la bitácora y muestra la lista actualizada. |

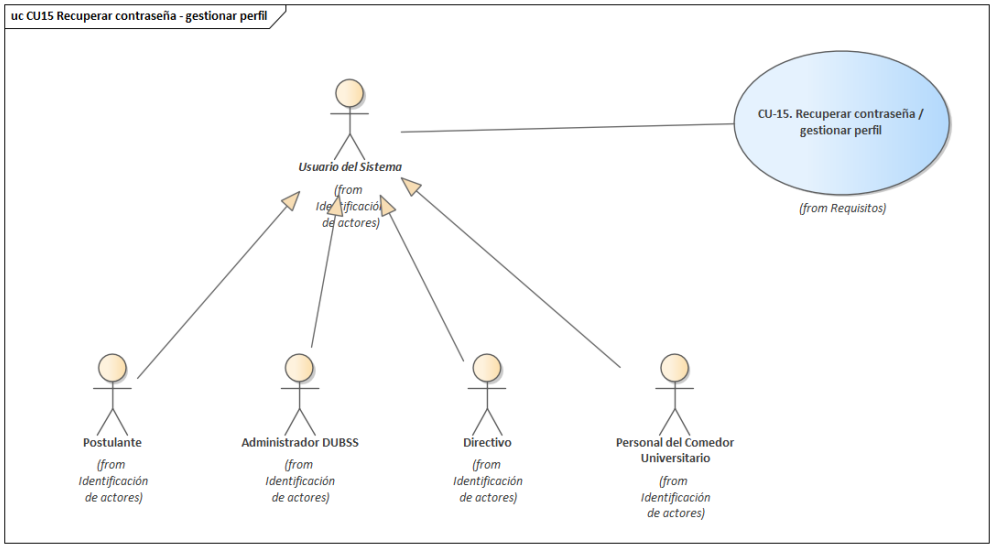
| **Caso de Uso** | **CU-14. Gestionar usuarios y roles** |
|---|---|
| | **Flujos alternativos.**<br>▪ A1 (paso 5): si el CI o el correo electrónico ya están registrados, el sistema rechaza el alta e indica el campo en conflicto.<br>▪ A2 (paso 5): si faltan datos obligatorios o su formato es inválido, el sistema marca los errores y permite corregirlos.<br>▪ A3 (paso 3): si se cambia el rol de una cuenta, el sistema aplica los nuevos permisos desde su siguiente inicio de sesión y conserva el registro del cambio (rol anterior, rol nuevo, fecha y responsable).<br>▪ A4 (paso 3): al desactivar una cuenta, el sistema impide su acceso e invalida sus sesiones activas; la cuenta no se elimina, de modo que se preserve la trazabilidad de sus acciones.<br>▪ A5 (paso 3): el sistema impide que el Administrador desactive su propia cuenta o la del último Administrador activo.<br>▪ A6 (paso 3): el Administrador puede reactivar una cuenta desactivada, que recupera su rol anterior. |
| **Postcondición** | ▪ La cuenta queda creada, modificada, desactivada o reactivada, y el sistema refleja el cambio de rol en los permisos del usuario.<br>▪ El cambio queda registrado en la bitácora. |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ Datos ingresados no válidos o CI/correo duplicados.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.15. CU-15 Recuperar contraseña / gestionar perfil*

**Figura 40**

*Diagrama de contexto del CU-15*




*Nota.* Elaboración propia.

**Tabla 55**

*Detalle del CU-15: Recuperar contraseña / gestionar perfil*

| **Caso de Uso** | **CU-15. Recuperar contraseña / gestionar perfil** |
|---|---|
| **Propósito** | Permitir a los usuarios recuperar el acceso a su cuenta cuando olvidan su contraseña y mantener actualizados sus datos de contacto y su contraseña desde su perfil. |
| **Descripción** | Este caso de uso describe dos funciones: la recuperación de contraseña mediante un enlace enviado al correo registrado (generado por el backend) y la gestión del perfil, donde el usuario autenticado actualiza sus datos de contacto y su contraseña. El CI no puede modificarse desde el perfil. Extiende al CU-01. Respalda el RF-10. |
| **Actores** | Usuario del Sistema (Postulante, Administrador DUBSS, Directivo, Personal del Comedor Universitario); Servicio de Correo Electrónico (actor secundario, entrega el enlace). |
| **Actor Iniciador** | Usuario del Sistema. |

| **Caso de Uso** | **CU-15. Recuperar contraseña / gestionar perfil** |
|---|---|
| **Precondición** | ▪ Recuperar contraseña: el usuario tiene una cuenta registrada con un correo electrónico al que puede acceder.<br>▪ Gestionar perfil: el usuario está autenticado en el sistema (CU-01). |
| **Proceso** | **Flujo principal A: recuperar contraseña.**<br>1. En la pantalla de inicio de sesión, el usuario selecciona «¿Olvidó su contraseña?».<br>2. El sistema solicita el correo electrónico de la cuenta.<br>3. El usuario ingresa su correo y confirma la solicitud.<br>4. El sistema genera un token de restablecimiento de un solo uso con tiempo de validez limitado, lo envía en un enlace al correo registrado y muestra un mensaje informando que, si el correo está registrado, recibirá las instrucciones.<br>5. El usuario abre el enlace recibido y define una nueva contraseña que cumpla la política de seguridad.<br>6. El sistema guarda la nueva contraseña, invalida el enlace, desbloquea la cuenta si estaba bloqueada, cierra todas las sesiones abiertas de la cuenta y permite al usuario iniciar sesión con la nueva contraseña.<br>**Flujo principal B: gestionar perfil.**<br>1. El usuario autenticado accede a la opción «Mi perfil».<br>2. El sistema muestra sus datos personales y de contacto; el CI se muestra solo de lectura.<br>3. El usuario modifica sus datos de contacto (el correo electrónico y, si es Postulante, el teléfono) y/o solicita cambiar su contraseña ingresando la contraseña actual y la nueva.<br>4. El sistema valida los datos, comprueba la unicidad del correo y la política de contraseña.<br>5. El sistema guarda los cambios, registra la acción en la bitácora y confirma la actualización; si cambió la contraseña, cierra las sesiones de los demás dispositivos y mantiene la actual. |

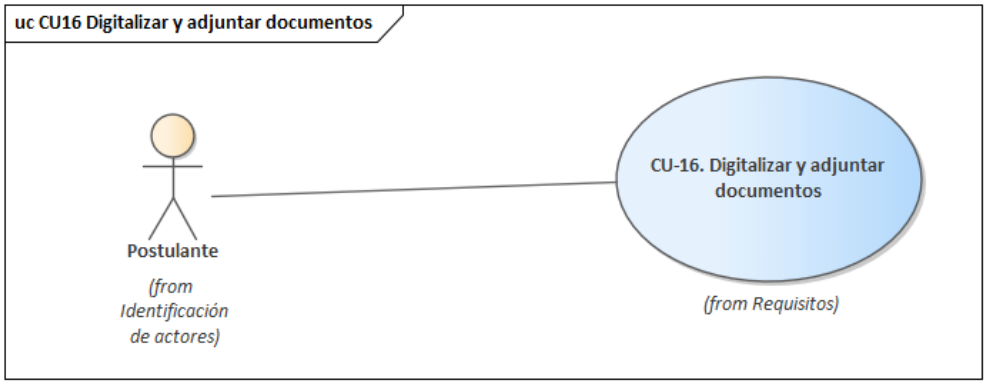
| **Caso de Uso** | **CU-15. Recuperar contraseña / gestionar perfil** |
|---|---|
| | **Flujos alternativos.**<br>▪ A1 (flujo A, paso 4): si el correo no está registrado, el sistema muestra el mismo mensaje neutro, para no revelar qué correos existen en el sistema.<br>▪ A2 (flujo A, paso 5): si el enlace expiró o ya fue utilizado, el sistema informa del hecho y permite solicitar uno nuevo.<br>▪ A3 (flujo A, paso 5): si la nueva contraseña no cumple la política de seguridad, el sistema muestra los requisitos y permite reintentar.<br>▪ A4 (flujo B, paso 3): si el usuario intenta modificar su CI, el sistema no lo permite (campo bloqueado).<br>▪ A5 (flujo B, paso 4): si el nuevo correo electrónico ya pertenece a otra cuenta, el sistema rechaza el cambio.<br>▪ A6 (flujo B, paso 4): si la contraseña actual ingresada es incorrecta, el sistema rechaza el cambio de contraseña.<br>▪ A7 (flujo B, paso 5): si se cambia el correo electrónico, el sistema exige la contraseña actual (reautenticación) antes de guardarlo. |
| **Postcondición** | ▪ Recuperar contraseña: la cuenta queda con una nueva contraseña y el usuario puede autenticarse.<br>▪ Gestionar perfil: los datos de contacto y/o la contraseña quedan actualizados y se utilizarán en las notificaciones al usuario. |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada (en la gestión de perfil).<br>▪ Datos ingresados no válidos.<br>▪ Enlace de restablecimiento inválido o expirado.<br>▪ Servicio de envío de correo electrónico no disponible.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.16. CU-16 Digitalizar y adjuntar documentos*

**Figura 41**

*Diagrama de contexto del CU-16*




*Nota.* Elaboración propia.

**Tabla 56**

*Detalle del CU-16: Digitalizar y adjuntar documentos*

| **Caso de Uso** | **CU-16. Digitalizar y adjuntar documentos** |
|---|---|
| **Propósito** | Permitir al Postulante digitalizar sus documentos con la cámara del teléfono, o cargarlos desde un archivo en la web, y asociarlos a su postulación como respaldo de los datos declarados. |
| **Descripción** | Este caso de uso describe cómo el Postulante captura o adjunta cada documento requerido por la convocatoria, valida su formato y tamaño, revisa una vista previa y lo asocia a su postulación, almacenándolo en MinIO. Es incluido por el CU-02 y también se utiliza al completar un borrador (CU-17). Permite evitar la presentación de carpetas físicas en la etapa inicial. Respalda el RF-11. |
| **Actores** | Postulante. |
| **Actor Iniciador** | Postulante. |
| **Precondición** | ▪ El Postulante está autenticado en el sistema (CU-01).<br>▪ El Postulante se encuentra completando una postulación o un borrador, con la convocatoria abierta.<br>▪ La convocatoria tiene definida la lista de documentos requeridos.<br>▪ En la aplicación móvil, el Postulante otorgó el permiso de uso de la cámara. |

| **Caso de Uso** | **CU-16. Digitalizar y adjuntar documentos** |
|---|---|
| **Proceso** | **Flujo principal.**<br>1. El sistema muestra la lista de documentos requeridos por la convocatoria, indicando cuáles están pendientes y cuáles ya fueron adjuntados.<br>2. El Postulante selecciona el documento que desea adjuntar.<br>3. En la aplicación móvil, el Postulante captura el documento con la cámara del dispositivo; en el portal web, selecciona un archivo de su equipo.<br>4. El sistema valida el formato y el tamaño del archivo (formatos y tamaño máximo por definir con la DUBSS).<br>5. El sistema muestra una vista previa del documento.<br>6. El Postulante confirma el documento, o bien lo reemplaza, lo vuelve a capturar o lo elimina.<br>7. El sistema almacena el archivo en MinIO y registra el documento adjunto (tipo y archivo) asociado a la postulación.<br>8. El sistema actualiza la lista de documentos con el estado «adjuntado».<br>**Flujos alternativos.**<br>▪ A1 (paso 3): si el Postulante no otorga el permiso de la cámara, el sistema le ofrece seleccionar el documento desde la galería o los archivos del dispositivo.<br>▪ A2 (paso 4): si el formato no es permitido o el tamaño excede el máximo, el sistema rechaza el archivo y muestra un mensaje de error.<br>▪ A3 (paso 6): el Postulante puede reemplazar o eliminar un documento ya adjuntado, siempre que la postulación no haya sido enviada o se encuentre devuelta (CU-19).<br>▪ A4 (paso 7): si falla la carga por problemas de conexión, el sistema permite reintentar sin perder los demás documentos ya adjuntados.<br>▪ A5 (paso 3): el Postulante puede adjuntar más de un archivo o página para un mismo documento (cantidad máxima por definir).<br>▪ A6 (general): si al enviar la postulación faltan documentos obligatorios, el CU-02 no permite el envío e indica los documentos faltantes. |
| **Postcondición** | ▪ Los documentos adjuntados quedan almacenados en MinIO y vinculados a la postulación (o al borrador).<br>▪ La lista de documentos requeridos refleja cuáles fueron cargados y cuáles faltan. |

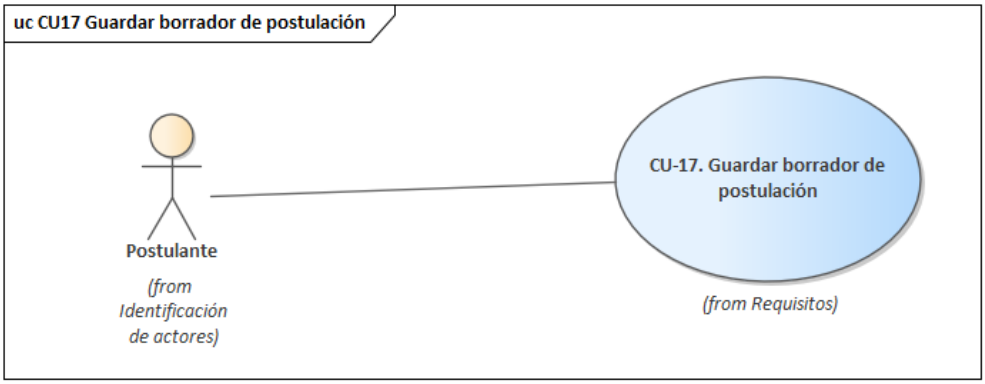
| Caso de Uso | CU-16. Digitalizar y adjuntar documentos |
|---|---|
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Permiso de cámara denegado.<br>▪ Formato o tamaño de archivo no permitido.<br>▪ Error al almacenar el archivo en MinIO.<br>▪ Convocatoria cerrada durante la carga de documentos.<br>▪ Error de conexión con el servidor. |

*Nota.* Elaboración propia.

### *5.4.17. CU-17 Guardar borrador de postulación*

**Figura 42**

*Diagrama de contexto del CU-17*




*Nota.* Elaboración propia.

**Tabla 57**

*Detalle del CU-17: Guardar borrador de postulación*

| Caso de Uso | CU-17. Guardar borrador de postulación |
|---|---|
| **Propósito** | Permitir al Postulante guardar su postulación incompleta como borrador y retomarla posteriormente sin perder lo ya ingresado. |
| **Descripción** | Este caso de uso describe cómo el Postulante guarda el avance del formulario socioeconómico y de los documentos adjuntos, sin necesidad de completar los campos obligatorios, y cómo lo retoma después para completarlo y enviarlo. Un borrador no es una postulación enviada, por lo que no se procesa con el módulo de IA ni cuenta como postulación vigente. Extiende al CU-02. Respalda el RF-12. |

| **Caso de Uso** | **CU-17. Guardar borrador de postulación** |
|---|---|
| **Actores** | Postulante. |
| **Actor Iniciador** | Postulante. |
| **Precondición** | ▪ El Postulante está autenticado en el sistema (CU-01).<br>▪ Existe una convocatoria abierta (CU-13).<br>▪ El Postulante no tiene una postulación vigente ya enviada en la misma gestión. |
| **Proceso** | **Flujo principal A: guardar borrador.**<br>1. Durante el llenado del formulario (CU-02), el Postulante selecciona «Guardar borrador».<br>2. El sistema verifica que la convocatoria continúe abierta.<br>3. El sistema guarda los datos ingresados hasta ese momento y las referencias a los documentos ya adjuntados, sin exigir los campos ni documentos obligatorios.<br>4. El sistema registra la fecha y hora del último guardado.<br>5. El sistema confirma que el borrador fue guardado.<br>**Flujo principal B: retomar borrador.**<br>1. El Postulante accede a la opción «Mi borrador» o «Nueva postulación».<br>2. El sistema detecta el borrador y lo muestra con los datos y documentos previamente guardados.<br>3. El Postulante continúa completando el formulario y puede guardar el borrador nuevamente o enviar la postulación (CU-02).<br>**Flujos alternativos.**<br>▪ A1 (flujo A, paso 2): si la convocatoria ya está cerrada, el sistema no permite guardar ni editar el borrador e informa que el plazo finalizó.<br>▪ A2 (flujo A, paso 3): si algún campo contiene un valor con formato inválido (por ejemplo, texto en un campo numérico), el sistema lo señala y no guarda ese valor.<br>▪ A3 (flujo A, paso 3): si falla la conexión, el sistema informa que no se guardó el borrador y permite reintentar.<br>▪ A4 (flujo B, paso 2): si no existe un borrador, el sistema inicia una postulación nueva.<br>▪ A5 (flujo B, paso 3): al enviarse la postulación, el borrador se convierte en postulación y deja de existir como borrador.<br>▪ A6 (general): al cerrar la convocatoria, el sistema descarta o bloquea los borradores no enviados (CU-13). |

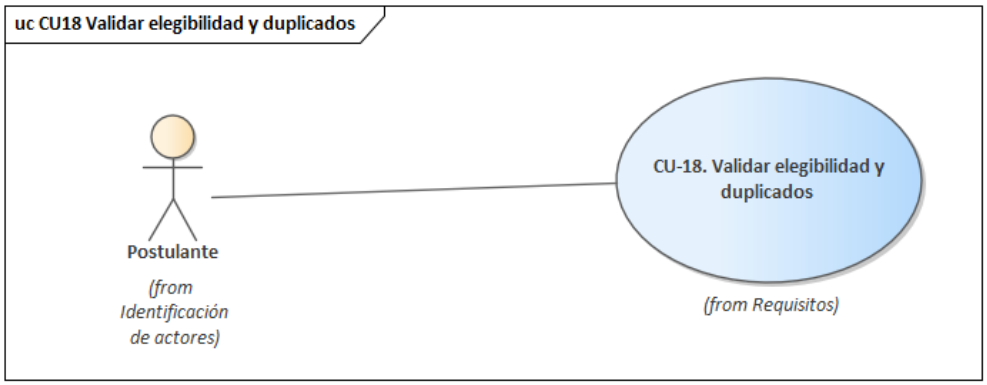
| Caso de Uso | CU-17. Guardar borrador de postulación |
|---|---|
| **Postcondición** | ▪ El borrador queda guardado con su fecha de último guardado y puede retomarse mientras la convocatoria esté abierta.<br>▪ El borrador no se considera enviado, no se procesa con el módulo de IA y no modifica el estado de ninguna postulación. |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Convocatoria cerrada.<br>▪ Datos con formato inválido.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.18. CU-18 Validar elegibilidad y duplicados*

**Figura 43**

*Diagrama de contexto del CU-18*




*Nota.* Elaboración propia.

**Tabla 58**

*Detalle del CU-18: Validar elegibilidad y duplicados*

| Caso de Uso | CU-18. Validar elegibilidad y duplicados |
|---|---|
| **Propósito** | Verificar, al momento de recibir una postulación, que el postulante cumple las condiciones de elegibilidad y que no existe otra postulación vigente suya en la misma gestión. |

| **Caso de Uso** | **CU-18. Validar elegibilidad y duplicados** |
|---|---|
| **Descripción** | Este caso de uso describe la validación automática que ejecuta el sistema cuando el Postulante envía su postulación: comprueba las condiciones de elegibilidad definidas por la DUBSS y la unicidad de la postulación por postulante y gestión. Evita registros duplicados o inválidos que distorsionen la priorización y la adjudicación. Es incluido por el CU-02. Respalda el RF-13. |
| **Actores** | Postulante. |
| **Actor Iniciador** | Postulante (a través del CU-02, al enviar su postulación; el sistema ejecuta la validación). |
| **Precondición** | ▪ El Postulante envió su postulación con los campos y documentos obligatorios completos (CU-02).<br>▪ Existe una convocatoria abierta a la que se asocia la postulación.<br>▪ Las condiciones de elegibilidad están configuradas en el sistema (por definir con la DUBSS). |
| **Proceso** | **Flujo principal.**<br>1. El sistema recibe la solicitud de validación con la identidad del Postulante y la convocatoria de destino.<br>2. El sistema consulta las postulaciones existentes del Postulante en la misma gestión académica.<br>3. El sistema verifica que no exista otra postulación vigente del Postulante en la convocatoria.<br>4. El sistema verifica las condiciones de elegibilidad (por ejemplo, la condición de estudiante de la UAGRM según su CI y perfil; reglas por definir con la DUBSS).<br>5. El sistema devuelve al CU-02 el resultado «válida».<br>6. El CU-02 continúa con el registro de la postulación.<br>**Flujos alternativos.**<br>▪ A1 (paso 3): si ya existe otra postulación vigente del Postulante en la gestión, el sistema devuelve el resultado «duplicada» y el CU-02 rechaza el envío e informa el motivo.<br>▪ A2 (paso 4): si el Postulante no cumple alguna condición de elegibilidad, el sistema devuelve el resultado «no elegible» con el motivo específico, y el CU-02 rechaza el envío.<br>▪ A3 (paso 3): si se trata del reenvío de una postulación devuelta (CU-19), el sistema no la considera duplicada y la vincula con la postulación original, conservando el historial.<br>▪ A4 (paso 3): si se recibe una solicitud duplicada, el sistema devuelve el mismo resultado sin generar registros adicionales (idempotencia).<br>▪ A5 (paso 2): si el servicio de validación no está disponible, el sistema no acepta la postulación e informa al Postulante que reintente. |

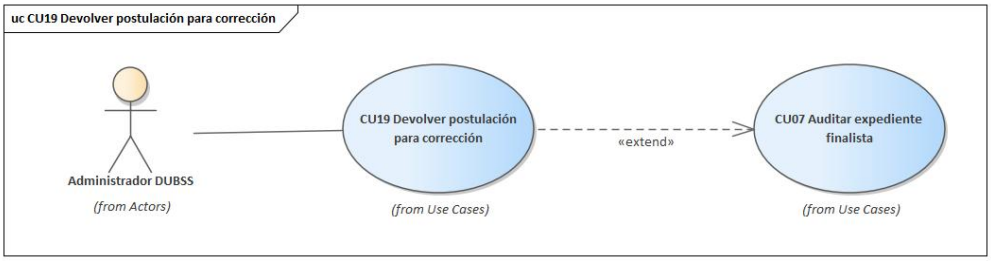
| Caso de Uso | CU-18. Validar elegibilidad y duplicados |
|---|---|
| **Postcondición** | ▪ El sistema determina si la postulación es válida y, en caso afirmativo, el CU-02 puede registrarla.<br>▪ Si la postulación es rechazada, no se crea ningún registro y el Postulante recibe el motivo del rechazo. |
| **Excepciones** | ▪ Postulación duplicada en la misma gestión.<br>▪ Postulante no elegible según las condiciones establecidas.<br>▪ Servicio de validación no disponible.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.19. CU-19 Devolver postulación para corrección*

**Figura 44**

*Diagrama de contexto del CU-19*




*Nota.* Elaboración propia.

**Tabla 59**

*Detalle del CU-19: Devolver postulación para corrección*

| Caso de Uso | CU-19. Devolver postulación para corrección |
|---|---|
| **Propósito** | Permitir al Administrador DUBSS devolver una postulación con observaciones para que el Postulante corrija o complete su documentación y la reenvíe, evitando descartarla por defectos subsanables. |

| **Caso de Uso** | **CU-19. Devolver postulación para corrección** |
|---|---|
| **Descripción** | Este caso de uso describe cómo el Administrador DUBSS, durante la auditoría de un expediente finalista, devuelve la postulación al Postulante desde el expediente del finalista, registrando un motivo de hasta 500 caracteres con las observaciones que debe subsanar. La postulación cambia al estado «devuelta», el cambio queda en el historial de la postulación y en la bitácora, y el Postulante recibe siempre el aviso por bandeja, push y correo. No hay plazo de reenvío ni límite de devoluciones: el Postulante puede corregir y reenviar la postulación cuando quiera mediante el CU-02. Extiende el CU-07. Respalda el RF-14. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01) con el rol Administrador DUBSS.<br>▪ El Administrador DUBSS se encuentra auditando el expediente de un finalista (CU-07).<br>▪ La postulación se encuentra en estado «calificada».<br>▪ El finalista tiene al menos una auditoría registrada después de su última devolución; no se exige entrega física. |
| **Proceso** | **Flujo principal.**<br>1. El Administrador DUBSS selecciona la opción «Devolver para corrección» en el expediente.<br>2. El sistema solicita el motivo de la devolución (máximo 500 caracteres).<br>3. El Administrador DUBSS registra el motivo y confirma la devolución.<br>4. El sistema valida que el motivo no esté vacío ni supere los 500 caracteres, que la postulación siga en estado «calificada» y que exista una auditoría posterior a la última devolución.<br>5. El sistema actualiza el estado de la postulación a «devuelta».<br>6. El sistema registra, en una sola operación, el cambio en el historial de la postulación (estado anterior, estado nuevo, usuario y motivo) y en la bitácora.<br>7. El sistema crea la notificación para el Postulante y se la envía siempre por bandeja, push y correo, con las observaciones.<br>8. El sistema confirma la devolución al Administrador DUBSS. |

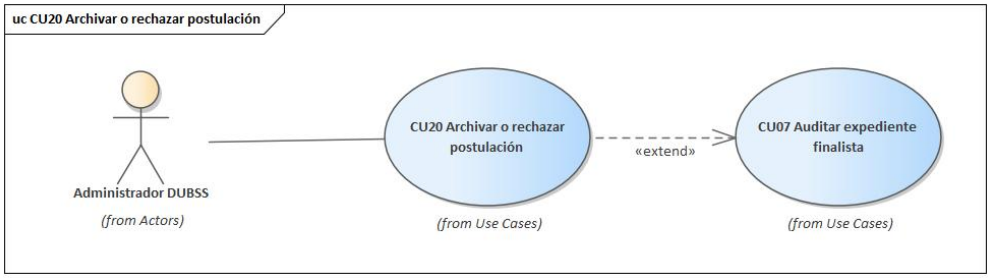
| **Caso de Uso** | **CU-19. Devolver postulación para corrección** |
|---|---|
| | **Flujos alternativos.**<br>▪ A1 (paso 4): si el motivo está vacío o supera los 500 caracteres, el sistema no permite devolver la postulación.<br>▪ A5 (paso 4): si el finalista no existe, el sistema informa que no se encontró.<br>▪ A6 (paso 4): si la postulación ya no está «calificada» (incluye a dos administradores actuando a la vez), el sistema rechaza la devolución por conflicto de estado.<br>▪ A7 (paso 4): si no hay una auditoría registrada después de la última devolución, el sistema rechaza la devolución por conflicto.<br>▪ A8 (paso 5): si la petición se repite por doble clic, la segunda se rechaza por estado inválido; el botón se deshabilita al enviar.<br>▪ A2 (paso 3): si el Administrador DUBSS cancela la operación, la postulación conserva su estado.<br>▪ A3 (paso 7): si falla el envío del push, la notificación queda en la bandeja del Postulante y el servicio de notificaciones la reintenta; el Postulante también puede verla al consultar su postulación (CU-04).<br>▪ A4 (posterior al paso 8): el reenvío es del CU-02; cuando el Postulante corrige y reenvía la postulación, esta vuelve al estado «pendiente», se incrementa su número de envíos, se conserva el historial de devoluciones y reenvíos, y sigue el mismo flujo del CU-07 (nueva entrega física si hace falta y nueva auditoría; las anteriores se conservan). |
| **Postcondición** | ▪ La postulación queda en estado «devuelta» con las observaciones registradas en su historial.<br>▪ El Postulante es notificado y puede consultar las observaciones (CU-04). |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ La postulación no se encuentra en un estado que permita su devolución (conflicto de estado).<br>▪ El finalista indicado no existe.<br>▪ Error al enviar la notificación push mediante Firebase Cloud Messaging.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.20. CU-20 Archivar o rechazar postulación*

**Figura 45**

*Diagrama de contexto del CU-20*




*Nota.* Elaboración propia.

**Tabla 60**

*Detalle del CU-20: Archivar o rechazar postulación*

| **Caso de Uso** | **CU-20. Archivar o rechazar postulación** |
|---|---|
| **Propósito** | Permitir al Administrador DUBSS cerrar formalmente una postulación que no procede, archivándola o rechazándola, y dejar constancia del motivo de la decisión. |
| **Descripción** | Este caso de uso describe cómo el Administrador DUBSS, durante la auditoría de un expediente finalista, archiva o rechaza una postulación registrando el motivo. Al archivarla, la postulación pasa al estado «archivada»; al rechazarla, pasa al estado «no adjudicada». En ambos casos deja de aparecer en la lista priorizada activa, el cambio queda en el historial de la postulación y esta se conserva para consulta histórica y reportes. Extiende el CU-07. Respalda el RF-15. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01) con el rol Administrador DUBSS.<br>▪ El Administrador DUBSS se encuentra auditando el expediente de un finalista (CU-07).<br>▪ La postulación se encuentra en estado «calificada». |

| Caso de Uso | CU-20. Archivar o rechazar postulación |
|---|---|
| **Proceso** | **Flujo principal A: archivar la postulación.**<br>1. El Administrador DUBSS selecciona la opción «Archivar» en el expediente.<br>2. El sistema solicita el motivo del archivo.<br>3. El Administrador DUBSS registra el motivo y confirma.<br>4. El sistema valida que el motivo no esté vacío.<br>5. El sistema actualiza el estado de la postulación a «archivada» y la retira de la lista priorizada activa.<br>6. El sistema registra el cambio en el historial de la postulación (estado anterior, estado nuevo, usuario y motivo) y en la bitácora, y confirma la operación.<br>**Flujo principal B: rechazar la postulación.**<br>1. El Administrador DUBSS selecciona la opción «Rechazar» en el expediente.<br>2. El sistema solicita el motivo del rechazo.<br>3. El Administrador DUBSS registra el motivo y confirma.<br>4. El sistema valida que el motivo no esté vacío.<br>5. El sistema actualiza el estado de la postulación a «no adjudicada» y la retira de la lista priorizada activa.<br>6. El sistema registra el cambio en el historial de la postulación (estado anterior, estado nuevo, usuario y motivo) y en la bitácora, y confirma la operación.<br>**Flujos alternativos.**<br>▪ A1 (flujos A y B, paso 4): si no se registra el motivo, el sistema no permite completar la operación.<br>▪ A2 (flujos A y B, paso 3): si el Administrador DUBSS cancela la operación, la postulación conserva su estado.<br>▪ A3 (flujos A y B, paso 1): si la postulación ya se encuentra en un estado final («archivada», «adjudicada» o «no adjudicada») o los resultados de la convocatoria fueron publicados (CU-22), el sistema no permite la operación. |
| **Postcondición** | ▪ La postulación queda en estado «archivada» o «no adjudicada», con el motivo registrado en su historial, y se conserva para consulta histórica y reportes.<br>▪ El Postulante puede consultar el estado y el motivo registrado (CU-04). |

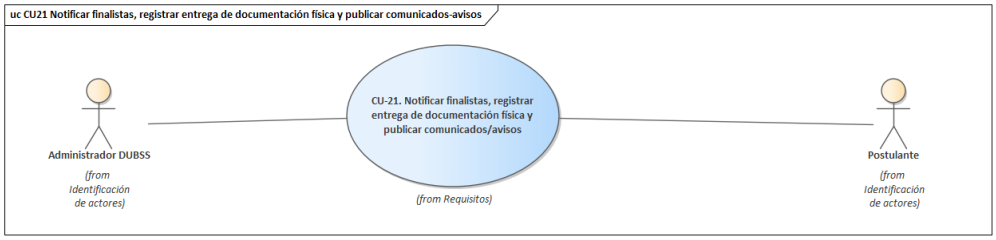
| Caso de Uso | CU-20. Archivar o rechazar postulación |
|---|---|
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ La postulación no se encuentra en un estado que permita archivarla o rechazarla.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.21. CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos*

**Figura 46**

*Diagrama de contexto del CU-21*




*Nota.* Elaboración propia.

**Tabla 61**

*Detalle del CU-21: Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos*

| Caso de Uso | CU-21. Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos |
|---|---|
| **Propósito** | Permitir al Administrador DUBSS notificar a los finalistas para que presenten su documentación física, registrar su recepción y publicar comunicados y avisos dirigidos a los postulantes. |

| | |
|---|---|
| **Caso de Uso** | **CU-21. Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos** |
| **Descripción** | Este caso de uso agrupa tres funciones que coordinan la auditoría física: notificar a los finalistas el plazo y los documentos que deben presentar, registrar la fecha en que se recibe la documentación física de cada uno y publicar comunicados o avisos generales. Las notificaciones se envían mediante notificación push (Firebase Cloud Messaging) y quedan consultables en la aplicación. La pantalla de finalistas muestra además la última auditoría de cada uno, la insignia «Devuelta» cuando la postulación fue devuelta y la acción «Auditar» (CU-07); todo aviso se envía por el servicio de notificaciones. Es incluido por el CU-07 (Auditar expediente finalista). Respalda el RF-16. |
| **Actores** | Administrador DUBSS; Postulante (receptor de las notificaciones). |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01) y cuenta con privilegios de administración.<br>▪ Para notificar finalistas y registrar entregas: existe una lista priorizada con finalistas definidos (CU-06), en el marco de la auditoría (CU-07).<br>▪ Para publicar comunicados: existe al menos un grupo de destinatarios (todos los postulantes o los finalistas). |
| **Proceso** | **Flujo principal A: notificar a los finalistas.**<br>1. El Administrador accede a la gestión de finalistas desde la auditoría de expedientes (CU-07).<br>2. El sistema muestra los finalistas de la lista priorizada (cantidad por definir con la DUBSS).<br>3. El Administrador selecciona los finalistas a notificar (todos o un subconjunto) y define el plazo, el lugar y horario de entrega (por definir con la DUBSS) y los documentos físicos que deben presentar.<br>4. El Administrador confirma el envío.<br>5. El sistema envía una notificación push a cada finalista con el plazo y los documentos requeridos, y guarda el aviso en su bandeja de la aplicación.<br>6. El sistema registra el envío (destinatarios, fecha y hora) en la bitácora (CU-25). |

| Caso de Uso | CU-21. Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos |
|---|---|

**Flujo principal B: registrar la entrega de documentación física.**

1. El Administrador busca al finalista por CI o nombre.
2. El sistema muestra el expediente del finalista y el estado de su entrega (pendiente o recibida).
3. El Administrador marca la documentación como recibida, indicando la fecha de recepción (por defecto, la fecha actual) y, opcionalmente, observaciones.
4. El sistema guarda el registro y habilita la continuación de la auditoría del expediente (CU-07).

**Flujo principal C: publicar comunicados y avisos.**

1. El Administrador selecciona «Nuevo comunicado».
2. El Administrador ingresa el título y el contenido, y elige los destinatarios (todos los postulantes o un grupo, por ejemplo los finalistas).
3. El sistema muestra una vista previa y solicita confirmación.
4. El Administrador confirma la publicación.
5. El sistema publica el comunicado, envía una notificación push a los destinatarios, lo deja consultable en la aplicación y registra la acción en la bitácora.

**Flujos alternativos.**

- A1 (flujo A, paso 5): si un finalista no tiene un dispositivo registrado para notificaciones push, el aviso queda disponible en su bandeja y se muestra al ingresar al sistema.
- A2 (flujo A, paso 5): si el envío push falla para algún destinatario, el sistema lo marca como fallido y permite reintentar el envío solo a esos destinatarios.
- A3 (flujo B, paso 3): si el finalista entrega la documentación de forma incompleta, el Administrador registra la observación y puede devolver la postulación (CU-19).
- A4 (flujo B, paso 3): si vence el plazo sin que el finalista entregue la documentación, el Administrador puede archivar o rechazar la postulación (CU-20).
- A5 (flujo B, paso 3): si el Administrador corrige una fecha de recepción ya registrada, el sistema conserva el valor anterior en la bitácora.
- A6 (flujo C, paso 2): si el título, el contenido o los destinatarios están vacíos, el sistema no permite la publicación e indica los campos faltantes.

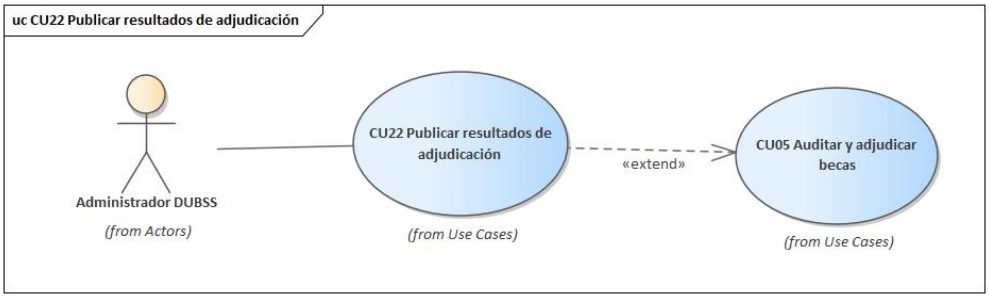
| Caso de Uso | CU-21. Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos |
|---|---|
| **Postcondición** | ▪ Los finalistas notificados reciben el aviso con el plazo y los documentos requeridos.<br>▪ La recepción de la documentación física queda registrada con su fecha.<br>▪ Los comunicados publicados quedan consultables por los destinatarios desde la aplicación.<br>▪ Las acciones quedan registradas en la bitácora; esta funcionalidad no modifica el estado de la postulación. |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ Datos ingresados no válidos o incompletos.<br>▪ Servicio de Firebase Cloud Messaging no disponible.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.22. CU-22 Publicar resultados de adjudicación*

**Figura 47**

*Diagrama de contexto del CU-22*




*Nota.* Elaboración propia.

**Tabla 62**

*Detalle del CU-22: Publicar resultados de adjudicación*

| **Caso de Uso** | **CU-22. Publicar resultados de adjudicación** |
|---|---|
| **Propósito** | Dar a conocer de forma oficial y transparente los resultados de la adjudicación de la Beca Comedor de una convocatoria y poner fin al proceso de selección. |
| **Descripción** | Este caso de uso describe cómo el Administrador DUBSS, una vez decididas todas las adjudicaciones, publica la lista oficial de adjudicados de la convocatoria. Al publicar, el sistema registra la publicación y notifica a cada postulante su resultado mediante notificación push. La lista publicada es consultable por los postulantes y, desde ese momento, las adjudicaciones no se modifican sin dejar registro en la bitácora. Extiende el CU-05. Respalda el RF-17. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01) con el rol Administrador DUBSS.<br>▪ Todas las adjudicaciones de los finalistas de la convocatoria están decididas (CU-05).<br>▪ Los resultados de la convocatoria no han sido publicados previamente. |
| **Proceso** | **Flujo principal.**<br>1. El Administrador DUBSS selecciona la opción «Publicar resultados» de la convocatoria.<br>2. El sistema obtiene y muestra las adjudicaciones de la convocatoria (adjudicados y no adjudicados) con el puntaje final de cada postulante.<br>3. El Administrador DUBSS revisa los resultados y confirma la publicación.<br>4. El sistema verifica que no existan adjudicaciones pendientes de decisión.<br>5. El sistema registra la publicación con el usuario que la realiza y la fecha, y deja constancia en la bitácora.<br>6. El sistema crea una notificación para cada postulante finalista con su resultado y le envía una notificación push.<br>7. El sistema confirma la publicación; la lista de adjudicados queda consultable por los postulantes (CU-04). |

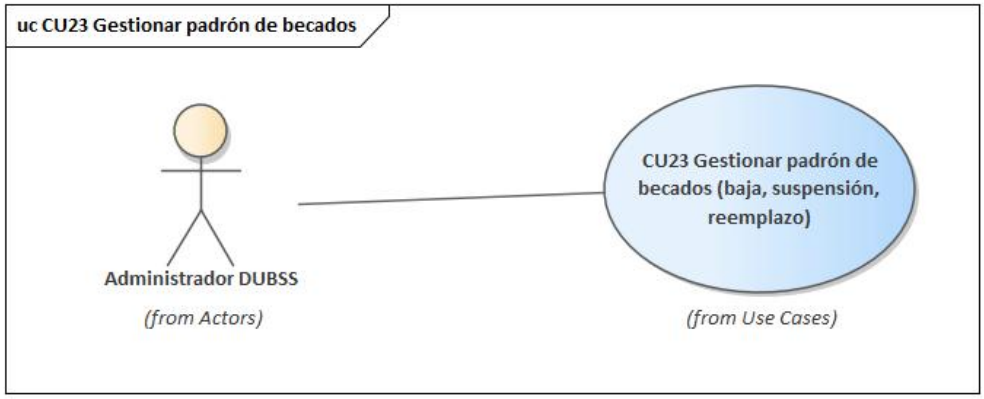
| Caso de Uso | CU-22. Publicar resultados de adjudicación |
|---|---|
| | **Flujos alternativos.**<br>▪ A1 (paso 4): si existen adjudicaciones pendientes de decisión, el sistema no permite publicar e indica cuántas faltan.<br>▪ A2 (paso 1): si los resultados de la convocatoria ya fueron publicados, el sistema no permite una nueva publicación y muestra la existente.<br>▪ A3 (paso 6): si falla el envío de alguna notificación push, la notificación queda registrada y el postulante puede ver su resultado al consultar su postulación (CU-04).<br>▪ A4 (paso 3): si el Administrador DUBSS cancela la operación, no se publica ningún resultado. |
| **Postcondición** | ▪ Los resultados de la convocatoria quedan publicados y registrados con su responsable y fecha.<br>▪ Cada postulante finalista es notificado de su resultado y los adjudicados conforman el padrón de becados de la gestión (CU-23).<br>▪ Las adjudicaciones de la convocatoria no se modifican sin dejar registro en la bitácora (CU-25). |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ Adjudicaciones pendientes de decisión.<br>▪ Error al enviar las notificaciones push mediante Firebase Cloud Messaging.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.23. CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo)*

**Figura 48**

*Diagrama de contexto del CU-23*




*Nota.* Elaboración propia.

**Tabla 63**

*Detalle del CU-23: Gestionar padrón de becados (baja, suspensión, reemplazo)*

| **Caso de Uso** | **CU-23. Gestionar padrón de becados (baja, suspensión, reemplazo)** |
|---|---|
| **Propósito** | Mantener actualizado el padrón de becados vigentes de la gestión, registrando bajas, suspensiones y reemplazos, del cual dependen las planillas de asistencia. |
| **Descripción** | Este caso de uso describe cómo el Administrador DUBSS consulta el padrón de becados de una convocatoria y registra sobre él las bajas, suspensiones y reemplazos que ocurren durante la gestión. Cada cambio actualiza el estado de la beca y queda registrado como un movimiento con su motivo, responsable y fecha. En un reemplazo, el cupo liberado se asigna al siguiente postulante priorizado. Los cambios se reflejan en las planillas de asistencia generadas posteriormente (CU-09). Respalda el RF-18. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01) con el rol Administrador DUBSS.<br>▪ Los resultados de adjudicación de la convocatoria fueron publicados (CU-22) y existen becas registradas. |

| Caso de Uso | **CU-23. Gestionar padrón de becados (baja, suspensión, reemplazo)** |
|---|---|
| **Proceso** | **Flujo principal A: consultar el padrón.**<br>1. El Administrador DUBSS selecciona la opción «Padrón de becados» y la convocatoria.<br>2. El sistema muestra las becas de la convocatoria con el nombre, CI y carrera del becado y el estado de la beca (vigente, suspendida o baja).<br>**Flujo principal B: dar de baja o suspender una beca.**<br>1. El Administrador DUBSS selecciona una beca vigente y la opción «Dar de baja» o «Suspender».<br>2. El sistema solicita el motivo.<br>3. El Administrador DUBSS registra el motivo y confirma.<br>4. El sistema actualiza el estado de la beca a «baja» o «suspendida», según corresponda.<br>5. El sistema registra el movimiento (tipo, motivo, usuario y fecha) y deja constancia en la bitácora.<br>**Flujo principal C: reemplazar un cupo liberado.**<br>1. El Administrador DUBSS selecciona una beca dada de baja y la opción «Reemplazar».<br>2. El sistema muestra al siguiente postulante de la lista priorizada disponible para ocupar el cupo.<br>3. El Administrador DUBSS confirma el reemplazo.<br>4. El sistema crea una nueva beca vigente para el postulante reemplazante.<br>5. El sistema registra el movimiento de reemplazo, vinculando la beca liberada con la beca reemplazante, y deja constancia en la bitácora.<br>**Flujos alternativos.**<br>▪ A1 (flujo B, paso 3): si no se registra el motivo, el sistema no permite completar la operación.<br>▪ A2 (flujo B, paso 1): si la beca no está vigente, el sistema no permite darla de baja ni suspenderla.<br>▪ A3 (flujo C, paso 1): si la beca seleccionada no ha sido dada de baja, el sistema no permite el reemplazo.<br>▪ A4 (flujo C, paso 2): si no existe un postulante disponible para ocupar el cupo, el sistema lo informa y no realiza el reemplazo.<br>▪ A5 (flujos B y C): si el Administrador DUBSS cancela la operación, el padrón no se modifica. |
| **Postcondición** | ▪ El padrón refleja el estado actualizado de cada beca y cada cambio queda registrado como movimiento y en la bitácora.<br>▪ Las planillas de asistencia que se generen posteriormente (CU-09) consideran solo las becas vigentes. |

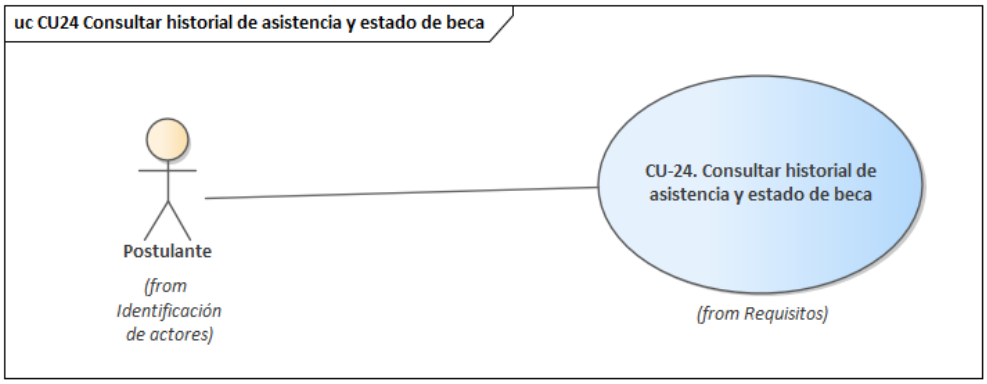
| Caso de Uso | **CU-23. Gestionar padrón de becados (baja, suspensión, reemplazo)** |
|---|---|
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ Operación no permitida para el estado actual de la beca.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.24. CU-24 Consultar historial de asistencia y estado de beca*

**Figura 49**

*Diagrama de contexto del CU-24*




*Nota.* Elaboración propia.

**Tabla 64**

*Detalle del CU-24: Consultar historial de asistencia y estado de beca*

| Caso de Uso | **CU-24. Consultar historial de asistencia y estado de beca** |
|---|---|
| **Propósito** | Permitir al Postulante adjudicado (becado) consultar su historial de asistencia al comedor y el estado vigente de su beca. |
| **Descripción** | Este caso de uso describe cómo el becado consulta, desde el portal web o la aplicación móvil, las fechas de asistencia registradas en la gestión vigente y el estado de su beca (vigente, suspendida o dada de baja) con su motivo, de modo que pueda verificar el cumplimiento de los requisitos de asistencia y conservar su beneficio. Es una consulta de solo lectura que utiliza los datos registrados en el CU-10 y en el padrón de becados (CU-23). Respalda el RF-19. |

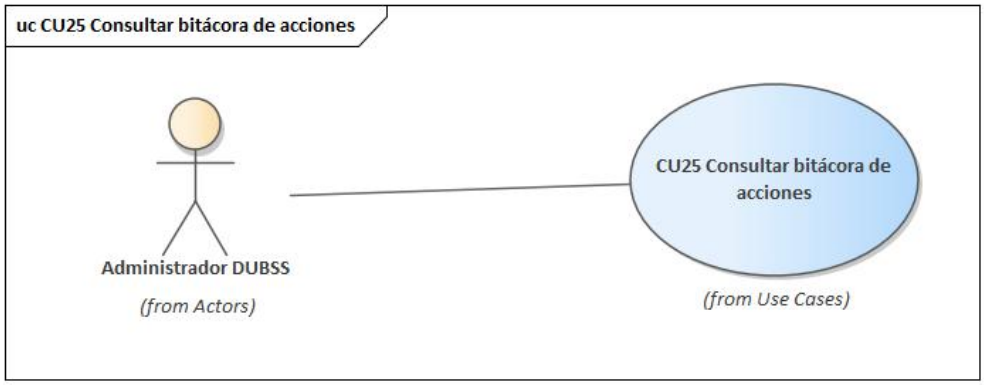
| **Caso de Uso** | **CU-24. Consultar historial de asistencia y estado de beca** |
|---|---|
| **Actores** | Postulante (becado, es decir, Postulante adjudicado). |
| **Actor Iniciador** | Postulante (becado). |
| **Precondición** | ▪ El Postulante está autenticado en el sistema (CU-01).<br>▪ La postulación del Postulante fue adjudicada y figura en el padrón de becados. |
| **Proceso** | **Flujo principal.**<br>1. El becado accede a la opción «Mi beca».<br>2. El sistema verifica que el usuario figure en el padrón de becados de la gestión.<br>3. El sistema muestra el estado de la beca (vigente, suspendida o dada de baja), la gestión académica y la fecha de adjudicación.<br>4. El sistema muestra el historial de asistencia de la gestión vigente: las fechas de asistencia registradas en el comedor y el total acumulado.<br>5. El becado revisa la información y, opcionalmente, filtra el historial por rango de fechas.<br>**Flujos alternativos.**<br>▪ A1 (paso 2): si el usuario no figura como becado, el sistema informa que no cuenta con una beca vigente y lo redirige a la consulta del estado de su postulación (CU-04).<br>▪ A2 (paso 4): si aún no existen asistencias registradas, el sistema muestra un mensaje informativo.<br>▪ A3 (paso 3): si la beca está suspendida o dada de baja, el sistema muestra el estado y el motivo registrado por el Administrador (CU-23); el historial de asistencia se conserva y sigue siendo consultable.<br>▪ A4 (paso 1): si el usuario intenta acceder a la información de otro becado, el sistema deniega el acceso. |
| **Postcondición** | ▪ El becado visualiza su historial de asistencia y el estado de su beca; la consulta no modifica ningún dato del sistema. |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario solo puede acceder a su propia información).<br>▪ Usuario sin beca adjudicada.<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.25. CU-25 Consultar bitácora de acciones*

**Figura 50**

*Diagrama de contexto del CU-25*




*Nota.* Elaboración propia.

**Tabla 65**

*Detalle del CU-25: Consultar bitácora de acciones*

| **Caso de Uso** | **CU-25. Consultar bitácora de acciones** |
|---|---|
| **Propósito** | Permitir al Administrador DUBSS consultar el registro de las acciones relevantes realizadas en el sistema, para garantizar la trazabilidad de las decisiones y facilitar las auditorías. |
| **Descripción** | Este caso de uso describe cómo el Administrador DUBSS consulta la bitácora del sistema, en la que se registran automáticamente las acciones relevantes (auditoría, devolución, adjudicación, publicación, cambios en el padrón y cambios de roles) con el usuario, la acción, la entidad o expediente afectado, la fecha y la hora. La bitácora puede filtrarse por usuario, tipo de acción y rango de fechas. Es una consulta de solo lectura: los registros no pueden editarse ni eliminarse. Respalda el RF-20. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01) con el rol Administrador DUBSS. |

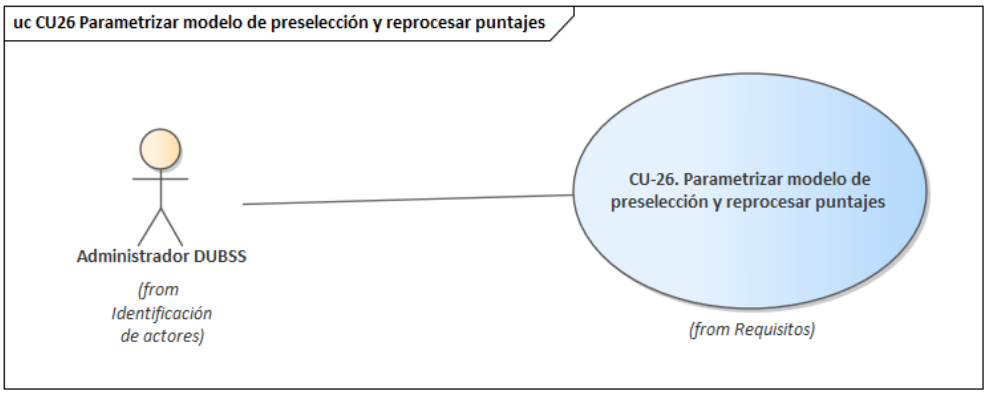
| **Caso de Uso** | **CU-25. Consultar bitácora de acciones** |
|---|---|
| **Proceso** | **Flujo principal.**<br>1. El Administrador DUBSS selecciona la opción «Bitácora de acciones».<br>2. El sistema muestra los registros de la bitácora ordenados del más reciente al más antiguo y paginados, con la fecha y hora, el usuario, la acción, la entidad o expediente afectado y el detalle de cada registro.<br>3. El Administrador DUBSS ingresa los filtros de búsqueda: usuario, tipo de acción y rango de fechas.<br>4. El sistema muestra los registros que cumplen los filtros.<br>**Flujos alternativos.**<br>▪ A1 (paso 4): si ningún registro cumple los filtros, el sistema muestra un mensaje informativo.<br>▪ A2 (paso 3): si la fecha final del rango es anterior a la inicial, el sistema lo indica y permite corregirla.<br>▪ A3 (paso 2): las acciones ejecutadas por el sistema o por el módulo de IA se muestran sin usuario asociado. |
| **Postcondición** | ▪ El Administrador DUBSS visualiza los registros de la bitácora; la consulta no modifica ningún dato del sistema. |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ Error de conexión con el servidor o la base de datos. |

*Nota.* Elaboración propia.

### *5.4.26. CU-26 Parametrizar modelo de preselección y reprocesar puntajes*

**Figura 51**

*Diagrama de contexto del CU-26*




*Nota.* Elaboración propia.

**Tabla 66**

*Detalle del CU-26: Parametrizar modelo de preselección y reprocesar puntajes*

| **Caso de Uso** | **CU-26. Parametrizar modelo de preselección y reprocesar puntajes** |
|---|---|
| **Propósito** | Permitir al Administrador DUBSS ajustar los criterios y ponderaciones del modelo de preselección y reprocesar los puntajes de las postulaciones de una convocatoria. |
| **Descripción** | Este caso de uso describe cómo el Administrador DUBSS consulta y modifica los parámetros del modelo de IA (criterios y ponderaciones de las variables socioeconómicas y académicas), los guarda como una nueva versión y, si lo decide, reprocesa los puntajes de una convocatoria. Cada recálculo conserva el puntaje anterior para fines de trazabilidad. Extiende al CU-03. Respalda el RF-21. |
| **Actores** | Administrador DUBSS. |
| **Actor Iniciador** | Administrador DUBSS. |
| **Precondición** | ▪ El Administrador DUBSS está autenticado en el sistema (CU-01) y cuenta con privilegios de administración.<br>▪ El módulo de IA está disponible.<br>▪ Para reprocesar: la convocatoria seleccionada tiene postulaciones precalificadas (CU-03). |

| **Caso de Uso** | **CU-26. Parametrizar modelo de preselección y reprocesar puntajes** |
|---|---|
| **Proceso** | **Flujo principal.**<br>1. El Administrador accede a la opción «Parámetros del modelo».<br>2. El sistema muestra los criterios y ponderaciones vigentes (variables: ingresos familiares, dependientes, procedencia, carrera, patrimonio, estado de vulnerabilidad y promedio académico), con su versión y fecha de vigencia.<br>3. El Administrador modifica los criterios o ponderaciones necesarios (valores y rangos por definir con la DUBSS).<br>4. El sistema valida los valores ingresados (rangos permitidos y consistencia entre ponderaciones).<br>5. El Administrador guarda los cambios y el sistema crea una nueva versión de los parámetros, conservando las versiones anteriores.<br>6. El Administrador selecciona la convocatoria y solicita «Reprocesar puntajes».<br>7. El sistema muestra el resumen del reprocesamiento (convocatoria y cantidad de postulaciones afectadas) y solicita confirmación.<br>8. El Administrador confirma.<br>9. El sistema ejecuta el reprocesamiento en lote, invocando el cálculo del CU-03 para cada postulación, y muestra el avance.<br>10. El sistema registra la fecha de cada recálculo y conserva el puntaje anterior de cada postulación.<br>11. El sistema muestra la lista priorizada actualizada (CU-06) y registra la acción en la bitácora (CU-25).<br>**Flujos alternativos.**<br>▪ A1 (paso 4): si los valores están fuera de rango o son inconsistentes, el sistema indica los errores y no guarda los parámetros.<br>▪ A2 (paso 5): si el Administrador guarda los parámetros sin reprocesar, la nueva versión se aplica únicamente a las postulaciones que se precalifiquen a partir de ese momento.<br>▪ A3 (paso 6): si ya se publicaron resultados de adjudicación en la convocatoria (CU-22), el sistema advierte o bloquea el reprocesamiento (regla por definir con la DUBSS).<br>▪ A4 (paso 6): si hay un reprocesamiento en curso para la misma convocatoria, el sistema impide lanzar otro de forma simultánea.<br>▪ A5 (paso 9): si el cálculo falla para algunas postulaciones, el sistema continúa con las demás, lista las fallidas y permite reintentarlas.<br>▪ A6 (paso 7): el Administrador puede cancelar el reprocesamiento antes de confirmarlo, sin que se modifique ningún puntaje. |

| **Caso de Uso** | **CU-26. Parametrizar modelo de preselección y reprocesar puntajes** |
|---|---|
| **Postcondición** | ▪ Los nuevos parámetros quedan guardados como una versión vigente, y las versiones anteriores se conservan.<br>▪ Los puntajes reprocesados quedan registrados con su fecha de recálculo y el puntaje anterior se conserva; la lista priorizada refleja el nuevo orden.<br>▪ La acción queda registrada en la bitácora. |
| **Excepciones** | ▪ Autenticación inválida o sesión expirada.<br>▪ Falta de permisos suficientes (el usuario no tiene el rol Administrador DUBSS).<br>▪ Parámetros ingresados no válidos.<br>▪ Módulo de IA no disponible durante el reprocesamiento.<br>▪ Error de conexión con el servidor o la base de datos. |

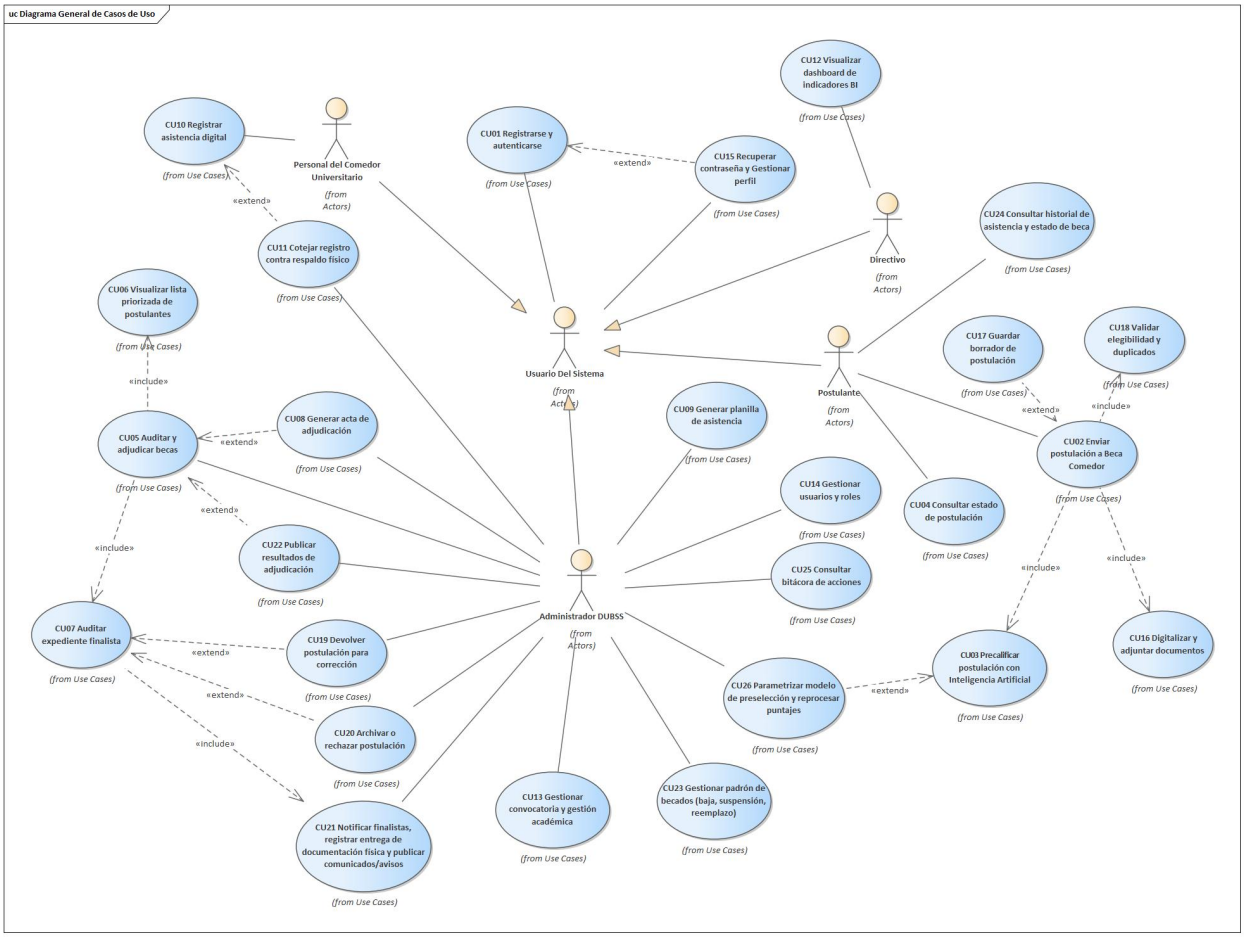
*Nota.* Elaboración propia.

## 5.5. Diagrama General de Casos de Uso

La Figura 52 presenta el diagrama general de casos de uso del sistema, que integra los actores identificados y los casos de uso priorizados y detallados en las secciones anteriores.

**Figura 52**

*Diagrama general de casos de uso del Sistema Web y Móvil de Postulación y Preselección de la Beca Comedor*




*Nota.* Elaboración propia.

Capítulo 6

# Análisis

# Capítulo 6: Análisis

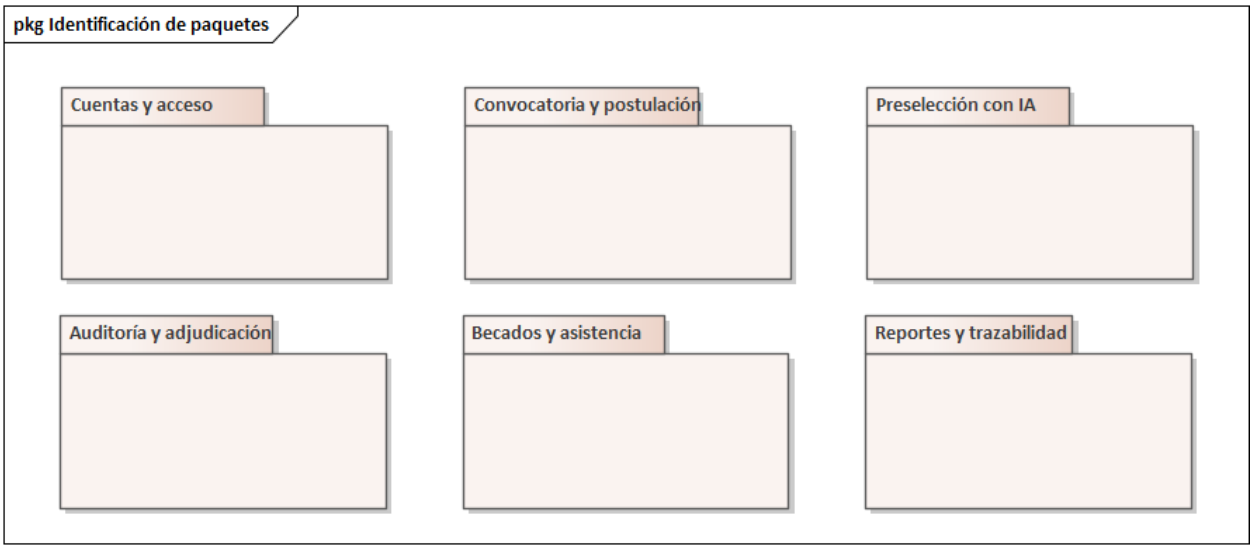
## 6.1. Análisis de la Arquitectura

### *6.1.1. Identificación de Paquetes*

Con el fin de organizar los veintiséis casos de uso del sistema, se agruparon por afinidad funcional en seis paquetes de análisis, de modo que cada uno concentre las responsabilidades de un área del proceso de la Beca Comedor. La Figura 53 presenta los paquetes identificados.

**Figura 53**

*Identificación de paquetes*




*Nota.* Elaboración propia.

Los paquetes identificados se describen a continuación:

- **Cuentas y acceso:** reúne el registro, la autenticación, la recuperación de contraseña, la gestión de perfil y la administración de usuarios y roles.
- **Convocatoria y postulación:** comprende la gestión de convocatorias y el proceso que realiza el postulante: envío de la postulación, digitalización de documentos, validación de elegibilidad, guardado de borradores y consulta del estado.

- **Preselección con IA:** contiene la precalificación automática de las postulaciones mediante Inteligencia Artificial y la parametrización del modelo.
- **Auditoría y adjudicación:** agrupa la lista priorizada de postulantes, la auditoría de expedientes finalistas, la devolución o archivo de postulaciones, la notificación a finalistas y la adjudicación con publicación de resultados.
- **Becados y asistencia:** abarca la gestión del padrón de becados, la generación de planillas, el registro y cotejo de la asistencia y la consulta del historial por el becado.
- **Reportes y trazabilidad:** incluye el dashboard de indicadores de Business Intelligence y la consulta de la bitácora de acciones.

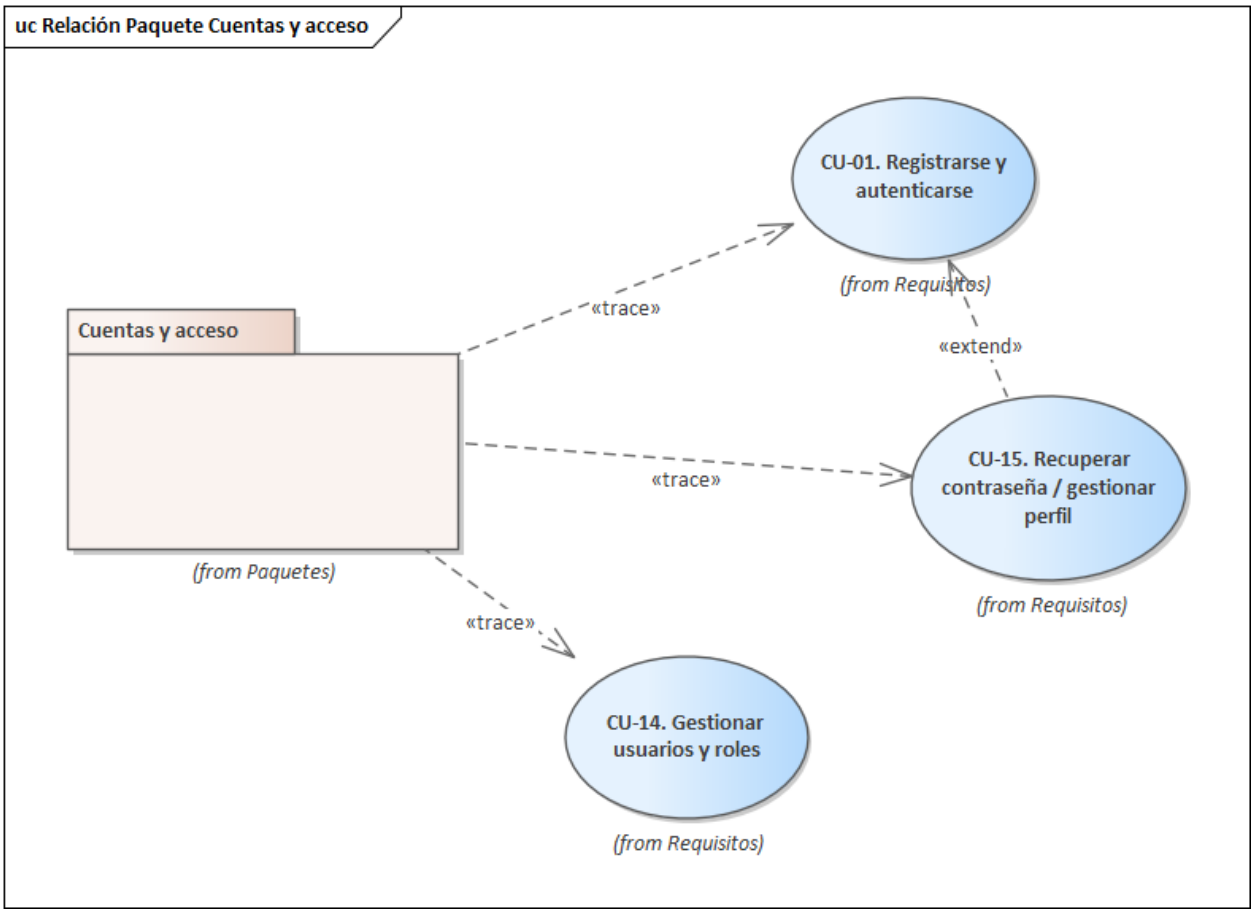
### *6.1.2. Relación de Paquetes*

Una vez identificados los paquetes, se establece la relación de cada uno con los casos de uso que contiene. Las Figuras 54, 55, 56, 57, 58 y 59 muestran, para cada paquete, sus casos de uso asociados mediante dependencias de trazabilidad («trace») y las relaciones de inclusión («include») y extensión («extend») entre ellos.

El paquete Cuentas y Acceso se relaciona, mediante dependencias de trazabilidad, con los casos de uso CU-01, CU-14 y CU-15; además, CU-15 extiende a CU-01, como se muestra en la Figura 54.

**Figura 54**

*Relación del paquete Cuentas y Acceso*

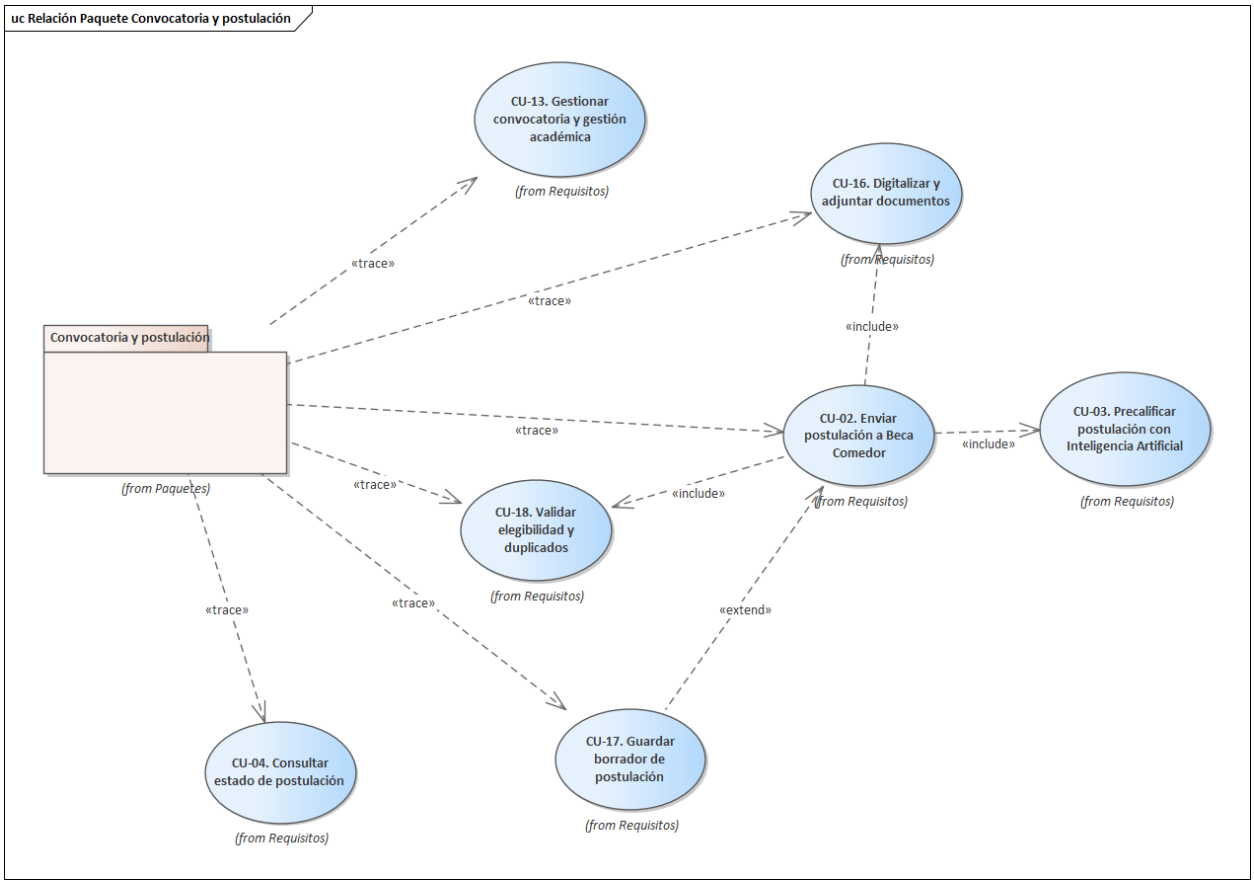



*Nota.* Elaboración propia.

El paquete Convocatoria y Postulación se relaciona con los casos de uso CU-02, CU-04, CU-13, CU-16, CU-17 y CU-18. En este paquete, CU-02 incluye a CU-16, CU-18 y CU-03 (este último perteneciente al paquete Preselección con IA), y CU-17 extiende a CU-02, como se muestra en la Figura 55.

**Figura 55**

*Relación del paquete Convocatoria y Postulación*

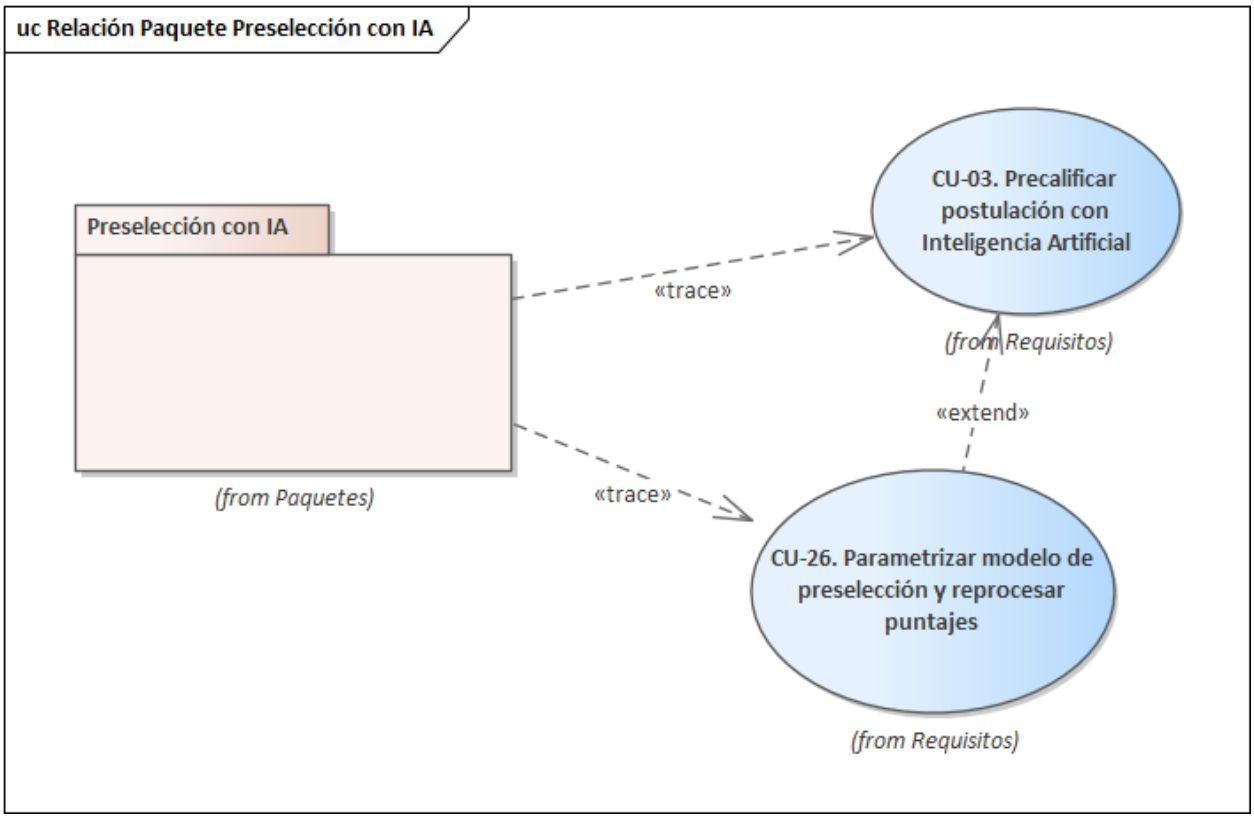



*Nota.* Elaboración propia.

El paquete Preselección con IA se relaciona con los casos de uso CU-03 y CU-26, donde CU-26 extiende a CU-03, como se muestra en la Figura 56.

**Figura 56**

*Relación del paquete Preselección con IA*

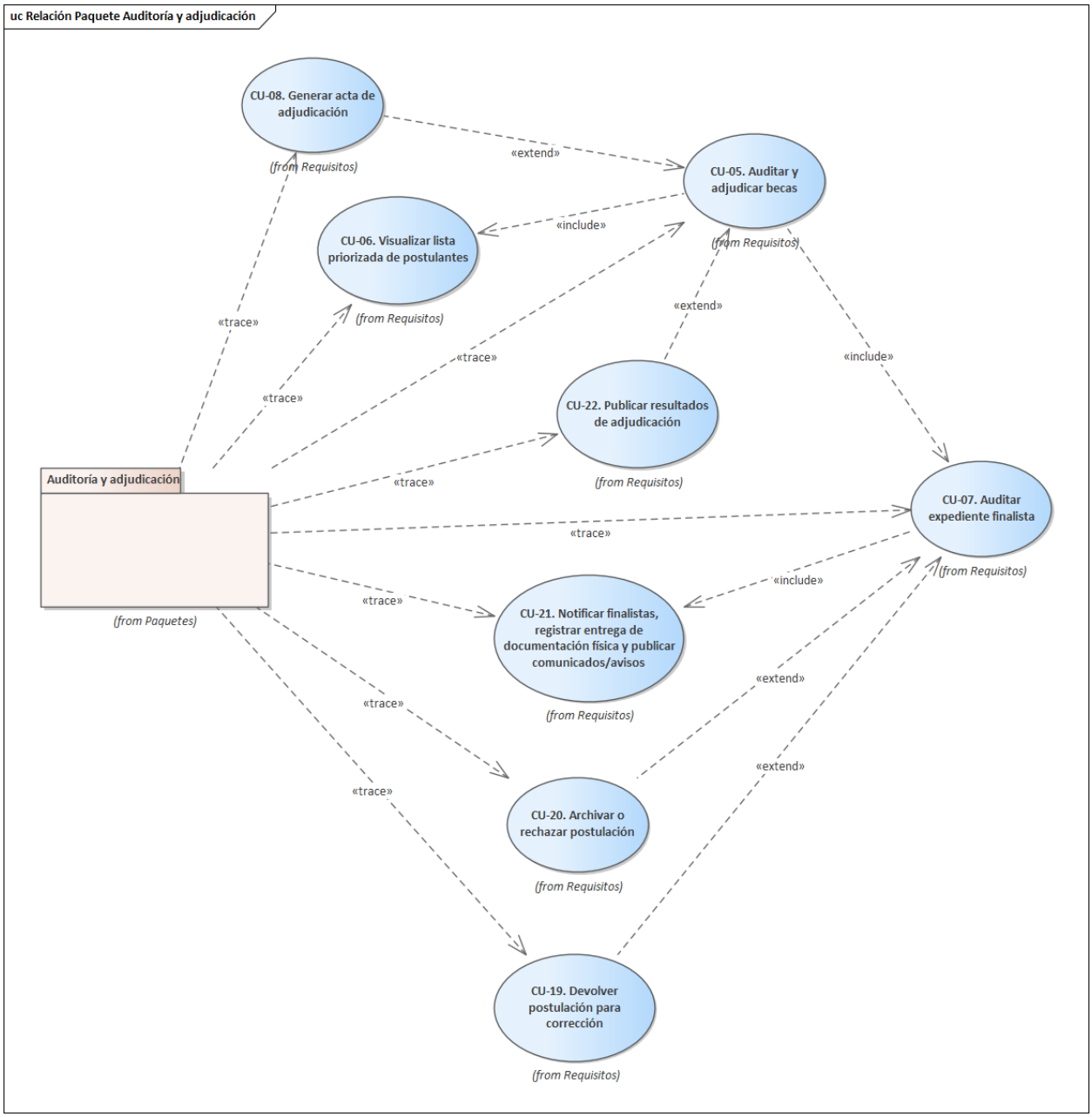



*Nota.* Elaboración propia.

El paquete Auditoría y Adjudicación se relaciona con los casos de uso CU-05, CU-06, CU-07, CU-08, CU-19, CU-20, CU-21 y CU-22. CU-05 incluye a CU-06 y CU-07; CU-07 incluye a CU-21; CU-19 y CU-20 extienden a CU-07; y CU-08 y CU-22 extienden a CU-05, como se muestra en la Figura 57.

**Figura 57**

*Relación del paquete Auditoría y Adjudicación*

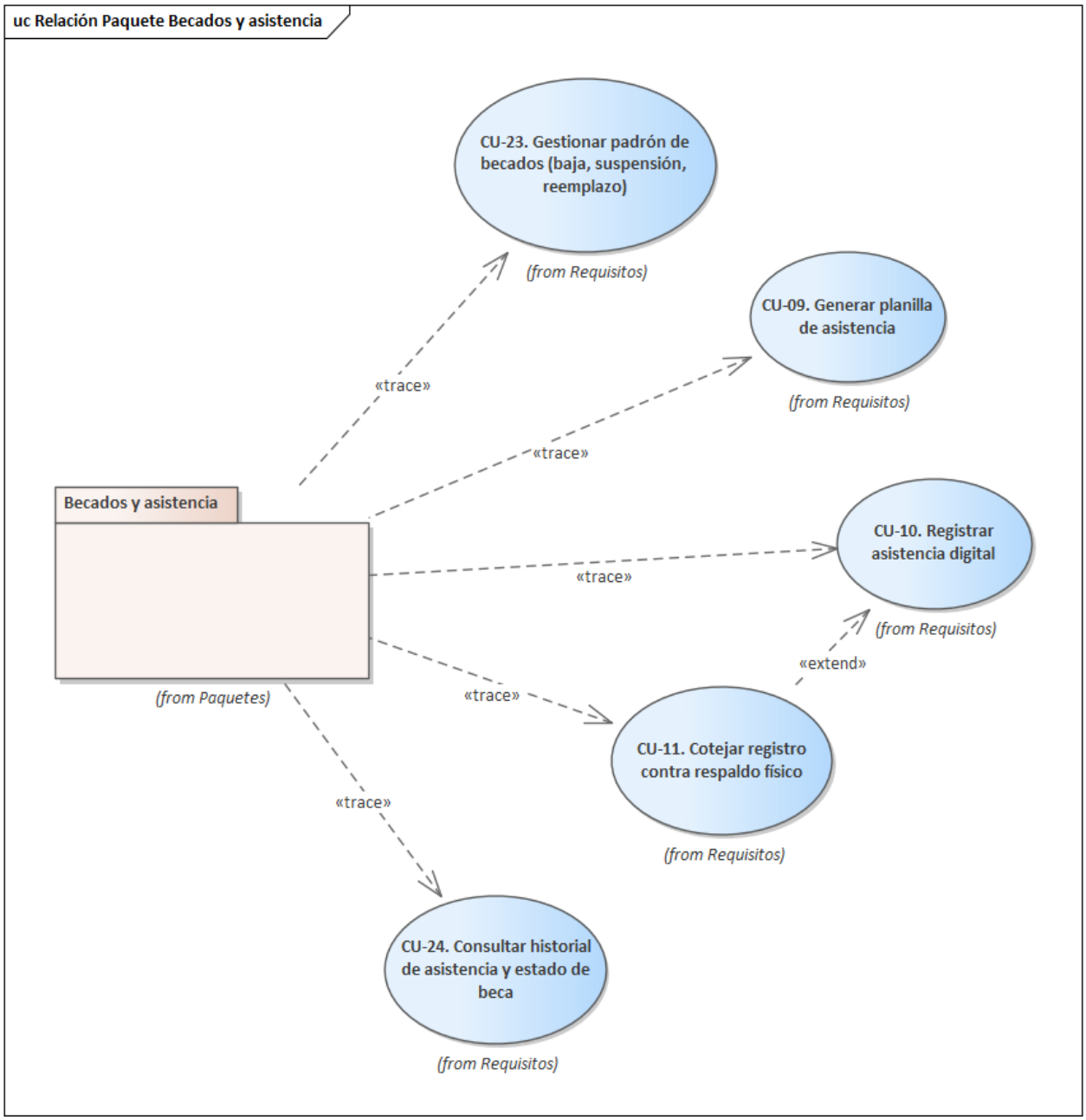



*Nota.* Elaboración propia.

El paquete Becados y Asistencia se relaciona con los casos de uso CU-09, CU-10, CU-11, CU-23 y CU-24, donde CU-11 extiende a CU-10, como se muestra en la Figura 58.

**Figura 58**

*Relación del paquete Becados y Asistencia*

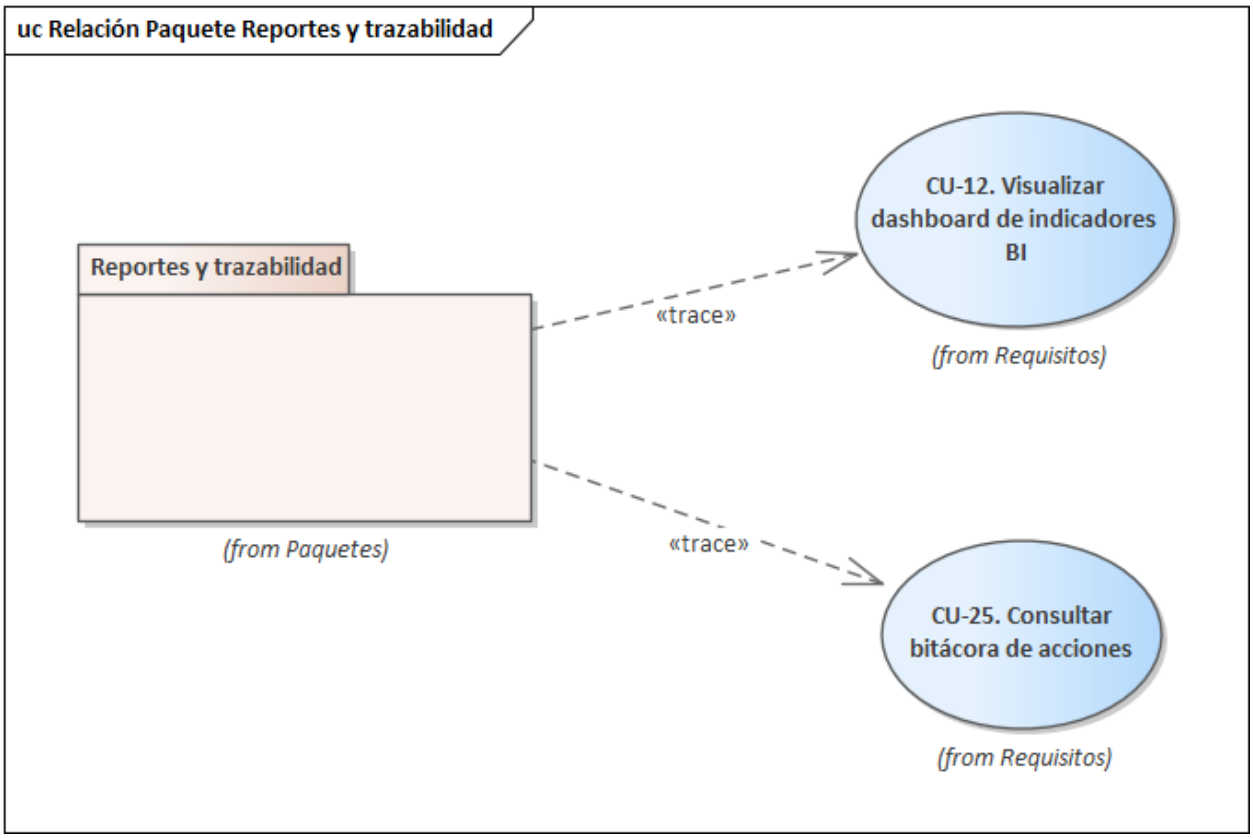



*Nota.* Elaboración propia.

El paquete Reportes y Trazabilidad se relaciona con los casos de uso CU-12 y CU-25, como se muestra en la Figura 59.

**Figura 59**

*Relación del paquete Reportes y Trazabilidad*




*Nota.* Elaboración propia.

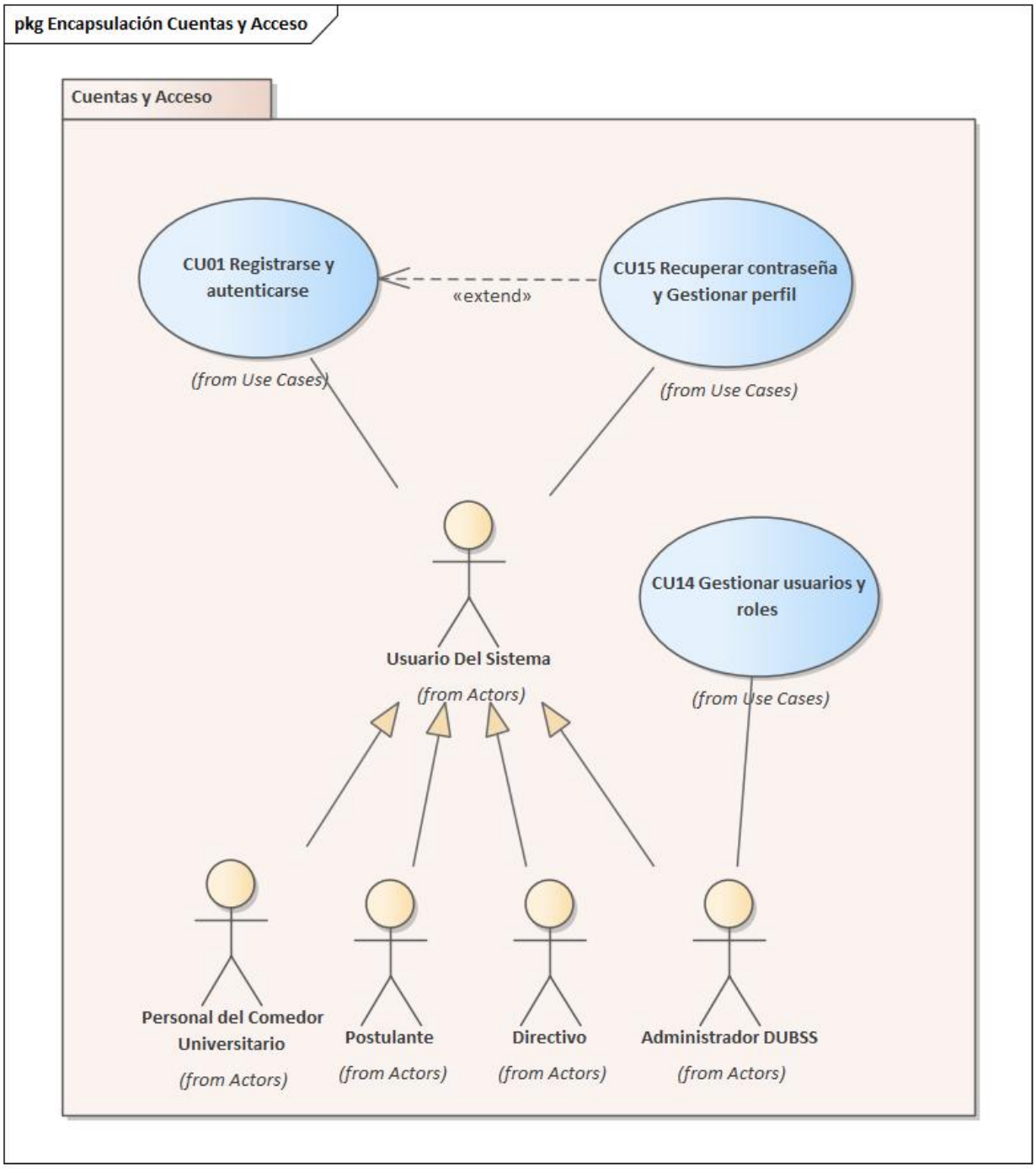
### *6.1.3. Encapsulamiento de Casos de Uso por Paquetes*

Las Figuras 60, 61, 62, 63, 64 y 65 muestran el encapsulamiento de los casos de uso en cada uno de los seis paquetes de análisis del sistema.

El paquete Cuentas y Acceso encapsula los casos de uso CU-01, CU-14 y CU-15, como se muestra en la Figura 60.

**Figura 60**

*Encapsulamiento de casos de uso del paquete Cuentas y Acceso*



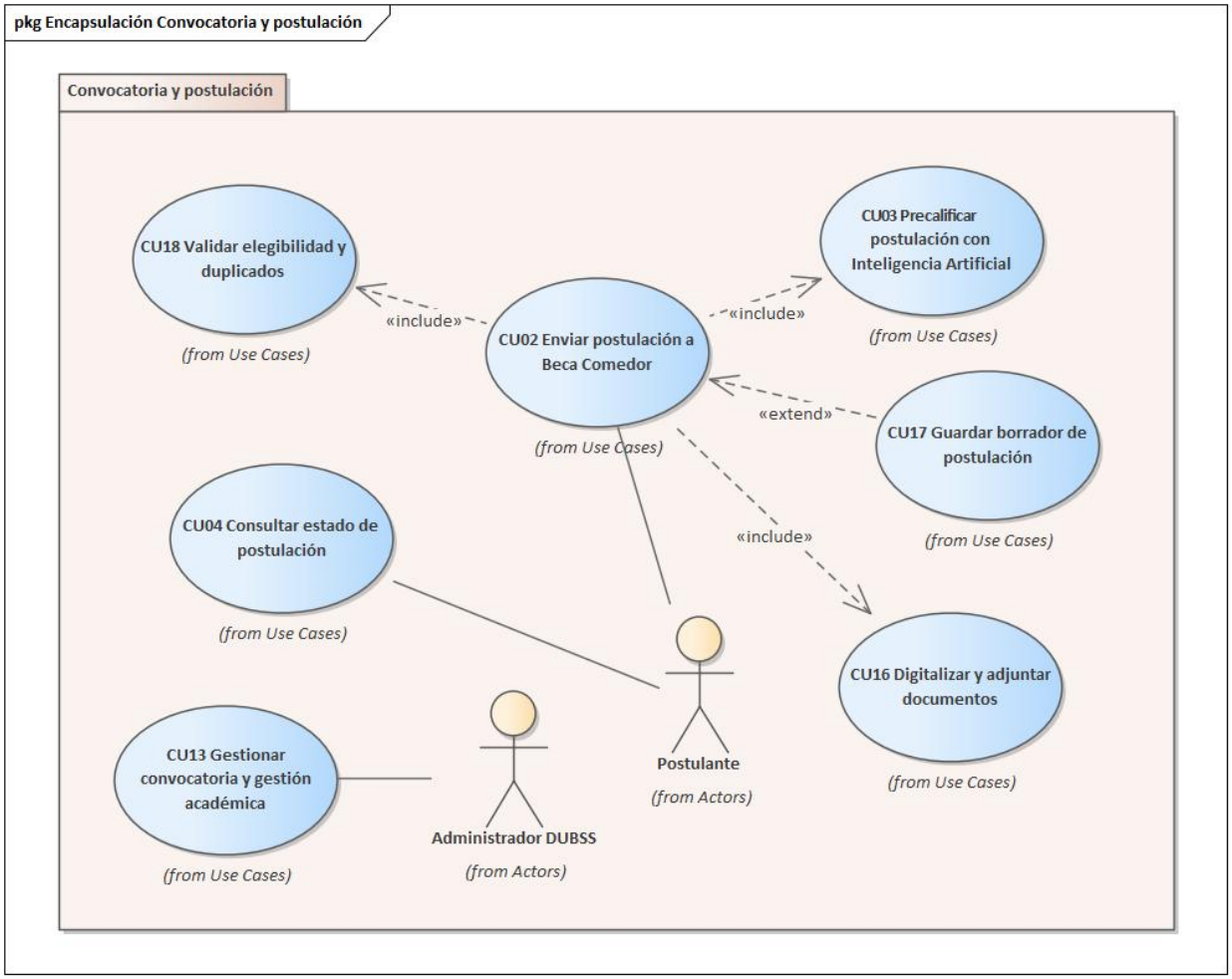

*Nota.* Elaboración propia.

El paquete Convocatoria y Postulación encapsula los casos de uso CU-02, CU-04,

CU-13, CU-16, CU-17 y CU-18, como se muestra en la Figura 61.

**Figura 61**

*Encapsulamiento de casos de uso del paquete Convocatoria y Postulación*

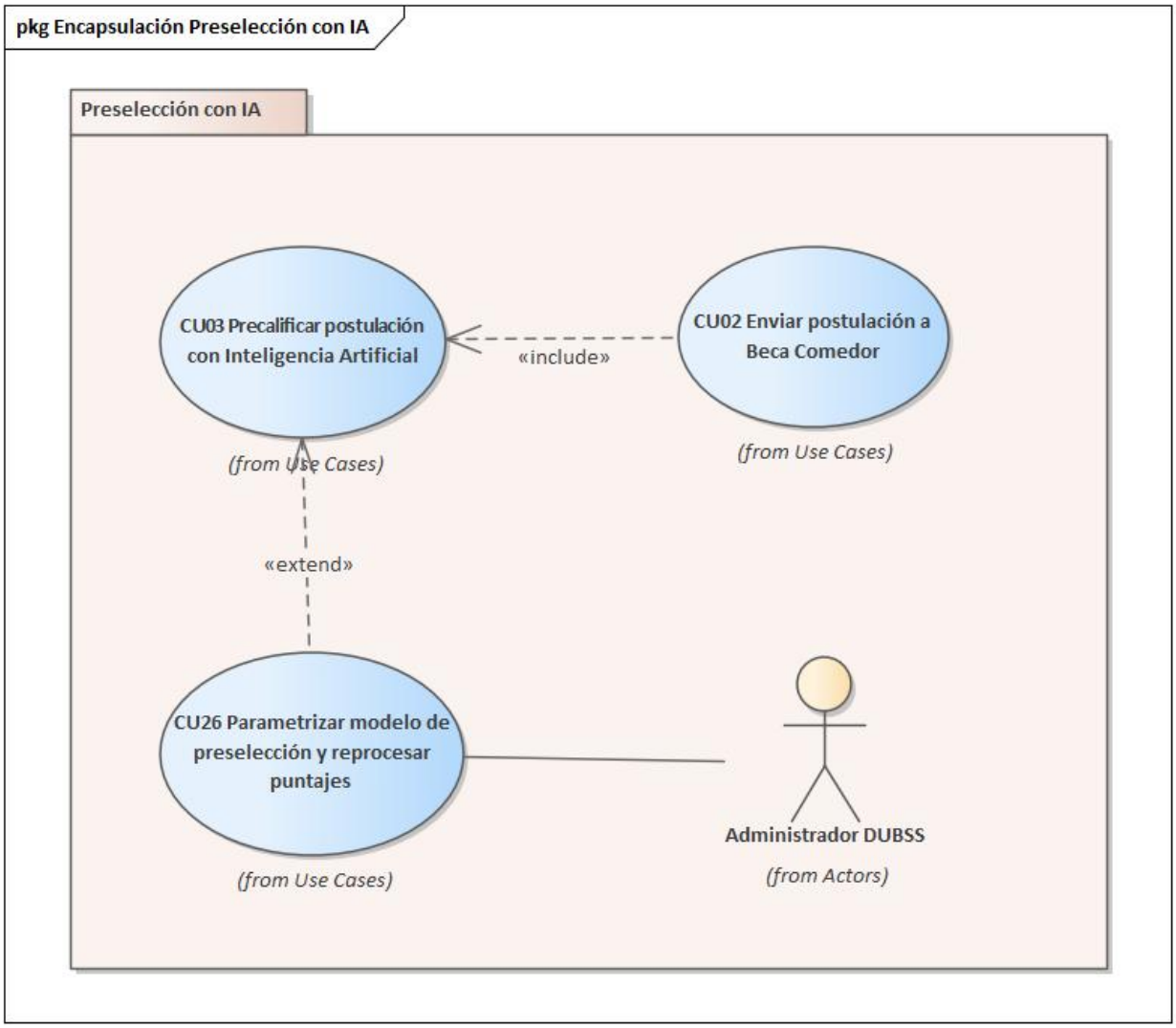



*Nota.* Elaboración propia.

El paquete Preselección con IA encapsula los casos de uso CU-03 y CU-26, como se muestra en la Figura 62.

**Figura 62**

*Encapsulamiento de casos de uso del paquete Preselección con IA*

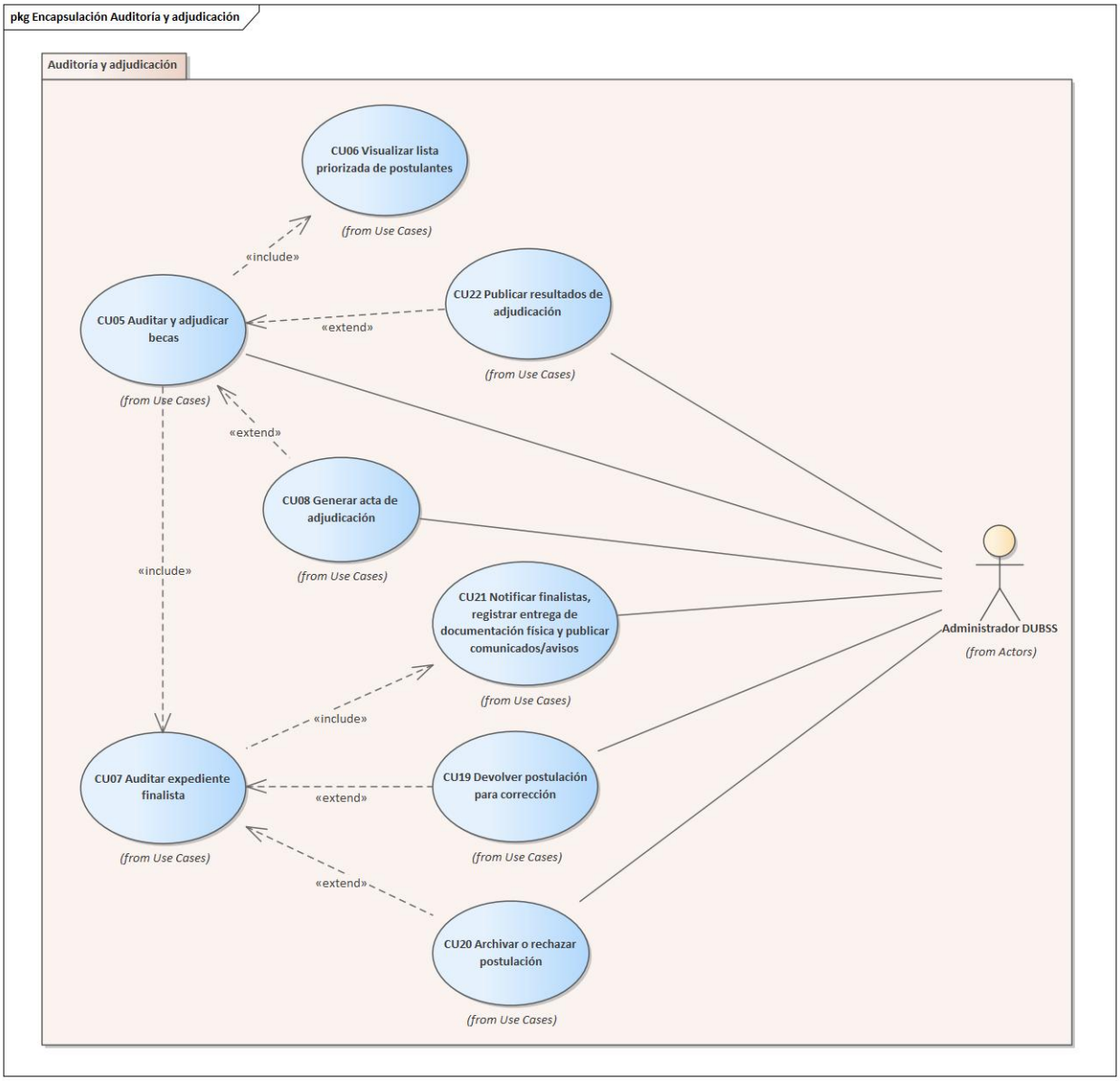



*Nota.* Elaboración propia.

El paquete Auditoría y Adjudicación encapsula los casos de uso CU-05, CU-06, CU-07, CU-08, CU-19, CU-20, CU-21 y CU-22, como se muestra en la Figura 63.

**Figura 63**

*Encapsulamiento de casos de uso del paquete Auditoría y Adjudicación*

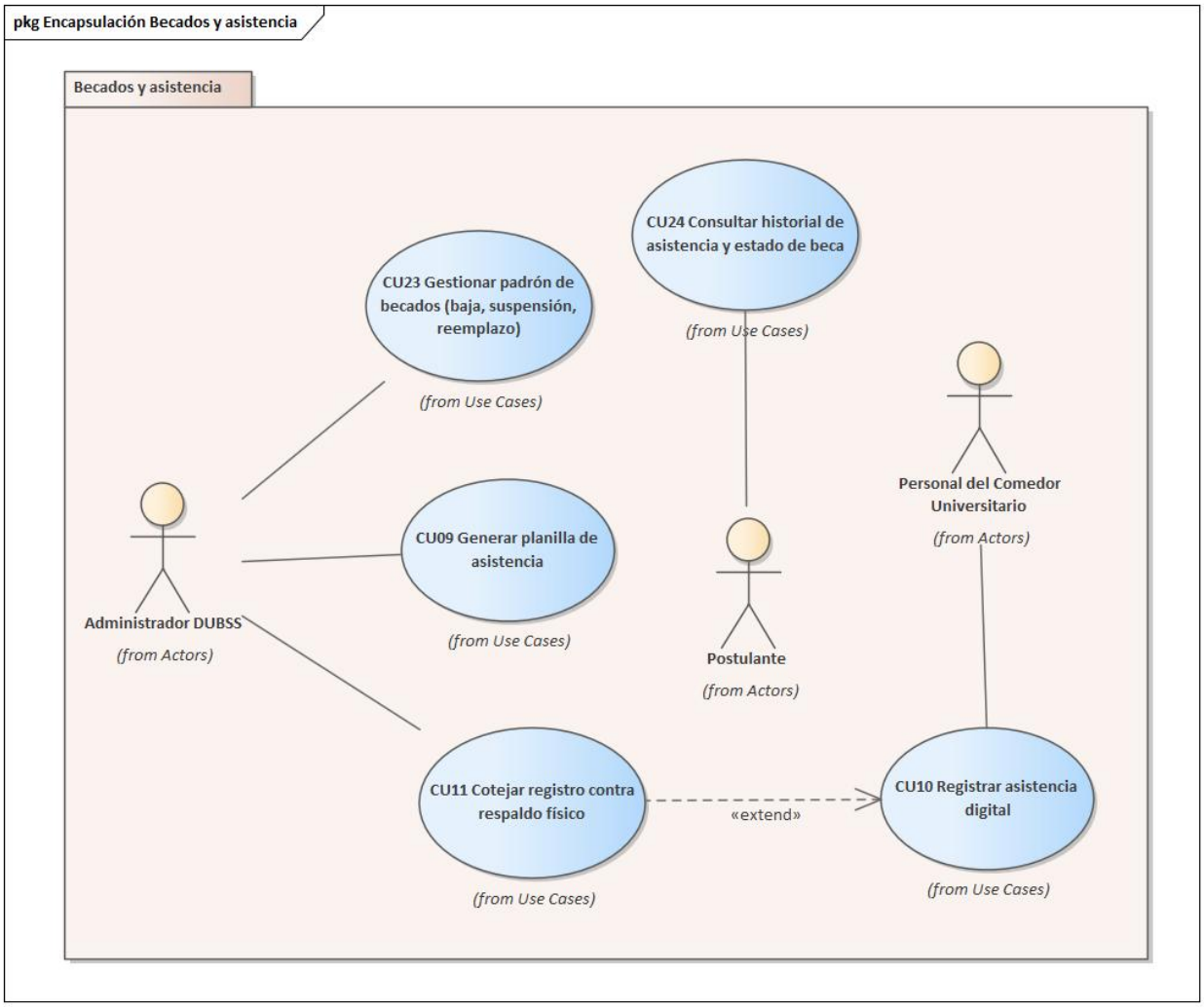



*Nota.* Elaboración propia.

El paquete Becados y Asistencia encapsula los casos de uso CU-09, CU-10, CU-11, CU-23 y CU-24, como se muestra en la Figura 64.

**Figura 64**

*Encapsulamiento de casos de uso del paquete Becados y Asistencia*

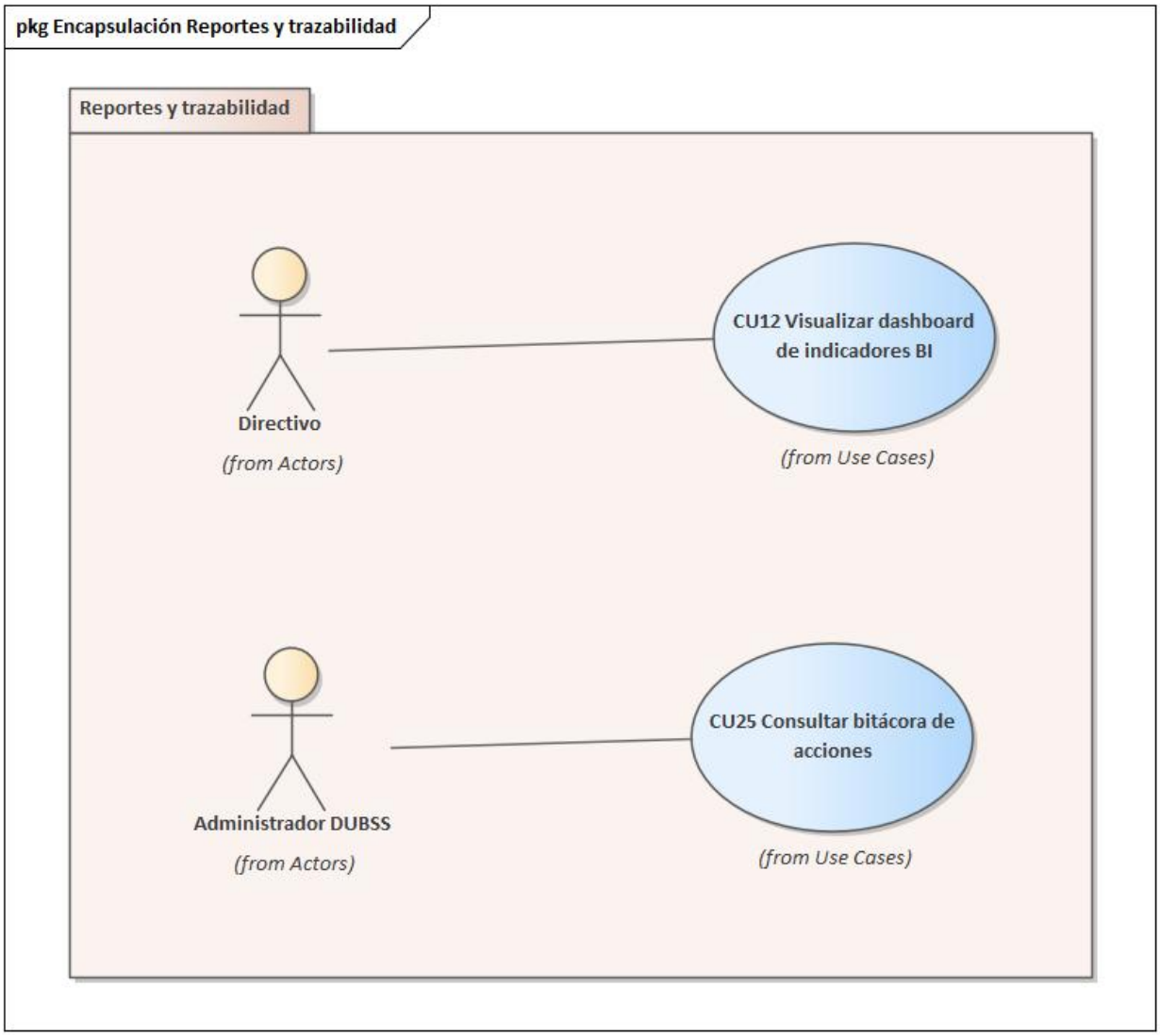



*Nota.* Elaboración propia.

El paquete Reportes y Trazabilidad encapsula los casos de uso CU-12 y CU-25, como se muestra en la Figura 65.

**Figura 65**

*Encapsulamiento de casos de uso del paquete Reportes y Trazabilidad*




*Nota.* Elaboración propia.

## 6.2. Análisis de Casos de Uso

El análisis de cada caso de uso se realizó mediante diagramas de colaboración que representan las clases de análisis que intervienen y los mensajes que intercambian. Siguiendo el Proceso Unificado de Desarrollo de Software, el modelo de análisis distingue tres tipos de clases: las clases de interfaz (*boundary*), que modelan la interacción del actor con el sistema;

las clases de control (*control*), que coordinan la lógica de cada caso de uso; y las clases de entidad (*entity*), que representan la información persistente del dominio (Jacobson et al., 1999). Estos diagramas se expresan en el Lenguaje Unificado de Modelado (Booch et al., 2005).

En todos los diagramas, el actor se comunica con la interfaz (clase terminada en *View*), esta con el controlador (clase terminada en *Controller*) y el controlador con las entidades; no existe comunicación directa entre el actor y el controlador ni entre la interfaz y las entidades. Los mensajes se numeran jerárquicamente: el número entero corresponde a la operación que solicita el actor, y los subniveles (1.1, 1.2, etc.) a los mensajes que se derivan de ella hacia el controlador y las entidades. Las flechas se diferencian por color según el tipo de operación. Los nombres de las entidades se corresponden con las tablas del modelo de datos del sistema.

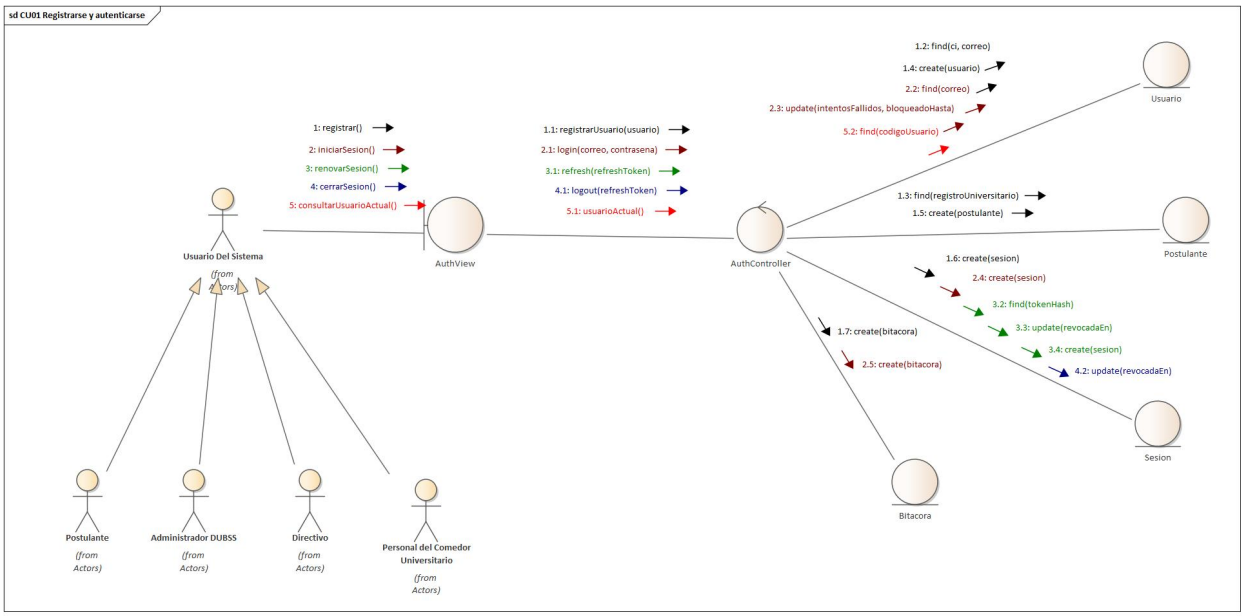
A continuación se presenta el análisis de los veintiséis casos de uso del sistema, en el orden de su numeración (CU-01 a CU-26).

### *6.2.1. Analizar CU-01 Registrarse y autenticarse*

En el análisis del CU-01 participa el actor Usuario del Sistema, generalizado por los cuatro roles del sistema (Postulante, Administrador DUBSS, Directivo y Personal del Comedor Universitario), de modo que cualquiera de ellos puede iniciar y cerrar sesión (el registro corresponde únicamente al Postulante). El actor interactúa con la interfaz *AuthView*, que delega las solicitudes en el controlador *AuthController*, y este accede a las entidades *Usuario*, *Postulante*, *Sesion* y *Bitacora*. La verificación de las credenciales, la emisión de los tokens y el bloqueo por intentos fallidos los resuelve el propio sistema. La Figura 66 presenta el diagrama de análisis.

**Figura 66**

*Análisis del CU-01 Registrarse y autenticarse*




*Nota.* Elaboración propia.

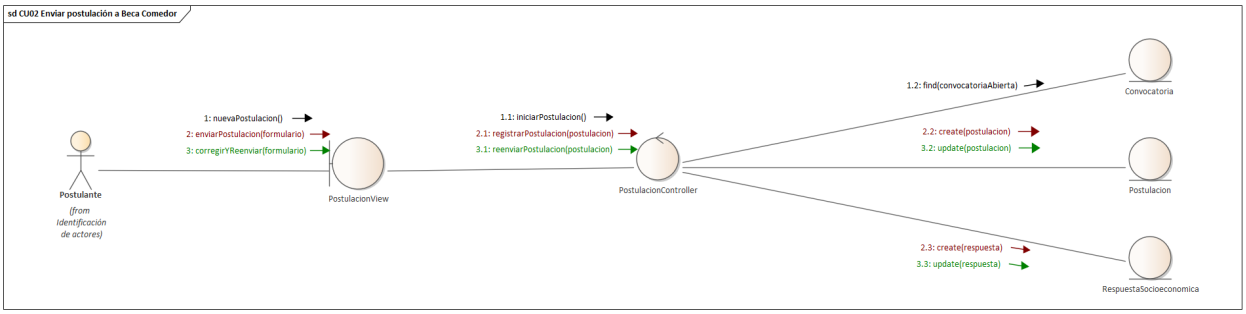
El registro (mensaje 1) verifica que el CI, el registro universitario y el correo electrónico no existan (1.2 y 1.3), crea el usuario con la contraseña almacenada como hash (1.4) y, para el rol Postulante, su perfil (1.5), abre la sesión (1.6) y anota el registro en la bitácora (1.7). El inicio de sesión (mensaje 2) consulta al usuario por su correo electrónico (2.2) y verifica la contraseña; si es incorrecta, actualiza el contador de intentos fallidos y, de ser necesario, el bloqueo (2.3); si es correcta, crea la sesión (2.4) y anota el acceso en la bitácora (2.5). La renovación de la sesión (mensaje 3) busca la sesión por el hash de su token (3.2), la revoca (3.3) y crea una nueva (3.4), de modo que el token de renovación se reemplaza en cada uso. El cierre de sesión (mensaje 4) revoca la sesión del dispositivo desde el que se solicita (4.2). La consulta del usuario actual (mensaje 5) busca al usuario por su código público (5.2). De este modo, la contraseña y el token de renovación se almacenan solo como hash, nunca en texto plano.

### *6.2.2. Analizar CU-02 Enviar postulación a Beca Comedor*

El CU-02 es iniciado por el Postulante a través de *PostulacionView*, que se comunica con *PostulacionController*. Este controlador utiliza las entidades *Convocatoria*, *Postulacion* y *RespuestaSocioeconomica*. La digitalización de documentos, la validación de elegibilidad y la precalificación se resuelven en los casos de uso incluidos (CU-16, CU-18 y CU-03), por lo que no se repiten en este diagrama. La Figura 67 presenta el diagrama de análisis.

**Figura 67**

*Análisis del CU-02 Enviar postulación a Beca Comedor*




*Nota.* Elaboración propia.

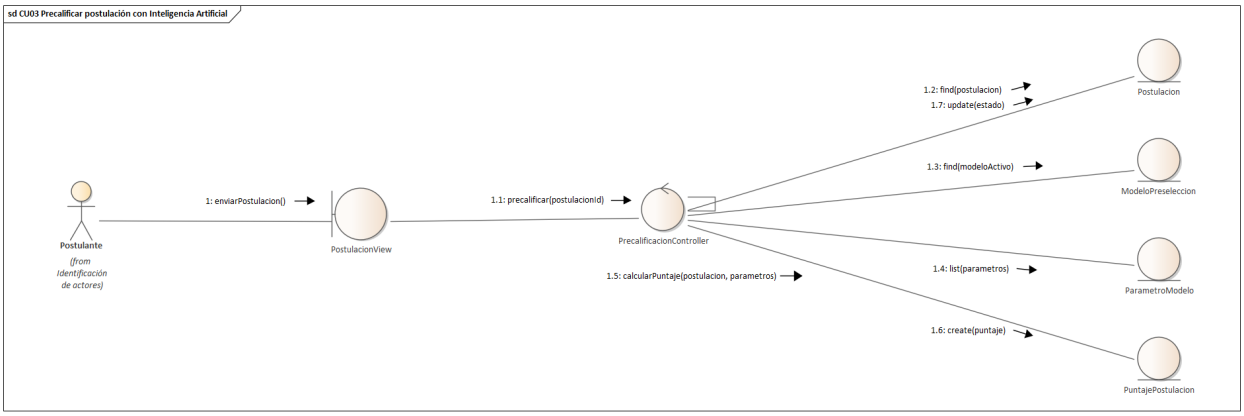
Al iniciar una nueva postulación (mensaje 1), el controlador consulta la convocatoria abierta (1.2). Al enviar el formulario (mensaje 2), crea la postulación (2.2) y su respuesta socioeconómica (2.3). Si la postulación fue devuelta para corrección, el Postulante la corrige y la reenvía (mensaje 3), con lo cual se actualizan la postulación (3.2) y la respuesta socioeconómica (3.3) sobre los mismos registros.

### *6.2.3. Analizar CU-03 Precalificar postulación con Inteligencia Artificial*

El CU-03 se dispara cuando el Postulante envía su postulación desde *PostulacionView*. El controlador *PrecalificacionController* recibe la solicitud de precalificación y colabora con las entidades *Postulacion*, *ModeloPreseleccion*, *ParametroModelo* y *PuntajePostulacion*. El módulo de Inteligencia Artificial no se representa como objeto independiente: su ejecución se modela como el mensaje 1.5 del propio controlador. La Figura 68 presenta el diagrama de análisis.

**Figura 68**

*Análisis del CU-03 Precalificar postulación con Inteligencia Artificial*




*Nota.* Elaboración propia.

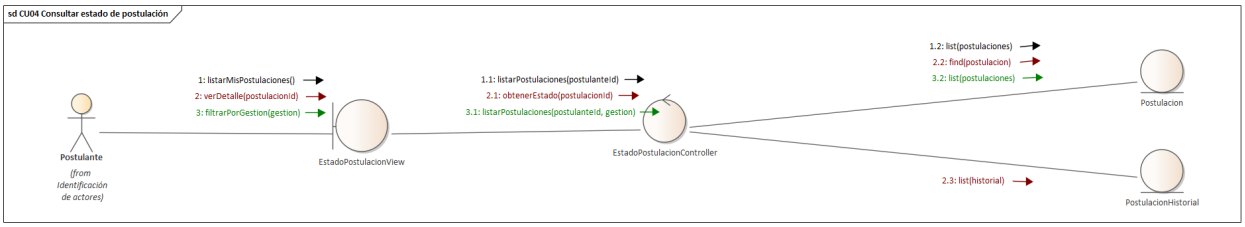
El controlador obtiene la postulación (1.2), el modelo de preselección activo (1.3) y sus parámetros (1.4); calcula el puntaje de vulnerabilidad (1.5); registra el puntaje con su fecha de cálculo (1.6) y actualiza el estado de la postulación (1.7), lo que permite conservar la trazabilidad de cada cálculo.

### *6.2.4. Analizar CU-04 Consultar estado de postulación*

En el CU-04, el Postulante consulta sus postulaciones mediante *EstadoPostulacionView*, que se apoya en *EstadoPostulacionController*. Las entidades involucradas son *Postulacion* y *PostulacionHistorial*. La Figura 69 presenta el diagrama de análisis.

**Figura 69**

*Análisis del CU-04 Consultar estado de postulación*




*Nota.* Elaboración propia.

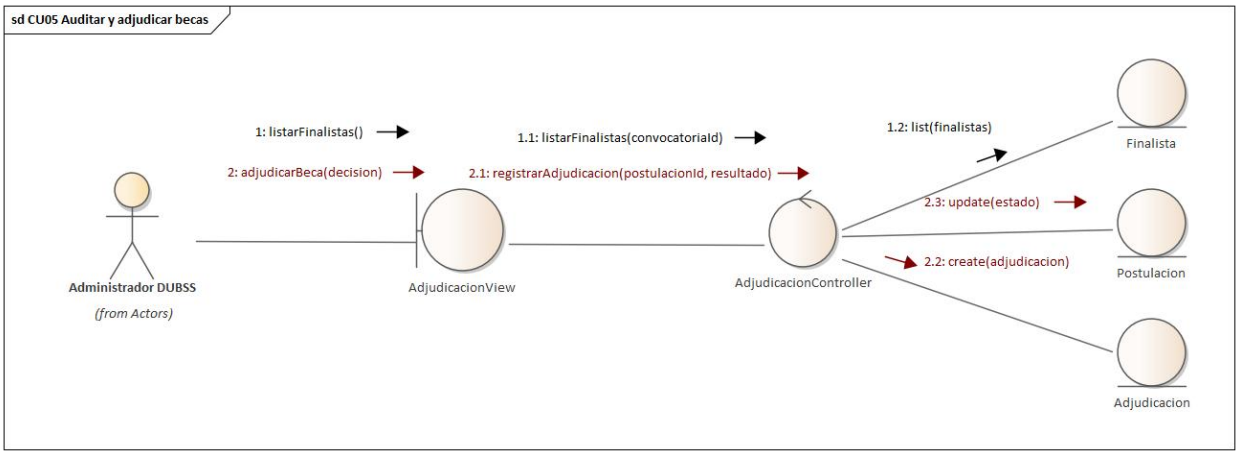
El actor puede listar sus postulaciones (mensaje 1), ver el detalle de una de ellas (mensaje 2), lo que implica obtener la postulación (2.2) y su historial de cambios de estado (2.3), y filtrar las postulaciones por gestión académica (mensaje 3). Se trata de operaciones de solo lectura.

### *6.2.5. Analizar CU-05 Auditar y adjudicar becas*

En el CU-05, el Administrador DUBSS conduce la adjudicación mediante *AdjudicacionView*, que delega en *AdjudicacionController*. Este controlador trabaja con las entidades *Finalista*, *Postulacion* y *Adjudicacion*. La visualización de la lista priorizada y la auditoría de cada expediente se resuelven en los casos de uso incluidos (CU-06 y CU-07), por lo que no se repiten en este diagrama. La Figura 70 presenta el diagrama de análisis.

**Figura 70**

*Análisis del CU-05 Auditar y adjudicar becas*




*Nota.* Elaboración propia.

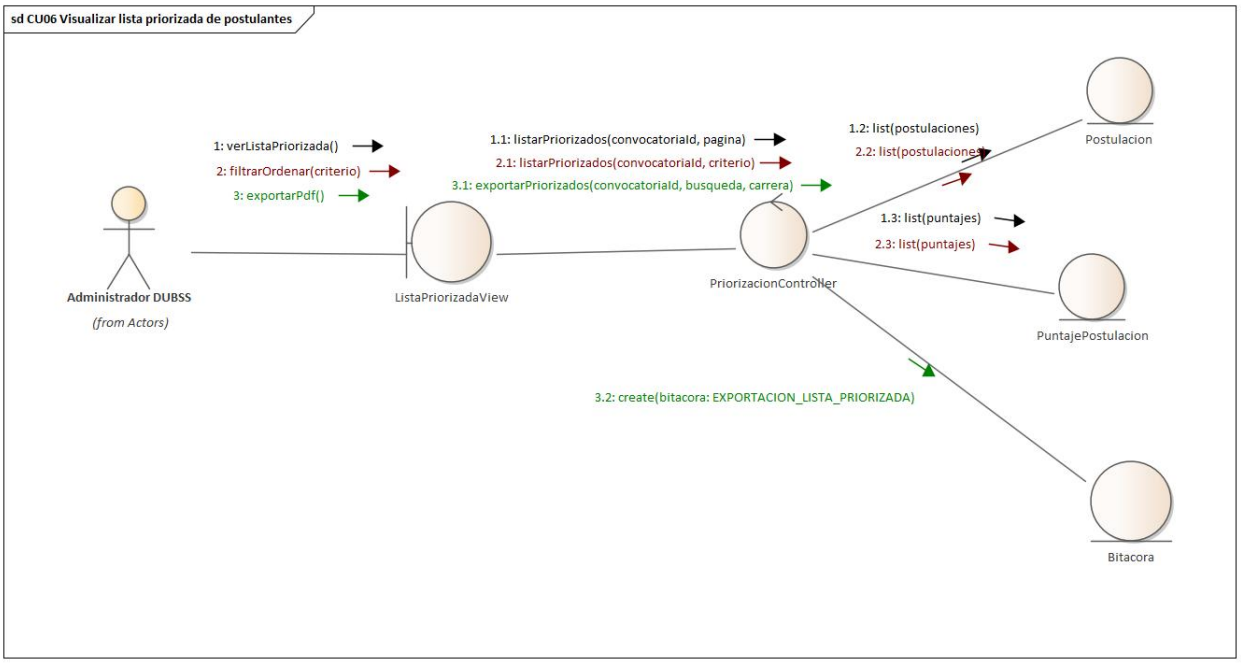
El Administrador puede listar los finalistas de la convocatoria (mensaje 1), con lo que el controlador obtiene los finalistas registrados (1.2). Al adjudicar una beca (mensaje 2), el controlador registra la decisión (2.1), crea la adjudicación con su resultado (2.2) y actualiza el estado de la postulación a «adjudicada» o «no adjudicada» (2.3).

### *6.2.6. Analizar CU-06 Visualizar lista priorizada de postulantes*

En el CU-06, el Administrador DUBSS consulta la lista priorizada mediante *ListaPriorizadaView*, que se comunica con *PriorizacionController*. Las entidades involucradas son *Postulacion* y *PuntajePostulacion*; la lista no se almacena, sino que se obtiene a partir de ambas. Para la exportación a PDF interviene además *Bitacora*, donde se registra la acción. La Figura 71 presenta el diagrama de análisis.

**Figura 71**

*Análisis del CU-06 Visualizar lista priorizada de postulantes*




*Nota.* Elaboración propia.

Al ver la lista priorizada (mensaje 1), el controlador solicita la página correspondiente de la convocatoria (1.1), obtiene las postulaciones calificadas (1.2) y sus puntajes vigentes (1.3), y las ordena de mayor a menor puntaje. Al filtrar u ordenar la lista (mensaje 2), se repite la consulta con el criterio indicado (2.1 a 2.3). Ambas operaciones son de solo lectura. Al exportar la lista (mensaje 3), el controlador recibe la solicitud con la convocatoria y los criterios de búsqueda y carrera vigentes (3.1), genera el documento PDF y registra la

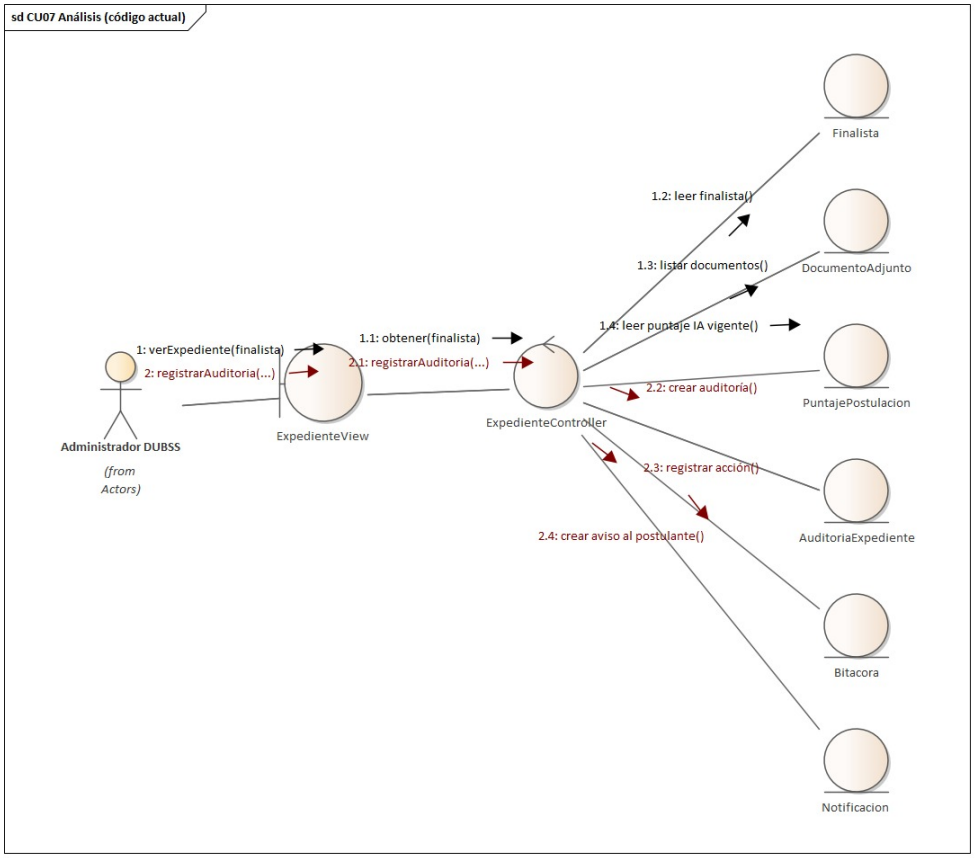
exportación en la bitácora con la acción EXPORTACION_LISTA_PRIORIZADA (3.2).

### *6.2.7. Analizar CU-07 Auditar expediente finalista*

En el CU-07, el Administrador DUBSS revisa el expediente de un finalista mediante *ExpedienteView* y *ExpedienteController*, que colaboran con las entidades *Finalista*, *DocumentoAdjunto*, *PuntajePostulacion*, *AuditoriaExpediente*, *Bitacora* y *Notificacion*. La notificación al finalista y el registro de la entrega física se resuelven en el CU-21, incluido por este caso de uso. La Figura 72 presenta el diagrama de análisis.

**Figura 72**

*Análisis del CU-07 Auditar expediente finalista*




*Nota.* Elaboración propia.

Al ver el expediente (mensaje 1), el controlador obtiene el finalista (1.2) y la lista de sus documentos adjuntos (1.3) y el puntaje vigente de la inteligencia artificial para

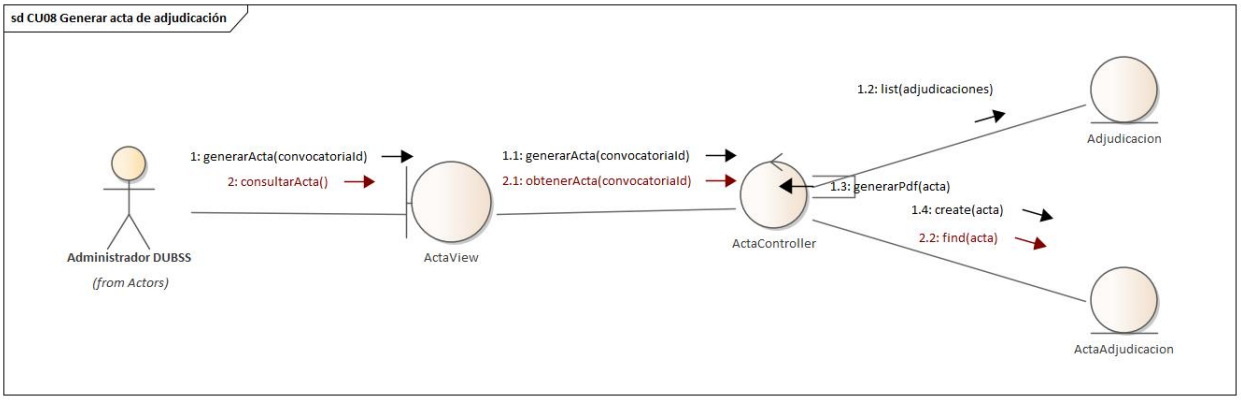
mostrar el total. Al registrar la auditoría (mensaje 2), el controlador recibe el resultado, las observaciones (hasta 500 caracteres) y el puntaje académico (0 a 30), y crea el registro de auditoría conservando las auditorías anteriores del expediente; luego deja constancia en la bitácora y crea la notificación que avisa el resultado al postulante.

### *6.2.8. Analizar CU-08 Generar acta de adjudicación*

En el CU-08, el Administrador DUBSS genera el acta mediante *ActaView* y *ActaController*, que utiliza las entidades *Adjudicacion* y *ActaAdjudicacion*. La generación del documento PDF se modela como un mensaje del propio controlador (1.3). La Figura 73 presenta el diagrama de análisis.

**Figura 73**

*Análisis del CU-08 Generar acta de adjudicación*




*Nota.* Elaboración propia.

Al generar el acta (mensaje 1), el controlador obtiene las adjudicaciones de la convocatoria (1.2), genera el documento PDF (1.3) y crea el registro del acta (1.4). Al consultar el acta (mensaje 2), el controlador obtiene el acta registrada de la convocatoria (2.2).

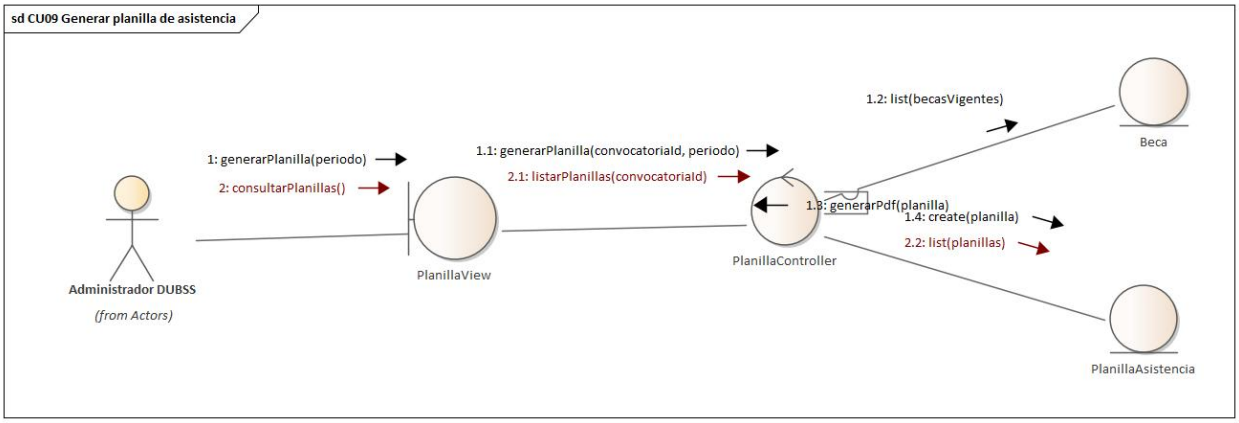
### *6.2.9. Analizar CU-09 Generar planilla de asistencia*

En el CU-09, el Administrador DUBSS genera las planillas mediante *PlanillaView* y *PlanillaController*, que trabajan con las entidades *Beca* y *PlanillaAsistencia*. La generación

del documento PDF imprimible se modela como un mensaje del propio controlador (1.3). La Figura 74 presenta el diagrama de análisis.

**Figura 74**

*Análisis del CU-09 Generar planilla de asistencia*




*Nota.* Elaboración propia.

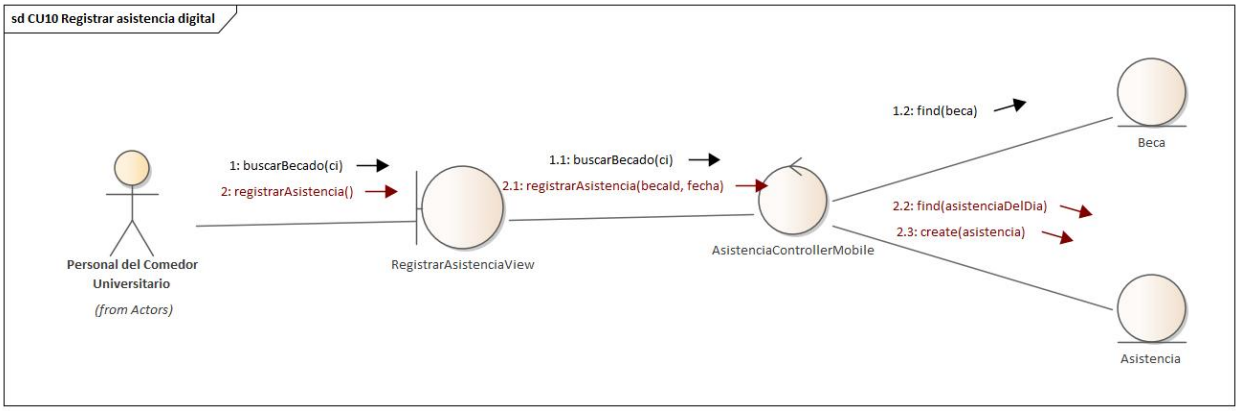
Al generar una planilla para un periodo (mensaje 1), el controlador obtiene las becas vigentes de la convocatoria (1.2), genera el PDF con los espacios de firma y fotocopia de carnet (1.3) y crea el registro de la planilla (1.4). Al consultar las planillas (mensaje 2), el controlador lista las planillas generadas para la convocatoria (2.2).

### *6.2.10. Analizar CU-10 Registrar asistencia digital*

En el CU-10, el Personal del Comedor Universitario registra la asistencia desde la aplicación móvil mediante *RegistrarAsistenciaView*, que delega en *AsistenciaControllerMobile*. Las entidades involucradas son *Beca* y *Asistencia*. La Figura 75 presenta el diagrama de análisis.

**Figura 75**

*Análisis del CU-10 Registrar asistencia digital*




*Nota.* Elaboración propia.

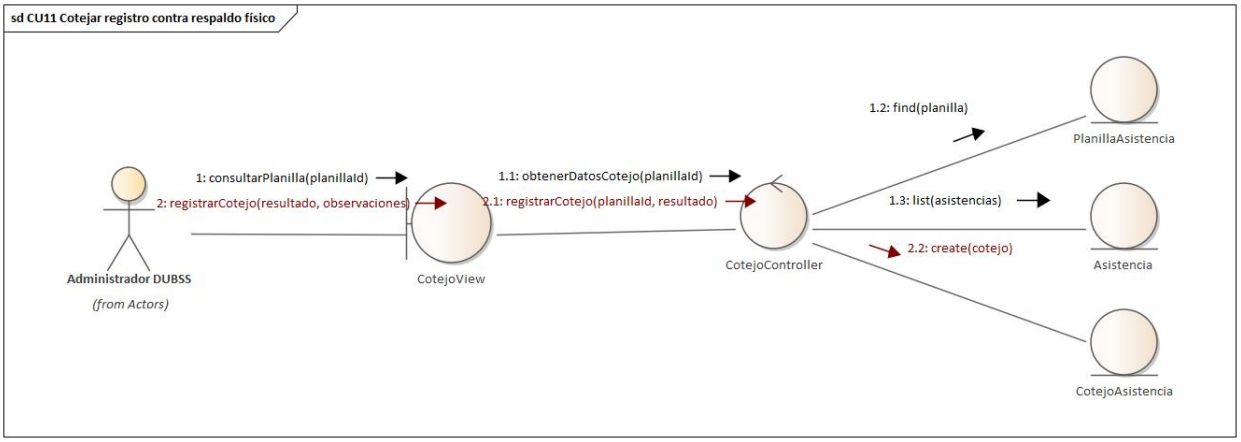
Al buscar al becado por su CI (mensaje 1), el controlador obtiene su beca (1.2) para verificar que esté vigente. Al registrar la asistencia (mensaje 2), el controlador verifica si ya existe una asistencia de esa beca en el día (2.2) y, si no existe, crea el registro de asistencia (2.3); de este modo, una petición duplicada no genera un segundo registro.

### *6.2.11. Analizar CU-11 Cotejar registro contra respaldo físico*

En el CU-11, el Administrador DUBSS realiza el cotejo mediante *CotejoView* y *CotejoController*, que colaboran con las entidades *PlanillaAsistencia*, *Asistencia* y *CotejoAsistencia*. La Figura 76 presenta el diagrama de análisis.

**Figura 76**

*Análisis del CU-11 Cotejar registro contra respaldo físico*




*Nota.* Elaboración propia.

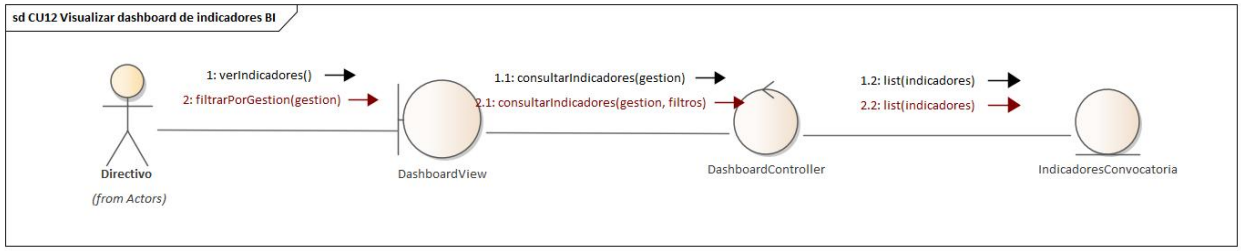
Al consultar una planilla (mensaje 1), el controlador obtiene la planilla (1.2) y la lista de asistencias registradas en su periodo (1.3). Al registrar el cotejo (mensaje 2), el controlador recibe el resultado (2.1) y crea el registro del cotejo (2.2), sin modificar los registros de asistencia.

### *6.2.12. Analizar CU-12 Visualizar dashboard de indicadores BI*

En el CU-12, el Directivo consulta el dashboard mediante *DashboardView* y *DashboardController*. La única entidad involucrada es *IndicadoresConvocatoria*, que representa los indicadores precalculados por lotes a partir de las postulaciones, adjudicaciones, becas y asistencias. La Figura 77 presenta el diagrama de análisis.

**Figura 77**

*Análisis del CU-12 Visualizar dashboard de indicadores BI*




*Nota.* Elaboración propia.

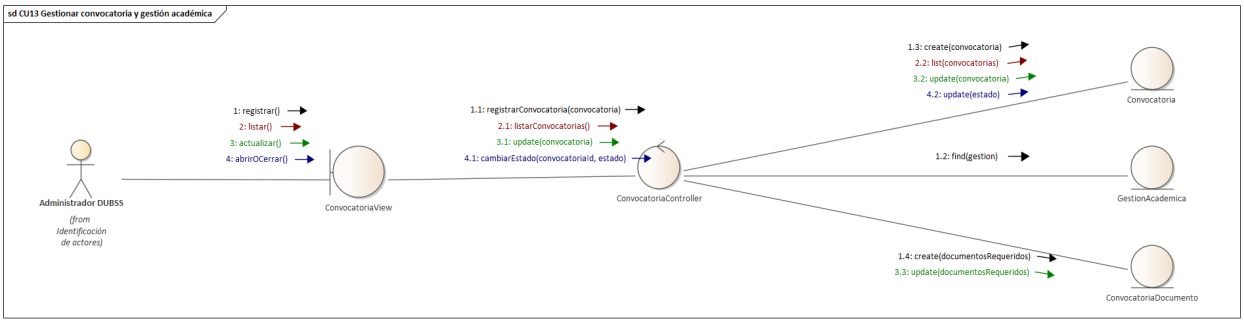
Al ver los indicadores (mensaje 1), el controlador los consulta para la gestión académica vigente (1.1) y obtiene la lista de indicadores (1.2). Al filtrar por gestión (mensaje 2), se repite la consulta con la gestión y los filtros seleccionados (2.1 y 2.2). Ambas operaciones son de solo lectura.

### *6.2.13. Analizar CU-13 Gestionar convocatoria y gestión académica*

En el CU-13, el Administrador DUBSS gestiona las convocatorias mediante *ConvocatoriaView* y *ConvocatoriaController*, que trabajan con las entidades *Convocatoria*, *GestionAcademica* y *ConvocatoriaDocumento*. La Figura 78 presenta el diagrama de análisis.

**Figura 78**

*Análisis del CU-13 Gestionar convocatoria y gestión académica*




*Nota.* Elaboración propia.

El registro de una convocatoria (mensaje 1) consulta la gestión académica (1.2), crea

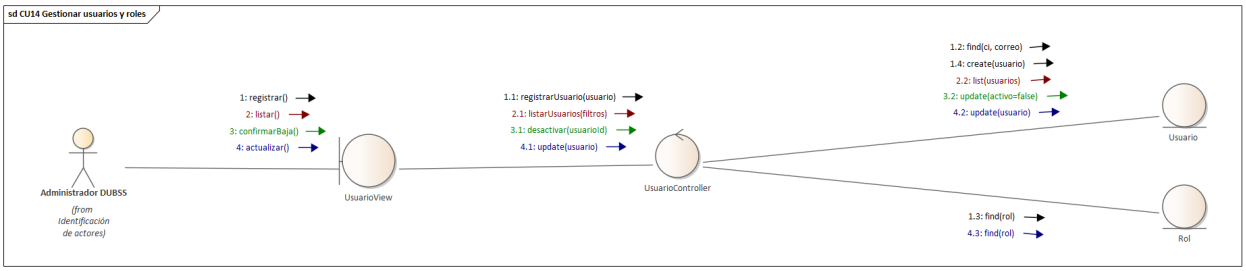
la convocatoria (1.3) y crea la lista de documentos requeridos (1.4). El listado (mensaje 2) recupera las convocatorias (2.2). La actualización (mensaje 3) modifica la convocatoria (3.2) y sus documentos requeridos (3.3). Finalmente, abrir o cerrar una convocatoria (mensaje 4) actualiza su estado (4.2). No se incluye una operación de eliminación, pues una convocatoria con postulaciones asociadas solo puede cerrarse.

### *6.2.14. Analizar CU-14 Gestionar usuarios y roles*

En el CU-14, el Administrador DUBSS administra las cuentas del personal mediante *UsuarioView* y *UsuarioController*, que utilizan las entidades *Usuario* y *Rol*. La Figura 79 presenta el diagrama de análisis.

**Figura 79**

*Análisis del CU-14 Gestionar usuarios y roles*




*Nota.* Elaboración propia.

Para registrar una cuenta (mensaje 1), el controlador verifica que el CI y el correo no existan (1.2), consulta el rol asignado (1.3) y crea el usuario (1.4). El listado (mensaje 2) recupera los usuarios según los filtros indicados (2.2). La baja (mensaje 3) desactiva la cuenta mediante la actualización del campo de actividad (3.2), sin eliminar el registro, de modo que se preserve la trazabilidad. La actualización (mensaje 4) modifica los datos del usuario (4.2) y su rol (4.3).

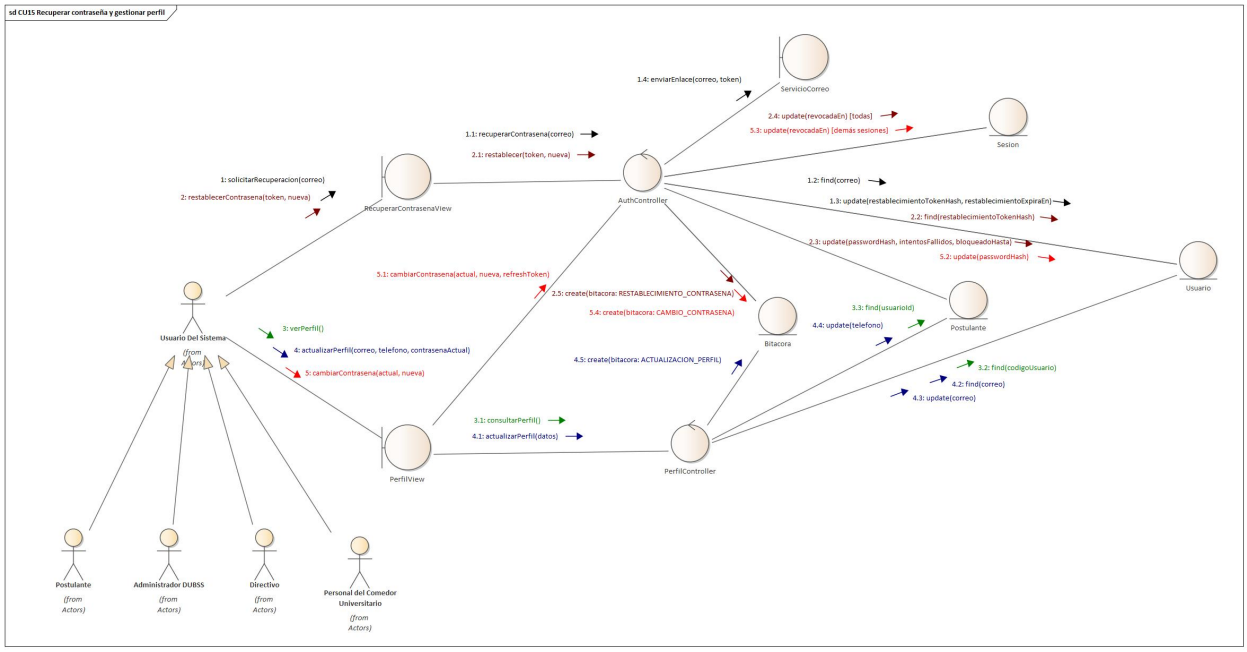
### *6.2.15. Analizar CU-15 Recuperar contraseña / gestionar perfil*

En el CU-15 participa el actor Usuario del Sistema, generalizado por los cuatro roles, quien interactúa con las vistas *RecuperarContrasenaView* y *PerfilView*. El controlador

*AuthController* atiende la recuperación, el restablecimiento y el cambio de contraseña, y *PerfilController* la consulta y actualización del perfil; ambos colaboran con las entidades *Usuario*, *Postulante*, *Sesion* y *Bitacora*, y el envío del enlace pasa por la interfaz *ServicioCorreo*. El enlace de restablecimiento no es una entidad aparte: su hash y su vencimiento se guardan en *Usuario*, con un solo enlace vigente por cuenta. La Figura 80 presenta el diagrama de análisis.

**Figura 80**

*Análisis del CU-15 Recuperar contraseña / gestionar perfil*




*Nota.* Elaboración propia.

La solicitud de recuperación (mensaje 1) consulta al usuario por su correo (1.2), guarda en él el hash del token de un solo uso y su vencimiento (1.3) y envía el enlace por correo (1.4); la respuesta es siempre el mismo mensaje neutro. El restablecimiento (mensaje 2) busca la cuenta por el hash del token (2.2), guarda la nueva contraseña y desbloquea la cuenta (2.3), revoca todas sus sesiones (2.4) y lo registra en la bitácora (2.5). La consulta del perfil (mensaje 3) obtiene los datos del usuario (3.2) y, si es Postulante, los suyos (3.3). La actualización (mensaje 4) verifica que el correo nuevo no pertenezca a otra cuenta (4.2),

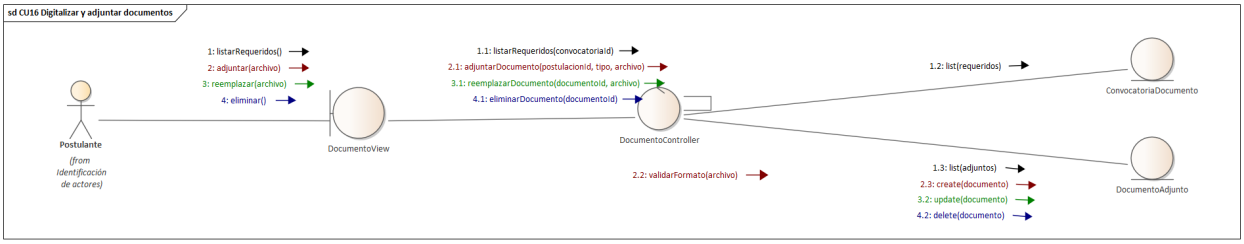
actualiza el correo (4.3) y el teléfono del postulante (4.4) y lo registra en la bitácora (4.5). El cambio de contraseña (mensaje 5) actualiza la contraseña (5.2), revoca las sesiones de los demás dispositivos (5.3) y lo registra en la bitácora (5.4).

### *6.2.16. Analizar CU-16 Digitalizar y adjuntar documentos*

En el CU-16, el Postulante adjunta los documentos de respaldo mediante *DocumentoView* y *DocumentoController*, que utilizan las entidades *ConvocatoriaDocumento* y *DocumentoAdjunto*. La Figura 81 presenta el diagrama de análisis.

**Figura 81**

*Análisis del CU-16 Digitalizar y adjuntar documentos*




*Nota.* Elaboración propia.

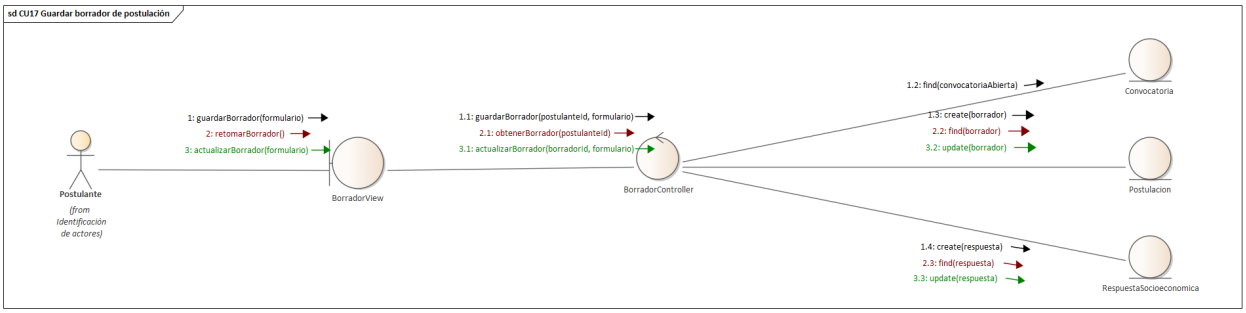
Primero se listan los documentos requeridos por la convocatoria y los ya adjuntados (mensaje 1, con los pasos 1.2 y 1.3). Al adjuntar un archivo (mensaje 2), el controlador valida su formato (2.2) y crea el documento adjunto (2.3). El reemplazo (mensaje 3) actualiza el documento (3.2) y la eliminación (mensaje 4) lo elimina (4.2), operaciones permitidas antes del envío de la postulación o cuando esta fue devuelta.

### *6.2.17. Analizar CU-17 Guardar borrador de postulación*

En el CU-17, el Postulante guarda y retoma su postulación incompleta mediante *BorradorView* y *BorradorController*, que trabajan con las entidades *Convocatoria*, *Postulacion* y *RespuestaSocioeconomica*. La Figura 82 presenta el diagrama de análisis.

**Figura 82**

*Análisis del CU-17 Guardar borrador de postulación*




*Nota.* Elaboración propia.

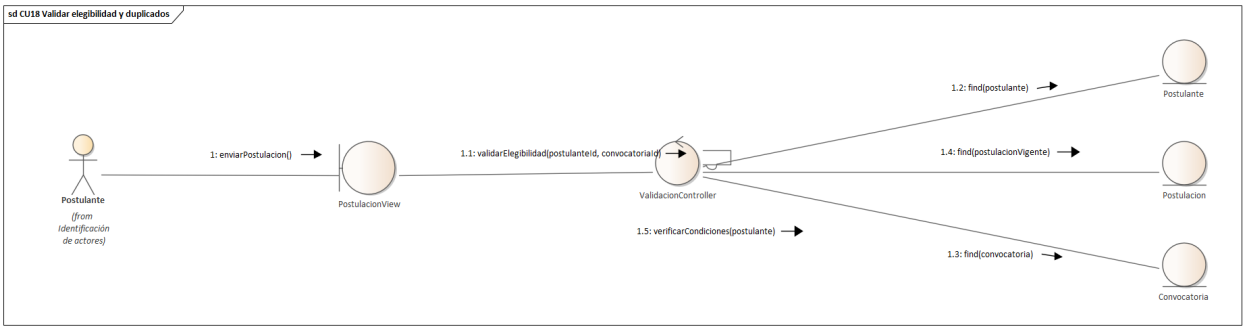
Al guardar el borrador (mensaje 1), el controlador verifica que la convocatoria esté abierta (1.2) y crea la postulación en estado de borrador (1.3) con las respuestas ingresadas (1.4), sin exigir los campos obligatorios. Al retomarlo (mensaje 2) se recuperan el borrador (2.2) y sus respuestas (2.3), y al continuar editándolo (mensaje 3) se actualizan ambos (3.2 y 3.3). El borrador no se envía al módulo de precalificación.

### *6.2.18. Analizar CU-18 Validar elegibilidad y duplicados*

El CU-18 es incluido por el CU-02: se ejecuta cuando el Postulante envía su postulación a través de *PostulacionView*. El controlador *ValidacionController* colabora con las entidades *Postulante*, *Convocatoria* y *Postulacion*. La Figura 83 presenta el diagrama de análisis.

**Figura 83**

*Análisis del CU-18 Validar elegibilidad y duplicados*




*Nota.* Elaboración propia.

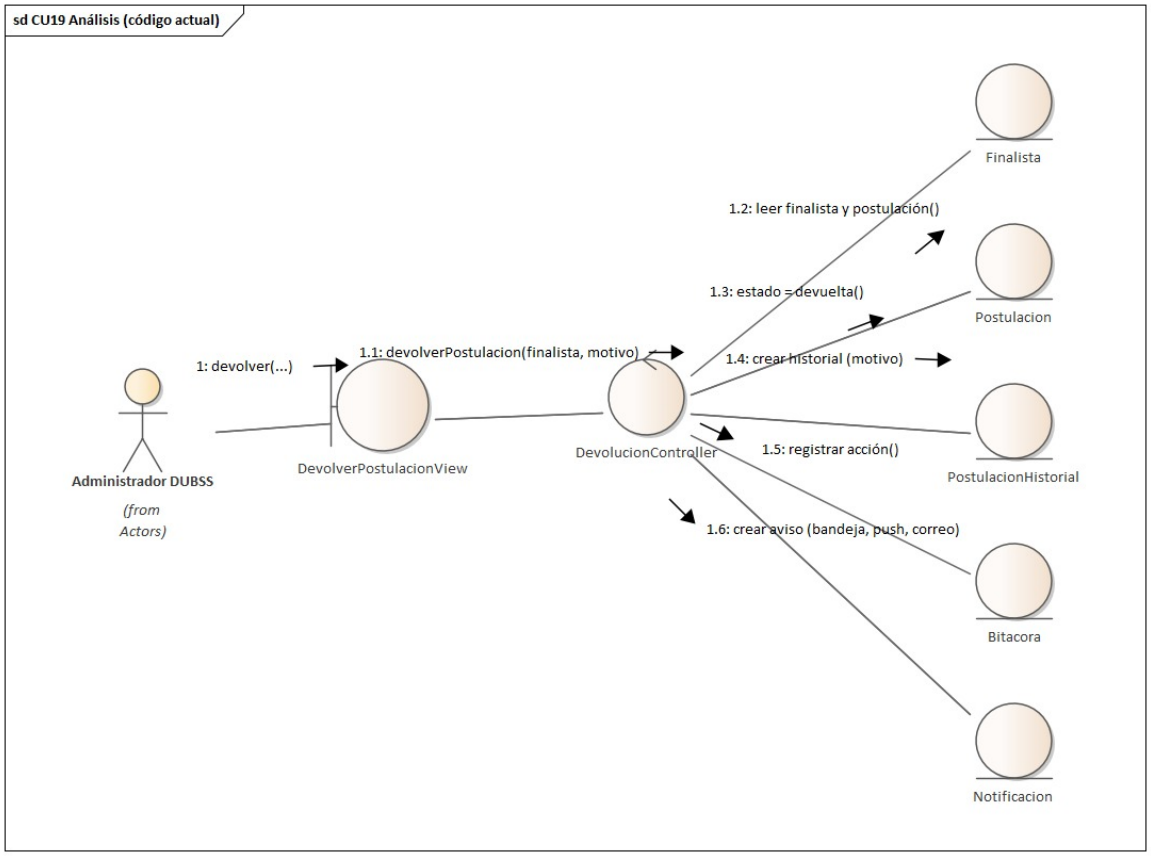
Con la solicitud de validación (mensaje 1.1), el controlador consulta al postulante (1.2), la convocatoria (1.3) y la postulación vigente del postulante en la misma gestión (1.4), y verifica las condiciones de elegibilidad (1.5). El resultado determina si el CU-02 puede registrar la postulación o debe rechazarla indicando el motivo.

### *6.2.19. Analizar CU-19 Devolver postulación para corrección*

En el CU-19, el Administrador DUBSS devuelve una postulación mediante *DevolverPostulacionView* y *DevolucionController*, que utiliza las entidades *Finalista*, *Postulacion*, *PostulacionHistorial*, *Bitacora* y *Notificacion*. La Figura 84 presenta el diagrama de análisis.

**Figura 84**

*Análisis del CU-19 Devolver postulación para corrección*




*Nota.* Elaboración propia.

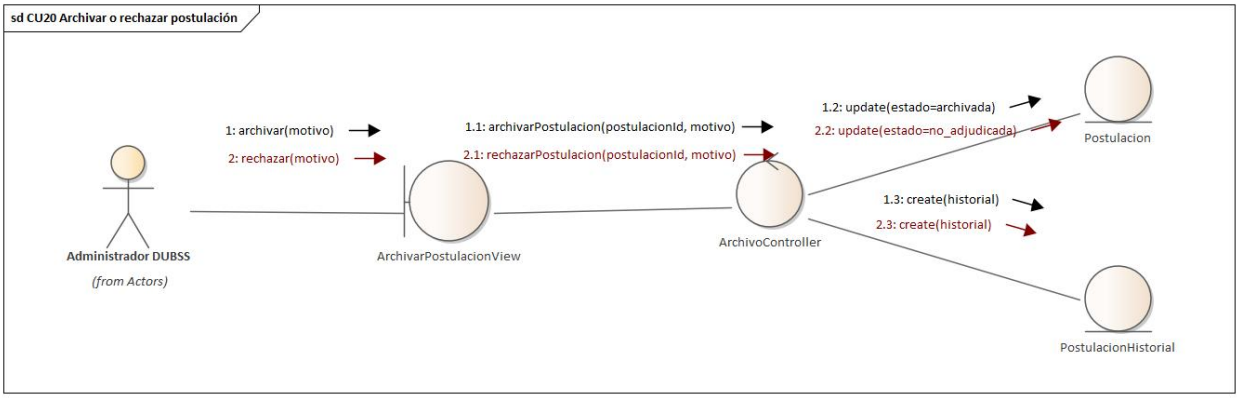
Al devolver la postulación con su motivo (mensaje 1), el controlador entra por el código del finalista, lee el finalista y su postulación y valida que siga «calificada»; a continuación actualiza su estado a «devuelta», crea el registro del cambio en el historial de la postulación con el motivo (hasta 500 caracteres), registra la acción en la bitácora y crea la notificación dirigida al Postulante, que se envía por bandeja, push y correo.

### *6.2.20. Analizar CU-20 Archivar o rechazar postulación*

En el CU-20, el Administrador DUBSS cierra una postulación mediante *ArchivarPostulacionView* y *ArchivoController*, que trabajan con las entidades *Postulacion* y *PostulacionHistorial*. La Figura 85 presenta el diagrama de análisis.

**Figura 85**

*Análisis del CU-20 Archivar o rechazar postulación*




*Nota.* Elaboración propia.

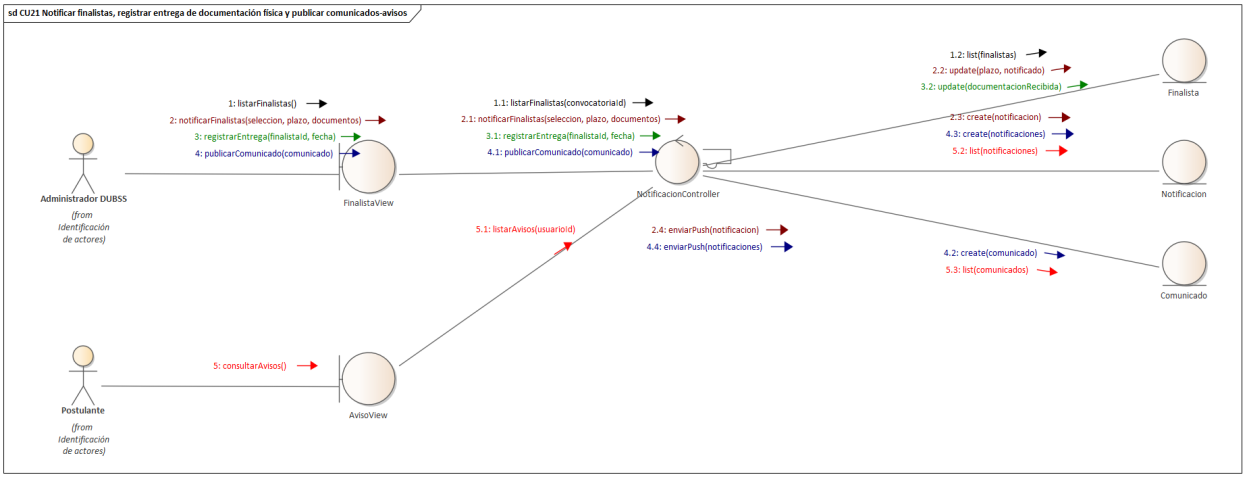
Al archivar la postulación (mensaje 1), el controlador actualiza su estado a «archivada» (1.2) y registra el cambio con su motivo en el historial (1.3). Al rechazarla (mensaje 2), el controlador actualiza su estado a «no adjudicada» (2.2) y registra igualmente el cambio en el historial (2.3).

### *6.2.21. Analizar CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos*

En el CU-21 participan dos actores: el Administrador DUBSS, que gestiona a los finalistas mediante *FinalistaView*, y el Postulante, que recibe los avisos mediante *AvisoView*. Ambas interfaces se comunican con un único controlador, *NotificacionController*, que utiliza las entidades *Finalista*, *Notificacion* y *Comunicado*. El envío de la notificación push se modela como un mensaje del propio controlador (2.4 y 4.4). La Figura 86 presenta el diagrama de análisis.

**Figura 86**

*Análisis del CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos*




*Nota.* Elaboración propia.

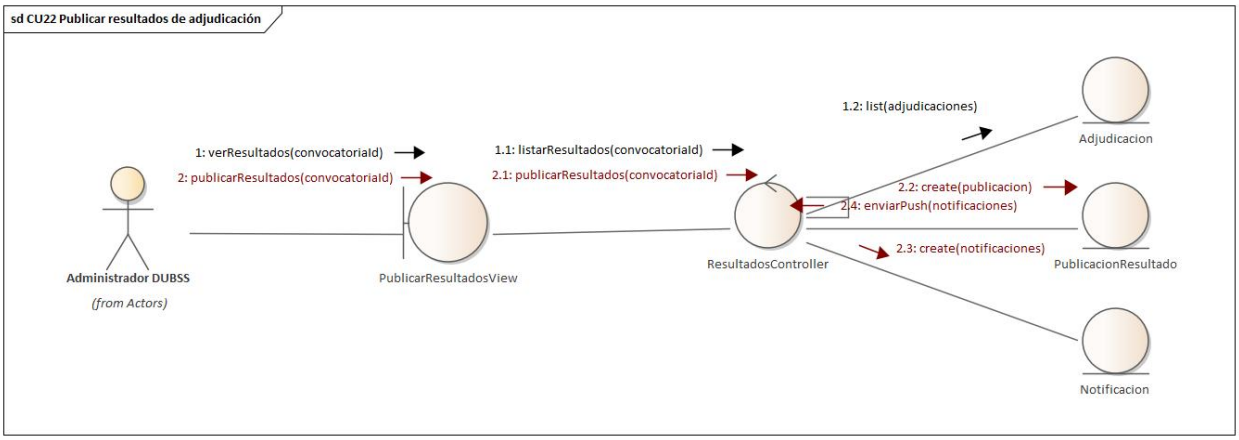
El Administrador puede listar los finalistas (mensaje 1), notificarlos con el plazo y los documentos que deben presentar (mensaje 2), lo que actualiza al finalista (2.2) y crea la notificación (2.3); registrar la entrega de la documentación física (mensaje 3), que actualiza al finalista (3.2); y publicar un comunicado (mensaje 4), que crea el comunicado (4.2) y las notificaciones correspondientes (4.3). El Postulante consulta sus avisos (mensaje 5), con lo que se listan sus notificaciones (5.2) y los comunicados (5.3).

### *6.2.22. Analizar CU-22 Publicar resultados de adjudicación*

En el CU-22, el Administrador DUBSS publica los resultados mediante *PublicarResultadosView* y *ResultadosController*, que colaboran con las entidades *Adjudicacion*, *PublicacionResultado* y *Notificacion*. El envío de las notificaciones push se modela como un mensaje del propio controlador (2.4). La Figura 87 presenta el diagrama de análisis.

**Figura 87**

*Análisis del CU-22 Publicar resultados de adjudicación*




*Nota.* Elaboración propia.

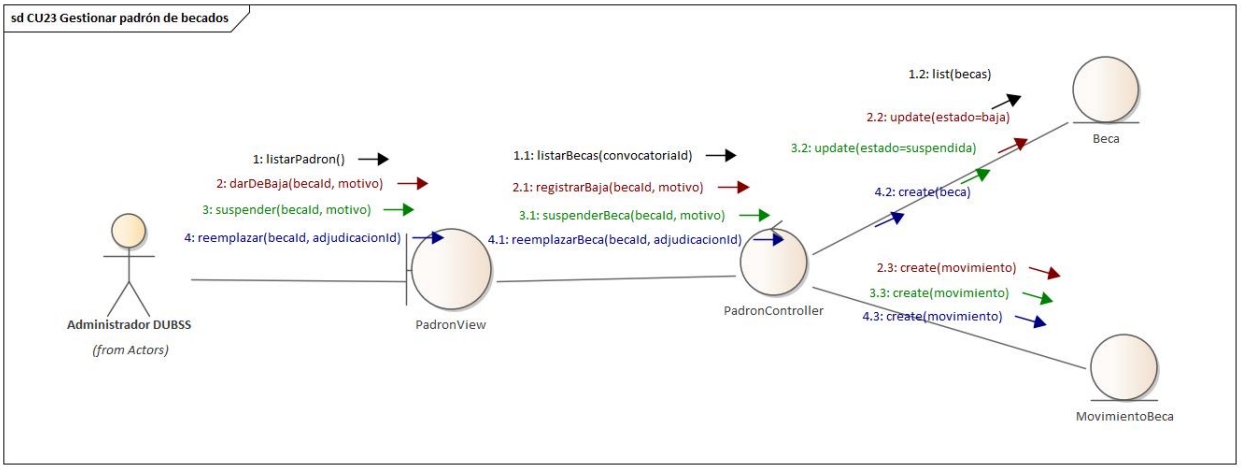
Al ver los resultados (mensaje 1), el controlador lista las adjudicaciones de la convocatoria (1.2). Al publicarlos (mensaje 2), el controlador crea el registro de la publicación (2.2), crea las notificaciones dirigidas a los postulantes (2.3) y las envía como notificaciones push (2.4).

### *6.2.23. Analizar CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo)*

En el CU-23, el Administrador DUBSS gestiona el padrón mediante *PadronView* y *PadronController*, que utilizan las entidades *Beca* y *MovimientoBeca*. La Figura 88 presenta el diagrama de análisis.

**Figura 88**

*Análisis del CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo)*




*Nota.* Elaboración propia.

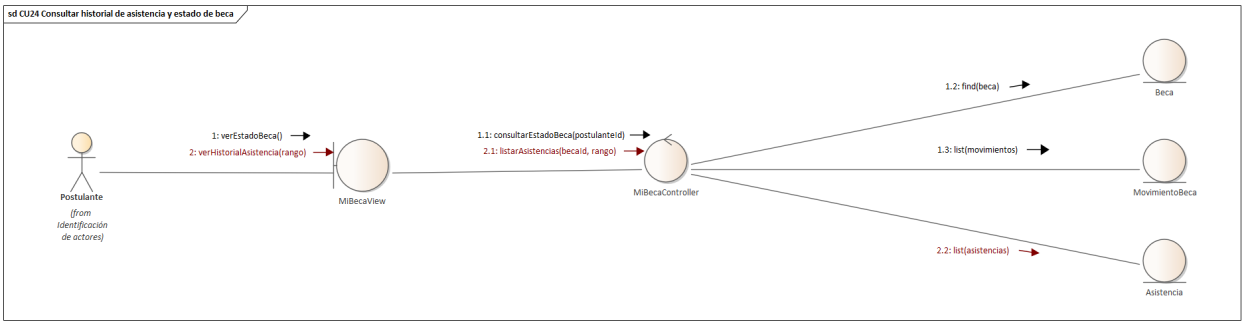
Al listar el padrón (mensaje 1), el controlador obtiene las becas de la convocatoria (1.2). Al dar de baja una beca (mensaje 2), actualiza su estado a «baja» (2.2) y crea el movimiento correspondiente (2.3). Al suspenderla (mensaje 3), actualiza su estado a «suspendida» (3.2) y crea el movimiento (3.3). Al reemplazar un cupo liberado (mensaje 4), crea una nueva beca para el postulante reemplazante (4.2) y registra el movimiento de reemplazo (4.3).

### *6.2.24. Analizar CU-24 Consultar historial de asistencia y estado de beca*

En el CU-24, el Postulante adjudicado (becado) consulta su beca mediante *MiBecaView*, que se apoya en *MiBecaController*. Las entidades involucradas son *Beca*, *MovimientoBeca* y *Asistencia*. La Figura 89 presenta el diagrama de análisis.

**Figura 89**

*Análisis del CU-24 Consultar historial de asistencia y estado de beca*




*Nota.* Elaboración propia.

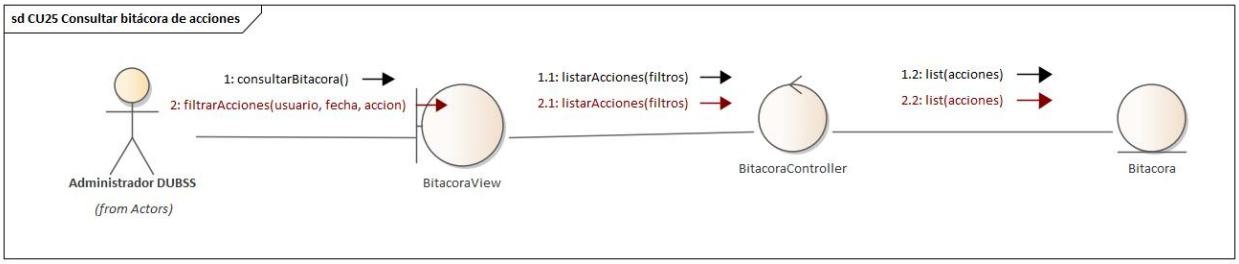
Para ver el estado de su beca (mensaje 1), el controlador obtiene la beca del postulante (1.2) y sus movimientos de baja, suspensión o reemplazo (1.3). Para ver el historial de asistencia (mensaje 2), lista las asistencias de la beca dentro del rango indicado (2.2). Ambas operaciones son de solo lectura y el becado accede únicamente a su propia información.

### *6.2.25. Analizar CU-25 Consultar bitácora de acciones*

En el CU-25, el Administrador DUBSS consulta la bitácora mediante *BitacoraView* y *BitacoraController*. La única entidad involucrada es *Bitacora*, que almacena los registros de las acciones relevantes del sistema. La Figura 90 presenta el diagrama de análisis.

**Figura 90**

*Análisis del CU-25 Consultar bitácora de acciones*




*Nota.* Elaboración propia.

Al consultar la bitácora (mensaje 1), el controlador lista las acciones registradas (1.2).

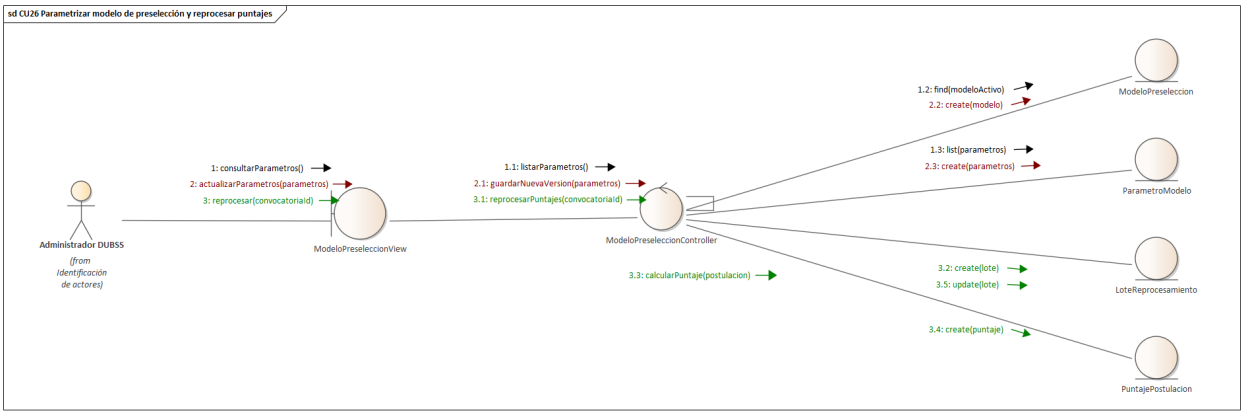
Al filtrar las acciones por usuario, fecha y tipo de acción (mensaje 2), el controlador repite la consulta con los filtros indicados (2.2). Ambas operaciones son de solo lectura y no permiten modificar los registros.

### *6.2.26. Analizar CU-26 Parametrizar modelo de preselección y reprocesar puntajes*

En el CU-26, el Administrador DUBSS ajusta el modelo de preselección mediante *ModeloPreseleccionView* y *ModeloPreseleccionController*, que colaboran con las entidades *ModeloPreseleccion*, *ParametroModelo*, *LoteReprocesamiento* y *PuntajePostulacion*. El recálculo de puntajes se modela como el mensaje 3.3 del propio controlador. La Figura 91 presenta el diagrama de análisis.

**Figura 91**

*Análisis del CU-26 Parametrizar modelo de preselección y reprocesar puntajes*




*Nota.* Elaboración propia.

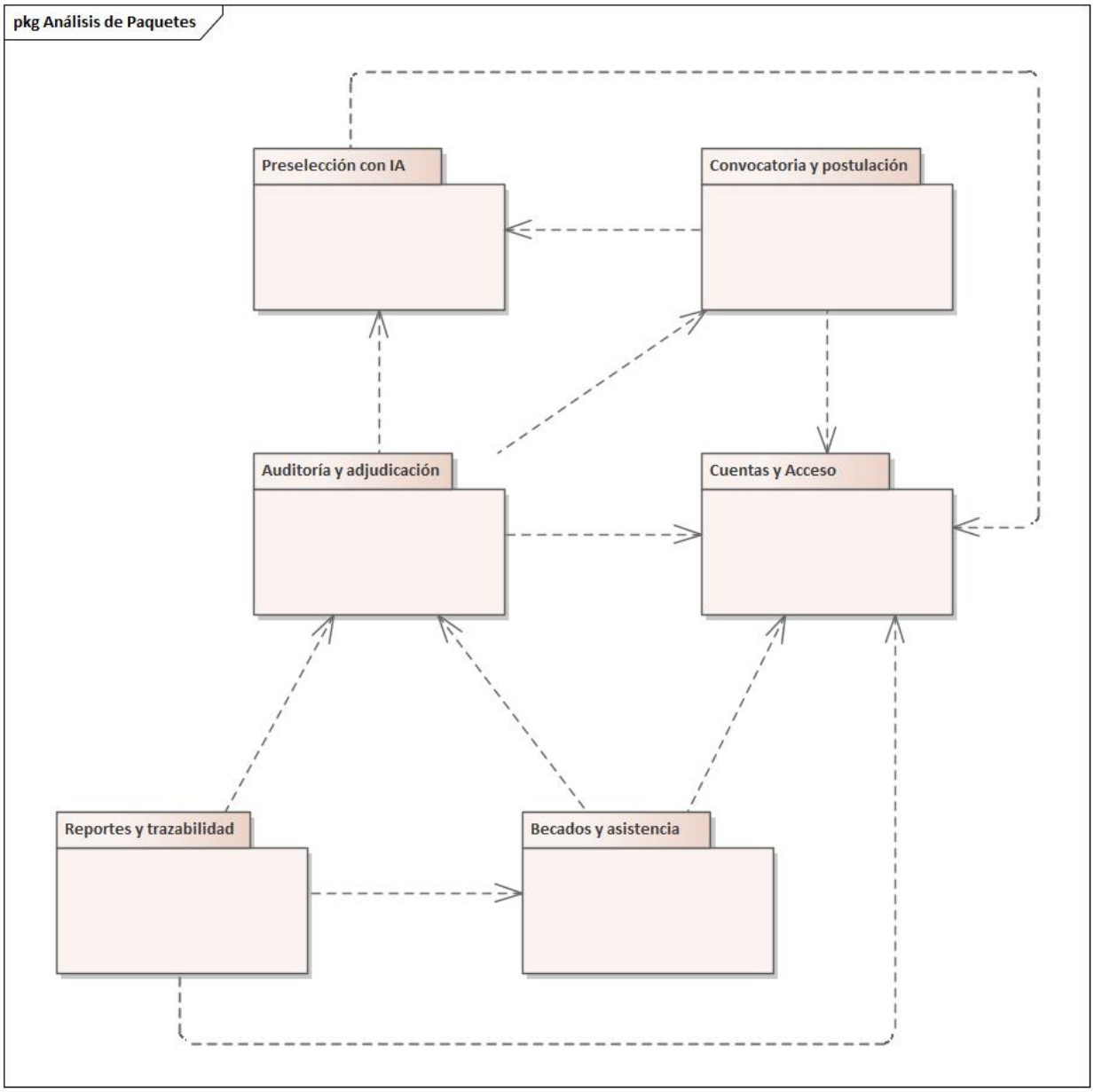
La consulta de parámetros (mensaje 1) obtiene el modelo activo (1.2) y sus parámetros (1.3). Al actualizarlos (mensaje 2), se crea una nueva versión del modelo (2.2) con sus parámetros (2.3), conservando las versiones anteriores. El reprocesamiento (mensaje 3) crea un lote (3.2), recalcula el puntaje de las postulaciones (3.3), registra los nuevos puntajes (3.4), conservando los anteriores, y actualiza el estado del lote (3.5).

## 6.3. Análisis de Paquetes

La Figura 92 presenta el diagrama de análisis de paquetes del sistema y las dependencias entre los seis paquetes identificados.

**Figura 92**

*Diagrama de análisis de paquetes del Sistema Web y Móvil de Postulación y Preselección de la Beca Comedor*




*Nota.* Elaboración propia.

Capítulo 7

# Diseño

# Capítulo 7: Diseño

## 7.1. Diseño de la Arquitectura

### *7.1.1. Diseño Físico*

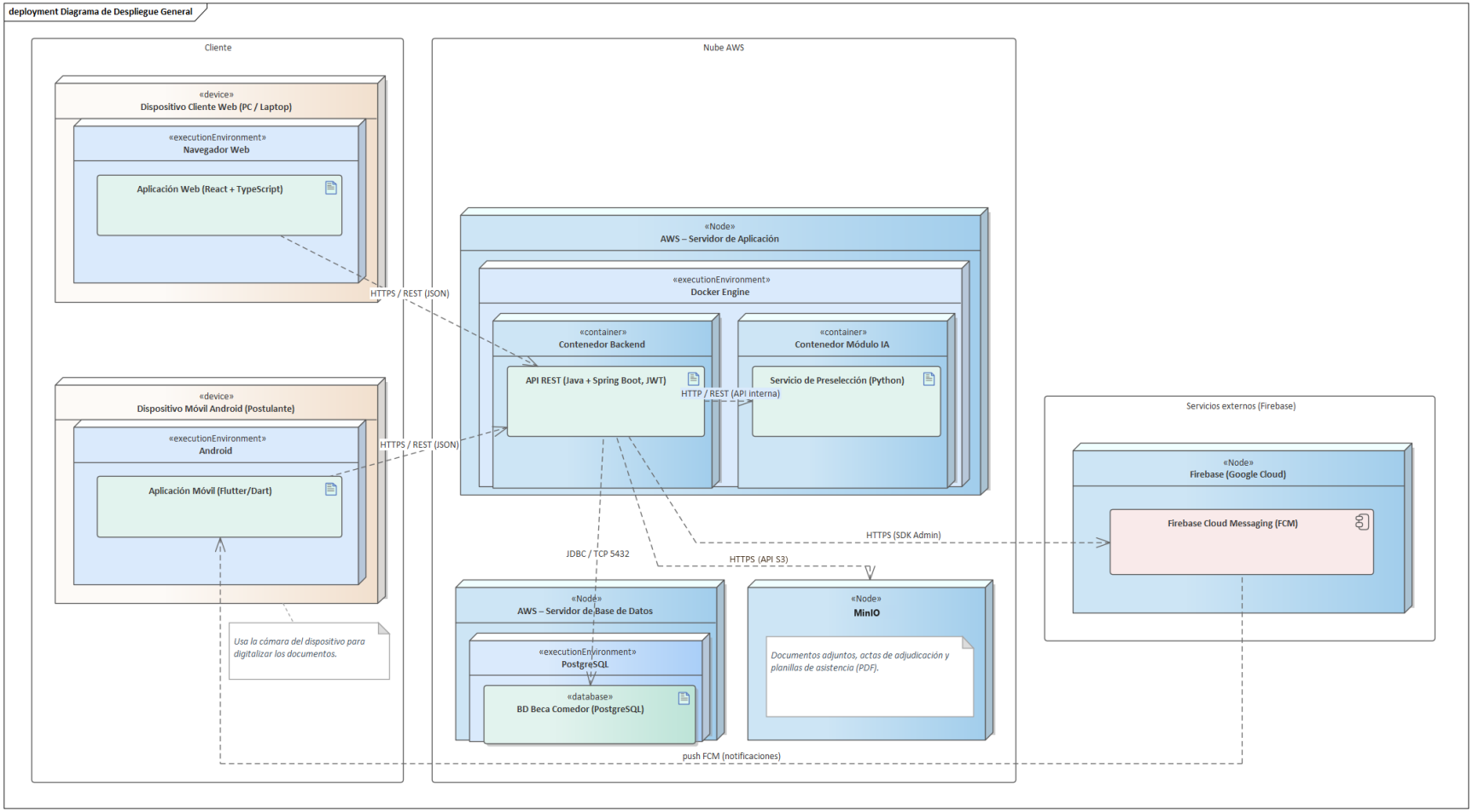
El diseño físico describe la arquitectura física del sistema, es decir, su topología: los nodos de hardware y los entornos de ejecución, junto con los artefactos de software que se ejecutan en cada uno y los protocolos de comunicación entre ellos. Se representa mediante un diagrama de despliegue del Lenguaje Unificado de Modelado (Booch et al., 2005; Rumbaugh et al., 2004).

El sistema se organiza en tres grupos de nodos. El primero corresponde al cliente: la aplicación web (React y TypeScript), que se ejecuta en el navegador de una computadora personal, y la aplicación móvil (Flutter), que se ejecuta en un dispositivo Android y utiliza la cámara para digitalizar los documentos de la postulación. El segundo grupo corresponde a la nube de Amazon Web Services (AWS), que contiene un servidor de aplicación con Docker Engine, donde se ejecutan en contenedores separados la API REST (Java y Spring Boot, que emite y verifica los tokens JWT de autenticación) y el servicio de preselección en Python; un servidor de base de datos con PostgreSQL; y MinIO (almacenamiento de objetos compatible con S3) para el almacenamiento de los documentos adjuntos, las actas de adjudicación y las planillas de asistencia en formato PDF. El tercer grupo reúne los servicios externos de Firebase, de los cuales se emplea Firebase Cloud Messaging (FCM) para el envío de notificaciones *push* al dispositivo móvil.

Las aplicaciones cliente se comunican con la API REST mediante HTTPS en formato JSON; el módulo de preselección solo es accesible desde la API, mediante una interfaz interna HTTP, y no se expone a los clientes. La API accede a PostgreSQL mediante JDBC, a MinIO mediante su API compatible con S3 y a FCM mediante el SDK de administración de Firebase. La Figura 93 presenta el diagrama de despliegue general del sistema.

## Figura 93

*Diagrama de despliegue general del sistema*




*Nota.* Elaboración propia.

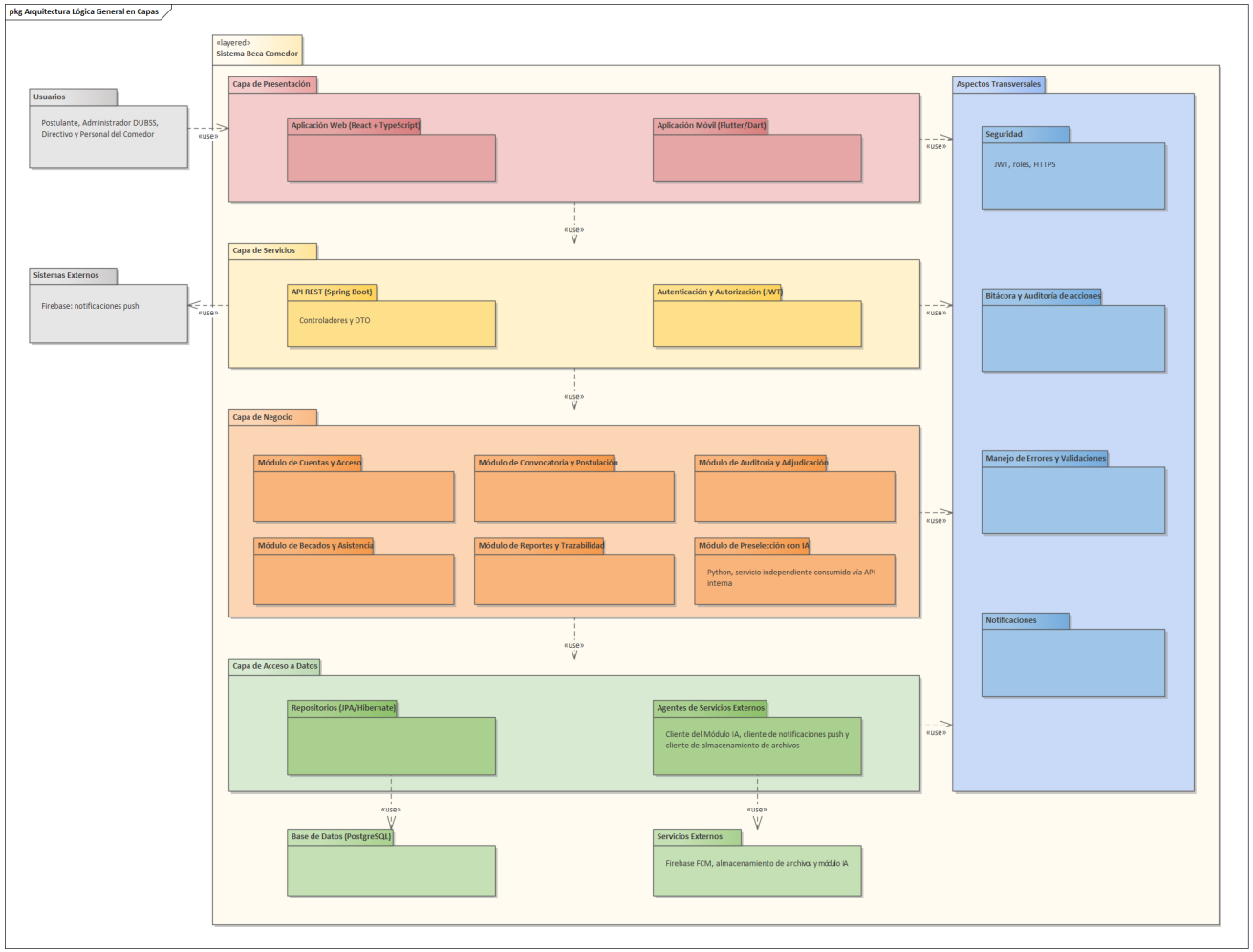
### *7.1.2. Diseño Lógico*

El diseño lógico describe la organización interna del sistema en capas y paquetes, independientemente de los nodos físicos en los que se ejecuta (sección 66). Se representa mediante diagramas de paquetes del Lenguaje Unificado de Modelado, en los que cada capa agrupa los paquetes con responsabilidades afines y las dependencias de uso (*use*) se dirigen de las capas superiores hacia las inferiores (Booch et al., 2005; Rumbaugh et al., 2004). Este criterio es coherente con el encapsulamiento de los casos de uso en paquetes realizado en el análisis (sección 66), de modo que los seis paquetes funcionales identificados se conservan como módulos de la capa de negocio (Jacobson et al., 1999).

**Arquitectura lógica general.** La Figura 94 presenta la arquitectura lógica general del sistema, organizada en cuatro capas y un paquete de aspectos transversales. La capa de presentación contiene la aplicación web (React y TypeScript) y la aplicación móvil (Flutter), con las que interactúan los usuarios del sistema. La capa de servicios expone la API REST de Spring Boot, con sus controladores y objetos de transferencia de datos, y el mecanismo de autenticación y autorización, que emite y verifica los tokens JWT y aplica el rol del usuario. La capa de negocio agrupa los módulos de cuentas y acceso, convocatoria y postulación, auditoría y adjudicación, becados y asistencia, reportes y trazabilidad, y preselección con inteligencia artificial; este último se implementa en Python como servicio independiente que se consume mediante una API interna. La capa de acceso a datos reúne los repositorios (JPA e Hibernate) y los agentes de servicios externos, que a su vez dependen de la base de datos PostgreSQL y de los servicios externos (notificaciones push, almacenamiento de archivos y módulo de inteligencia artificial). Finalmente, los aspectos transversales (seguridad, bitácora y auditoría de acciones, manejo de errores y validaciones, y notificaciones) son utilizados por todas las capas.

**Figura 94**

*Arquitectura lógica general del sistema en capas*

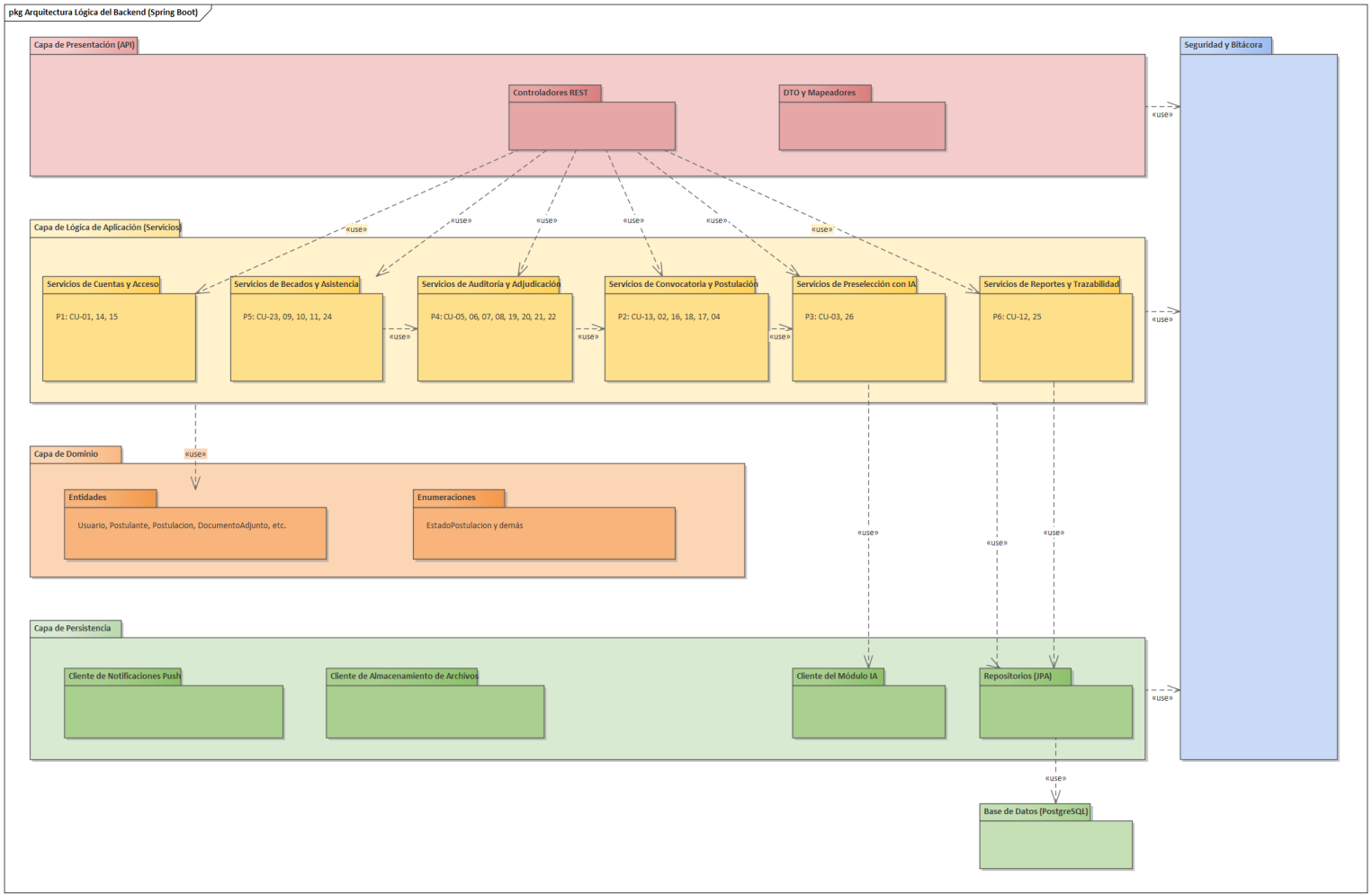



*Nota.* Elaboración propia.

**Arquitectura lógica del *backend*.** La Figura 95 detalla la organización del *backend* desarrollado con Spring Boot. La capa de presentación (API) contiene los controladores REST y los objetos de transferencia de datos con sus mapeadores; los controladores invocan a los servicios de la capa de lógica de aplicación, que se organizan según los seis paquetes funcionales: cuentas y acceso (P1), convocatoria y postulación (P2), preselección con IA (P3), auditoría y adjudicación (P4), becados y asistencia (P5), y reportes y trazabilidad (P6). Entre estos paquetes existen dependencias que reflejan el flujo del proceso: los servicios de becados y asistencia utilizan los de auditoría y adjudicación, estos utilizan los de convocatoria y postulación, y estos últimos los de preselección con IA. Los servicios operan sobre la capa de dominio, que contiene las entidades y las enumeraciones (por ejemplo, el estado de la postulación), y acceden a la capa de persistencia, conformada por los repositorios JPA, el cliente del módulo de inteligencia artificial y los clientes de notificaciones push y de almacenamiento de archivos. Los repositorios acceden a la base de datos PostgreSQL, y el paquete de seguridad y bitácora es utilizado por todas las capas.

**Figura 95**

*Arquitectura lógica del backend (Spring Boot)*




*Nota.* Elaboración propia. P1 a P6 corresponden a los paquetes de casos de uso definidos en el análisis.

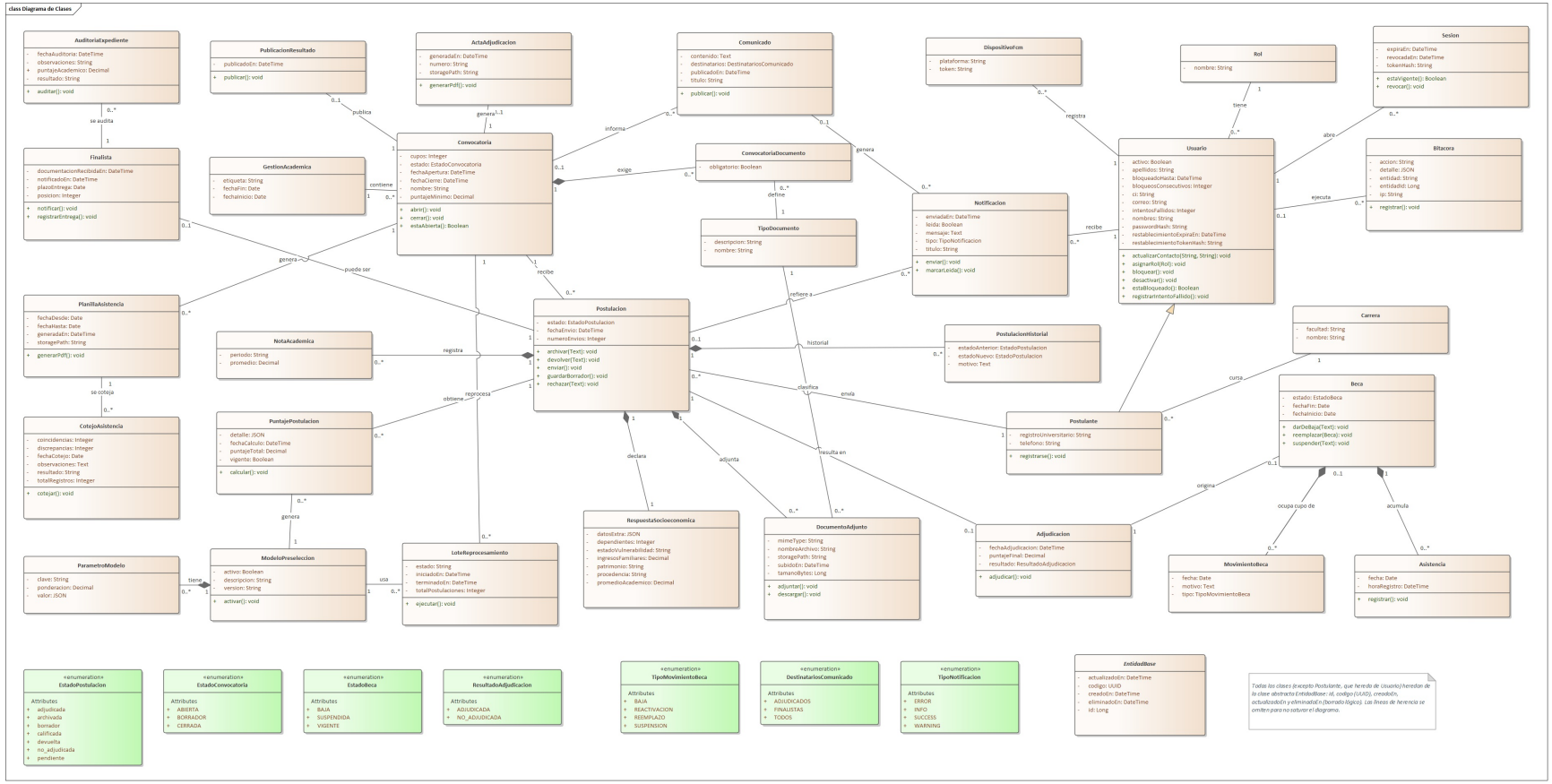
## 7.2. Diagrama de Clase

El diagrama de clases de diseño integra las clases de entidad identificadas en el análisis de los casos de uso (Capítulo 6) y las refina con sus atributos, tipos de datos, operaciones y relaciones de asociación, composición y herencia, expresadas en el Lenguaje Unificado de Modelado (Booch et al., 2005). Las clases representan la información persistente del dominio de la Beca Comedor y se corresponden con las tablas del diseño de datos (sección 66); los valores admitidos para los estados y tipos se modelan como enumeraciones.

Todas las clases, excepto *Postulante*, que hereda de *Usuario*, heredan de la clase abstracta *EntidadBase*, que aporta el identificador interno, el código público (UUID) y las fechas de creación, actualización y eliminación lógica; las líneas de esa herencia se omiten en el diagrama para no saturarlo. La Figura 96 presenta el diagrama de clases del sistema.

## Figura 96

*Diagrama de clases del sistema*




*Nota.* Elaboración propia.

## 7.3. Diseño de Datos

El diseño de datos traduce las clases persistentes del diagrama de clases (sección 66) a un modelo relacional implementado en PostgreSQL. El modelo consta de 32 tablas agrupadas según los seis paquetes del análisis: Cuentas y acceso (5), Convocatoria y postulación (9), Preselección con IA (4), Auditoría y adjudicación (8), Becados y asistencia (5) y Reportes y trazabilidad (1). Los nombres de tablas y columnas se escriben en minúsculas, en singular y separados por guion bajo; cada tabla tiene una clave primaria interna *id* y las claves foráneas se nombran *<tabla>_id*.

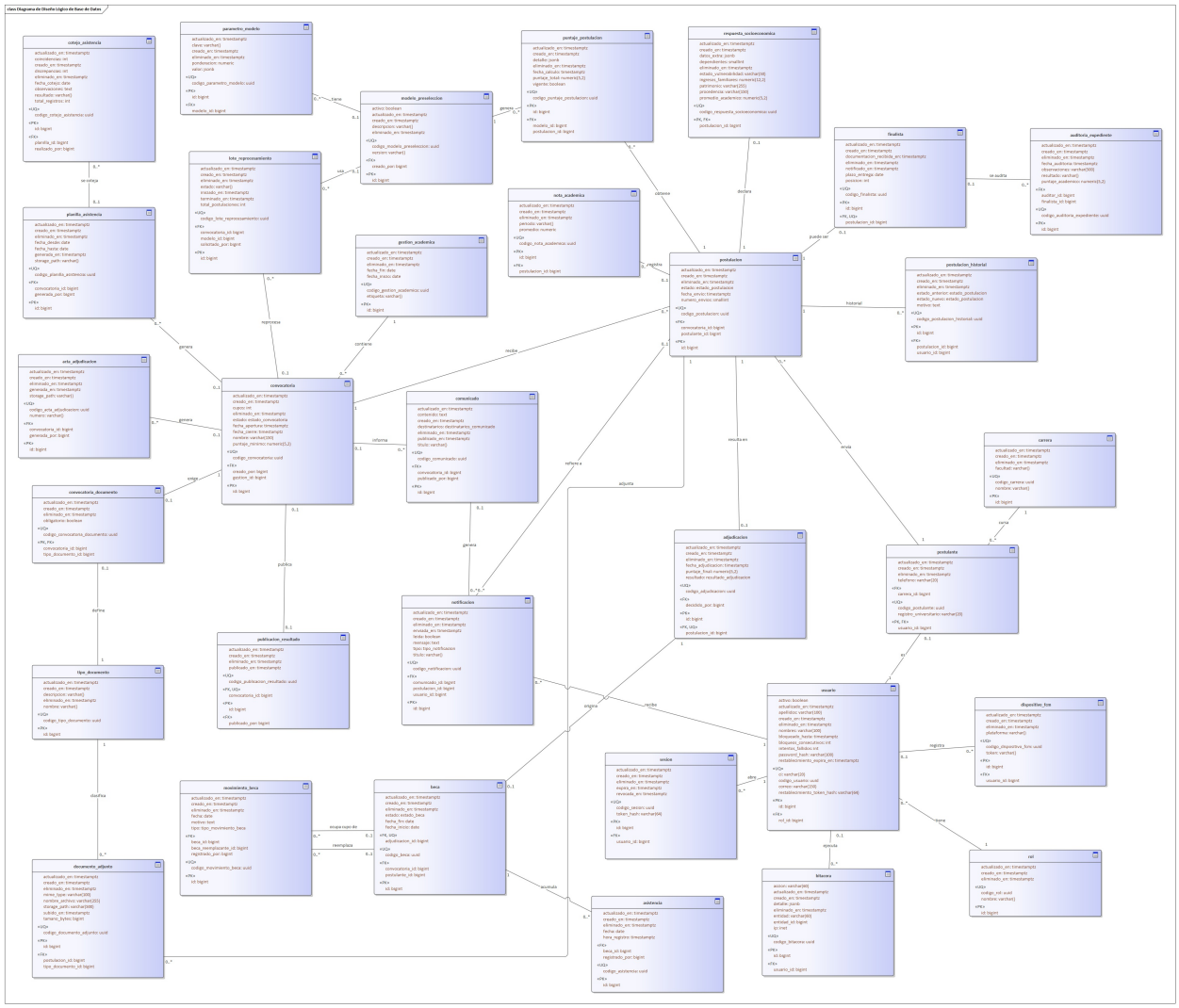
Todas las tablas incorporan cuatro columnas comunes: *codigo_<tabla>*, un identificador público UUID versión 7, único y no nulo, que es el único identificador que se expone a la API y a los clientes web y móvil; *creado_en* y *actualizado_en*, que registran la fecha y hora de creación y de la última modificación; y *eliminado_en*, que implementa el borrado lógico, de modo que ningún registro se elimina físicamente. Las contraseñas se almacenan únicamente como hash (BCrypt) y los tokens de renovación de sesión y de recuperación de contraseña solo como hash (SHA-256), y los archivos se guardan en MinIO, del que la base de datos solo conserva la ruta.

### *7.3.1. Diseño de Datos Lógico*

**7.3.1.1. Diseño lógico de la Base de Datos.** El diseño lógico de la base de datos representa cada tabla con sus columnas, tipos de datos, claves primarias (PK), claves foráneas (FK) y restricciones de unicidad (UQ), así como las relaciones y cardinalidades entre las tablas. La Figura 97 presenta el diagrama del diseño lógico.

**Figura 97**

*Diseño lógico de la base de datos*



*Nota.* Elaboración propia.

**7.3.1.2. Mapeo.** El mapeo presenta, para cada una de las 32 tablas del diseño lógico, sus atributos y el tipo de dato asignado a cada uno, indicando entre paréntesis las claves primarias (PK), foráneas (FK) y únicas (UQ). Las tablas se presentan agrupadas por paquete, en el mismo orden del diseño lógico.

**Tabla 67**

*Mapeo de la tabla rol*

| id (PK) | nombre (UQ) | codigo_rol (UQ) | creado_en | actualizado_en |
|---|---|---|---|---|
| bigint | varchar(50) | uuid | timestamptz | timestamptz |

| eliminado_en |
|---|
| timestamptz |

*Nota.* Elaboración propia.

**Tabla 68**

*Mapeo de la tabla usuario*

| id (PK) | ci (UQ) | correo (UQ) | nombres | apellidos |
|---|---|---|---|---|
| bigint | varchar(20) | varchar(150) | varchar(100) | varchar(100) |

| password_hash | activo | rol_id (FK) | codigo_usuario (UQ) | intentos_fallidos |
|---|---|---|---|---|
| varchar(100) | boolean | bigint | uuid | integer |

| bloqueos_consecutivos | bloqueado_hasta | restablecimiento_token_hash (UQ) | restablecimiento_expira_en | creado_en |
|---|---|---|---|---|
| integer | timestamptz | varchar(64) | timestamptz | timestamptz |

| actualizado_en | eliminado_en |
|---|---|
| timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 69**

*Mapeo de la tabla carrera*

| id (PK) | nombre (UQ) | facultad | codigo_carrera (UQ) | creado_en |
|---|---|---|---|---|
| bigint | varchar(150) | varchar(150) | uuid | timestamptz |

| actualizado_en | eliminado_en |
|---|---|
| timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 70**

*Mapeo de la tabla sesion*

| id (PK) | usuario_id (FK) | codigo_sesion (UQ) | token_hash (UQ) | expira_en |
|---|---|---|---|---|
| bigint | bigint | uuid | varchar(64) | timestamptz |

| revocada_en | creado_en | actualizado_en | eliminado_en |
|---|---|---|---|
| timestamptz | timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 71**

*Mapeo de la tabla postulante*

| usuario_id (PK, FK) | carrera_id (FK) | registro_ universitario (UQ) | telefono | codigo_ postulante (UQ) |
|---|---|---|---|---|
| bigint | bigint | varchar(20) | varchar(20) | uuid |

| creado_en | actualizado_en | eliminado_en |
|---|---|---|
| timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 72**

*Mapeo de la tabla gestion_academica*

| id (PK) | etiqueta (UQ) | fecha_inicio | fecha_fin | codigo_ gestion_ academica (UQ) |
|---|---|---|---|---|
| bigint | varchar(20) | date | date | uuid |

| creado_en | actualizado_en | eliminado_en |
|---|---|---|
| timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 73**

*Mapeo de la tabla convocatoria*

| id (PK) | gestion_id (FK) | nombre | fecha_apertura | fecha_cierre |
|---|---|---|---|---|
| bigint | bigint | varchar(150) | timestamptz | timestamptz |

| cupos | puntaje_ minimo | estado | creado_por (FK) | codigo_ convocatoria (UQ) |
|---|---|---|---|---|
| integer | numeric(5,2) | estado_ convocatoria | bigint | uuid |

| creado_en | actualizado_en | eliminado_en |
|---|---|---|
| timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 74**

*Mapeo de la tabla tipo_documento*

| id (PK) | nombre | descripcion | codigo_tipo_documento (UQ) | creado_en |
|---|---|---|---|---|
| bigint | varchar(100) | varchar(255) | uuid | timestamptz |

| actualizado_en | eliminado_en |
|---|---|
| timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 75**

*Mapeo de la tabla convocatoria_documento*

| convocatoria_id (PK, FK) | tipo_documento_id (PK, FK) | obligatorio | codigo_convocatoria_documento (UQ) | creado_en |
|---|---|---|---|---|
| bigint | bigint | boolean | uuid | timestamptz |

| actualizado_en | eliminado_en |
|---|---|
| timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 76**

*Mapeo de la tabla postulacion*

| id (PK) | postulante_id (FK) | convocatoria_id (FK) | estado | fecha_envio |
|---|---|---|---|---|
| bigint | bigint | bigint | estado_postulacion | timestamptz |

| numero_envios | codigo_postulacion (UQ) | creado_en | actualizado_en | eliminado_en |
|---|---|---|---|---|
| smallint | uuid | timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 77**

*Mapeo de la tabla respuesta_socioeconomica*

| postulacion_id (PK, FK) | ingresos_ familiares | dependientes | procedencia | patrimonio |
|---|---|---|---|---|
| bigint | numeric(12,2) | smallint | varchar(150) | varchar(255) |

| estado_ vulnerabilidad | promedio_ academico | datos_extra | codigo_ respuesta_ socioeconomica (UQ) | creado_en |
|---|---|---|---|---|
| varchar(50) | numeric(5,2) | jsonb | uuid | timestamptz |

| actualizado_en | eliminado_en |
|---|---|
| timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 78**

*Mapeo de la tabla nota_academica*

| id (PK) | postulacion_id (FK) | periodo | promedio | codigo_nota_ academica (UQ) |
|---|---|---|---|---|
| bigint | bigint | varchar(20) | numeric | uuid |

| creado_en | actualizado_en | eliminado_en |
|---|---|---|
| timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 79**

*Mapeo de la tabla documento_adjunto*

| id (PK) | postulacion_id (FK) | tipo_ documento_id (FK) | nombre_ archivo | storage_path |
|---|---|---|---|---|
| bigint | bigint | bigint | varchar(255) | varchar(500) |

| mime_type | tamano_bytes | subido_en | codigo_ documento_ adjunto (UQ) | creado_en |
|---|---|---|---|---|
| varchar(100) | bigint | timestamptz | uuid | timestamptz |

| actualizado_en | eliminado_en |
|---|---|
| timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 80**

*Mapeo de la tabla postulacion_historial*

| id (PK) | postulacion_id (FK) | estado_anterior | estado_nuevo | usuario_id (FK) |
|---|---|---|---|---|
| bigint | bigint | estado_ postulacion | estado_ postulacion | bigint |

| motivo | codigo_ postulacion_ historial (UQ) | creado_en | actualizado_en | eliminado_en |
|---|---|---|---|---|
| text | uuid | timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 81**

*Mapeo de la tabla modelo_preseleccion*

| id (PK) | version (UQ) | descripcion | activo | creado_por (FK) |
|---|---|---|---|---|
| bigint | varchar(20) | varchar(255) | boolean | bigint |

| codigo_ modelo_ preseleccion (UQ) | creado_en | actualizado_en | eliminado_en | |
|---|---|---|---|---|
| uuid | timestamptz | timestamptz | timestamptz | |

*Nota.* Elaboración propia.

**Tabla 82**

*Mapeo de la tabla parametro_modelo*

| id (PK) | modelo_id (FK) | clave | ponderacion | valor |
|---|---|---|---|---|
| bigint | bigint | varchar(100) | numeric | jsonb |

| codigo_ parametro_ modelo (UQ) | creado_en | actualizado_en | eliminado_en | |
|---|---|---|---|---|
| uuid | timestamptz | timestamptz | timestamptz | |

*Nota.* Elaboración propia.

**Tabla 83**

*Mapeo de la tabla puntaje_postulacion*

| id (PK) | postulacion_id (FK) | modelo_id (FK) | puntaje_total | detalle |
|---|---|---|---|---|
| bigint | bigint | bigint | numeric(5,2) | jsonb |

| fecha_calculo | vigente | codigo_puntaje_postulacion (UQ) | creado_en | actualizado_en |
|---|---|---|---|---|
| timestamptz | boolean | uuid | timestamptz | timestamptz |

| eliminado_en |
|---|
| timestamptz |

*Nota.* Elaboración propia.

**Tabla 84**

*Mapeo de la tabla lote_reprocesamiento*

| id (PK) | convocatoria_id (FK) | modelo_id (FK) | solicitado_por (FK) | estado |
|---|---|---|---|---|
| bigint | bigint | bigint | bigint | varchar(20) |

| total_postulaciones | iniciado_en | terminado_en | codigo_lote_reprocesamiento (UQ) | creado_en |
|---|---|---|---|---|
| integer | timestamptz | timestamptz | uuid | timestamptz |

| actualizado_en | eliminado_en |
|---|---|
| timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 85**

*Mapeo de la tabla finalista*

| id (PK) | postulacion_id (FK, UQ) | posicion | notificado_en | plazo_entrega |
|---|---|---|---|---|
| bigint | bigint | integer | timestamptz | date |

| documentacion_recibida_en | codigo_finalista (UQ) | creado_en | actualizado_en | eliminado_en |
|---|---|---|---|---|
| timestamptz | uuid | timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 86**

*Mapeo de la tabla auditoria_expediente*

| id (PK) | finalista_id (FK) | auditor_id (FK) | resultado | puntaje_academico |
|---|---|---|---|---|
| bigint | bigint | bigint | varchar(20) | numeric(5,2) |

| observaciones | fecha_auditoria | codigo_auditoria_expediente (UQ) | creado_en | actualizado_en | eliminado_en |
|---|---|---|---|---|---|
| varchar(500) | timestamptz | uuid | timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 87**

*Mapeo de la tabla adjudicacion*

| id (PK) | postulacion_id (FK, UQ) | resultado | puntaje_final | decidido_por (FK) |
|---|---|---|---|---|
| bigint | bigint | resultado_ adjudicacion | numeric(5,2) | bigint |

| fecha_ adjudicacion | codigo_ adjudicacion (UQ) | creado_en | actualizado_en | eliminado_en |
|---|---|---|---|---|
| timestamptz | uuid | timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 88**

*Mapeo de la tabla acta_adjudicacion*

| id (PK) | convocatoria_ id (FK) | numero (UQ) | storage_path | generada_por (FK) |
|---|---|---|---|---|
| bigint | bigint | varchar(30) | varchar(500) | bigint |

| generada_en | codigo_acta_ adjudicacion (UQ) | creado_en | actualizado_en | eliminado_en |
|---|---|---|---|---|
| timestamptz | uuid | timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 89**

*Mapeo de la tabla publicacion_resultado*

| id (PK) | convocatoria_ id (FK, UQ) | publicado_por (FK) | publicado_en | codigo_ publicacion_ resultado (UQ) |
|---|---|---|---|---|
| bigint | bigint | bigint | timestamptz | uuid |

| creado_en | actualizado_en | eliminado_en |
|---|---|---|
| timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 90**

*Mapeo de la tabla comunicado*

| id (PK) | convocatoria_ id (FK) | titulo | contenido | destinatarios |
|---|---|---|---|---|
| bigint | bigint | varchar(150) | text | destinatarios_ comunicado |

| publicado_por (FK) | publicado_en | codigo_ comunicado (UQ) | creado_en | actualizado_en |
|---|---|---|---|---|
| bigint | timestamptz | uuid | timestamptz | timestamptz |

| eliminado_en |
|---|
| timestamptz |

*Nota.* Elaboración propia.

**Tabla 91**

*Mapeo de la tabla notificacion*

| id (PK) | usuario_id (FK) | tipo | titulo | mensaje |
|---|---|---|---|---|
| bigint | bigint | tipo_notificacion | varchar(150) | text |

| postulacion_id (FK) | comunicado_id (FK) | leida | enviada_en | codigo_ notificacion (UQ) |
|---|---|---|---|---|
| bigint | bigint | boolean | timestamptz | uuid |

| creado_en | actualizado_en | eliminado_en |
|---|---|---|
| timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 92**

*Mapeo de la tabla dispositivo_fcm*

| id (PK) | usuario_id (FK) | token (UQ) | plataforma | codigo_ dispositivo_fcm (UQ) |
|---|---|---|---|---|
| bigint | bigint | varchar(255) | varchar(10) | uuid |

| creado_en | actualizado_en | eliminado_en |
|---|---|---|
| timestamptz | timestamptz | timestamptz |

*Nota.* Elaboración propia.

**Tabla 93**

*Mapeo de la tabla beca*

| id (PK) | adjudicacion_id (FK, UQ) | postulante_id (FK) | convocatoria_ id (FK) | estado |
|---|---|---|---|---|
| bigint | bigint | bigint | bigint | estado_beca |

| fecha_inicio | fecha_fin | codigo_beca (UQ) | creado_en | actualizado_en |
|---|---|---|---|---|
| date | date | uuid | timestamptz | timestamptz |

| eliminado_en |
|---|
| timestamptz |

*Nota.* Elaboración propia.

**Tabla 94**

*Mapeo de la tabla movimiento_beca*

| id (PK) | beca_id (FK) | tipo | motivo | beca_ reemplazante_ id (FK) |
|---|---|---|---|---|
| bigint | bigint | tipo_ movimiento_beca | text | bigint |

| registrado_por (FK) | fecha | codigo_ movimiento_ beca (UQ) | creado_en | actualizado_en |
|---|---|---|---|---|
| bigint | date | uuid | timestamptz | timestamptz |

| eliminado_en |
|---|
| timestamptz |

*Nota.* Elaboración propia.

**Tabla 95**

*Mapeo de la tabla planilla_asistencia*

| id (PK) | convocatoria_ id (FK) | fecha_desde | fecha_hasta | storage_path |
|---|---|---|---|---|
| bigint | bigint | date | date | varchar(500) |

| generada_por (FK) | generada_en | codigo_ planilla_ asistencia (UQ) | creado_en | actualizado_en |
|---|---|---|---|---|
| bigint | timestamptz | uuid | timestamptz | timestamptz |

| eliminado_en |
|---|
| timestamptz |

*Nota.* Elaboración propia.

**Tabla 96**

*Mapeo de la tabla asistencia*

| id (PK) | beca_id (FK) | fecha | hora_registro | registrado_por (FK) |
|---|---|---|---|---|
| bigint | bigint | date | timestamptz | bigint |

| codigo_asistencia (UQ) | creado_en | actualizado_en | eliminado_en | |
|---|---|---|---|---|
| uuid | timestamptz | timestamptz | timestamptz | |

*Nota.* Elaboración propia.

**Tabla 97**

*Mapeo de la tabla cotejo_asistencia*

| id (PK) | planilla_id (FK) | realizado_por (FK) | fecha_cotejo | total_registros |
|---|---|---|---|---|
| bigint | bigint | bigint | date | integer |

| coincidencias | discrepancias | observaciones | resultado | codigo_cotejo_asistencia (UQ) |
|---|---|---|---|---|
| integer | integer | text | varchar(20) | uuid |

| creado_en | actualizado_en | eliminado_en | | |
|---|---|---|---|---|
| timestamptz | timestamptz | timestamptz | | |

*Nota.* Elaboración propia.

**Tabla 98**

*Mapeo de la tabla bitacora*

| id (PK) | usuario_id (FK) | accion | entidad | entidad_id |
|---|---|---|---|---|
| bigint | bigint | varchar(60) | varchar(60) | bigint |

| detalle | ip | codigo_ bitacora (UQ) | creado_en | actualizado_en |
|---|---|---|---|---|
| jsonb | inet | uuid | timestamptz | timestamptz |

| eliminado_en |
|---|
| timestamptz |

*Nota.* Elaboración propia.

### *7.3.2. Diseño de Datos Físico*

**7.3.2.1. Volumen.** El volumen detalla cada atributo de las tablas con su tipo de dato, descripción, tamaño (longitud máxima en los tipos *varchar* y precisión y escala en los tipos *numeric*), si admite valores nulos y el tipo de llave. El diseño se dimensiona para un volumen de hasta 20,000 postulaciones por gestión académica.

**Tabla 99**

*Volumen de la tabla rol*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| nombre | varchar(50) | Nombre del rol (POSTULANTE, ADMINISTRADOR_ DUBSS, DIRECTIVO o PERSONAL_COMEDOR) | 50 | No | UQ |
| codigo_rol | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 100**

*Volumen de la tabla usuario*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| ci | varchar(20) | Cédula de identidad del usuario | 20 | No | UQ |
| correo | varchar(150) | Correo electrónico del usuario | 150 | No | UQ |
| nombres | varchar(100) | Nombres del usuario | 100 | No | No |
| apellidos | varchar(100) | Apellidos del usuario | 100 | No | No |
| password_hash | varchar(100) | Hash BCrypt de la contraseña (nunca el texto plano) | 100 | No | No |
| activo | boolean | Indica si la cuenta está habilitada | — | No | No |
| rol_id | bigint | Identificador del rol asignado | — | No | FK |
| codigo_usuario | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| intentos_fallidos | integer | Intentos de inicio de sesión fallidos consecutivos | — | No | No |
| bloqueos_ consecutivos | integer | Cantidad de bloqueos seguidos, que define la duración del siguiente | — | No | No |
| bloqueado_hasta | timestamptz | Fecha y hora hasta la que la cuenta está bloqueada (nula si no lo está) | — | Sí | No |
| restablecimiento_ token_hash | varchar(64) | Hash SHA-256 del token del enlace vigente de recuperación de contraseña (nunca el token; nulo si no hay enlace vigente) | 64 | Sí | UQ |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| restablecimiento_ expira_en | timestamptz | Fecha y hora de vencimiento del enlace de recuperación (nula si no hay enlace vigente) | — | Sí | No |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 101**

*Volumen de la tabla carrera*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| nombre | varchar(150) | Nombre de la carrera | 150 | No | UQ |
| facultad | varchar(150) | Facultad a la que pertenece la carrera | 150 | No | No |
| codigo_carrera | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 102**

*Volumen de la tabla sesion*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| usuario_id | bigint | Identificador del usuario dueño de la sesión | — | No | FK |
| codigo_sesion | uuid | Identificador público (UUID v7) de la sesión | — | No | UQ |
| token_hash | varchar(64) | Hash SHA-256 del token de renovación (nunca el token) | 64 | No | UQ |
| expira_en | timestamptz | Fecha y hora de vencimiento del token de renovación | — | No | No |
| revocada_en | timestamptz | Fecha y hora en que se revocó la sesión (nula si sigue vigente) | — | Sí | No |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 103**

*Volumen de la tabla postulante*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| usuario_id | bigint | Identificador del usuario (clave primaria y foránea) | — | No | PK, FK |
| carrera_id | bigint | Identificador de la carrera que cursa | — | No | FK |
| registro_ universitario | varchar(20) | Registro universitario del estudiante (formato por definir con la DUBSS) | 20 | No | UQ |
| telefono | varchar(20) | Teléfono de contacto | 20 | No | No |
| codigo_postulante | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 104**

*Volumen de la tabla gestion_academica*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| etiqueta | varchar(20) | Identificación de la gestión, p. ej. 2026-II (formato por definir con la DUBSS) | 20 | No | UQ |
| fecha_inicio | date | Fecha de inicio de la gestión | — | No | No |
| fecha_fin | date | Fecha de finalización de la gestión | — | No | No |
| codigo_gestion_academica | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 105**

*Volumen de la tabla convocatoria*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| gestion_id | bigint | Identificador de la gestión académica | — | No | FK |
| nombre | varchar(150) | Nombre de la convocatoria | 150 | No | No |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| fecha_apertura | timestamptz | Fecha y hora de apertura de la convocatoria | — | No | No |
| fecha_cierre | timestamptz | Fecha y hora de cierre de la convocatoria | — | No | No |
| cupos | integer | Número de becas disponibles (por definir con la DUBSS) | — | Sí | No |
| puntaje_minimo | numeric(5,2) | Puntaje mínimo requerido para adjudicar (por definir con la DUBSS) | 5,2 | Sí | No |
| estado | estado_ convocatoria | Estado de la convocatoria (BORRADOR, ABIERTA o CERRADA) | — | No | No |
| creado_por | bigint | Identificador del administrador que registró la convocatoria | — | No | FK |
| codigo_ convocatoria | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 106**

*Volumen de la tabla tipo_documento*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| nombre | varchar(100) | Nombre del tipo de documento (por definir con la DUBSS) | 100 | No | No |
| descripcion | varchar(255) | Descripción del tipo de documento | 255 | Sí | No |
| codigo_tipo_ documento | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 107**

*Volumen de la tabla convocatoria_documento*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| convocatoria_id | bigint | Identificador de la convocatoria (clave primaria compuesta) | — | No | PK, FK |
| tipo_documento_id | bigint | Identificador del tipo de documento (clave primaria compuesta) | — | No | PK, FK |
| obligatorio | boolean | Indica si el documento es obligatorio | — | No | No |
| codigo_convocatoria_documento | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 108**

*Volumen de la tabla postulacion*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| postulante_id | bigint | Identificador del postulante | — | No | FK |
| convocatoria_id | bigint | Identificador de la convocatoria | — | No | FK |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| estado | estado_ postulacion | Estado de la postulación | — | No | No |
| fecha_envio | timestamptz | Fecha y hora del último envío (nula mientras es borrador) | — | Sí | No |
| numero_envios | smallint | Cantidad de envíos (aumenta con cada reenvío tras una devolución) | — | No | No |
| codigo_postulacion | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 109**

*Volumen de la tabla respuesta_socioeconomica*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| postulacion_id | bigint | Identificador de la postulación (clave primaria y foránea) | — | No | PK, FK |
| ingresos_familiares | numeric(12,2) | Ingresos familiares declarados | 12,2 | Sí | No |
| dependientes | smallint | Número de dependientes del grupo familiar | — | Sí | No |
| procedencia | varchar(150) | Lugar de procedencia del postulante | 150 | Sí | No |
| patrimonio | varchar(255) | Descripción o categoría del patrimonio (por definir con la DUBSS) | 255 | Sí | No |
| estado_ vulnerabilidad | varchar(50) | Estado de vulnerabilidad declarado (catálogo por definir con la DUBSS) | 50 | Sí | No |
| promedio_ academico | numeric(5,2) | Promedio académico declarado | 5,2 | Sí | No |
| datos_extra | jsonb | Variables adicionales del formulario | — | Sí | No |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| codigo_respuesta_socioeconomica | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 110**

*Volumen de la tabla nota_academica*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| postulacion_id | bigint | Identificador de la postulación | — | No | FK |
| periodo | varchar(20) | Periodo académico de la nota | 20 | No | No |
| promedio | numeric | Promedio obtenido en el periodo | — | No | No |
| codigo_nota_academica | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 111**

*Volumen de la tabla documento_adjunto*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| postulacion_id | bigint | Identificador de la postulación | — | No | FK |
| tipo_documento_id | bigint | Identificador del tipo de documento | — | No | FK |
| nombre_archivo | varchar(255) | Nombre original del archivo | 255 | No | No |
| storage_path | varchar(500) | Ruta del archivo en MinIO | 500 | No | No |
| mime_type | varchar(100) | Tipo de contenido del archivo | 100 | No | No |
| tamano_bytes | bigint | Tamaño del archivo en bytes | — | No | No |
| subido_en | timestamptz | Fecha y hora de carga del archivo | — | No | No |
| codigo_documento_adjunto | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 112**

*Volumen de la tabla postulacion_historial*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| postulacion_id | bigint | Identificador de la postulación | — | No | FK |
| estado_anterior | estado_postulacion | Estado previo al cambio (nulo en el primer registro) | — | Sí | No |
| estado_nuevo | estado_postulacion | Estado resultante del cambio | — | No | No |
| usuario_id | bigint | Identificador del usuario que realizó el cambio (nulo si fue el sistema) | — | Sí | FK |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| motivo | text | Motivo u observaciones del cambio (obligatorio al devolver, archivar o rechazar; el servicio valida un máximo de 500 caracteres) | — | Sí | No |
| codigo_ postulacion_ historial | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 113**

*Volumen de la tabla modelo_preseleccion*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| version | varchar(20) | Versión del modelo | 20 | No | UQ |
| descripcion | varchar(255) | Descripción de la versión del modelo | 255 | Sí | No |
| activo | boolean | Indica si es la versión vigente (solo una activa) | — | No | No |
| creado_por | bigint | Identificador del administrador que registró la versión | — | Sí | FK |
| codigo_modelo_ preseleccion | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 114**

*Volumen de la tabla parametro_modelo*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| modelo_id | bigint | Identificador del modelo de preselección | — | No | FK |
| clave | varchar(100) | Nombre del criterio o variable | 100 | No | No |
| ponderacion | numeric | Ponderación del criterio (por definir con la DUBSS) | — | Sí | No |
| valor | jsonb | Valores o rangos adicionales del parámetro | — | Sí | No |
| codigo_parametro_modelo | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 115**

*Volumen de la tabla puntaje_postulacion*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| postulacion_id | bigint | Identificador de la postulación | — | No | FK |
| modelo_id | bigint | Identificador de la versión del modelo que calculó el puntaje | — | No | FK |
| puntaje_total | numeric(5,2) | Puntaje de vulnerabilidad obtenido de la IA (0 a 70) | 5,2 | No | No |
| detalle | jsonb | Desglose del puntaje por variable | — | Sí | No |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| fecha_calculo | timestamptz | Fecha y hora del cálculo | — | No | No |
| vigente | boolean | Indica si es el puntaje vigente de la postulación | — | No | No |
| codigo_puntaje_postulacion | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 116**

*Volumen de la tabla lote_reprocesamiento*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| convocatoria_id | bigint | Identificador de la convocatoria reprocesada | — | No | FK |
| modelo_id | bigint | Identificador de la versión del modelo aplicada | — | No | FK |
| solicitado_por | bigint | Identificador del administrador que solicitó el reprocesamiento | — | No | FK |
| estado | varchar(20) | Estado del lote (PENDIENTE, EN_PROCESO, COMPLETADO o FALLIDO) | 20 | No | No |
| total_postulaciones | integer | Cantidad de postulaciones del lote | — | No | No |
| iniciado_en | timestamptz | Fecha y hora de inicio del procesamiento | — | Sí | No |
| terminado_en | timestamptz | Fecha y hora de finalización del procesamiento | — | Sí | No |
| codigo_lote_reprocesamiento | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 117**

*Volumen de la tabla finalista*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| postulacion_id | bigint | Identificador de la postulación | — | No | FK, UQ |
| posicion | integer | Posición en la lista priorizada al ser seleccionada | — | No | No |
| notificado_en | timestamptz | Fecha y hora de la notificación al finalista | — | Sí | No |
| plazo_entrega | date | Plazo de entrega de la documentación física (por definir con la DUBSS) | — | Sí | No |
| documentacion_recibida_en | timestamptz | Fecha y hora de recepción de la documentación física | — | Sí | No |
| codigo_finalista | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 118**

*Volumen de la tabla auditoria_expediente*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| finalista_id | bigint | Identificador del finalista auditado | — | No | FK |
| auditor_id | bigint | Identificador del administrador que auditó | — | No | FK |
| resultado | varchar(20) | Resultado de la auditoría (VALIDADO, OBSERVADO o NO_VALIDO) | 20 | No | No |
| puntaje_academico | numeric(5,2) | Puntaje de rendimiento académico registrado por el auditor (0 a 30); solo con resultado VALIDADO | 5,2 | Sí | No |
| observaciones | varchar(500) | Observaciones de la auditoría | 500 | Sí | No |
| fecha_auditoria | timestamptz | Fecha y hora de la auditoría | — | No | No |
| codigo_auditoria_expediente | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 119**

*Volumen de la tabla adjudicacion*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| postulacion_id | bigint | Identificador de la postulación | — | No | FK, UQ |
| resultado | resultado_adjudicacion | Resultado (ADJUDICADA o NO_ADJUDICADA) | — | No | No |
| puntaje_final | numeric(5,2) | Puntaje vigente al momento de la decisión | 5,2 | Sí | No |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| decidido_por | bigint | Identificador del administrador que decidió | — | No | FK |
| fecha_adjudicacion | timestamptz | Fecha y hora de la adjudicación | — | No | No |
| codigo_ adjudicacion | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 120**

*Volumen de la tabla acta_adjudicacion*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| convocatoria_id | bigint | Identificador de la convocatoria | — | No | FK |
| numero | varchar(30) | Número único del acta | 30 | No | UQ |
| storage_path | varchar(500) | Ruta del PDF del acta en MinIO | 500 | No | No |
| generada_por | bigint | Identificador del administrador que generó el acta | — | No | FK |
| generada_en | timestamptz | Fecha y hora de generación del acta | — | No | No |
| codigo_acta_ adjudicacion | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |

| | | | | | |
|---|---|---|---|---|---|
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 121**

*Volumen de la tabla publicacion_resultado*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| convocatoria_id | bigint | Identificador de la convocatoria | — | No | FK, UQ |
| publicado_por | bigint | Identificador del administrador que publicó | — | No | FK |
| publicado_en | timestamptz | Fecha y hora de la publicación | — | No | No |
| codigo_ publicacion_ resultado | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 122**

*Volumen de la tabla comunicado*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| convocatoria_id | bigint | Identificador de la convocatoria relacionada (opcional) | — | Sí | FK |
| titulo | varchar(150) | Título del comunicado | 150 | No | No |
| contenido | text | Contenido del comunicado | — | No | No |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| destinatarios | destinatarios_ comunicado | Destinatarios (TODOS, FINALISTAS o ADJUDICADOS) | — | No | No |
| publicado_por | bigint | Identificador del administrador que publicó | — | No | FK |
| publicado_en | timestamptz | Fecha y hora de publicación | — | No | No |
| codigo_comunicado | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 123**

*Volumen de la tabla notificacion*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| usuario_id | bigint | Identificador del usuario destinatario | — | No | FK |
| tipo | tipo_ notificacion | Tipo de notificación | — | No | No |
| titulo | varchar(150) | Título de la notificación | 150 | No | No |
| mensaje | text | Mensaje de la notificación | — | No | No |
| postulacion_id | bigint | Identificador de la postulación relacionada (opcional) | — | Sí | FK |
| comunicado_id | bigint | Identificador del comunicado relacionado (opcional) | — | Sí | FK |
| leida | boolean | Indica si el usuario leyó la notificación | — | No | No |
| enviada_en | timestamptz | Fecha y hora de envío del push (nula si falló) | — | Sí | No |
| codigo_notificacion | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 124**

*Volumen de la tabla dispositivo_fcm*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| usuario_id | bigint | Identificador del usuario propietario del dispositivo | — | No | FK |
| token | varchar(255) | Token de Firebase Cloud Messaging del dispositivo | 255 | No | UQ |
| plataforma | varchar(10) | Plataforma del dispositivo (ANDROID, IOS o WEB) | 10 | No | No |
| codigo_dispositivo_fcm | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 125**

*Volumen de la tabla beca*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| adjudicacion_id | bigint | Identificador de la adjudicación que origina la beca | — | No | FK, UQ |
| postulante_id | bigint | Identificador del postulante becado | — | No | FK |
| convocatoria_id | bigint | Identificador de la convocatoria | — | No | FK |
| estado | estado_beca | Estado de la beca (VIGENTE, SUSPENDIDA o BAJA) | — | No | No |
| fecha_inicio | date | Fecha de inicio del beneficio | — | No | No |
| fecha_fin | date | Fecha de finalización del beneficio | — | Sí | No |
| codigo_beca | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 126**

*Volumen de la tabla movimiento_beca*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| beca_id | bigint | Identificador de la beca afectada | — | No | FK |
| tipo | tipo_ movimiento_ beca | Tipo de movimiento | — | No | No |
| motivo | text | Motivo del movimiento | — | No | No |
| beca_ reemplazante_id | bigint | Identificador de la beca que ocupa el cupo liberado (solo en reemplazos) | — | Sí | FK |
| registrado_por | bigint | Identificador del administrador que registró el movimiento | — | No | FK |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| fecha | date | Fecha del movimiento | — | No | No |
| codigo_ movimiento_beca | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 127**

*Volumen de la tabla planilla_asistencia*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| convocatoria_id | bigint | Identificador de la convocatoria | — | No | FK |
| fecha_desde | date | Fecha inicial del periodo de la planilla | — | No | No |
| fecha_hasta | date | Fecha final del periodo de la planilla | — | No | No |
| storage_path | varchar(500) | Ruta del PDF de la planilla en MinIO | 500 | No | No |
| generada_por | bigint | Identificador del administrador que generó la planilla | — | No | FK |
| generada_en | timestamptz | Fecha y hora de generación | — | No | No |
| codigo_planilla_ asistencia | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 128**

*Volumen de la tabla asistencia*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| beca_id | bigint | Identificador de la beca | — | No | FK |
| fecha | date | Fecha de la asistencia | — | No | No |
| hora_registro | timestamptz | Fecha y hora del registro | — | No | No |
| registrado_por | bigint | Identificador del personal del comedor que registró | — | No | FK |
| codigo_asistencia | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 129**

*Volumen de la tabla cotejo_asistencia*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| planilla_id | bigint | Identificador de la planilla cotejada | — | No | FK |
| realizado_por | bigint | Identificador del administrador que realizó el cotejo | — | No | FK |
| fecha_cotejo | date | Fecha del cotejo | — | No | No |
| total_registros | integer | Cantidad de registros cotejados | — | No | No |
| coincidencias | integer | Cantidad de registros coincidentes | — | No | No |
| discrepancias | integer | Cantidad de registros con discrepancias | — | No | No |
| observaciones | text | Observaciones del cotejo | — | Sí | No |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| resultado | varchar(20) | Resultado del cotejo (CONFORME o CON_ DISCREPANCIAS) | 20 | No | No |
| codigo_cotejo_ asistencia | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**Tabla 130**

*Volumen de la tabla bitacora*

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| id | bigint | Identificador interno de la tabla (no se expone en la API) | — | No | PK |
| usuario_id | bigint | Identificador del usuario que ejecutó la acción (nulo si fue el sistema) | — | Sí | FK |
| accion | varchar(60) | Acción realizada (p. ej. INICIO_SESION, CAMBIO_ROL, RESTABLECIMIENTO_ CONTRASENA) | 60 | No | No |
| entidad | varchar(60) | Tabla o entidad afectada (nula si la acción no afecta a un registro concreto) | 60 | Sí | No |
| entidad_id | bigint | Identificador del registro afectado | — | Sí | No |
| detalle | jsonb | Valores anteriores y posteriores a la acción | — | Sí | No |
| ip | inet | Dirección IP desde la que se realizó la acción | — | Sí | No |
| codigo_bitacora | uuid | Identificador público (UUID v7) usado por la API y los clientes | — | No | UQ |
| creado_en | timestamptz | Fecha y hora de creación del registro | — | No | No |

| Atributo | Tipo de dato | Descripción | Tamaño | Nulos | Llave |
|---|---|---|---|---|---|
| actualizado_en | timestamptz | Fecha y hora de la última actualización | — | No | No |
| eliminado_en | timestamptz | Fecha y hora de eliminación lógica (nula si el registro está activo) | — | Sí | No |

*Nota.* Elaboración propia.

**7.3.2.2. Script.** El script de creación de la base de datos, escrito para PostgreSQL 18, crea en orden de dependencias los tipos enumerados, las 32 tablas con sus claves y restricciones de verificación, los índices únicos parciales que respetan el borrado lógico (por ejemplo, una sola postulación por postulante y convocatoria, y un solo registro de asistencia por becado y día), los índices de consulta, el disparador que actualiza la columna *actualizado_en*, la protección de las tablas de solo inserción y la vista materializada que alimenta el dashboard de indicadores.

```
-- ============================================================================
-- Sistema de Postulacion y Preseleccion de la Beca Comedor (DUBSS - UAGRM)
-- Script de creacion de la base de datos - PostgreSQL 18
-- Generado a partir del diseno logico de la base de datos (32 tablas).
-- ============================================================================

-- Requiere PostgreSQL 18 o superior (funcion uuidv7()).
-- Ejecutar conectado a la base de datos destino, p. ej.:
--   CREATE DATABASE beca_comedor ENCODING 'UTF8';

BEGIN;

-- ----------------------------------------------------------------------------
-- 1. Tipos enumerados
-- ----------------------------------------------------------------------------
CREATE TYPE estado_postulacion AS ENUM (
    'borrador', 'pendiente', 'devuelta', 'archivada', 'calificada', 'adjudicada', 'no_adjudicada');
CREATE TYPE estado_convocatoria AS ENUM (
    'BORRADOR', 'ABIERTA', 'CERRADA');
CREATE TYPE estado_beca AS ENUM (
    'VIGENTE', 'SUSPENDIDA', 'BAJA');
CREATE TYPE resultado_adjudicacion AS ENUM (
    'ADJUDICADA', 'NO_ADJUDICADA');
```

```
CREATE TYPE tipo_movimiento_beca AS ENUM (
    'BAJA', 'SUSPENSION', 'REACTIVACION', 'REEMPLAZO');
CREATE TYPE destinatarios_comunicado AS ENUM (
    'TODOS', 'FINALISTAS', 'ADJUDICADOS');
CREATE TYPE tipo_notificacion AS ENUM (
    'SUCCESS', 'INFO', 'WARNING', 'ERROR');

-- ------------------------------------------------------------------------
-- 2. Tablas (en orden de dependencias)
--    Columnas comunes: codigo_<tabla> (UUID v7 publico), creado_en,
--    actualizado_en y eliminado_en (borrado logico).
-- ------------------------------------------------------------------------

-- P1 - Cuentas y acceso

CREATE TABLE rol (
    id                      bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    nombre                  varchar(50) NOT NULL,
    codigo_rol              uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en               timestamptz NOT NULL DEFAULT now(),
    actualizado_en          timestamptz NOT NULL DEFAULT now(),
    eliminado_en            timestamptz,
    CONSTRAINT pk_rol PRIMARY KEY (id)
);

CREATE TABLE usuario (
    id                      bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    ci                      varchar(20) NOT NULL,
    correo                  varchar(150) NOT NULL,
    password_hash           varchar(100) NOT NULL,
    nombres                 varchar(100) NOT NULL,
    apellidos               varchar(100) NOT NULL,
    activo                  boolean NOT NULL DEFAULT true,
    intentos_fallidos       integer NOT NULL DEFAULT 0,
    bloqueos_consecutivos   integer NOT NULL DEFAULT 0,
    bloqueado_hasta         timestamptz,
    restablecimiento_token_hash varchar(64),
    restablecimiento_expira_en  timestamptz,
    rol_id                  bigint NOT NULL REFERENCES rol(id) ON DELETE RESTRICT,
    codigo_usuario          uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en               timestamptz NOT NULL DEFAULT now(),
```

```
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_usuario PRIMARY KEY (id)
);

CREATE TABLE sesion (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    usuario_id                bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    token_hash                varchar(64) NOT NULL,
    expira_en                 timestamptz NOT NULL,
    revocada_en               timestamptz,
    codigo_sesion             uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_sesion PRIMARY KEY (id)
);

CREATE TABLE carrera (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    nombre                    varchar(150) NOT NULL,
    facultad                  varchar(150) NOT NULL,
    codigo_carrera            uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_carrera PRIMARY KEY (id)
);

CREATE TABLE postulante (
    usuario_id                bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    carrera_id                bigint NOT NULL REFERENCES carrera(id) ON DELETE RESTRICT,
    registro_universitario    varchar(20) NOT NULL,
    telefono                  varchar(20) NOT NULL,
    codigo_postulante         uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_postulante PRIMARY KEY (usuario_id)
);
```

```
-- P2 - Convocatoria y postulacion

CREATE TABLE gestion_academica (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    etiqueta                  varchar(20) NOT NULL,
    fecha_inicio              date NOT NULL,
    fecha_fin                 date NOT NULL,
    codigo_gestion_academica  uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_gestion_academica PRIMARY KEY (id),
    CONSTRAINT ck_gestion_academica_1 CHECK (fecha_fin >= fecha_inicio)
);

CREATE TABLE convocatoria (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    gestion_id                bigint NOT NULL REFERENCES gestion_academica(id) ON DELETE RESTRICT,
    nombre                    varchar(150) NOT NULL,
    fecha_apertura            timestamptz NOT NULL,
    fecha_cierre              timestamptz NOT NULL,
    cupos                     integer,
    puntaje_minimo            numeric(5,2),
    estado                    estado_convocatoria NOT NULL DEFAULT 'BORRADOR',
    creado_por                bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    codigo_convocatoria       uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_convocatoria PRIMARY KEY (id),
    CONSTRAINT ck_convocatoria_1 CHECK (fecha_cierre > fecha_apertura),
    CONSTRAINT ck_convocatoria_2 CHECK (cupos > 0),
    CONSTRAINT ck_convocatoria_3 CHECK (puntaje_minimo BETWEEN 0 AND 100)
);

CREATE TABLE tipo_documento (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    nombre                    varchar(100) NOT NULL,
    descripcion               varchar(255),
    codigo_tipo_documento     uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
```

```
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_tipo_documento PRIMARY KEY (id)
);

CREATE TABLE convocatoria_documento (
    convocatoria_id           bigint NOT NULL REFERENCES convocatoria(id) ON DELETE RESTRICT,
    tipo_documento_id         bigint NOT NULL REFERENCES tipo_documento(id) ON DELETE RESTRICT,
    obligatorio               boolean NOT NULL DEFAULT true,
    codigo_convocatoria_documento uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_convocatoria_documento PRIMARY KEY (convocatoria_id, tipo_documento_id)
);

CREATE TABLE postulacion (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    postulante_id             bigint NOT NULL REFERENCES postulante(usuario_id) ON DELETE RESTRICT,
    convocatoria_id           bigint NOT NULL REFERENCES convocatoria(id) ON DELETE RESTRICT,
    estado                    estado_postulacion NOT NULL DEFAULT 'borrador',
    fecha_envio               timestamptz,
    numero_envios             smallint NOT NULL DEFAULT 0,
    codigo_postulacion        uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_postulacion PRIMARY KEY (id),
    CONSTRAINT ck_postulacion_1 CHECK (numero_envios >= 0)
);

CREATE TABLE respuesta_socioeconomica (
    postulacion_id            bigint NOT NULL REFERENCES postulacion(id) ON DELETE RESTRICT,
    ingresos_familiares       numeric(12,2),
    dependientes              smallint,
    procedencia               varchar(150),
    patrimonio                varchar(255),
    estado_vulnerabilidad     varchar(50),
    promedio_academico        numeric(5,2),
    datos_extra               jsonb,
    codigo_respuesta_socioeconomica uuid NOT NULL DEFAULT uuidv7() UNIQUE,
```

```
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_respuesta_socioeconomica PRIMARY KEY (postulacion_id),
    CONSTRAINT ck_respuesta_socioeconomica_1 CHECK (promedio_academico BETWEEN 0 AND 100),
    CONSTRAINT ck_respuesta_socioeconomica_2 CHECK (ingresos_familiares >= 0),
    CONSTRAINT ck_respuesta_socioeconomica_3 CHECK (dependientes >= 0)
);

CREATE TABLE nota_academica (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    postulacion_id            bigint NOT NULL REFERENCES postulacion(id) ON DELETE RESTRICT,
    periodo                   varchar(20) NOT NULL,
    promedio                  numeric NOT NULL,
    codigo_nota_academica     uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_nota_academica PRIMARY KEY (id)
);

CREATE TABLE documento_adjunto (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    postulacion_id            bigint NOT NULL REFERENCES postulacion(id) ON DELETE RESTRICT,
    tipo_documento_id         bigint NOT NULL REFERENCES tipo_documento(id) ON DELETE RESTRICT,
    nombre_archivo            varchar(255) NOT NULL,
    storage_path              varchar(500) NOT NULL,
    mime_type                 varchar(100) NOT NULL,
    tamano_bytes              bigint NOT NULL,
    subido_en                 timestamptz NOT NULL DEFAULT now(),
    codigo_documento_adjunto  uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_documento_adjunto PRIMARY KEY (id),
    CONSTRAINT ck_documento_adjunto_1 CHECK (tamano_bytes > 0)
);

CREATE TABLE postulacion_historial (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    postulacion_id            bigint NOT NULL REFERENCES postulacion(id) ON DELETE RESTRICT,
```

```
    estado_anterior           estado_postulacion,
    estado_nuevo              estado_postulacion NOT NULL,
    usuario_id                bigint REFERENCES usuario(id) ON DELETE RESTRICT,
    motivo                    text,
    codigo_postulacion_historial uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_postulacion_historial PRIMARY KEY (id)
);

-- P3 - Preseleccion con IA

CREATE TABLE modelo_preseleccion (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    version                   varchar(20) NOT NULL,
    descripcion               varchar(255),
    activo                    boolean NOT NULL DEFAULT false,
    creado_por                bigint REFERENCES usuario(id) ON DELETE RESTRICT,
    codigo_modelo_preseleccion uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_modelo_preseleccion PRIMARY KEY (id)
);

CREATE TABLE parametro_modelo (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    modelo_id                 bigint NOT NULL REFERENCES modelo_preseleccion(id) ON DELETE RESTRICT,
    clave                     varchar(100) NOT NULL,
    ponderacion               numeric,
    valor                     jsonb,
    codigo_parametro_modelo   uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_parametro_modelo PRIMARY KEY (id)
);

CREATE TABLE puntaje_postulacion (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
```

```
    postulacion_id            bigint NOT NULL REFERENCES postulacion(id) ON DELETE RESTRICT,
    modelo_id                 bigint NOT NULL REFERENCES modelo_preseleccion(id) ON DELETE RESTRICT,
    puntaje_total             numeric(5,2) NOT NULL,
    detalle                   jsonb,
    fecha_calculo             timestamptz NOT NULL DEFAULT now(),
    vigente                   boolean NOT NULL DEFAULT true,
    codigo_puntaje_postulacion uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_puntaje_postulacion PRIMARY KEY (id),
    CONSTRAINT ck_puntaje_postulacion_1 CHECK (puntaje_total BETWEEN 0 AND 100)
);

CREATE TABLE lote_reprocesamiento (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    convocatoria_id           bigint NOT NULL REFERENCES convocatoria(id) ON DELETE RESTRICT,
    modelo_id                 bigint NOT NULL REFERENCES modelo_preseleccion(id) ON DELETE RESTRICT,
    solicitado_por            bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    estado                    varchar(20) NOT NULL DEFAULT 'PENDIENTE',
    total_postulaciones       integer NOT NULL DEFAULT 0,
    iniciado_en               timestamptz,
    terminado_en              timestamptz,
    codigo_lote_reprocesamiento uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_lote_reprocesamiento PRIMARY KEY (id),
    CONSTRAINT ck_lote_reprocesamiento_1
        CHECK (estado IN ('PENDIENTE', 'EN_PROCESO', 'COMPLETADO', 'FALLIDO')),
    CONSTRAINT ck_lote_reprocesamiento_2 CHECK (total_postulaciones >= 0)
);

-- P4 - Auditoria y adjudicacion

CREATE TABLE finalista (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    postulacion_id            bigint NOT NULL REFERENCES postulacion(id) ON DELETE RESTRICT,
    posicion                  integer NOT NULL,
    notificado_en             timestamptz,
    plazo_entrega             date,
```

```
    documentacion_recibida_en timestamptz,
    codigo_finalista          uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_finalista PRIMARY KEY (id),
    CONSTRAINT ck_finalista_1 CHECK (posicion > 0)
);

CREATE TABLE auditoria_expediente (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    finalista_id              bigint NOT NULL REFERENCES finalista(id) ON DELETE RESTRICT,
    auditor_id                bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    resultado                 varchar(20) NOT NULL,
    observaciones             text,
    fecha_auditoria           timestamptz NOT NULL DEFAULT now(),
    codigo_auditoria_expediente uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_auditoria_expediente PRIMARY KEY (id),
    CONSTRAINT ck_auditoria_expediente_1
        CHECK (resultado IN ('VALIDADO', 'OBSERVADO', 'NO_VALIDO'))
);

CREATE TABLE adjudicacion (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    postulacion_id            bigint NOT NULL REFERENCES postulacion(id) ON DELETE RESTRICT,
    resultado                 resultado_adjudicacion NOT NULL,
    puntaje_final             numeric(5,2),
    decidido_por              bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    fecha_adjudicacion        timestamptz NOT NULL DEFAULT now(),
    codigo_adjudicacion       uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_adjudicacion PRIMARY KEY (id),
    CONSTRAINT ck_adjudicacion_1 CHECK (puntaje_final BETWEEN 0 AND 100)
);

CREATE TABLE acta_adjudicacion (
```

```
    id                          bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    convocatoria_id             bigint NOT NULL REFERENCES convocatoria(id) ON DELETE RESTRICT,
    numero                      varchar(30) NOT NULL,
    storage_path                varchar(500) NOT NULL,
    generada_por                bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    generada_en                 timestamptz NOT NULL DEFAULT now(),
    codigo_acta_adjudicacion    uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                   timestamptz NOT NULL DEFAULT now(),
    actualizado_en              timestamptz NOT NULL DEFAULT now(),
    eliminado_en                timestamptz,
    CONSTRAINT pk_acta_adjudicacion PRIMARY KEY (id)
);

CREATE TABLE publicacion_resultado (
    id                          bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    convocatoria_id             bigint NOT NULL REFERENCES convocatoria(id) ON DELETE RESTRICT,
    publicado_por               bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    publicado_en                timestamptz NOT NULL DEFAULT now(),
    codigo_publicacion_resultado uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                   timestamptz NOT NULL DEFAULT now(),
    actualizado_en              timestamptz NOT NULL DEFAULT now(),
    eliminado_en                timestamptz,
    CONSTRAINT pk_publicacion_resultado PRIMARY KEY (id)
);

CREATE TABLE comunicado (
    id                          bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    convocatoria_id             bigint REFERENCES convocatoria(id) ON DELETE RESTRICT,
    titulo                      varchar(150) NOT NULL,
    contenido                   text NOT NULL,
    destinatarios               destinatarios_comunicado NOT NULL,
    publicado_por               bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    publicado_en                timestamptz NOT NULL DEFAULT now(),
    codigo_comunicado           uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                   timestamptz NOT NULL DEFAULT now(),
    actualizado_en              timestamptz NOT NULL DEFAULT now(),
    eliminado_en                timestamptz,
    CONSTRAINT pk_comunicado PRIMARY KEY (id)
);

CREATE TABLE notificacion (
```

```
    id                          bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    usuario_id                  bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    tipo                        tipo_notificacion NOT NULL,
    titulo                      varchar(150) NOT NULL,
    mensaje                     text NOT NULL,
    postulacion_id              bigint REFERENCES postulacion(id) ON DELETE RESTRICT,
    comunicado_id               bigint REFERENCES comunicado(id) ON DELETE RESTRICT,
    leida                       boolean NOT NULL DEFAULT false,
    enviada_en                  timestamptz,
    codigo_notificacion         uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                   timestamptz NOT NULL DEFAULT now(),
    actualizado_en              timestamptz NOT NULL DEFAULT now(),
    eliminado_en                timestamptz,
    CONSTRAINT pk_notificacion PRIMARY KEY (id)
);

CREATE TABLE dispositivo_fcm (
    id                          bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    usuario_id                  bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    token                       varchar(255) NOT NULL,
    plataforma                  varchar(10) NOT NULL,
    codigo_dispositivo_fcm      uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                   timestamptz NOT NULL DEFAULT now(),
    actualizado_en              timestamptz NOT NULL DEFAULT now(),
    eliminado_en                timestamptz,
    CONSTRAINT pk_dispositivo_fcm PRIMARY KEY (id),
    CONSTRAINT ck_dispositivo_fcm_1 CHECK (plataforma IN ('ANDROID', 'IOS', 'WEB'))
);

-- P5 - Becados y asistencia

CREATE TABLE beca (
    id                          bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    adjudicacion_id             bigint NOT NULL REFERENCES adjudicacion(id) ON DELETE RESTRICT,
    postulante_id               bigint NOT NULL REFERENCES postulante(usuario_id) ON DELETE RESTRICT,
    convocatoria_id             bigint NOT NULL REFERENCES convocatoria(id) ON DELETE RESTRICT,
    estado                      estado_beca NOT NULL DEFAULT 'VIGENTE',
    fecha_inicio                date NOT NULL,
    fecha_fin                   date,
    codigo_beca                 uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                   timestamptz NOT NULL DEFAULT now(),
```

```
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_beca PRIMARY KEY (id),
    CONSTRAINT ck_beca_1 CHECK (fecha_fin IS NULL OR fecha_fin >= fecha_inicio)
);

CREATE TABLE movimiento_beca (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    beca_id                   bigint NOT NULL REFERENCES beca(id) ON DELETE RESTRICT,
    tipo                      tipo_movimiento_beca NOT NULL,
    motivo                    text NOT NULL,
    beca_reemplazante_id      bigint REFERENCES beca(id) ON DELETE RESTRICT,
    registrado_por            bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    fecha                     date NOT NULL DEFAULT CURRENT_DATE,
    codigo_movimiento_beca    uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_movimiento_beca PRIMARY KEY (id),
    CONSTRAINT ck_movimiento_beca_1
        CHECK ((tipo = 'REEMPLAZO') = (beca_reemplazante_id IS NOT NULL))
);

CREATE TABLE planilla_asistencia (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    convocatoria_id           bigint NOT NULL REFERENCES convocatoria(id) ON DELETE RESTRICT,
    fecha_desde               date NOT NULL,
    fecha_hasta               date NOT NULL,
    storage_path              varchar(500) NOT NULL,
    generada_por              bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    generada_en               timestamptz NOT NULL DEFAULT now(),
    codigo_planilla_asistencia uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_planilla_asistencia PRIMARY KEY (id),
    CONSTRAINT ck_planilla_asistencia_1 CHECK (fecha_hasta >= fecha_desde)
);

CREATE TABLE asistencia (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
```

```
    beca_id                   bigint NOT NULL REFERENCES beca(id) ON DELETE RESTRICT,
    fecha                     date NOT NULL,
    hora_registro             timestamptz NOT NULL DEFAULT now(),
    registrado_por            bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    codigo_asistencia         uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_asistencia PRIMARY KEY (id)
);

CREATE TABLE cotejo_asistencia (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    planilla_id               bigint NOT NULL REFERENCES planilla_asistencia(id) ON DELETE RESTRICT,
    realizado_por             bigint NOT NULL REFERENCES usuario(id) ON DELETE RESTRICT,
    fecha_cotejo              date NOT NULL DEFAULT CURRENT_DATE,
    total_registros           integer NOT NULL,
    coincidencias             integer NOT NULL,
    discrepancias             integer NOT NULL,
    observaciones             text,
    resultado                 varchar(20) NOT NULL,
    codigo_cotejo_asistencia  uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_cotejo_asistencia PRIMARY KEY (id),
    CONSTRAINT ck_cotejo_asistencia_1 CHECK (total_registros >= 0),
    CONSTRAINT ck_cotejo_asistencia_2 CHECK (coincidencias >= 0),
    CONSTRAINT ck_cotejo_asistencia_3 CHECK (discrepancias >= 0),
    CONSTRAINT ck_cotejo_asistencia_4
        CHECK (resultado IN ('CONFORME', 'CON_DISCREPANCIAS'))
);

-- P6 - Reportes y trazabilidad

CREATE TABLE bitacora (
    id                        bigint GENERATED ALWAYS AS IDENTITY NOT NULL,
    usuario_id                bigint REFERENCES usuario(id) ON DELETE RESTRICT,
    accion                    varchar(60) NOT NULL,
    entidad                   varchar(60),
    entidad_id                bigint,
```

```
    detalle                   jsonb,
    ip                        inet,
    codigo_bitacora           uuid NOT NULL DEFAULT uuidv7() UNIQUE,
    creado_en                 timestamptz NOT NULL DEFAULT now(),
    actualizado_en            timestamptz NOT NULL DEFAULT now(),
    eliminado_en              timestamptz,
    CONSTRAINT pk_bitacora PRIMARY KEY (id)
);

-- ------------------------------------------------------------------------
-- 3. Restricciones de unicidad (indices parciales que respetan el borrado logico)
-- ------------------------------------------------------------------------
CREATE UNIQUE INDEX uq_rol_nombre
    ON rol (nombre)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_usuario_ci
    ON usuario (ci)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_usuario_correo
    ON usuario (correo)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_usuario_restablecimiento_token_hash
    ON usuario (restablecimiento_token_hash)
    WHERE restablecimiento_token_hash IS NOT NULL AND eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_sesion_token_hash
    ON sesion (token_hash)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_carrera_nombre
    ON carrera (nombre)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_postulante_registro_universitario
    ON postulante (registro_universitario)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_gestion_etiqueta
    ON gestion_academica (etiqueta)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_convocatoria_abierta_por_gestion
    ON convocatoria (gestion_id)
    WHERE estado = 'ABIERTA' AND eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_postulacion_postulante_convocatoria
    ON postulacion (postulante_id, convocatoria_id)
```

```
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_nota_postulacion_periodo
    ON nota_academica (postulacion_id, periodo)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_modelo_version
    ON modelo_preseleccion (version)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_modelo_activo
    ON modelo_preseleccion (activo)
    WHERE activo AND eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_parametro_modelo_clave
    ON parametro_modelo (modelo_id, clave)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_puntaje_vigente
    ON puntaje_postulacion (postulacion_id)
    WHERE vigente AND eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_finalista_postulacion
    ON finalista (postulacion_id)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_adjudicacion_postulacion
    ON adjudicacion (postulacion_id)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_acta_numero
    ON acta_adjudicacion (numero)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_publicacion_convocatoria
    ON publicacion_resultado (convocatoria_id)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_dispositivo_token
    ON dispositivo_fcm (token)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_beca_adjudicacion
    ON beca (adjudicacion_id)
    WHERE eliminado_en IS NULL;
CREATE UNIQUE INDEX uq_asistencia_beca_fecha
    ON asistencia (beca_id, fecha)
    WHERE eliminado_en IS NULL;

-- ------------------------------------------------------------------------
-- 4. Indices de consulta
-- ------------------------------------------------------------------------
```

```
CREATE INDEX ix_usuario_rol_id
    ON usuario (rol_id)
    WHERE eliminado_en IS NULL;
CREATE INDEX ix_sesion_usuario_id
    ON sesion (usuario_id)
    WHERE eliminado_en IS NULL;
CREATE INDEX ix_postulante_carrera_id
    ON postulante (carrera_id)
    WHERE eliminado_en IS NULL;
CREATE INDEX ix_postulacion_convocatoria_estado
    ON postulacion (convocatoria_id, estado)
    WHERE eliminado_en IS NULL;
CREATE INDEX ix_puntaje_lista_priorizada
    ON puntaje_postulacion (puntaje_total DESC)
    WHERE vigente AND eliminado_en IS NULL;
CREATE INDEX ix_postulacion_historial_postulacion
    ON postulacion_historial (postulacion_id, creado_en);
CREATE INDEX ix_documento_adjunto_postulacion
    ON documento_adjunto (postulacion_id)
    WHERE eliminado_en IS NULL;
CREATE INDEX ix_beca_convocatoria_estado
    ON beca (convocatoria_id, estado)
    WHERE eliminado_en IS NULL;
CREATE INDEX ix_beca_postulante
    ON beca (postulante_id)
    WHERE eliminado_en IS NULL;
CREATE INDEX ix_notificacion_no_leidas
    ON notificacion (usuario_id, leida, creado_en DESC)
    WHERE eliminado_en IS NULL;
CREATE INDEX ix_bitacora_usuario_fecha
    ON bitacora (usuario_id, creado_en);
CREATE INDEX ix_bitacora_accion_fecha
    ON bitacora (accion, creado_en);


-- ------------------------------------------------------------------------
-- 5. Actualizacion automatica de actualizado_en
-- ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fn_set_actualizado_en() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    NEW.actualizado_en := now();
```

```
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_rol_actualizado_en BEFORE UPDATE ON rol
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_usuario_actualizado_en BEFORE UPDATE ON usuario
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_sesion_actualizado_en BEFORE UPDATE ON sesion
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_carrera_actualizado_en BEFORE UPDATE ON carrera
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_postulante_actualizado_en BEFORE UPDATE ON postulante
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_gestion_academica_actualizado_en BEFORE UPDATE ON gestion_academica
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_convocatoria_actualizado_en BEFORE UPDATE ON convocatoria
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_tipo_documento_actualizado_en BEFORE UPDATE ON tipo_documento
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_convocatoria_documento_actualizado_en BEFORE UPDATE ON convocatoria_documento
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_postulacion_actualizado_en BEFORE UPDATE ON postulacion
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_respuesta_socioeconomica_actualizado_en BEFORE UPDATE ON respuesta_socioeconomica
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_nota_academica_actualizado_en BEFORE UPDATE ON nota_academica
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_documento_adjunto_actualizado_en BEFORE UPDATE ON documento_adjunto
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_modelo_preseleccion_actualizado_en BEFORE UPDATE ON modelo_preseleccion
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_parametro_modelo_actualizado_en BEFORE UPDATE ON parametro_modelo
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_puntaje_postulacion_actualizado_en BEFORE UPDATE ON puntaje_postulacion
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_lote_reprocesamiento_actualizado_en BEFORE UPDATE ON lote_reprocesamiento
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_finalista_actualizado_en BEFORE UPDATE ON finalista
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_auditoria_expediente_actualizado_en BEFORE UPDATE ON auditoria_expediente
```

```
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_adjudicacion_actualizado_en BEFORE UPDATE ON adjudicacion
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_acta_adjudicacion_actualizado_en BEFORE UPDATE ON acta_adjudicacion
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_publicacion_resultado_actualizado_en BEFORE UPDATE ON publicacion_resultado
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_comunicado_actualizado_en BEFORE UPDATE ON comunicado
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_notificacion_actualizado_en BEFORE UPDATE ON notificacion
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_dispositivo_fcm_actualizado_en BEFORE UPDATE ON dispositivo_fcm
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_beca_actualizado_en BEFORE UPDATE ON beca
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_movimiento_beca_actualizado_en BEFORE UPDATE ON movimiento_beca
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_planilla_asistencia_actualizado_en BEFORE UPDATE ON planilla_asistencia
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_asistencia_actualizado_en BEFORE UPDATE ON asistencia
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();
CREATE TRIGGER trg_cotejo_asistencia_actualizado_en BEFORE UPDATE ON cotejo_asistencia
    FOR EACH ROW EXECUTE FUNCTION fn_set_actualizado_en();

-- ----------------------------------------------------------------------------
-- 6. Tablas de solo insercion (append-only): bitacora y postulacion_historial
--    El rol de la aplicacion (app_user) no puede modificar ni eliminar filas.
-- ----------------------------------------------------------------------------
DO $$
BEGIN
    IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'app_user') THEN
        REVOKE UPDATE, DELETE ON bitacora, postulacion_historial FROM app_user;
    END IF;
END;
$$;

-- ----------------------------------------------------------------------------
-- 7. Vista materializada de indicadores BI (CU-12), refrescada por lotes:
--    REFRESH MATERIALIZED VIEW mv_indicadores_convocatoria;
-- ----------------------------------------------------------------------------
CREATE MATERIALIZED VIEW mv_indicadores_convocatoria AS
```

```
SELECT
    g.id                AS gestion_id,
    g.etiqueta          AS gestion,
    c.id                AS convocatoria_id,
    c.nombre            AS convocatoria,
    (SELECT count(*) FROM postulacion p
      WHERE p.convocatoria_id = c.id AND p.estado <> 'borrador'
        AND p.eliminado_en IS NULL)                     AS postulaciones_recibidas,
    (SELECT count(*) FROM finalista f JOIN postulacion p ON p.id = f.postulacion_id
      WHERE p.convocatoria_id = c.id AND f.eliminado_en IS NULL) AS finalistas,
    (SELECT count(*) FROM adjudicacion a JOIN postulacion p ON p.id = a.postulacion_id
      WHERE p.convocatoria_id = c.id AND a.resultado = 'ADJUDICADA'
        AND a.eliminado_en IS NULL)                     AS becas_adjudicadas,
    (SELECT count(*) FROM beca b
      WHERE b.convocatoria_id = c.id AND b.estado = 'VIGENTE'
        AND b.eliminado_en IS NULL)                     AS becas_vigentes,
    (SELECT count(*) FROM asistencia s JOIN beca b ON b.id = s.beca_id
      WHERE b.convocatoria_id = c.id AND s.eliminado_en IS NULL) AS asistencias_registradas,
    now()               AS actualizado_en
FROM convocatoria c
JOIN gestion_academica g ON g.id = c.gestion_id
WHERE c.eliminado_en IS NULL;

CREATE UNIQUE INDEX uq_mv_indicadores_convocatoria ON mv_indicadores_convocatoria (convocatoria_id);

-- ------------------------------------------------------------------------
-- 8. Datos iniciales del catalogo de roles
-- ------------------------------------------------------------------------
INSERT INTO rol (nombre) VALUES
    ('POSTULANTE'), ('ADMINISTRADOR_DUBSS'), ('DIRECTIVO'), ('PERSONAL_COMEDOR');

COMMIT;
```

## 7.4. Diseñar Caso de Uso

El diseño de cada caso de uso se expresa mediante diagramas de secuencia, que muestran el orden temporal de los mensajes intercambiados entre el actor y los objetos que realizan el caso de uso (Booch et al., 2005). Siguiendo el Proceso Unificado de Desarrollo de Software, cada diagrama de secuencia refina el diagrama de colaboración del análisis (sección 66)

y conserva sus clases de interfaz (*View*), control (*Controller*) y entidad, incorporando los mensajes de retorno y los fragmentos combinados que representan las alternativas del flujo (Jacobson et al., 1999).

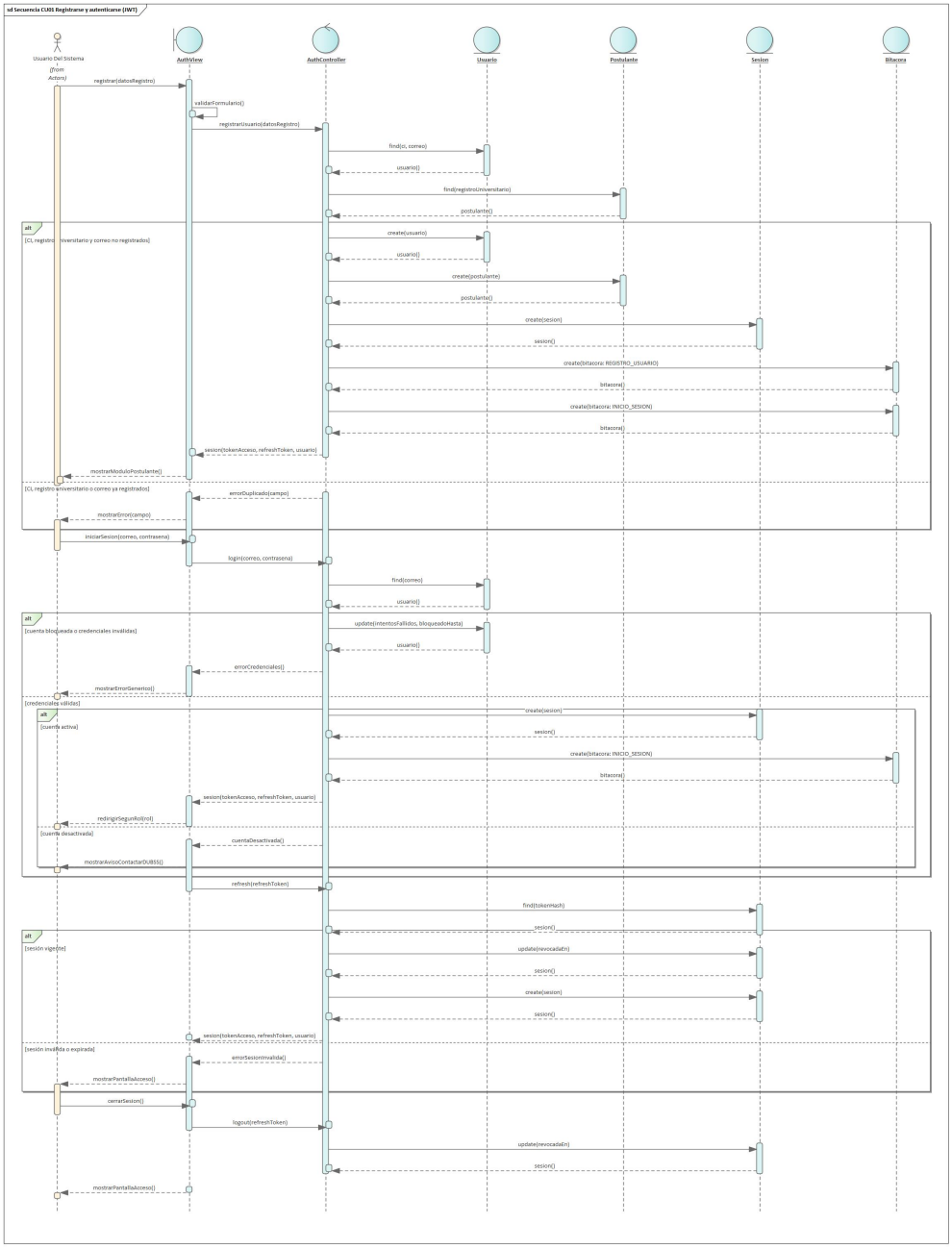
A continuación se presenta el diseño de los veintiséis casos de uso del sistema, en el orden de su numeración (CU-01 a CU-26).

### *7.4.1. Diseñar CU-01 Registrarse y autenticarse*

La Figura 98 presenta el diagrama de secuencia del CU-01.

**Figura 98**

*Diagrama de secuencia del CU-01 Registrarse y autenticarse*




*Nota.* Elaboración propia.

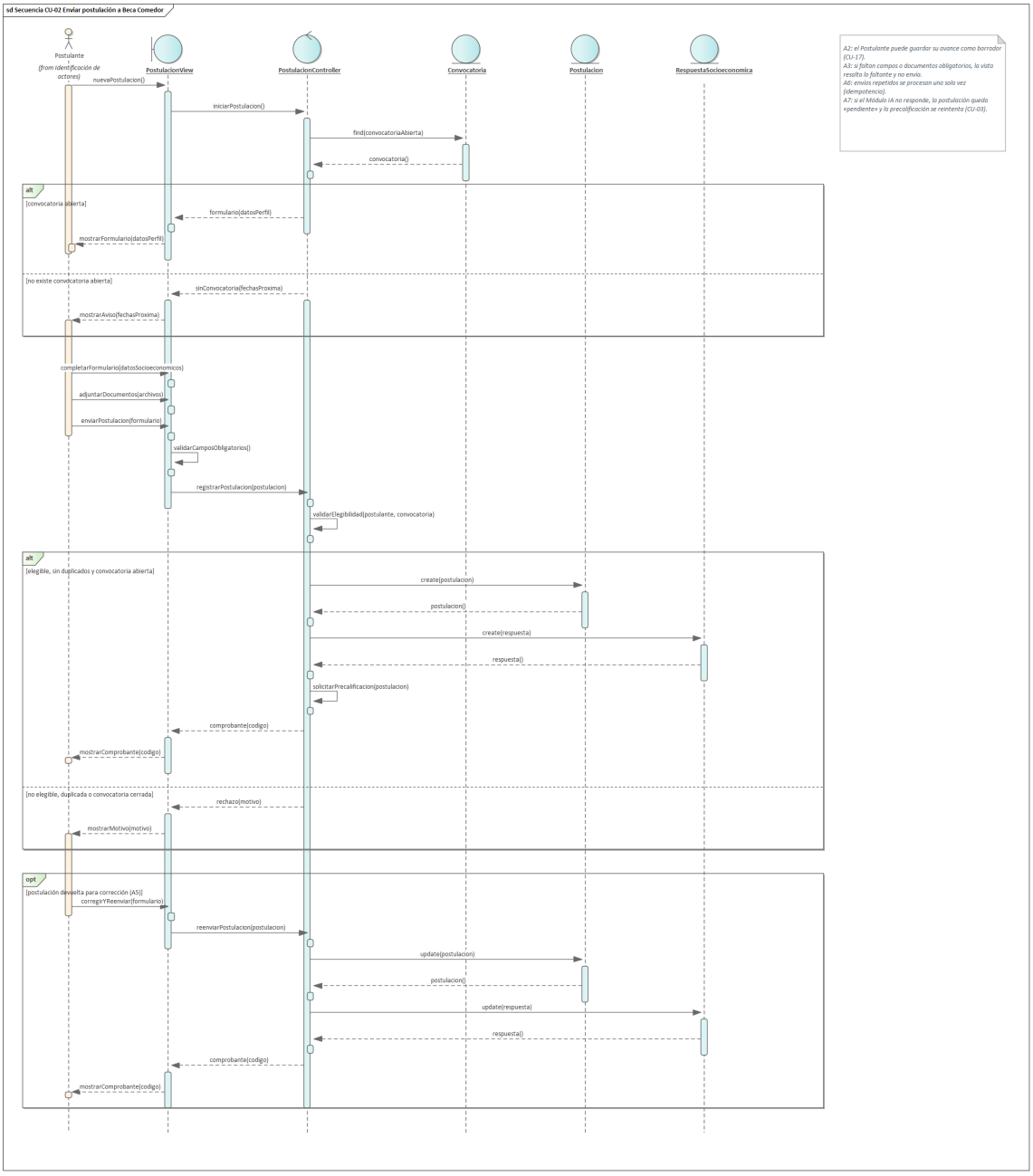
En el registro, el cliente envía los datos al backend; *AuthController* comprueba que el CI, el registro universitario y el correo electrónico no existan, crea el *Usuario* con el rol Postulante y la contraseña almacenada como hash, crea su perfil de *Postulante* (con registro universitario y teléfono), abre la *Sesion*, anota en la bitácora el registro y el inicio de sesión, y devuelve el token de acceso, el token de renovación y el usuario; si alguno de esos datos ya existe, informa el campo en conflicto y no crea la cuenta. En el inicio de sesión, el controlador busca al *Usuario* por su correo electrónico; si las credenciales no son válidas o la cuenta está bloqueada, actualiza el contador de intentos fallidos y el bloqueo y responde con un mismo mensaje de error. Si son válidas y la cuenta está activa, abre la *Sesion*, anota el acceso en la bitácora y devuelve los tokens y el usuario con su rol, según el cual se redirige al módulo correspondiente; si la cuenta está desactivada, niega el acceso e indica que debe comunicarse con la DUBSS. Cuando el token de acceso vence, el cliente solicita su renovación con el token de renovación: el controlador busca la *Sesion* por el hash del token, la revoca y crea una nueva con un token de renovación nuevo; si la sesión no es válida, el usuario vuelve a la pantalla de acceso. Para cerrar sesión, el controlador revoca la *Sesion* del dispositivo, sin afectar las de otros dispositivos. La contraseña (mínimo de 6 caracteres) se almacena solo como hash y el bloqueo tras 5 intentos fallidos consecutivos lo aplica el propio sistema.

### *7.4.2. Diseñar CU-02 Enviar postulación a Beca Comedor*

La Figura 99 presenta el diagrama de secuencia del CU-02.

**Figura 99**

*Diagrama de secuencia del CU-02 Enviar postulación a Beca Comedor*



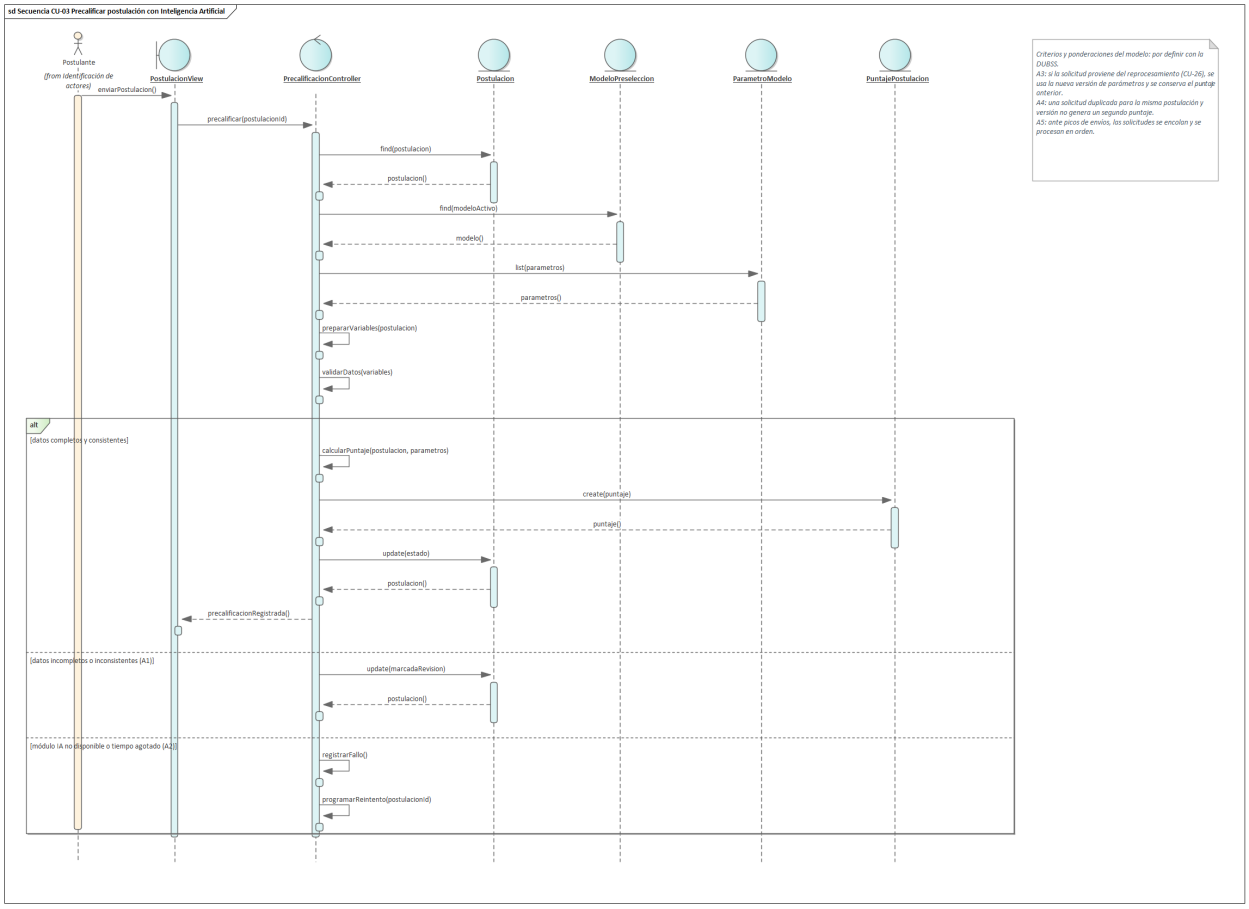

*Nota.* Elaboración propia.

### *7.4.3. Diseñar CU-03 Precalificar postulación con Inteligencia Artificial*

La Figura 100 presenta el diagrama de secuencia del CU-03.

**Figura 100**

*Diagrama de secuencia del CU-03 Precalificar postulación con Inteligencia Artificial*



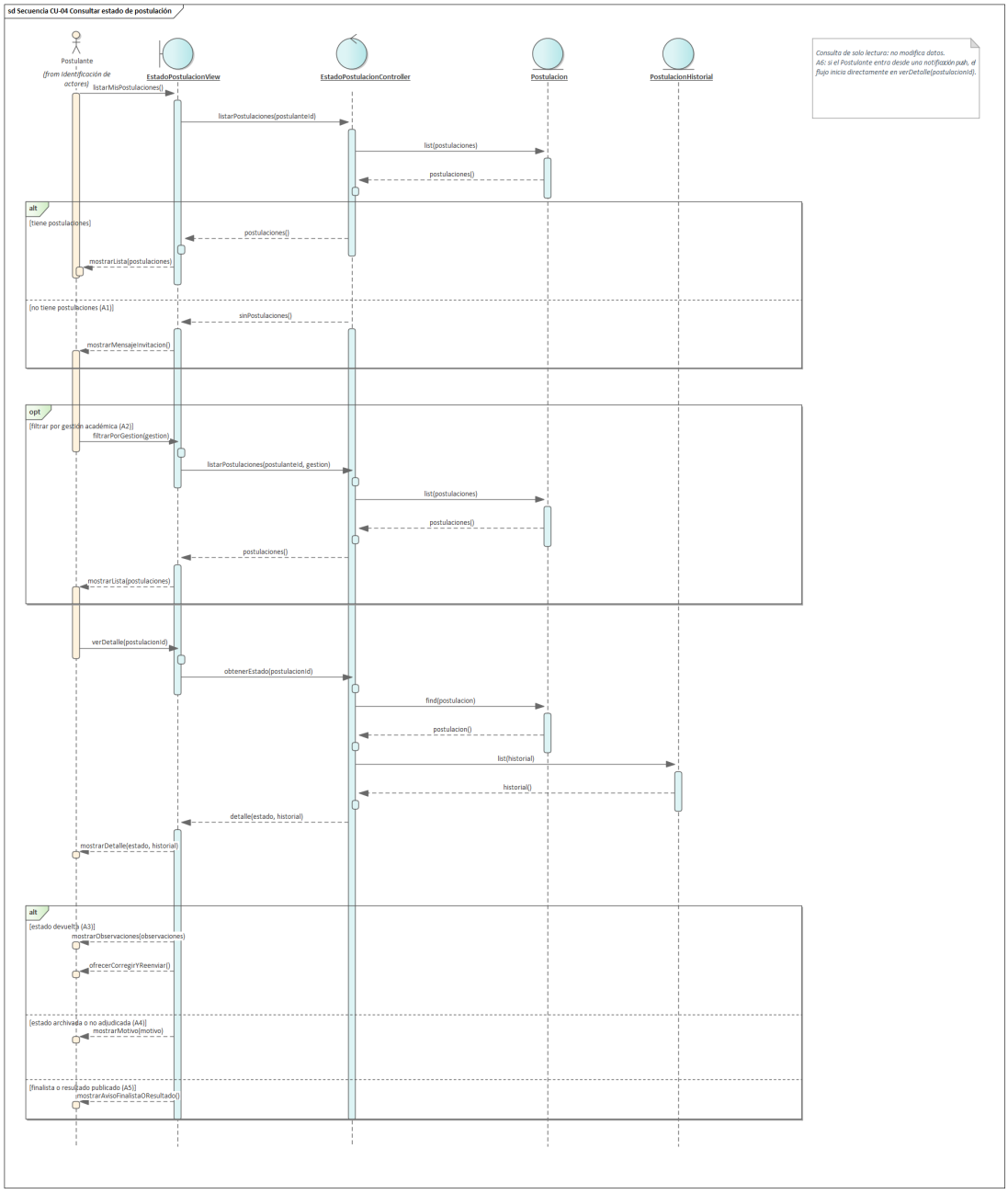

*Nota.* Elaboración propia.

### *7.4.4. Diseñar CU-04 Consultar estado de postulación*

La Figura 101 presenta el diagrama de secuencia del CU-04.

**Figura 101**

*Diagrama de secuencia del CU-04 Consultar estado de postulación*



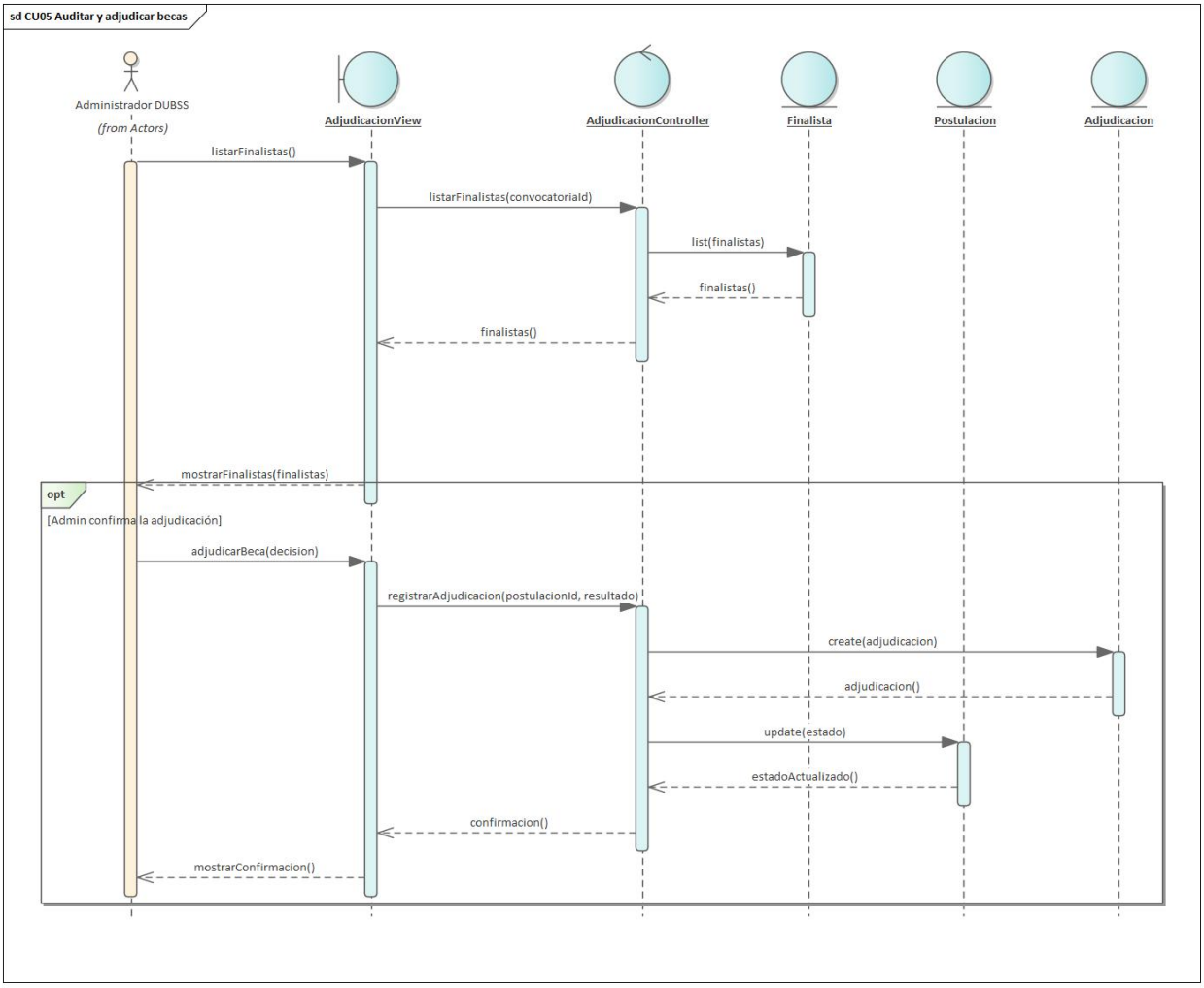

*Nota.* Elaboración propia.

### *7.4.5. Diseñar CU-05 Auditar y adjudicar becas*

La Figura 102 presenta el diagrama de secuencia del CU-05.

**Figura 102**

*Diagrama de secuencia del CU-05 Auditar y adjudicar becas*



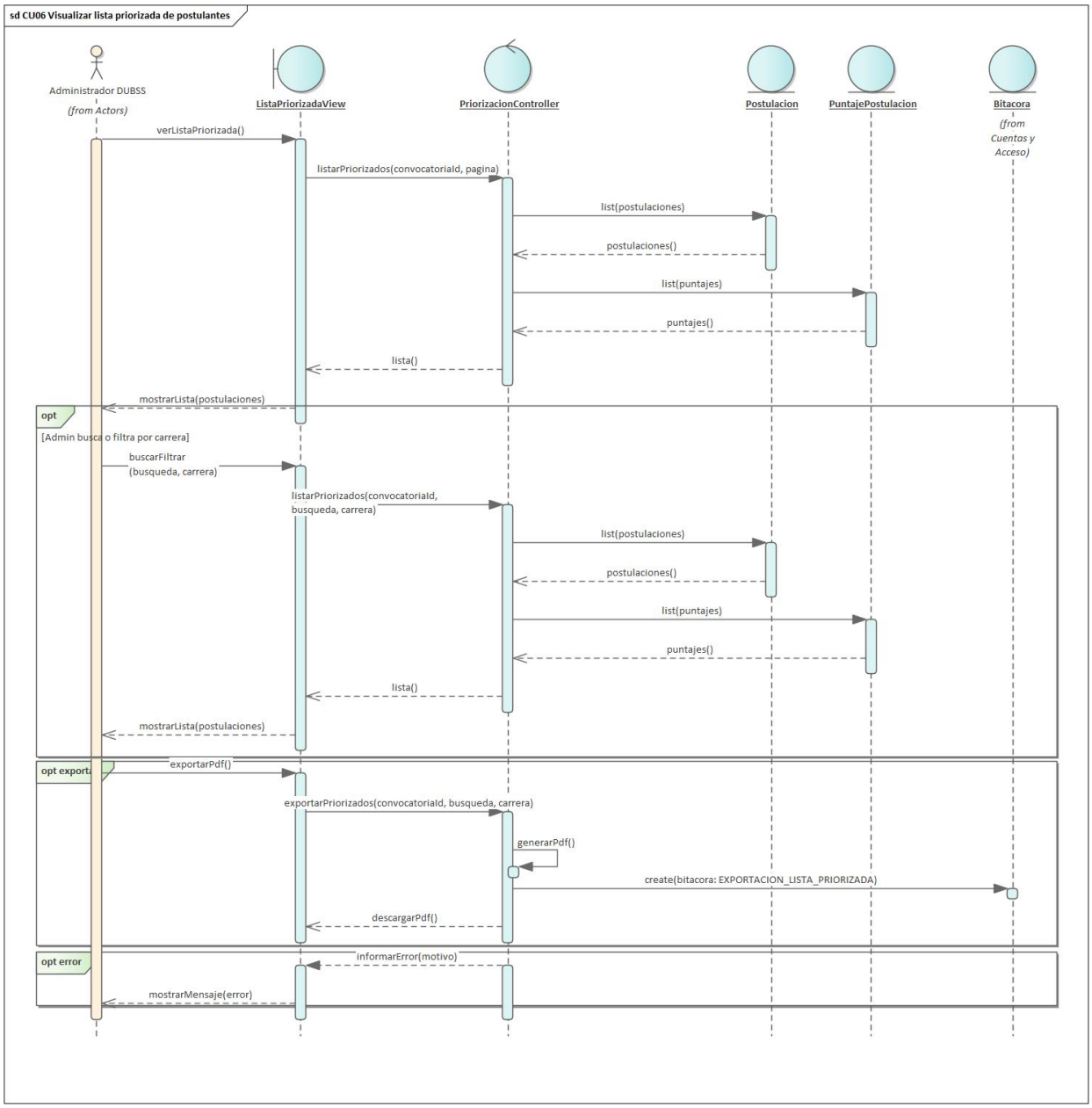

*Nota.* Elaboración propia.

### *7.4.6. Diseñar CU-06 Visualizar lista priorizada de postulantes*

La Figura 103 presenta el diagrama de secuencia del CU-06.

**Figura 103**

*Diagrama de secuencia del CU-06 Visualizar lista priorizada de postulantes*



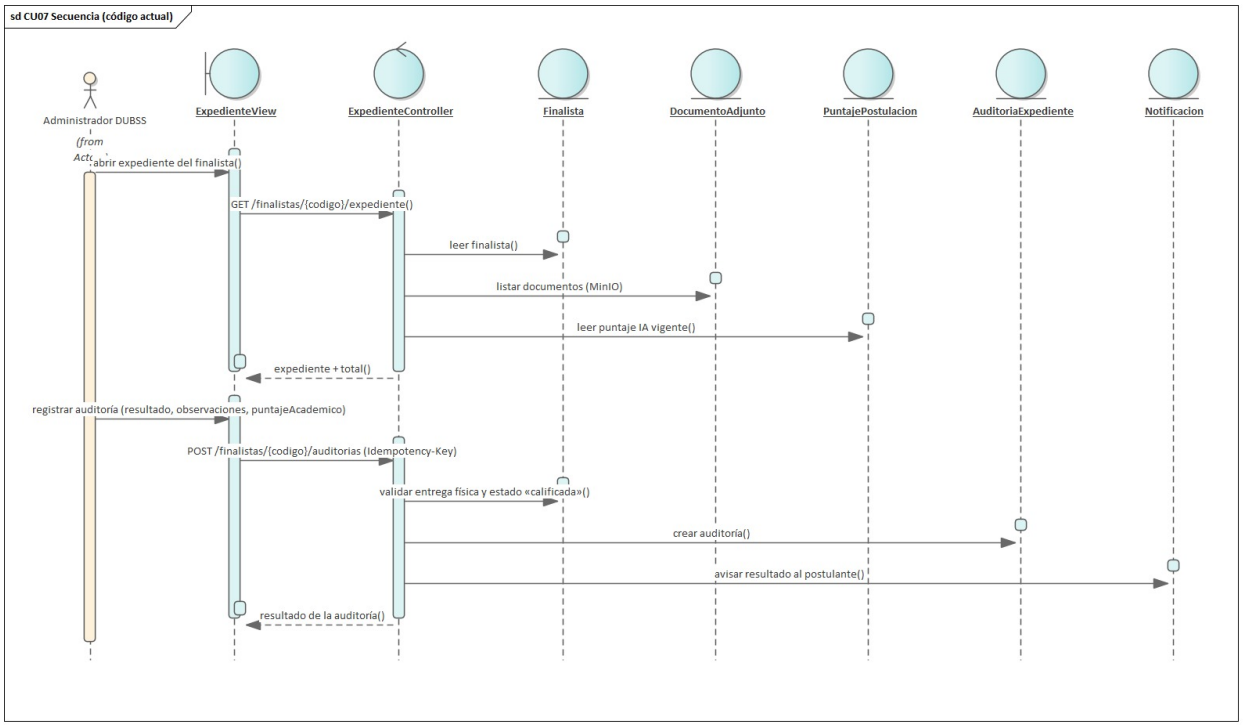

*Nota.* Elaboración propia.

### *7.4.7. Diseñar CU-07 Auditar expediente finalista*

La Figura 104 presenta el diagrama de secuencia del CU-07.

**Figura 104**

*Diagrama de secuencia del CU-07 Auditar expediente finalista*



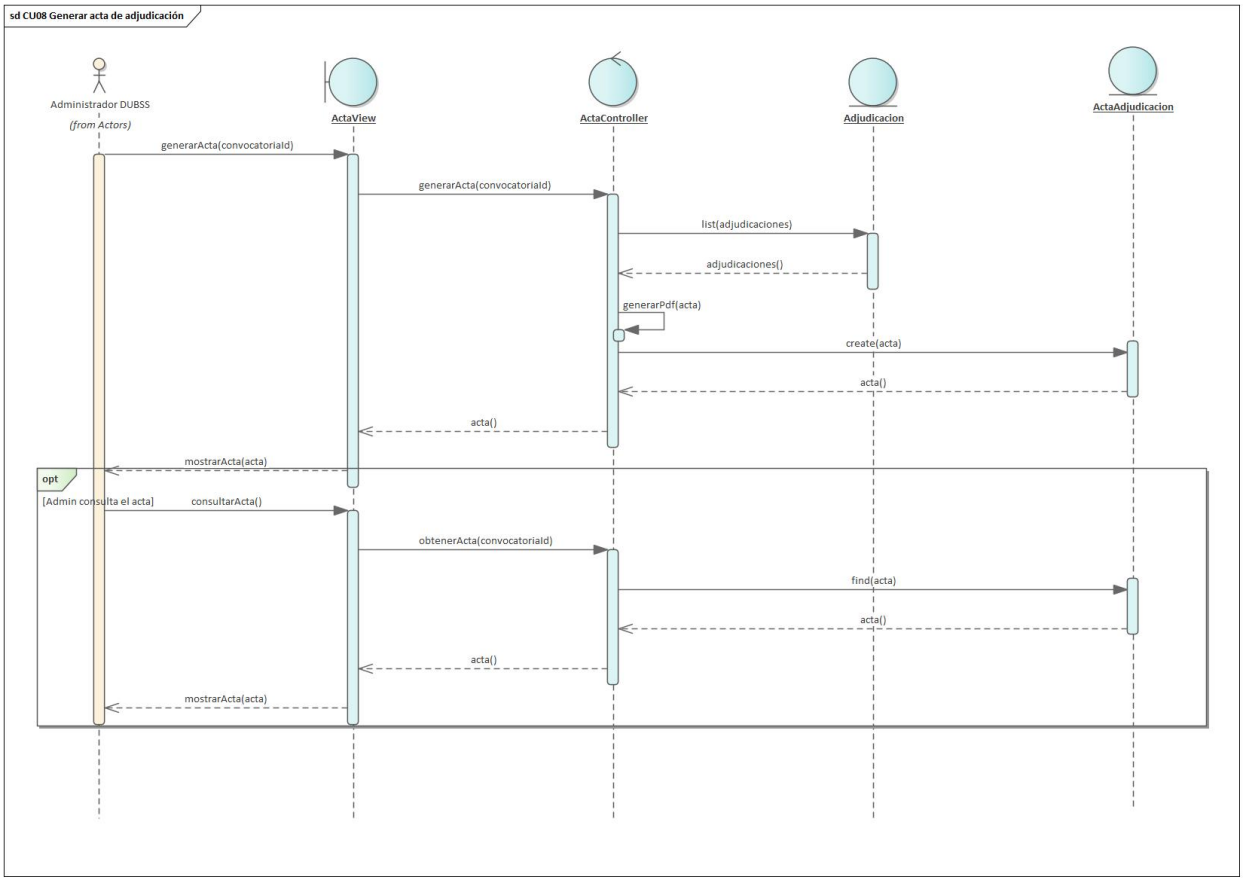

*Nota.* Elaboración propia.

### *7.4.8. Diseñar CU-08 Generar acta de adjudicación*

La Figura 105 presenta el diagrama de secuencia del CU-08.

**Figura 105**

*Diagrama de secuencia del CU-08 Generar acta de adjudicación*



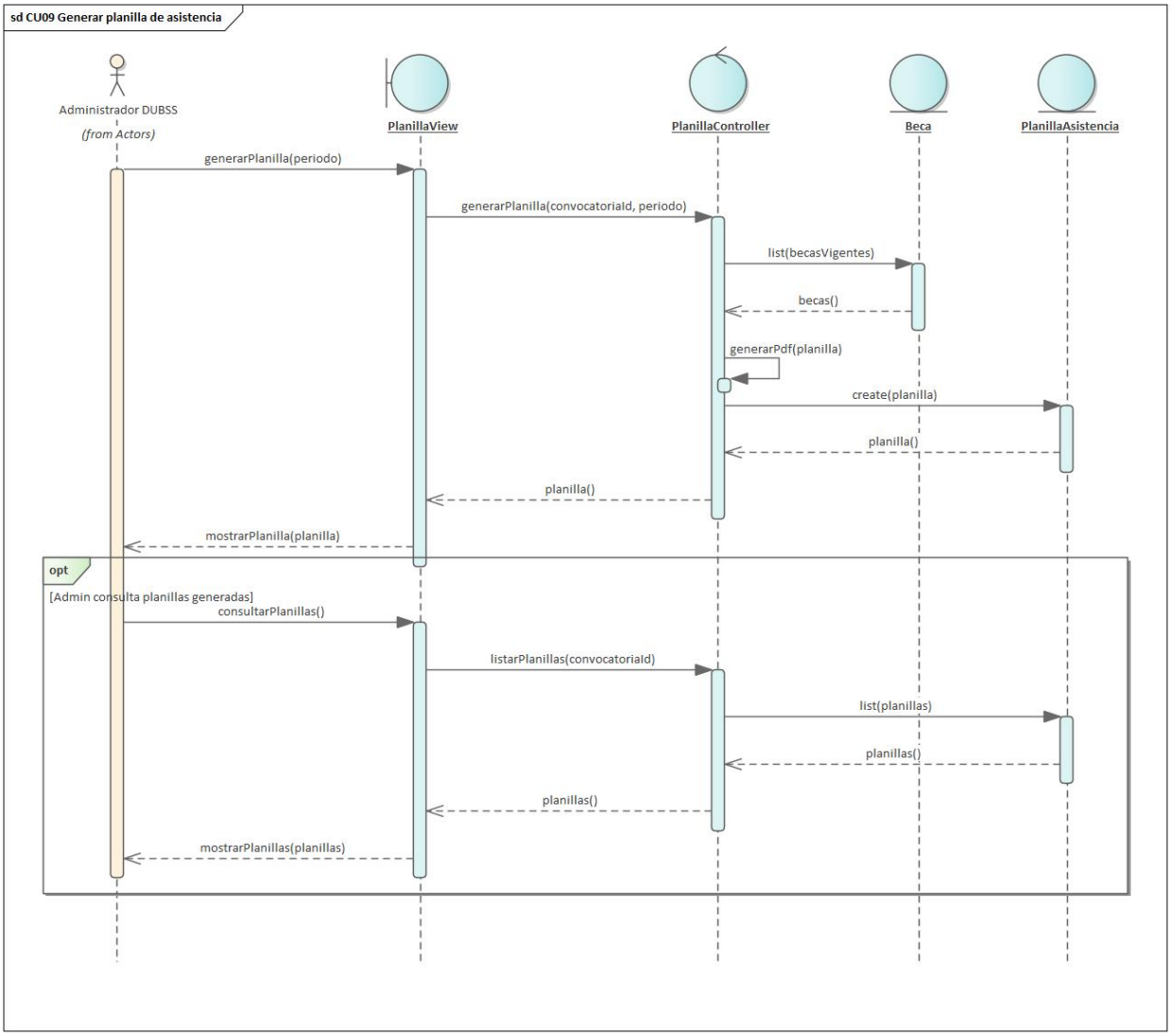

*Nota.* Elaboración propia.

### *7.4.9. Diseñar CU-09 Generar planilla de asistencia*

La Figura 106 presenta el diagrama de secuencia del CU-09.

**Figura 106**

*Diagrama de secuencia del CU-09 Generar planilla de asistencia*



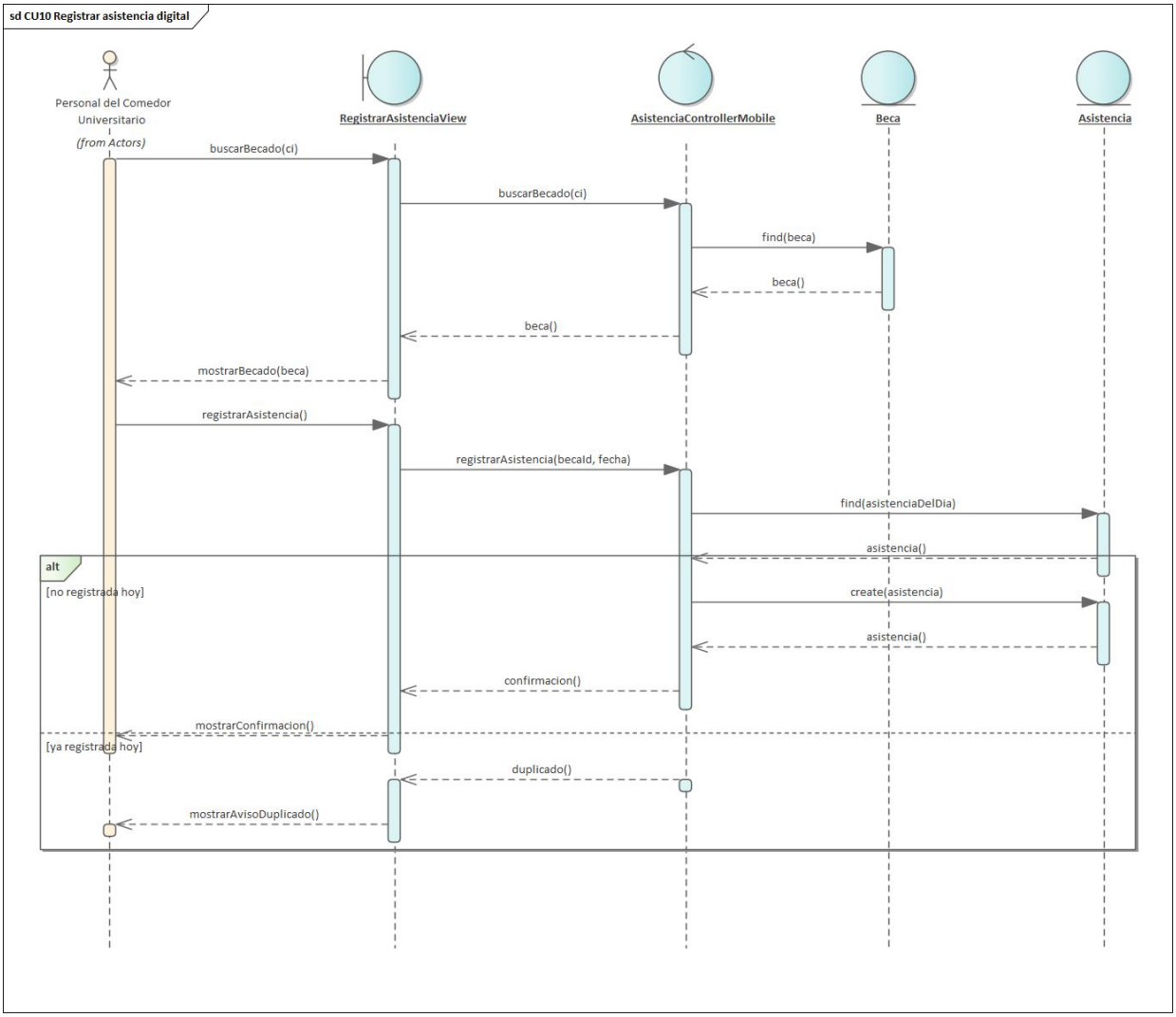

*Nota.* Elaboración propia.

### *7.4.10. Diseñar CU-10 Registrar asistencia digital*

La Figura 107 presenta el diagrama de secuencia del CU-10.

**Figura 107**

*Diagrama de secuencia del CU-10 Registrar asistencia digital*



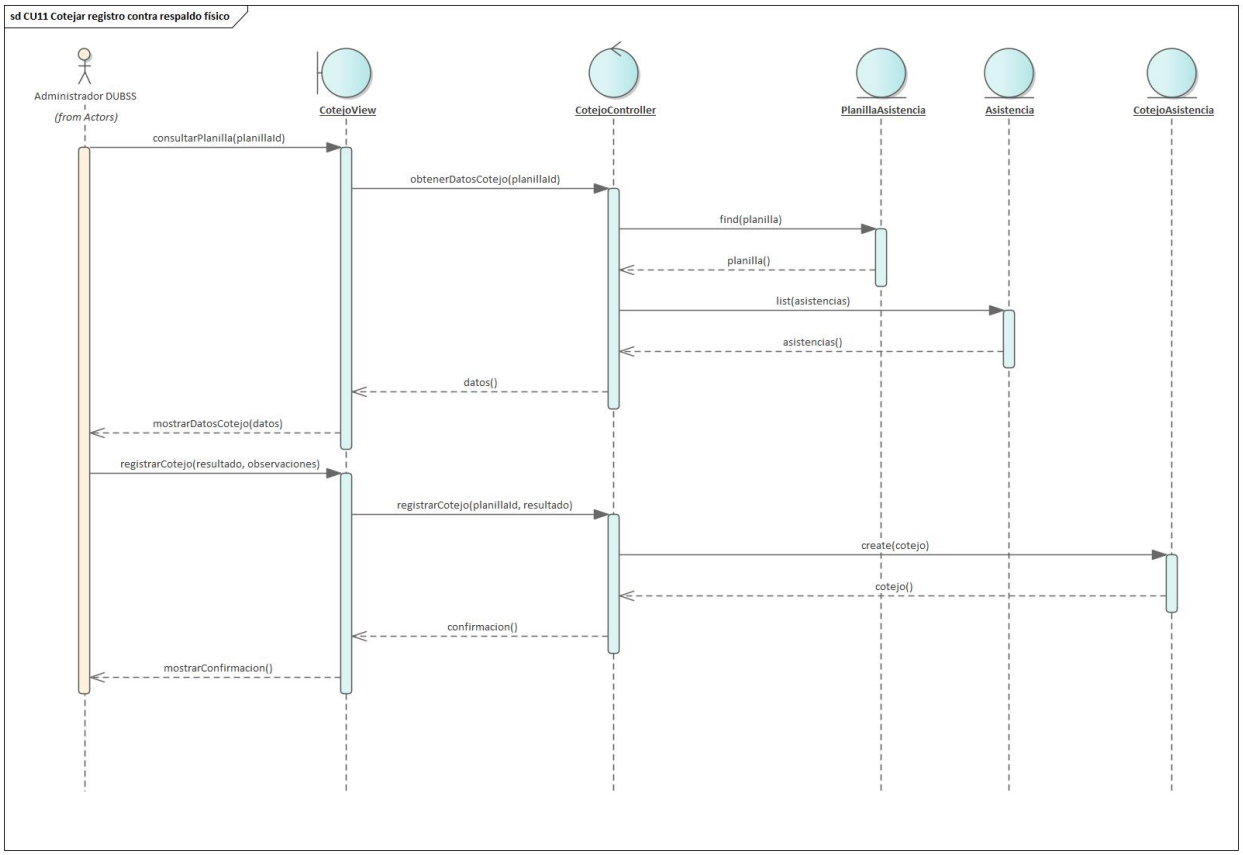

*Nota.* Elaboración propia.

### *7.4.11. Diseñar CU-11 Cotejar registro contra respaldo físico*

La Figura 108 presenta el diagrama de secuencia del CU-11.

**Figura 108**

*Diagrama de secuencia del CU-11 Cotejar registro contra respaldo físico*



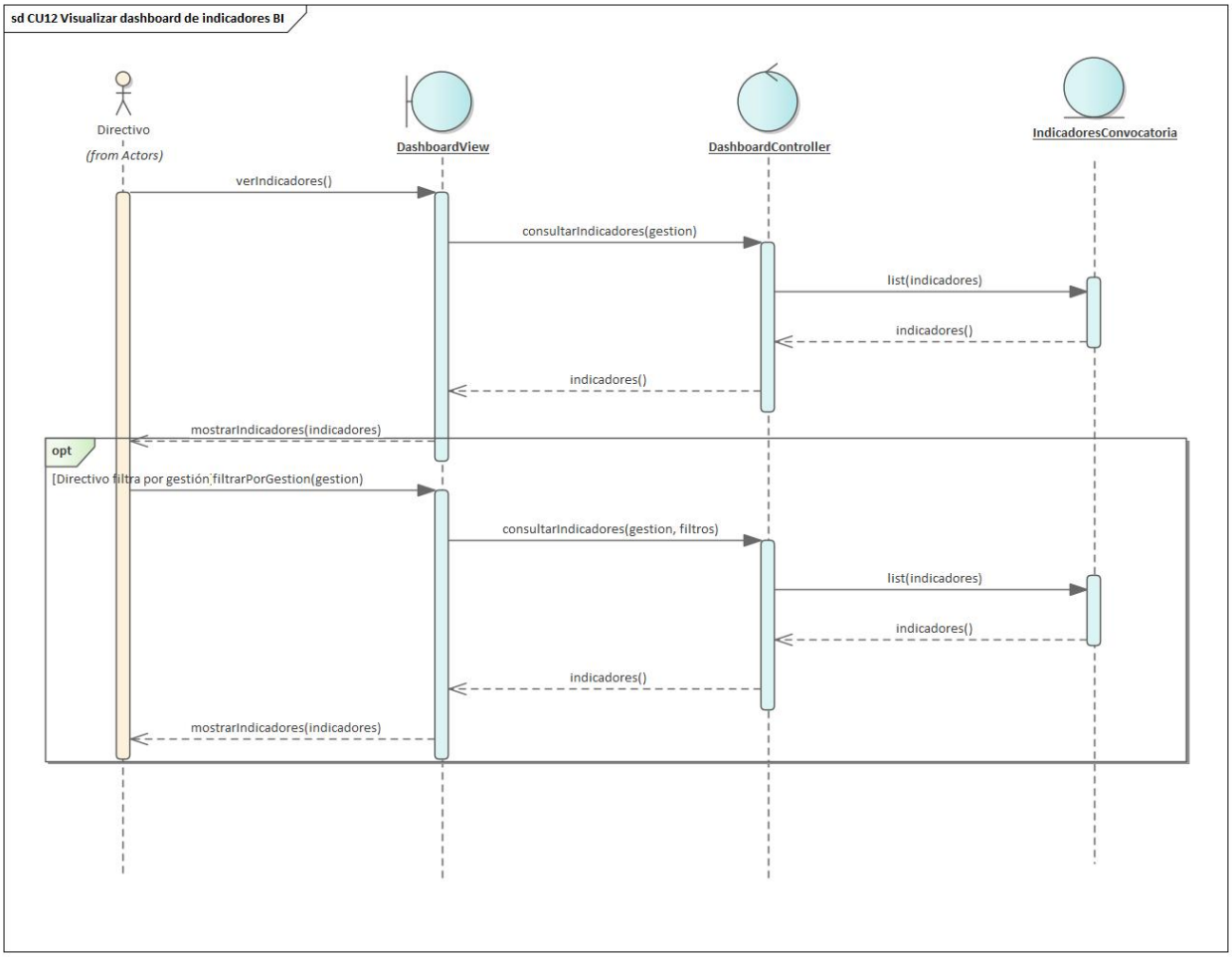

*Nota.* Elaboración propia.

### *7.4.12. Diseñar CU-12 Visualizar dashboard de indicadores BI*

La Figura 109 presenta el diagrama de secuencia del CU-12.

**Figura 109**

*Diagrama de secuencia del CU-12 Visualizar dashboard de indicadores BI*



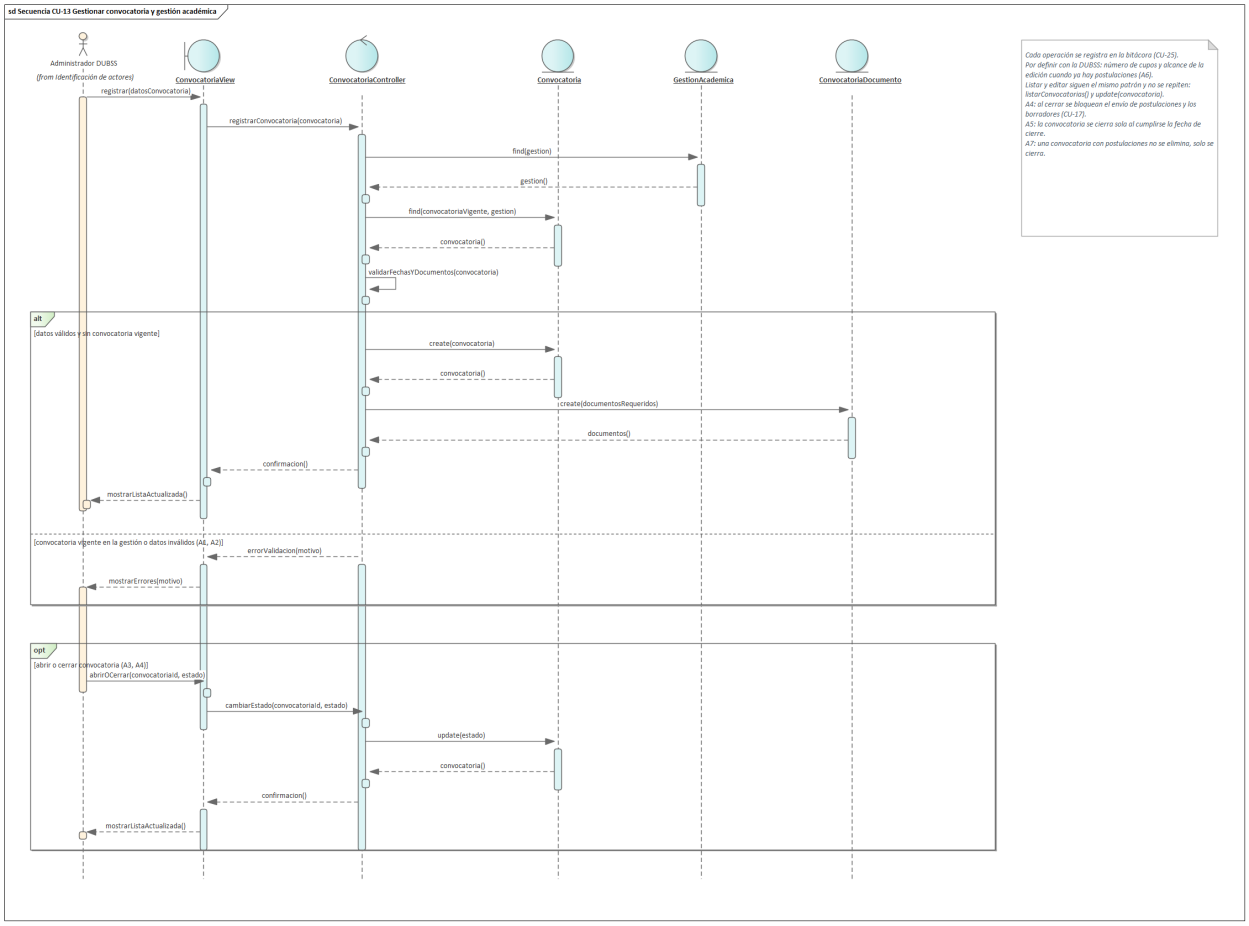

*Nota.* Elaboración propia.

### *7.4.13. Diseñar CU-13 Gestionar convocatoria y gestión académica*

La Figura 110 presenta el diagrama de secuencia del CU-13.

**Figura 110**

*Diagrama de secuencia del CU-13 Gestionar convocatoria y gestión académica*



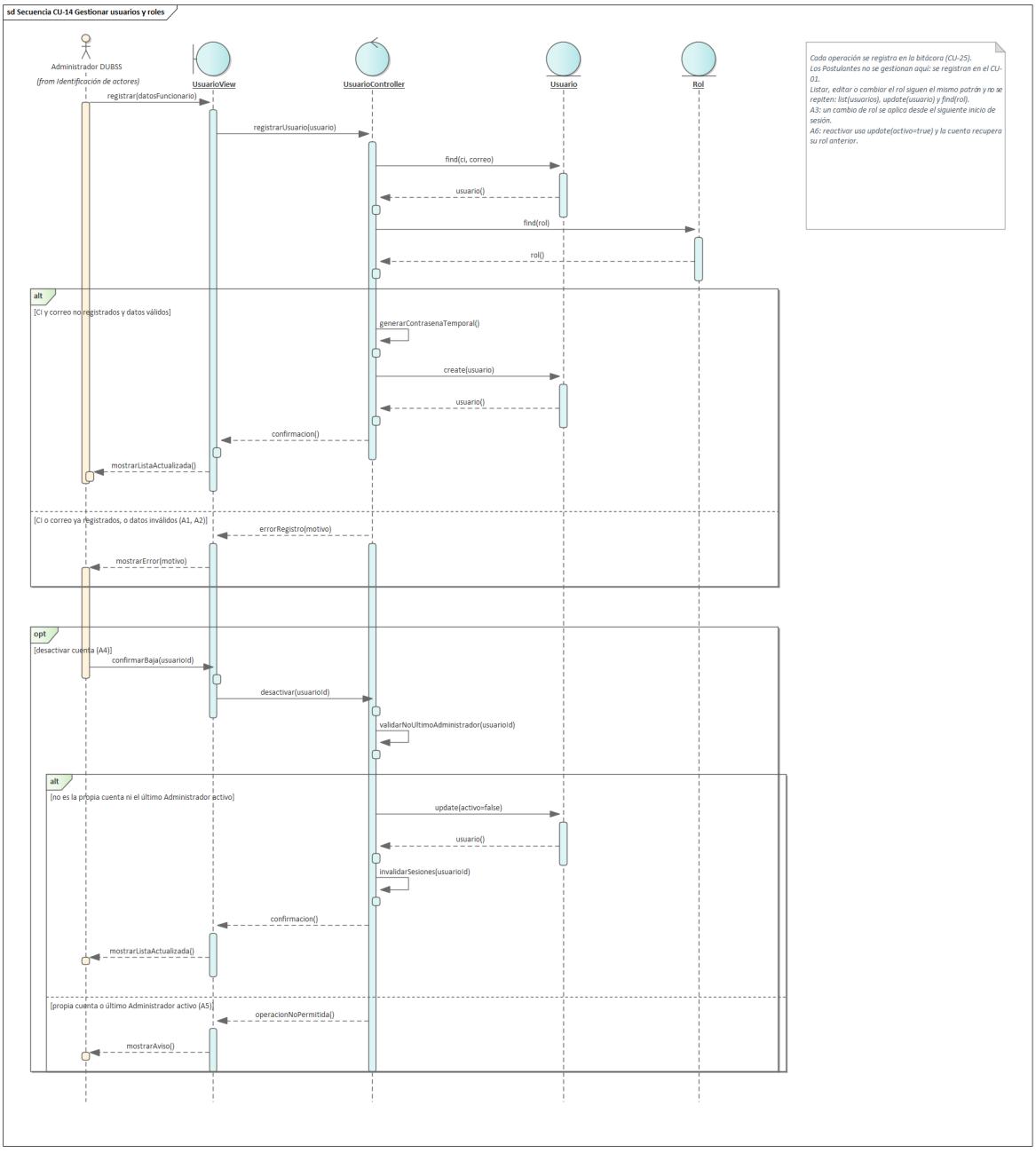

*Nota.* Elaboración propia.

### *7.4.14. Diseñar CU-14 Gestionar usuarios y roles*

La Figura 111 presenta el diagrama de secuencia del CU-14.

**Figura 111**

*Diagrama de secuencia del CU-14 Gestionar usuarios y roles*



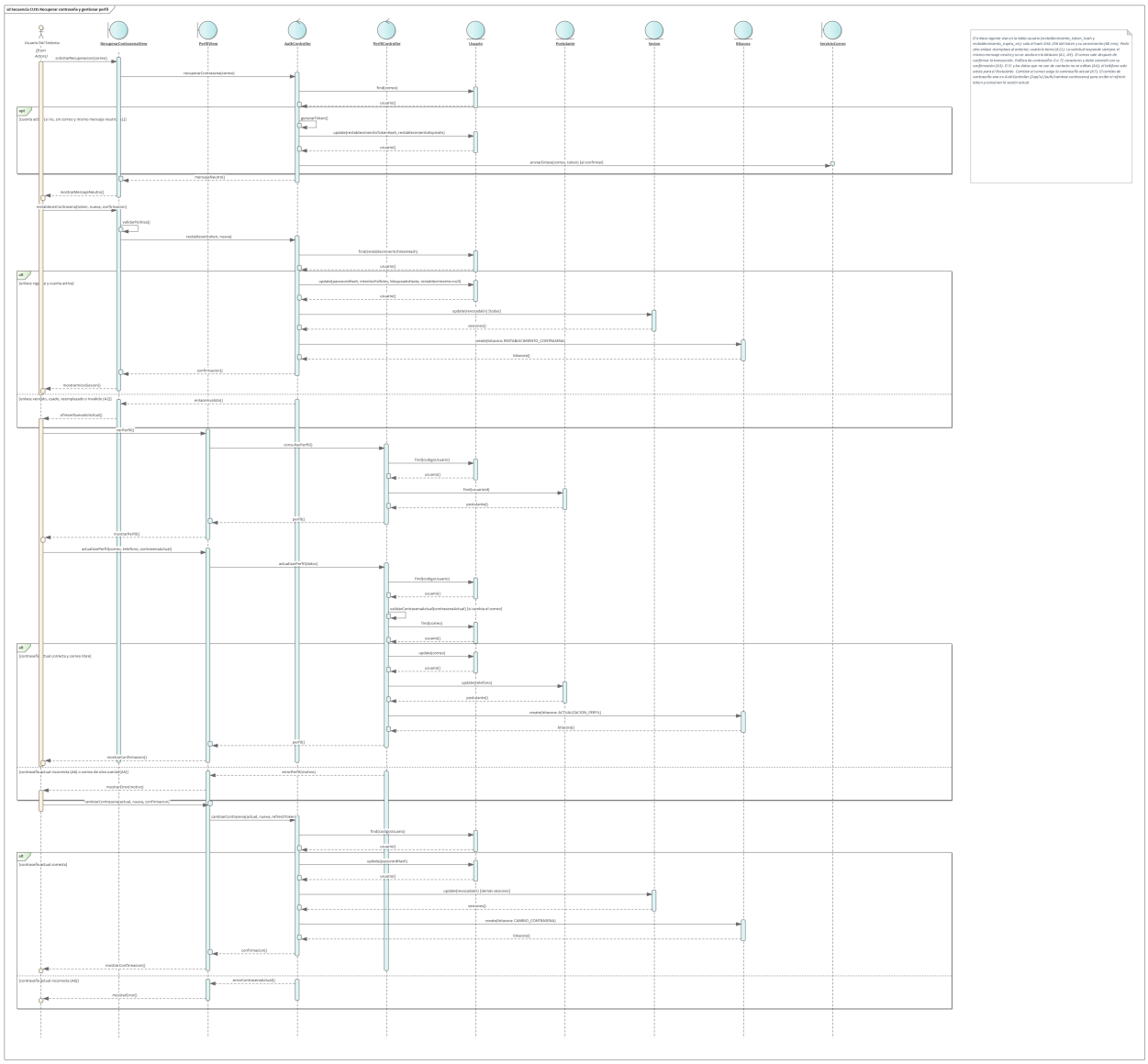

*Nota.* Elaboración propia.

### *7.4.15. Diseñar CU-15 Recuperar contraseña / gestionar perfil*

La Figura 112 presenta el diagrama de secuencia del CU-15. El enlace de restablecimiento se guarda en la propia cuenta (hash del token y vencimiento), restablecer la contraseña cierra todas las sesiones y cambiarla desde el perfil cierra solo las de los demás dispositivos.

**Figura 112**

*Diagrama de secuencia del CU-15 Recuperar contraseña / gestionar perfil*



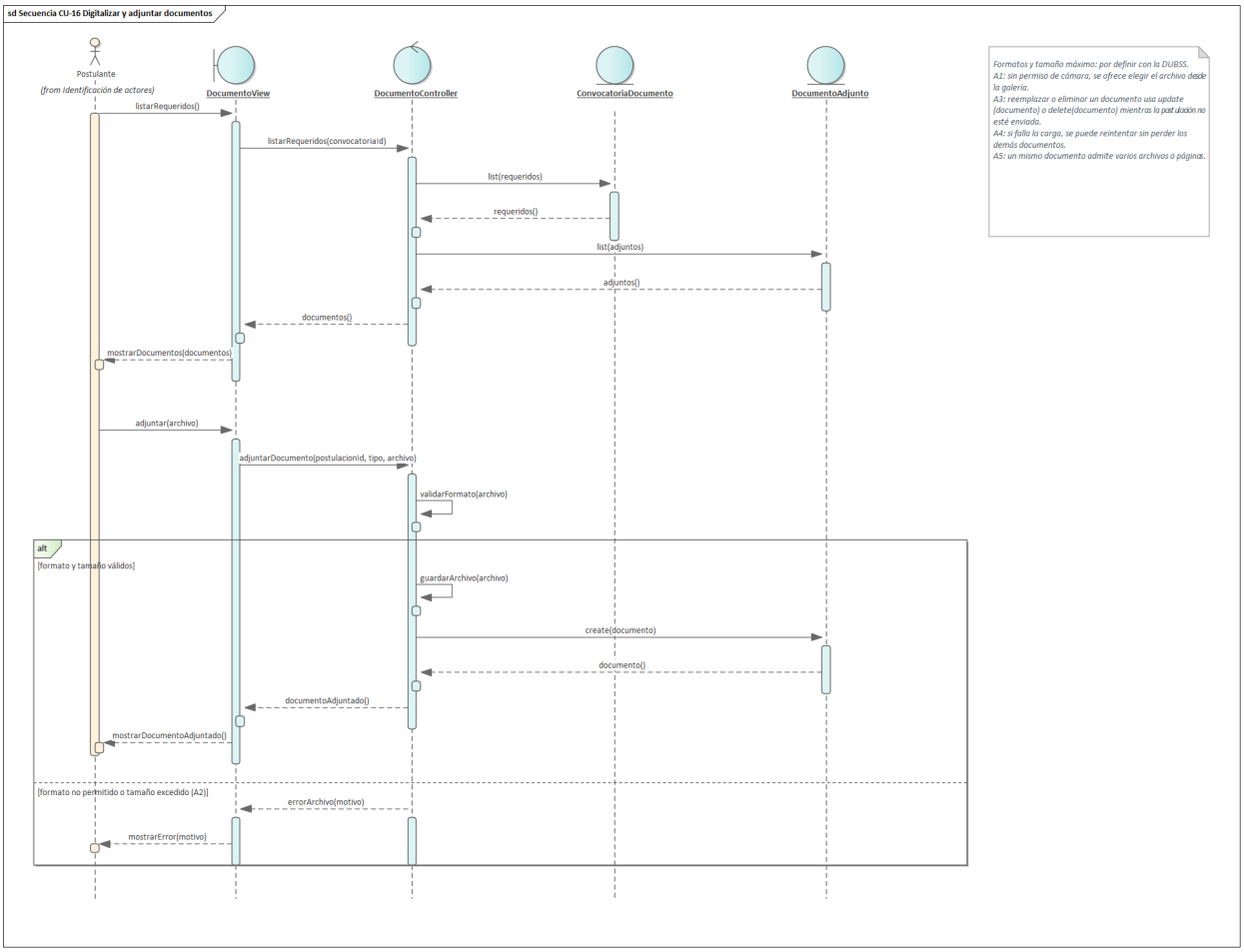
*Nota.* Elaboración propia.

### *7.4.16. Diseñar CU-16 Digitalizar y adjuntar documentos*

La Figura 113 presenta el diagrama de secuencia del CU-16.

**Figura 113**

*Diagrama de secuencia del CU-16 Digitalizar y adjuntar documentos*



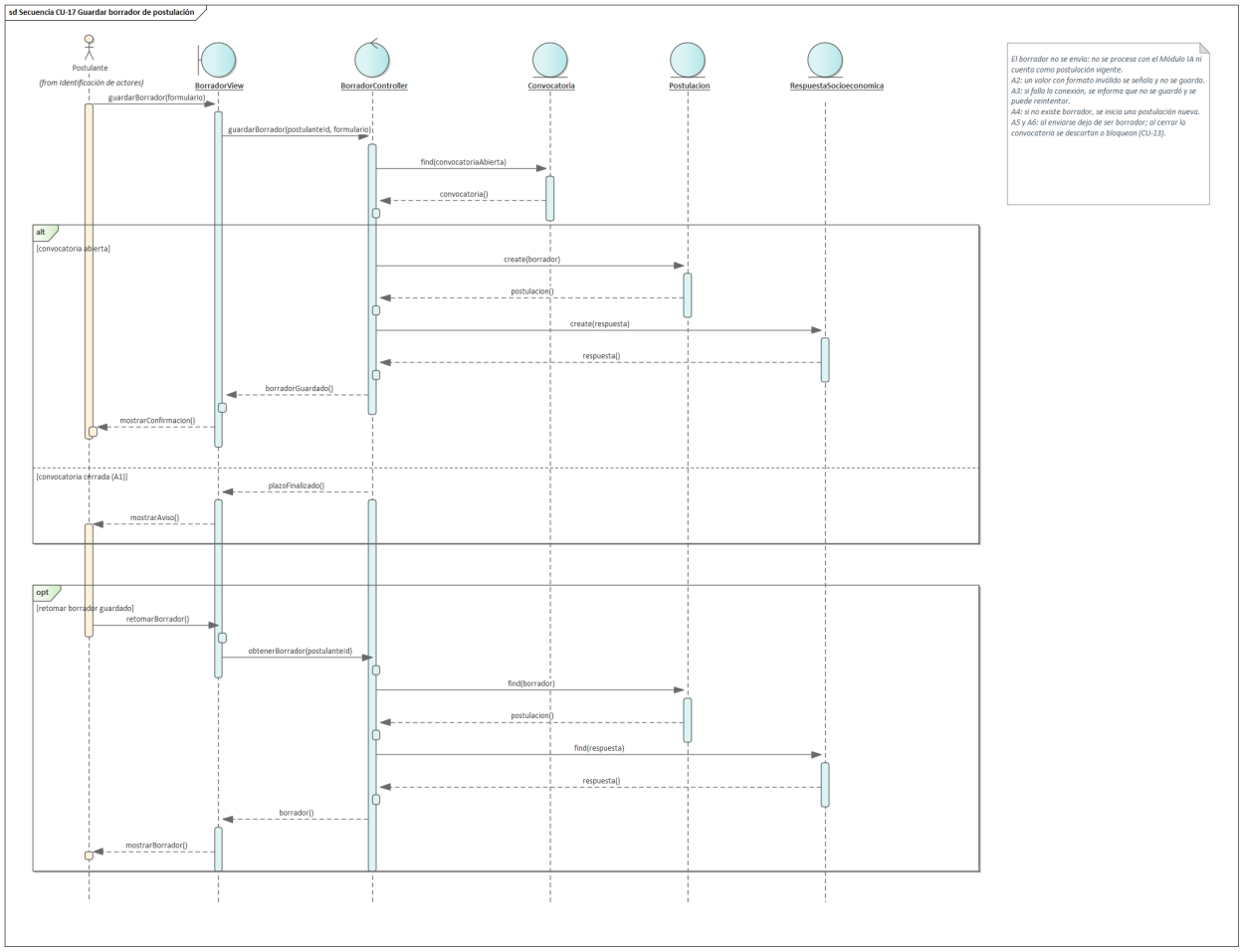

*Nota.* Elaboración propia.

### *7.4.17. Diseñar CU-17 Guardar borrador de postulación*

La Figura 114 presenta el diagrama de secuencia del CU-17.

**Figura 114**

*Diagrama de secuencia del CU-17 Guardar borrador de postulación*



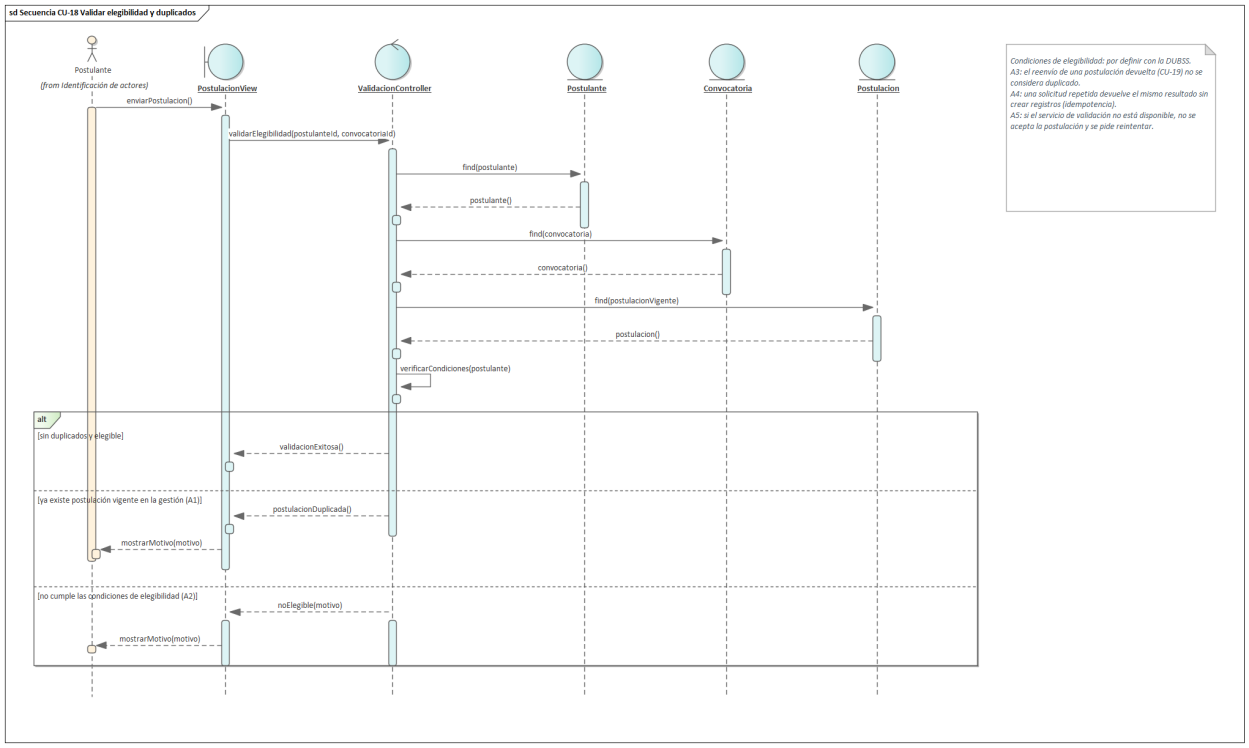

*Nota.* Elaboración propia.

### *7.4.18. Diseñar CU-18 Validar elegibilidad y duplicados*

La Figura 115 presenta el diagrama de secuencia del CU-18.

**Figura 115**

*Diagrama de secuencia del CU-18 Validar elegibilidad y duplicados*



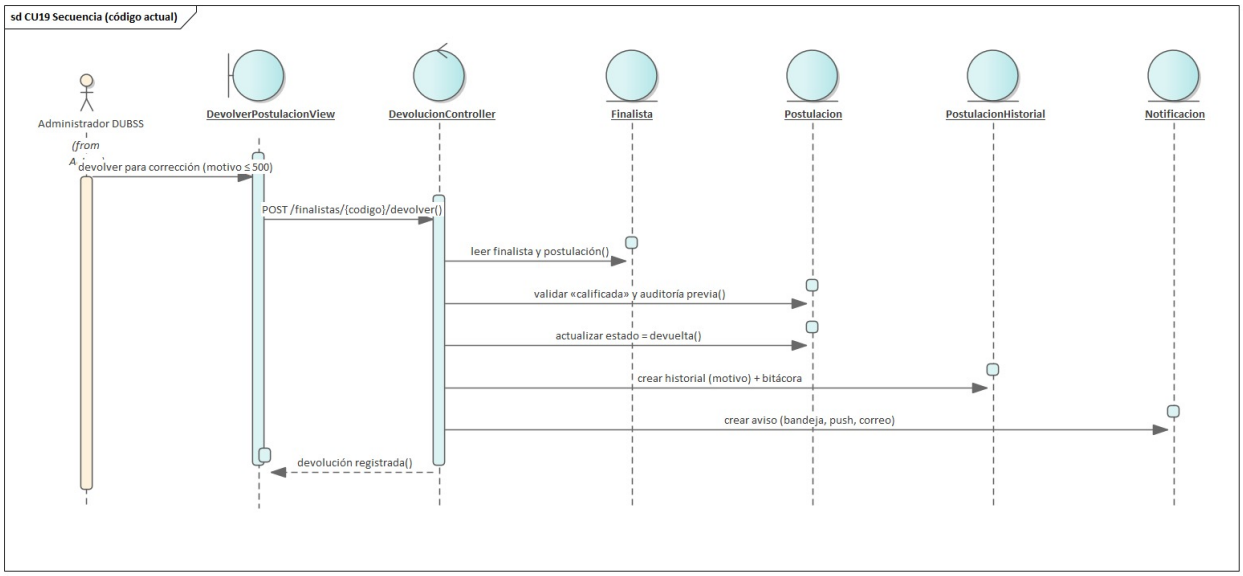

*Nota.* Elaboración propia.

### *7.4.19. Diseñar CU-19 Devolver postulación para corrección*

La Figura 116 presenta el diagrama de secuencia del CU-19.

**Figura 116**

*Diagrama de secuencia del CU-19 Devolver postulación para corrección*



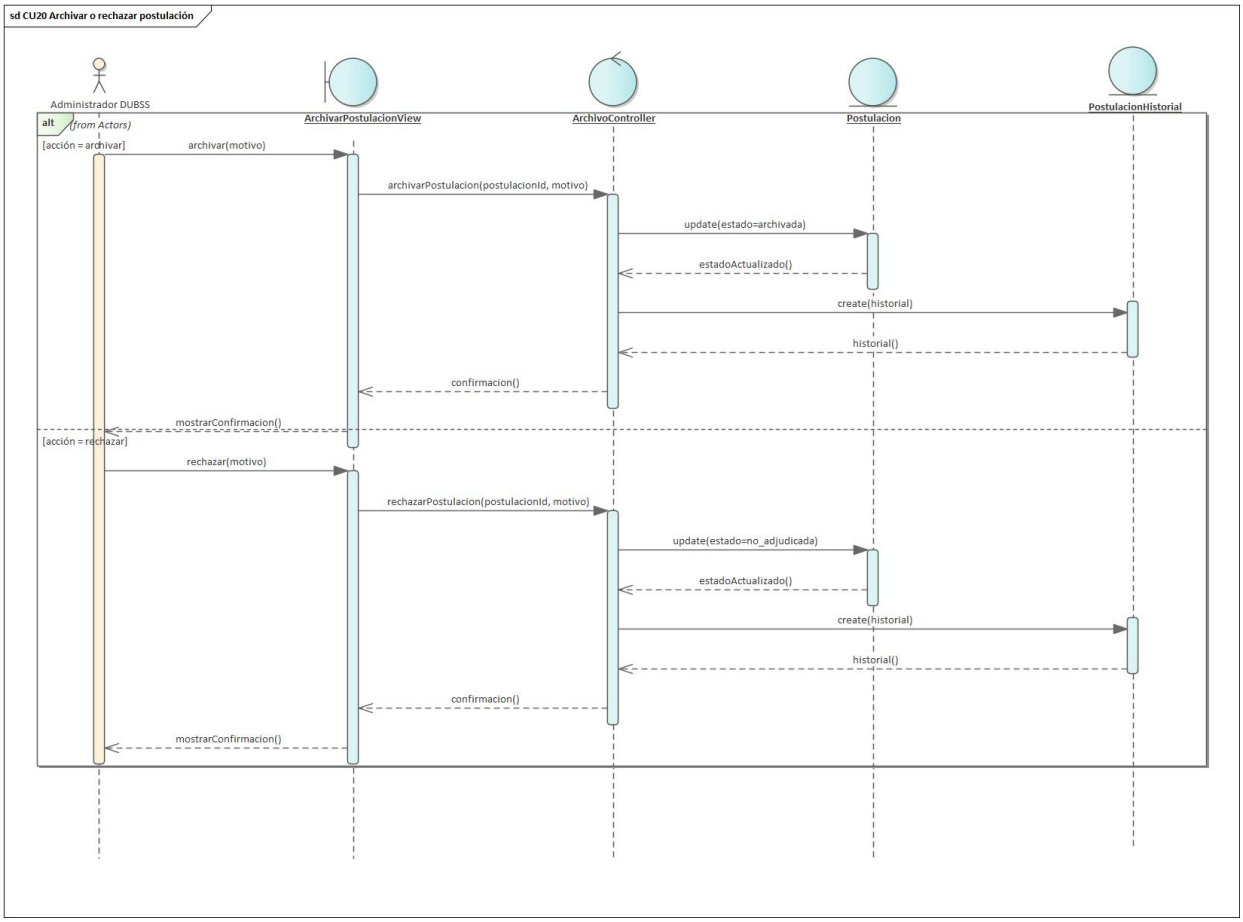

*Nota.* Elaboración propia.

### *7.4.20. Diseñar CU-20 Archivar o rechazar postulación*

La Figura 117 presenta el diagrama de secuencia del CU-20.

**Figura 117**

*Diagrama de secuencia del CU-20 Archivar o rechazar postulación*



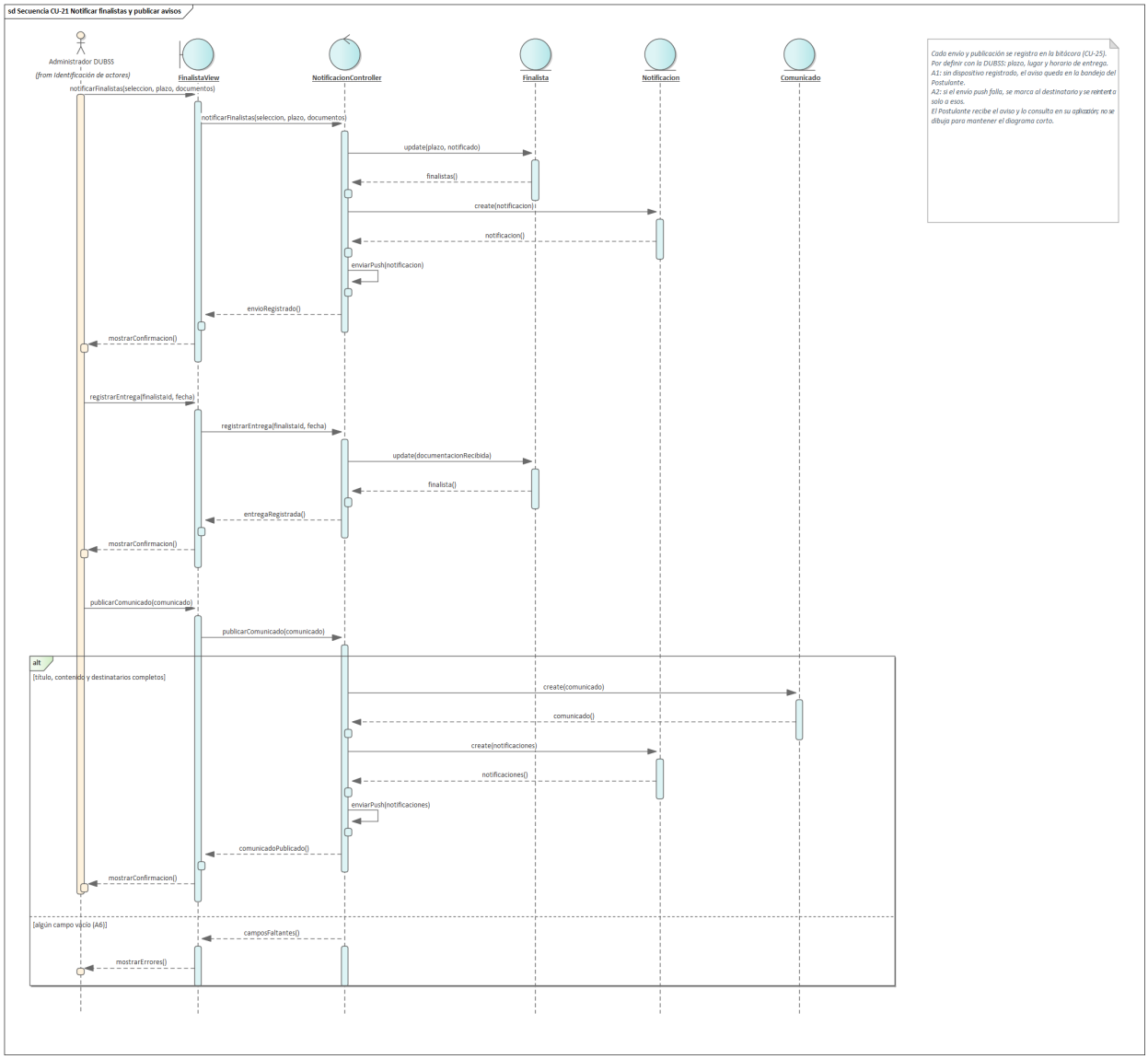

*Nota.* Elaboración propia.

### *7.4.21. Diseñar CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos*

La Figura 118 presenta el diagrama de secuencia del CU-21.

**Figura 118**

*Diagrama de secuencia del CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos*



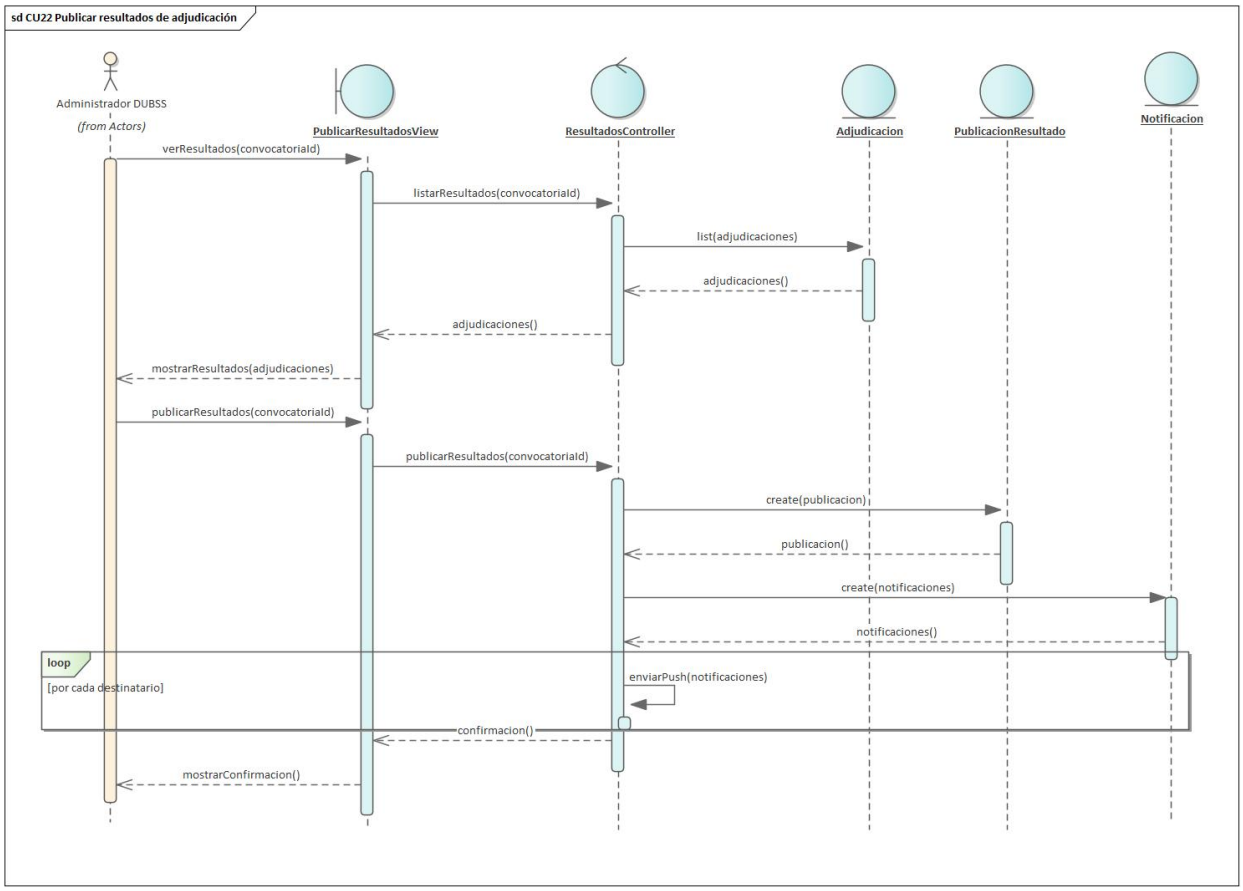

*Nota.* Elaboración propia.

### *7.4.22. Diseñar CU-22 Publicar resultados de adjudicación*

La Figura 119 presenta el diagrama de secuencia del CU-22.

**Figura 119**

*Diagrama de secuencia del CU-22 Publicar resultados de adjudicación*



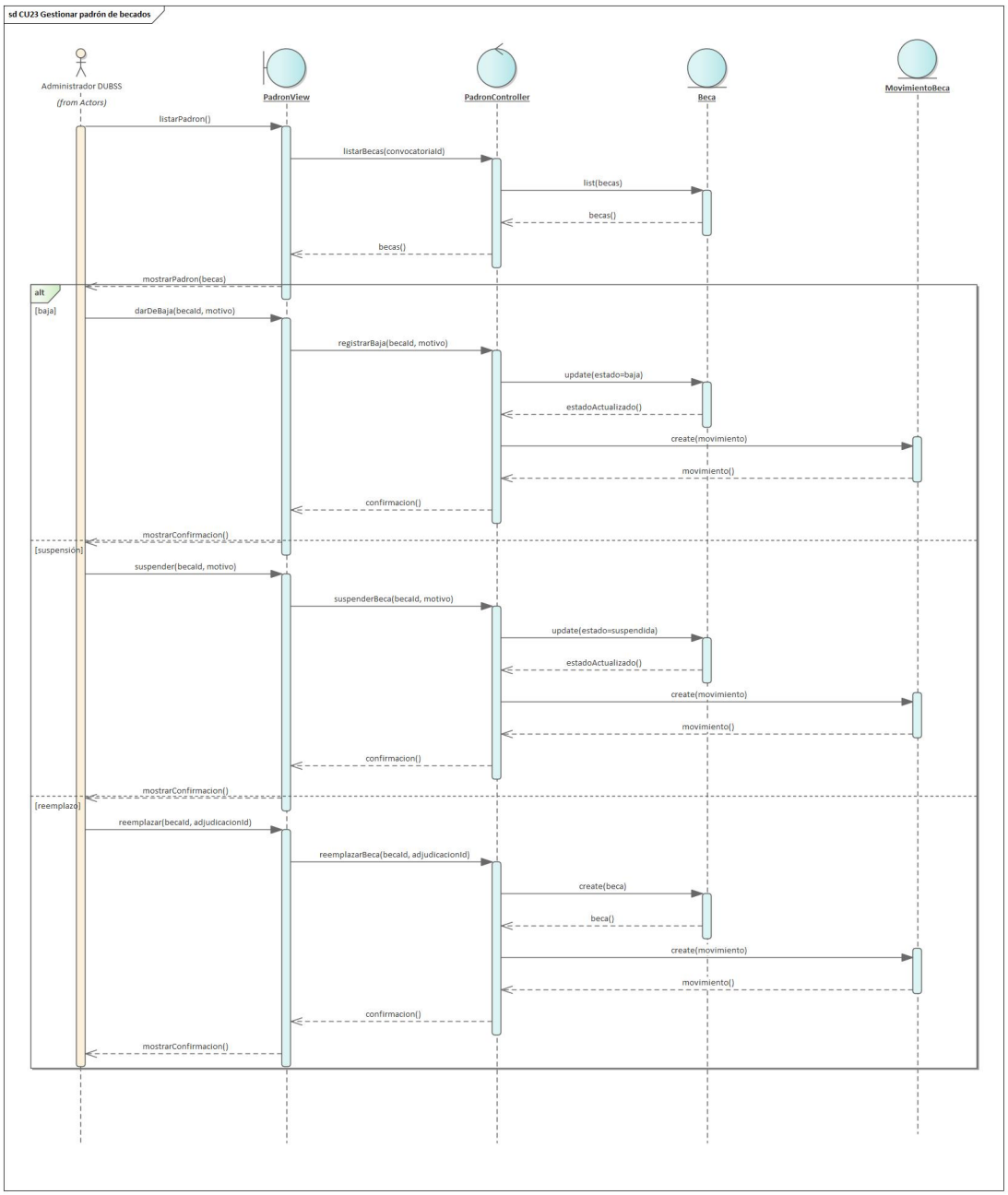

*Nota.* Elaboración propia.

### *7.4.23. Diseñar CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo)*

La Figura 120 presenta el diagrama de secuencia del CU-23.

**Figura 120**

*Diagrama de secuencia del CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo)*



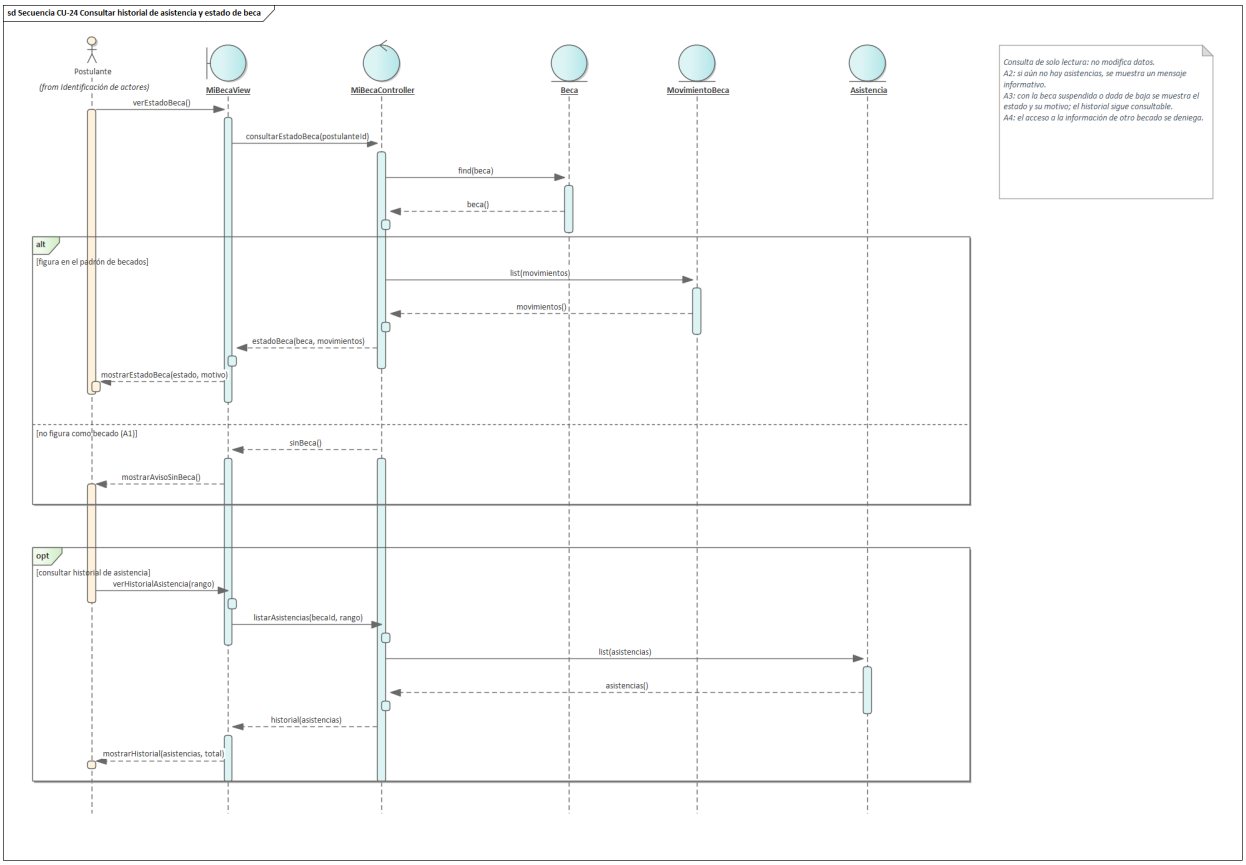

*Nota.* Elaboración propia.

### *7.4.24. Diseñar CU-24 Consultar historial de asistencia y estado de beca*

La Figura 121 presenta el diagrama de secuencia del CU-24.

**Figura 121**

*Diagrama de secuencia del CU-24 Consultar historial de asistencia y estado de beca*



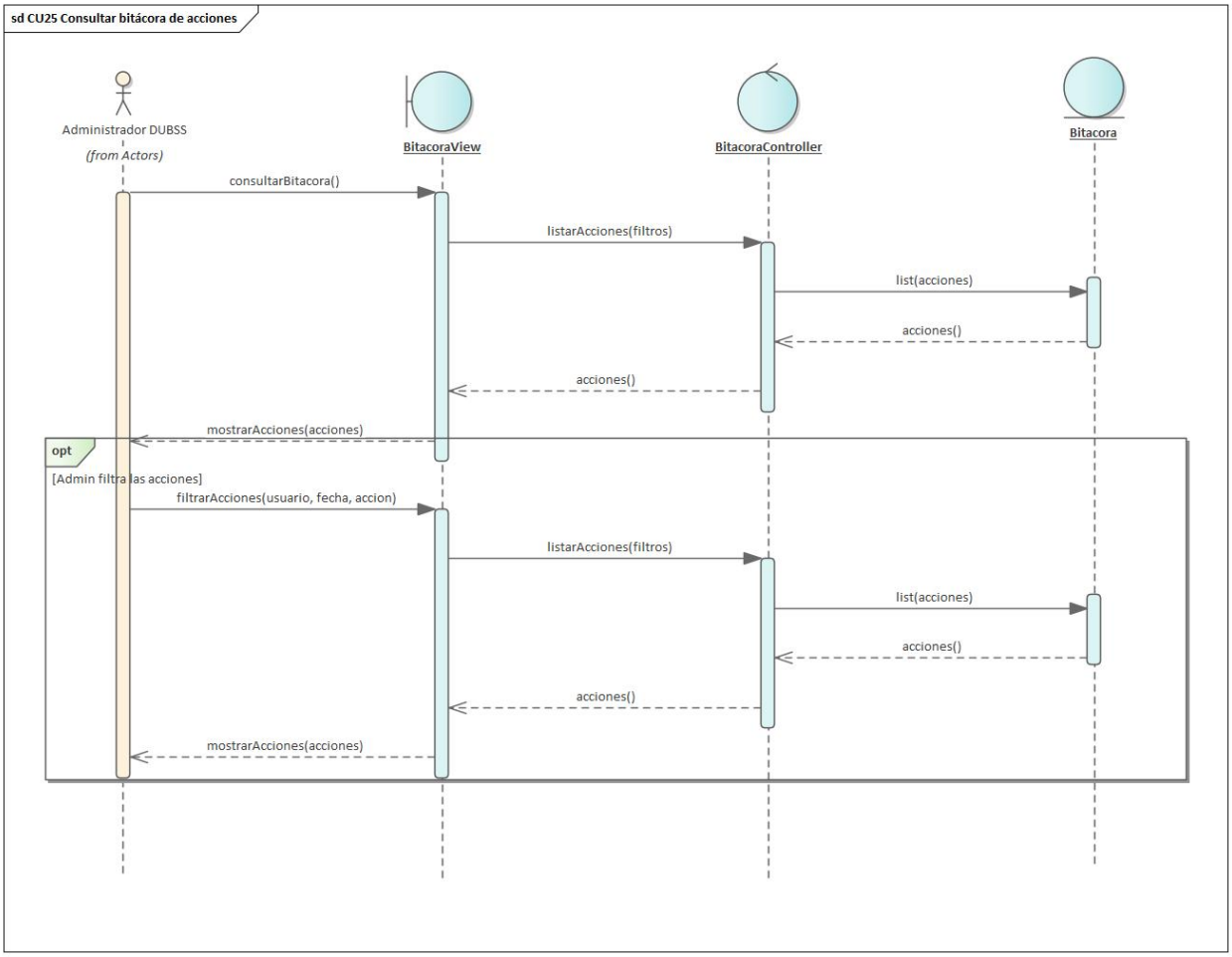

*Nota.* Elaboración propia.

### *7.4.25. Diseñar CU-25 Consultar bitácora de acciones*

La Figura 122 presenta el diagrama de secuencia del CU-25.

**Figura 122**

*Diagrama de secuencia del CU-25 Consultar bitácora de acciones*



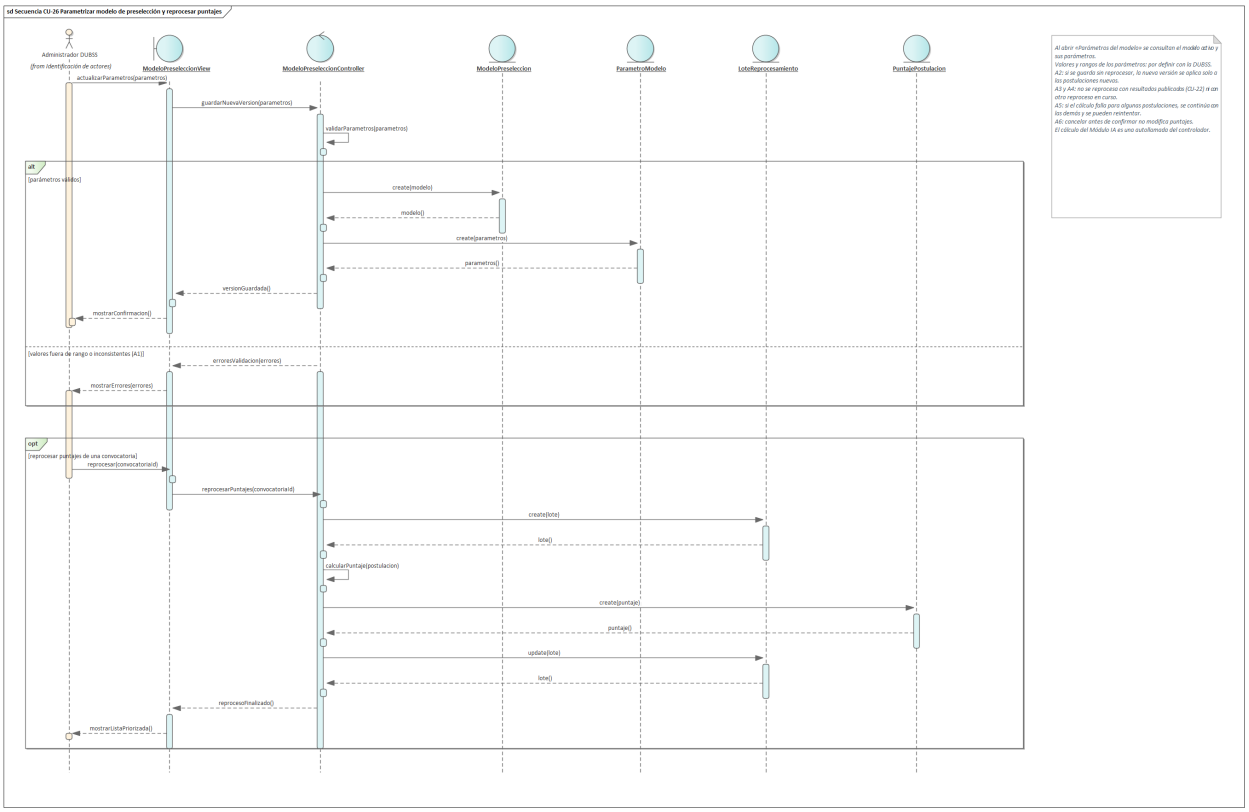

*Nota.* Elaboración propia.

### *7.4.26. Diseñar CU-26 Parametrizar modelo de preselección y reprocesar puntajes*

La Figura 123 presenta el diagrama de secuencia del CU-26.

**Figura 123**

*Diagrama de secuencia del CU-26 Parametrizar modelo de preselección y reprocesar puntajes*




*Nota.* Elaboración propia.

## 7.5. Diseño de Interfaces

El diseño de interfaces expresa, mediante maquetas (*mockups*), la disposición de los elementos visuales con los que el actor interactúa en cada caso de uso, de modo que las clases de interfaz (*View*) identificadas en el análisis cuenten con una representación concreta (Jacobson et al., 1999).

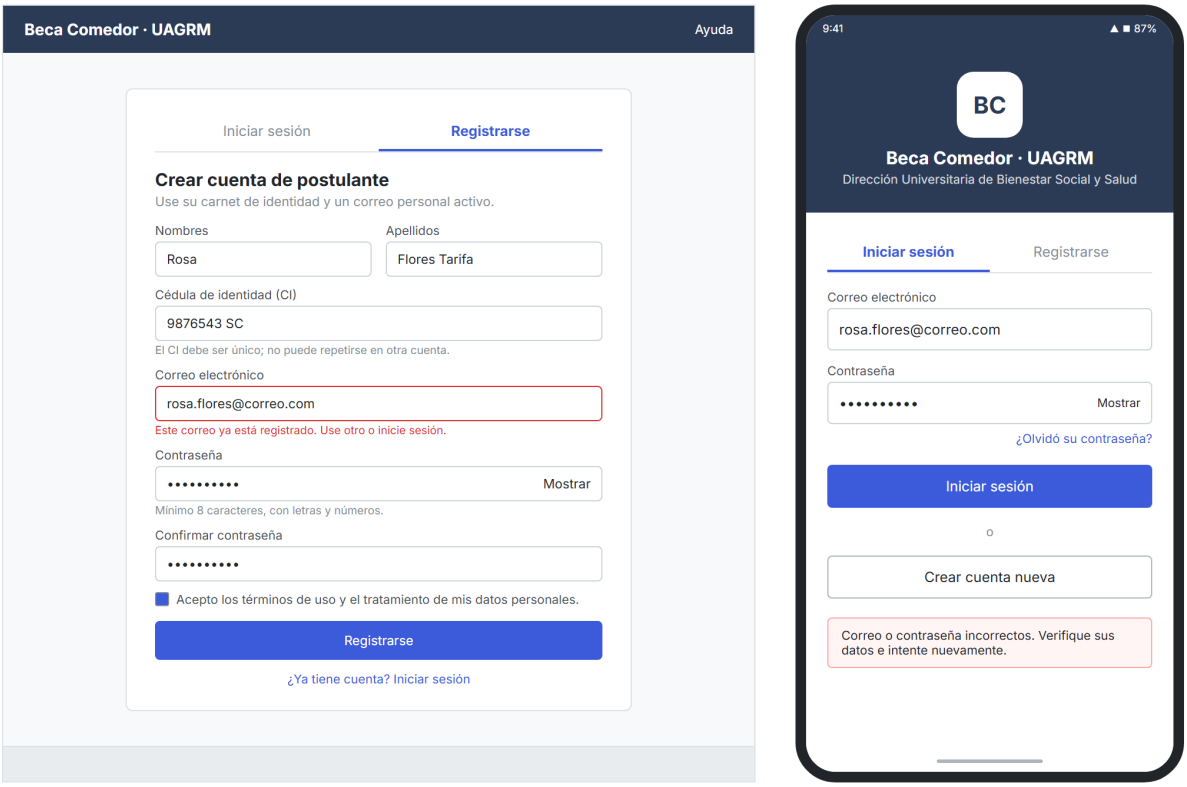
A continuación se presenta el diseño de interfaces de los veintiséis casos de uso del sistema, en el orden de su numeración (CU-01 a CU-26).

### *7.5.1. Interfaz CU-01 Registrarse y autenticarse*

La Figura 124 presenta la maqueta de interfaz del CU-01.

**Figura 124**

*Maqueta de interfaz del CU-01 Registrarse y autenticarse*



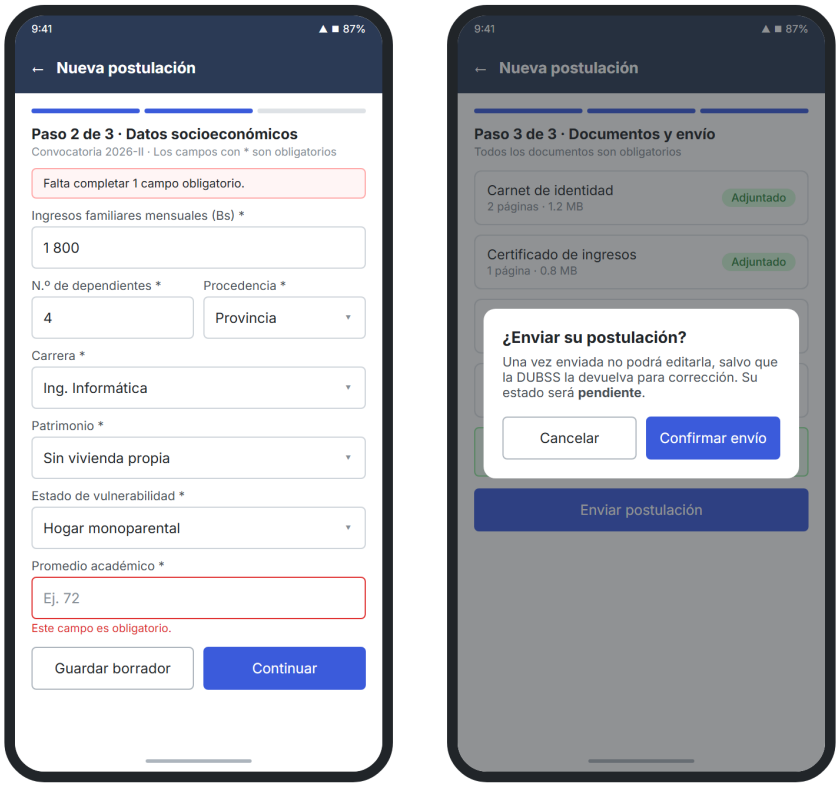

*Nota.* Elaboración propia.

### *7.5.2. Interfaz CU-02 Enviar postulación a Beca Comedor*

La Figura 125 presenta la maqueta de interfaz del CU-02.

**Figura 125**

*Maqueta de interfaz del CU-02 Enviar postulación a Beca Comedor*



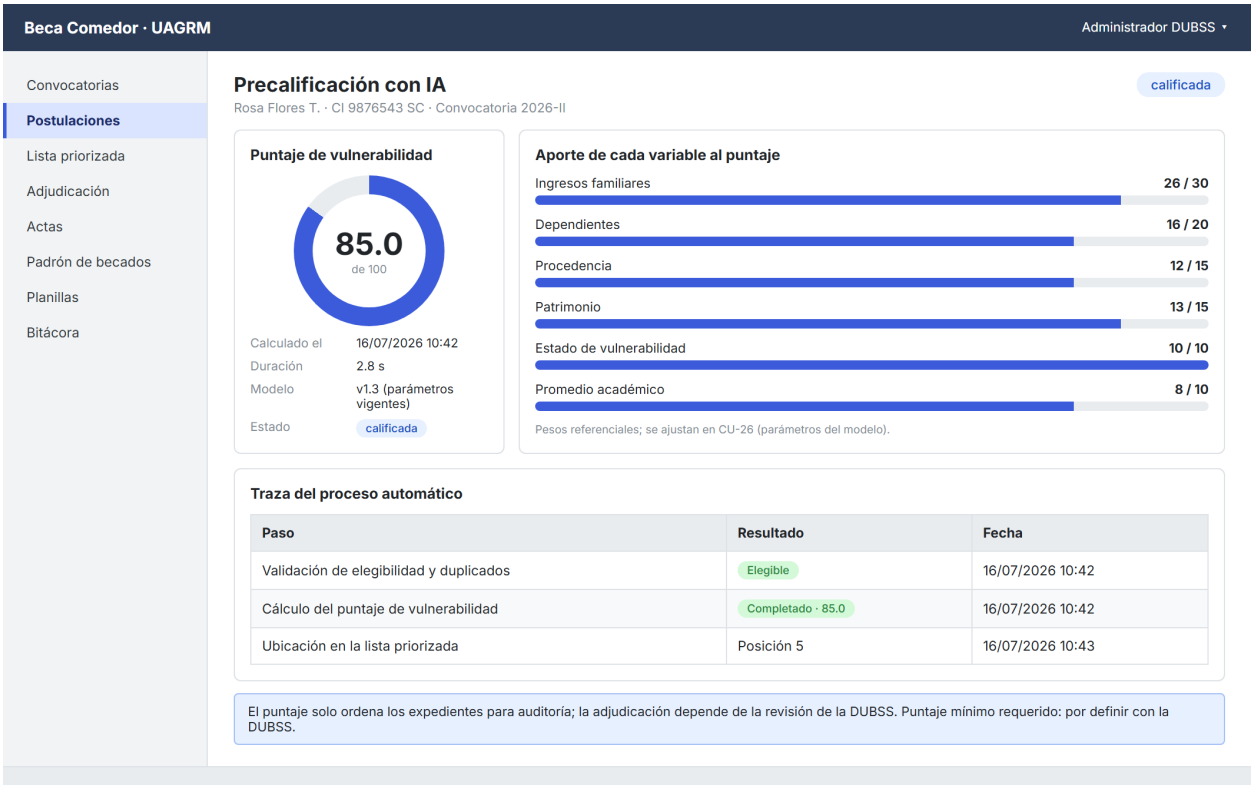

*Nota.* Elaboración propia.

### *7.5.3. Interfaz CU-03 Precalificar postulación con Inteligencia Artificial*

La Figura 126 presenta la maqueta de interfaz del CU-03.

**Figura 126**

*Maqueta de interfaz del CU-03 Precalificar postulación con Inteligencia Artificial*



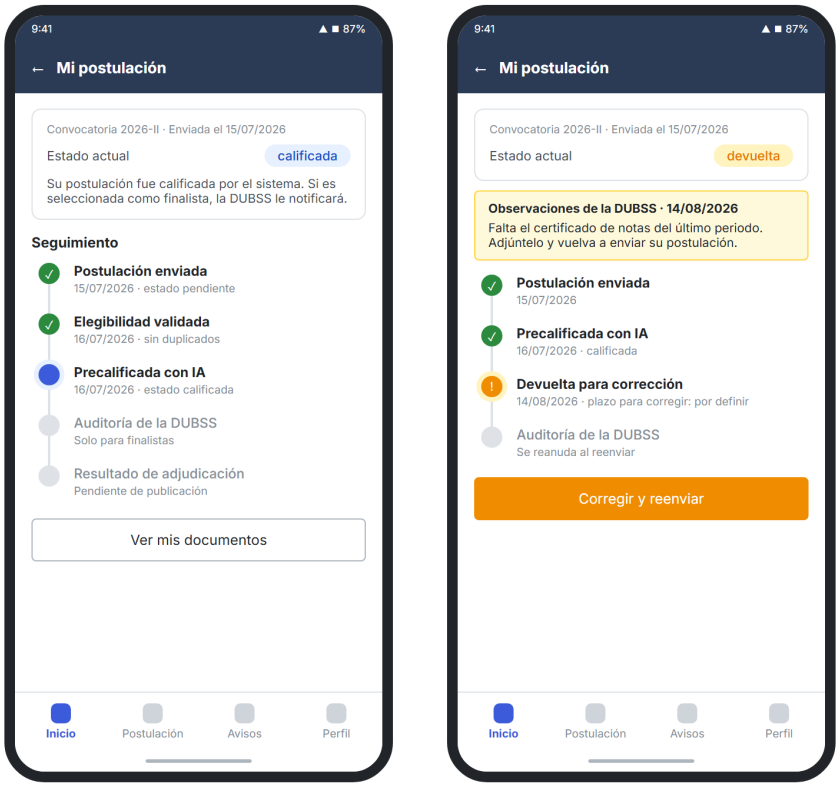

*Nota.* Elaboración propia.

### *7.5.4. Interfaz CU-04 Consultar estado de postulación*

La Figura 127 presenta la maqueta de interfaz del CU-04.

**Figura 127**

*Maqueta de interfaz del CU-04 Consultar estado de postulación*



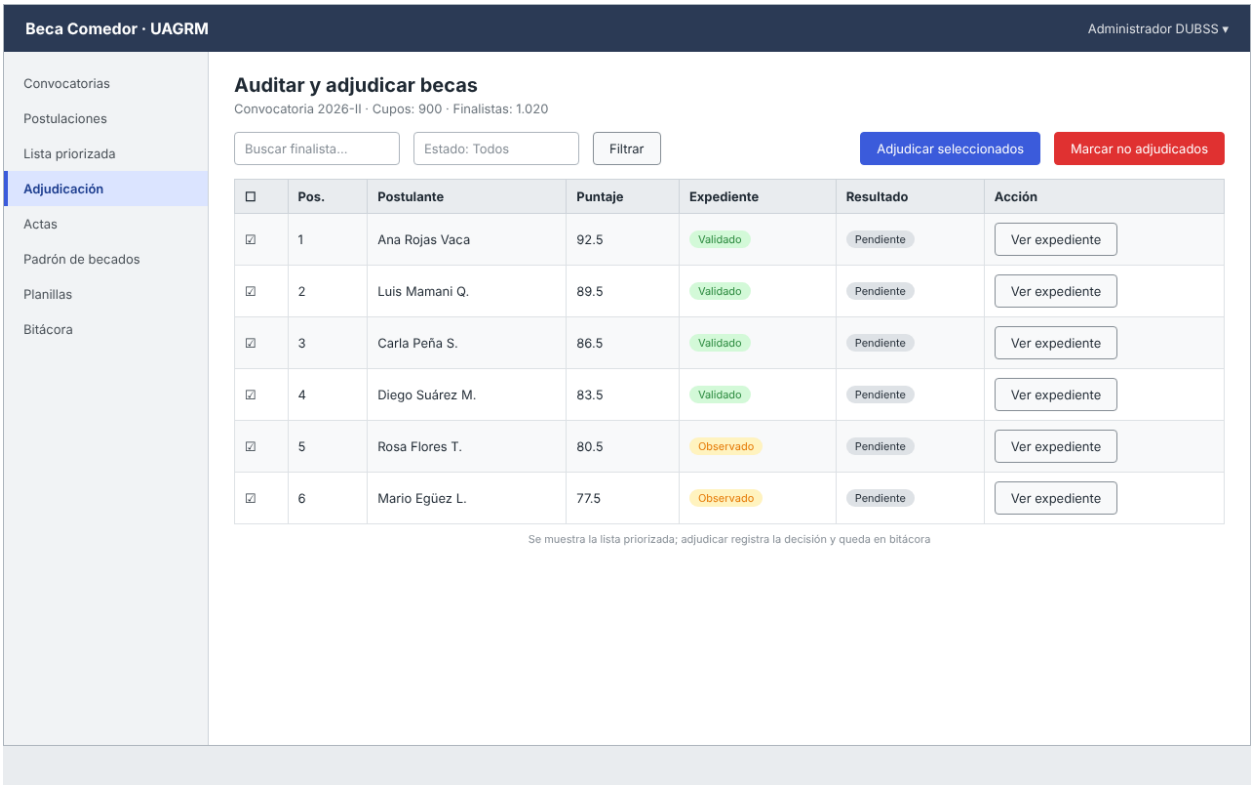

*Nota.* Elaboración propia.

### *7.5.5. Interfaz CU-05 Auditar y adjudicar becas*

La Figura 128 presenta la maqueta de interfaz del CU-05.

**Figura 128**

*Maqueta de interfaz del CU-05 Auditar y adjudicar becas*



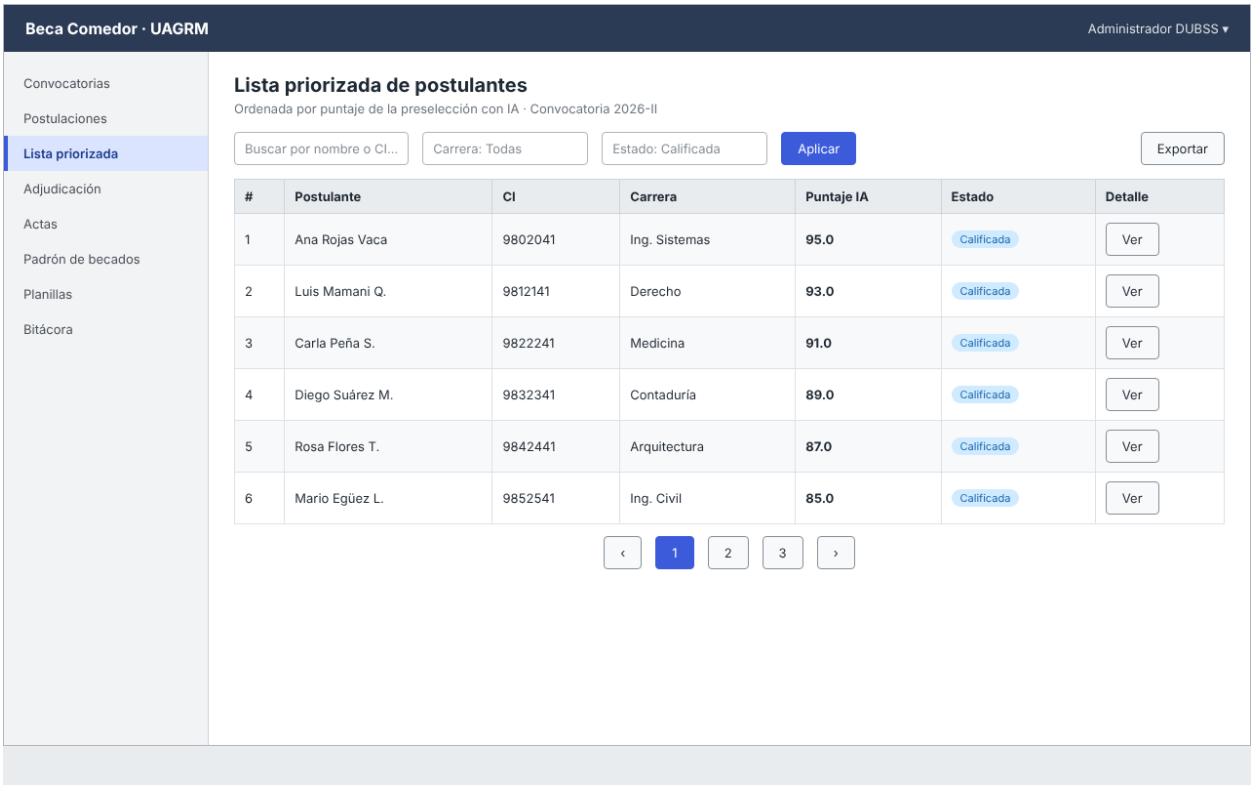

*Nota.* Elaboración propia.

### *7.5.6. Interfaz CU-06 Visualizar lista priorizada de postulantes*

La Figura 129 presenta la maqueta de interfaz del CU-06.

**Figura 129**

*Maqueta de interfaz del CU-06 Visualizar lista priorizada de postulantes*



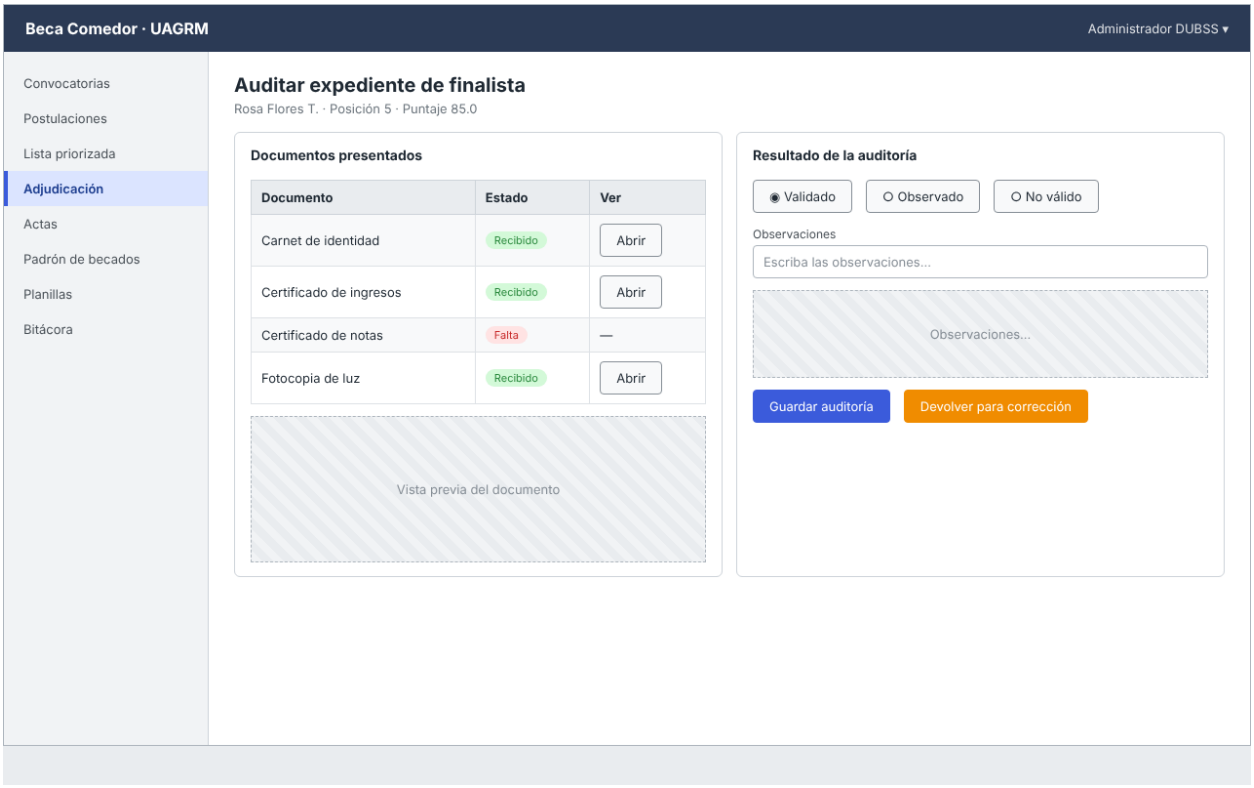

*Nota.* Elaboración propia.

### *7.5.7. Interfaz CU-07 Auditar expediente finalista*

La Figura 130 presenta la maqueta de interfaz del CU-07.

**Figura 130**

*Maqueta de interfaz del CU-07 Auditar expediente finalista*



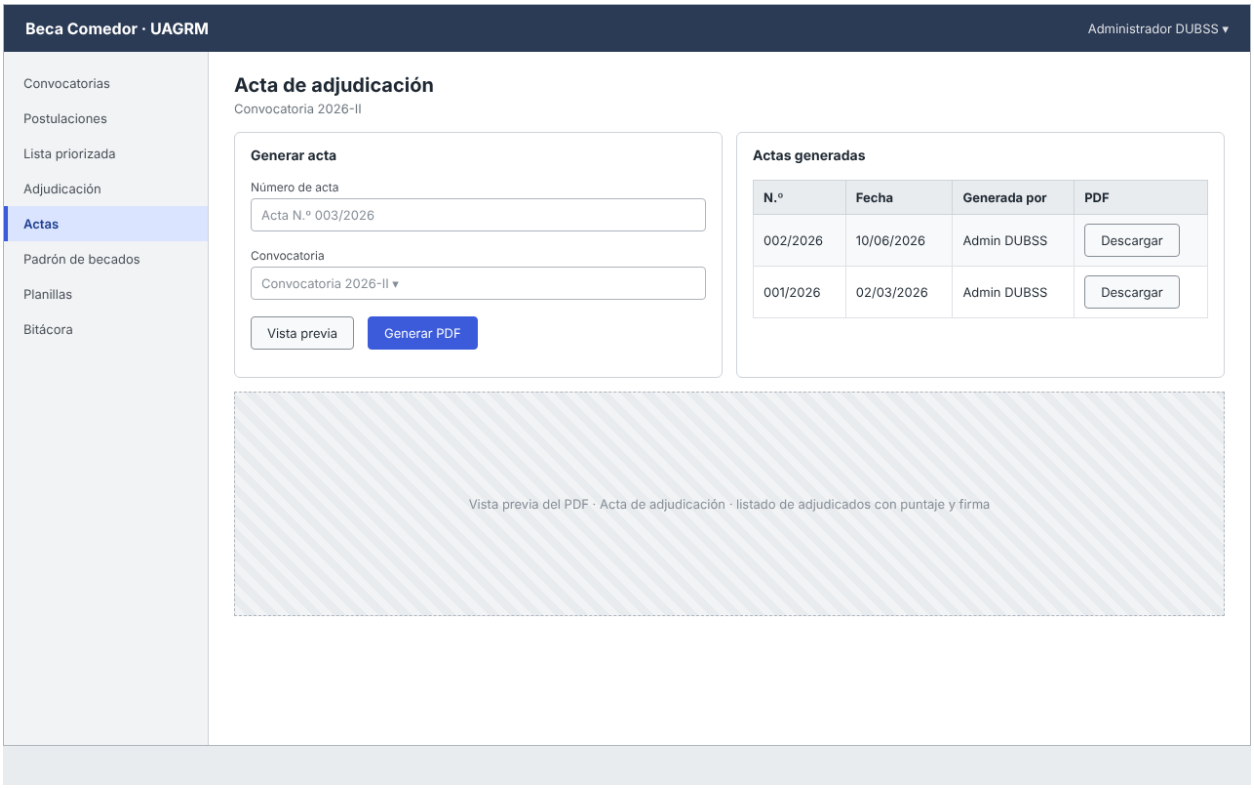

*Nota.* Elaboración propia.

### *7.5.8. Interfaz CU-08 Generar acta de adjudicación*

La Figura 131 presenta la maqueta de interfaz del CU-08.

**Figura 131**

*Maqueta de interfaz del CU-08 Generar acta de adjudicación*



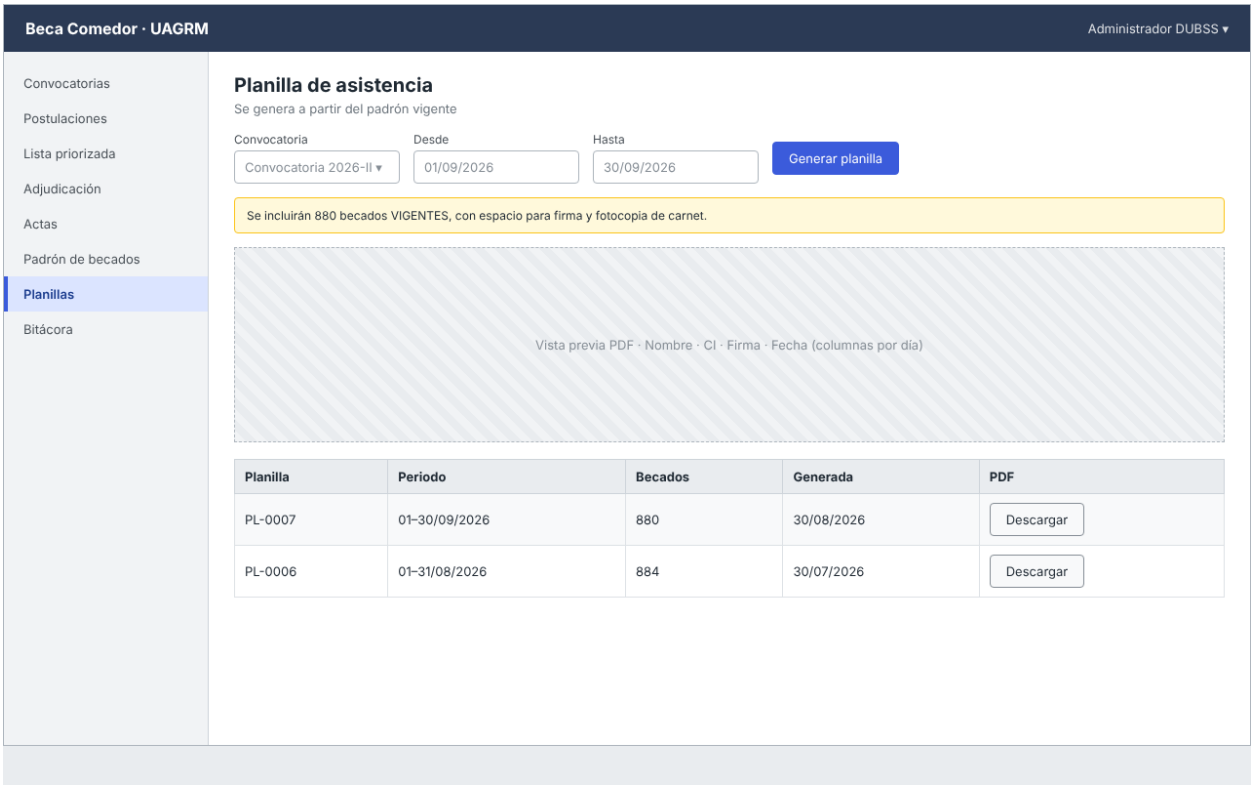

*Nota.* Elaboración propia.

### *7.5.9. Interfaz CU-09 Generar planilla de asistencia*

La Figura 132 presenta la maqueta de interfaz del CU-09.

**Figura 132**

*Maqueta de interfaz del CU-09 Generar planilla de asistencia*



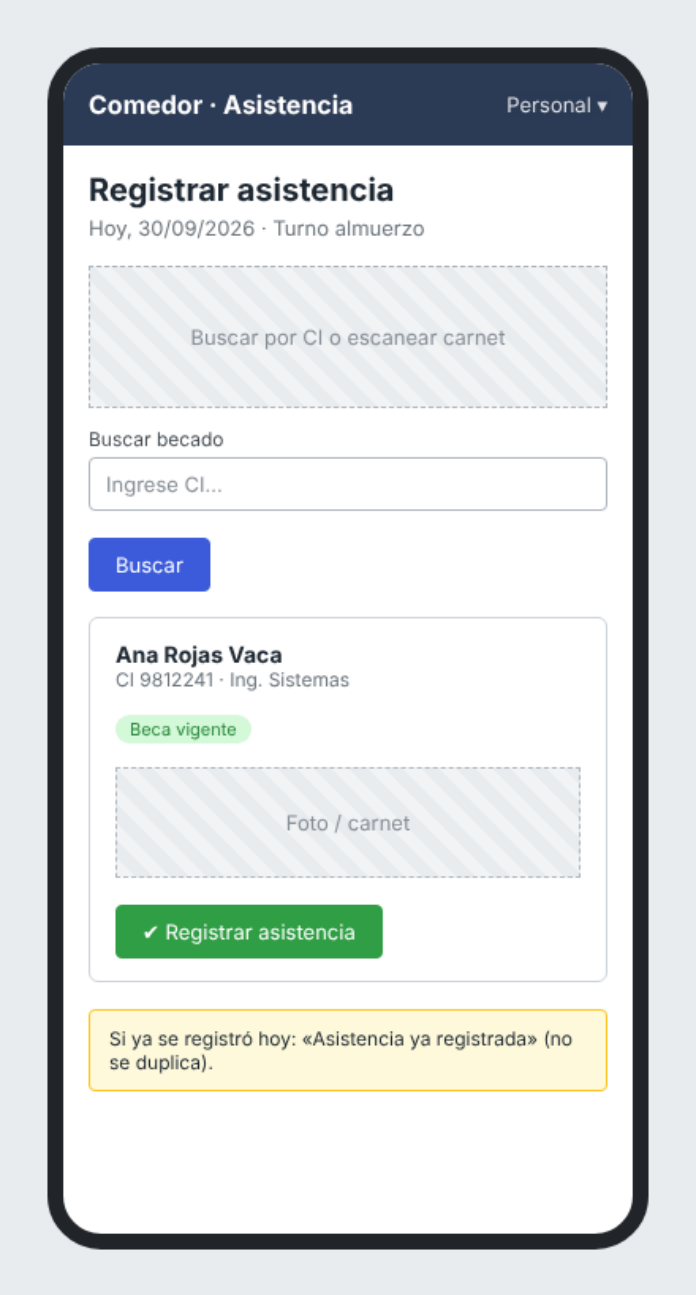

*Nota.* Elaboración propia.

### *7.5.10. Interfaz CU-10 Registrar asistencia digital*

La Figura 133 presenta la maqueta de interfaz del CU-10.

**Figura 133**

*Maqueta de interfaz del CU-10 Registrar asistencia digital*



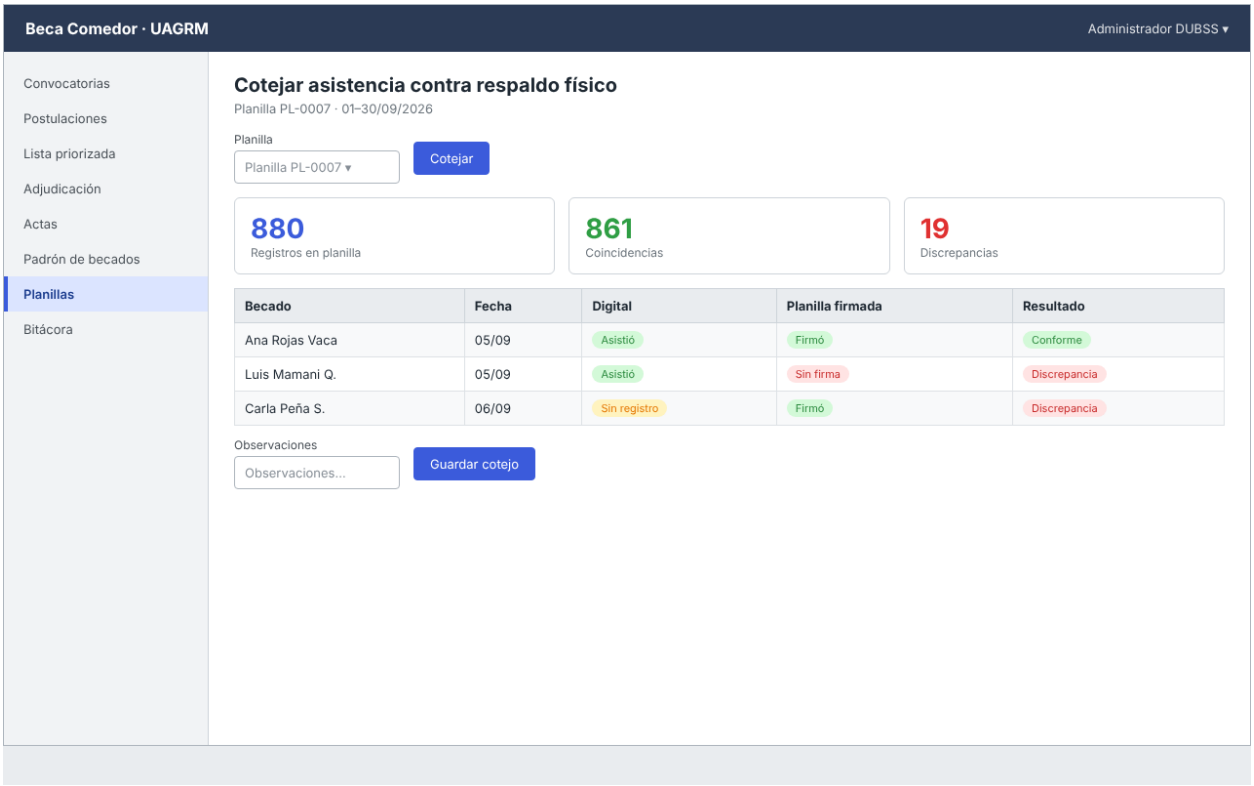

*Nota.* Elaboración propia.

### *7.5.11. Interfaz CU-11 Cotejar registro contra respaldo físico*

La Figura 134 presenta la maqueta de interfaz del CU-11.

**Figura 134**

*Maqueta de interfaz del CU-11 Cotejar registro contra respaldo físico*



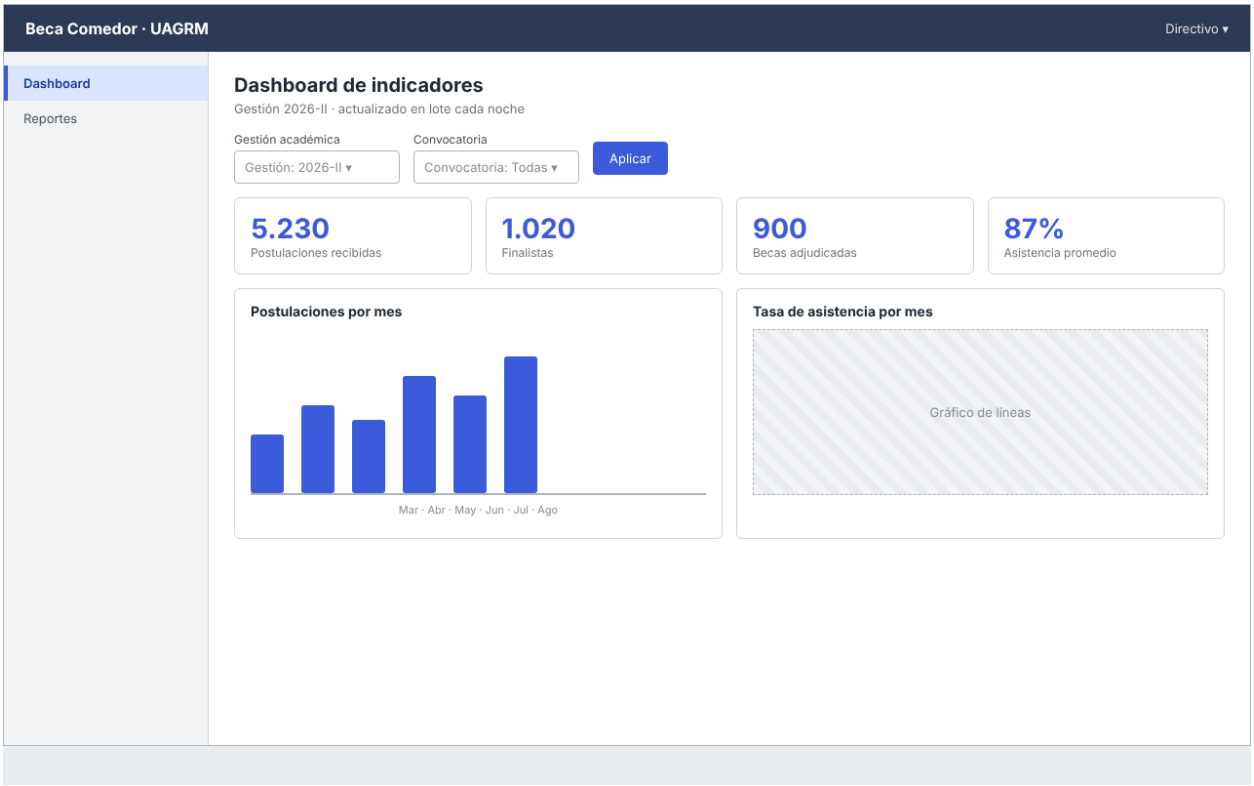

*Nota.* Elaboración propia.

### *7.5.12. Interfaz CU-12 Visualizar dashboard de indicadores BI*

La Figura 135 presenta la maqueta de interfaz del CU-12.

**Figura 135**

*Maqueta de interfaz del CU-12 Visualizar dashboard de indicadores BI*



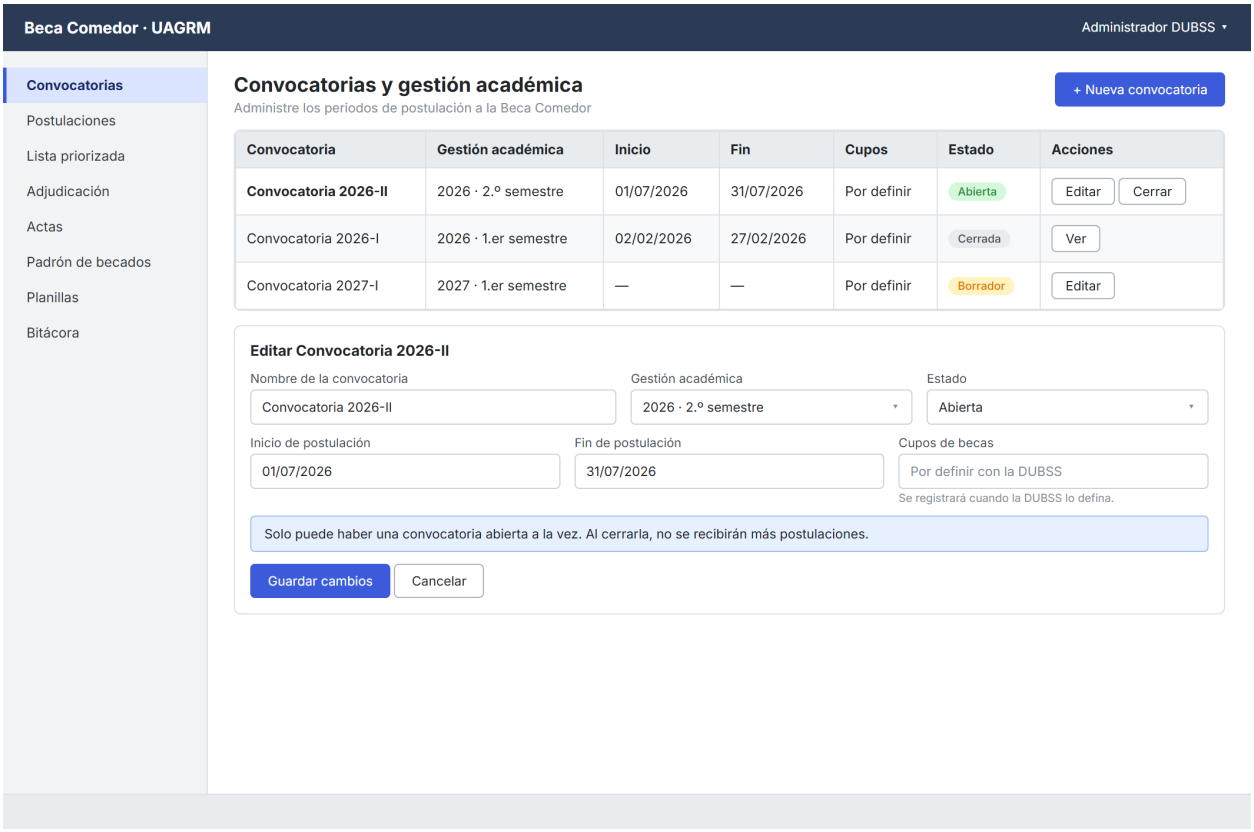

*Nota.* Elaboración propia.

### *7.5.13. Interfaz CU-13 Gestionar convocatoria y gestión académica*

La Figura 136 presenta la maqueta de interfaz del CU-13.

**Figura 136**

*Maqueta de interfaz del CU-13 Gestionar convocatoria y gestión académica*



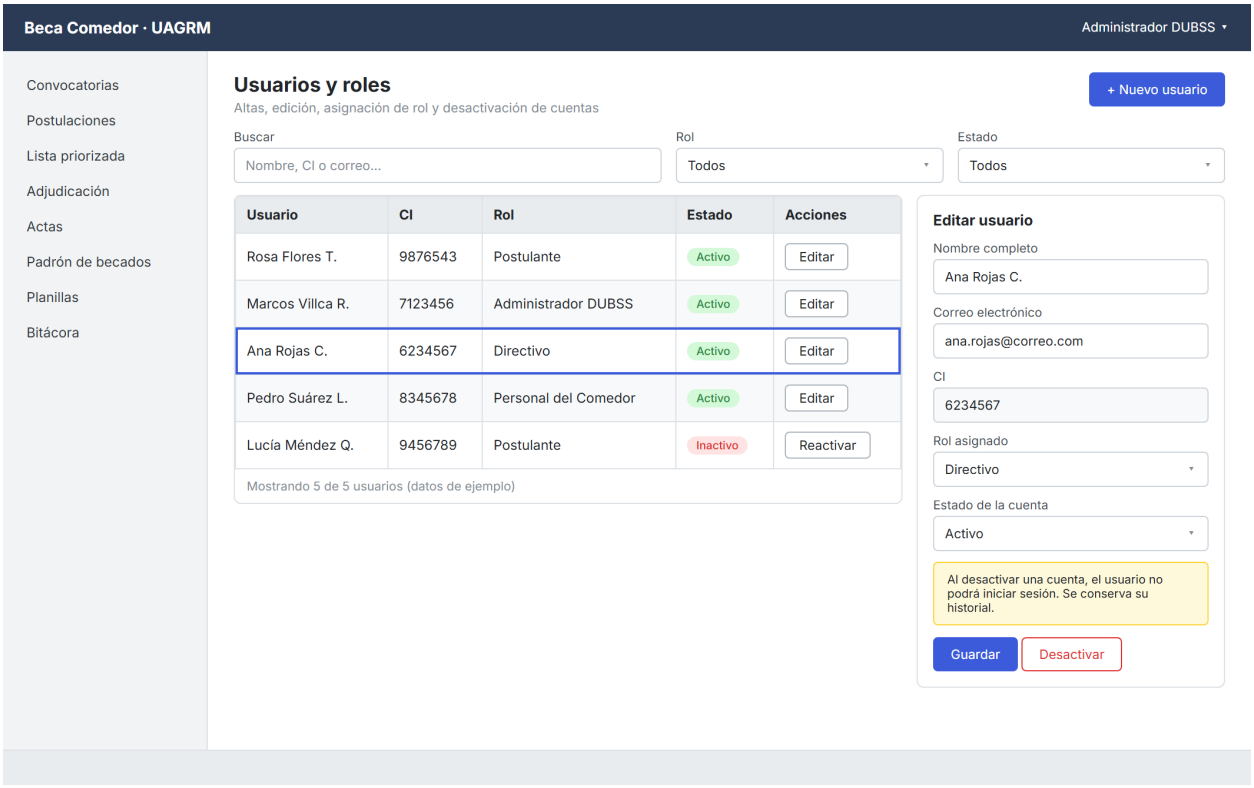

*Nota.* Elaboración propia.

### *7.5.14. Interfaz CU-14 Gestionar usuarios y roles*

La Figura 137 presenta la maqueta de interfaz del CU-14.

**Figura 137**

*Maqueta de interfaz del CU-14 Gestionar usuarios y roles*



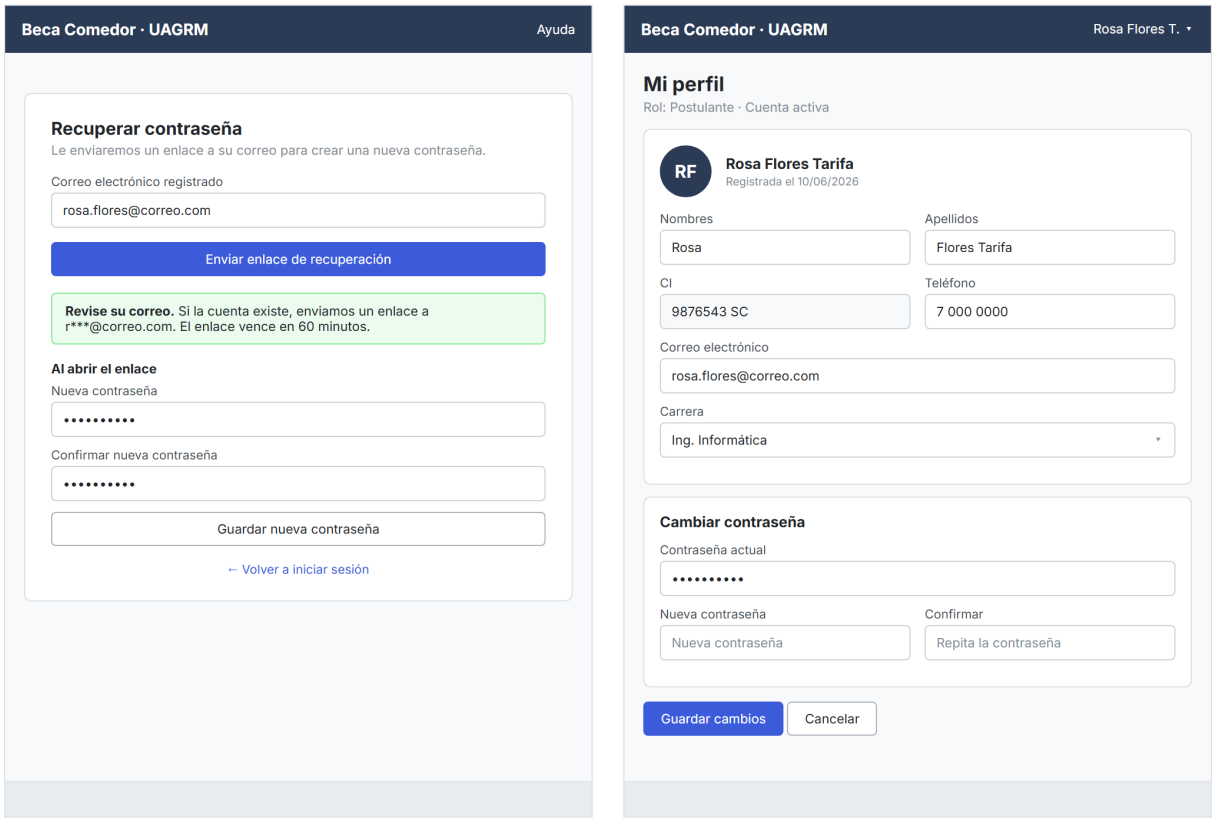

*Nota.* Elaboración propia.

### *7.5.15. Interfaz CU-15 Recuperar contraseña / gestionar perfil*

La Figura 138 presenta la maqueta de interfaz del CU-15.

**Figura 138**

*Maqueta de interfaz del CU-15 Recuperar contraseña / gestionar perfil*



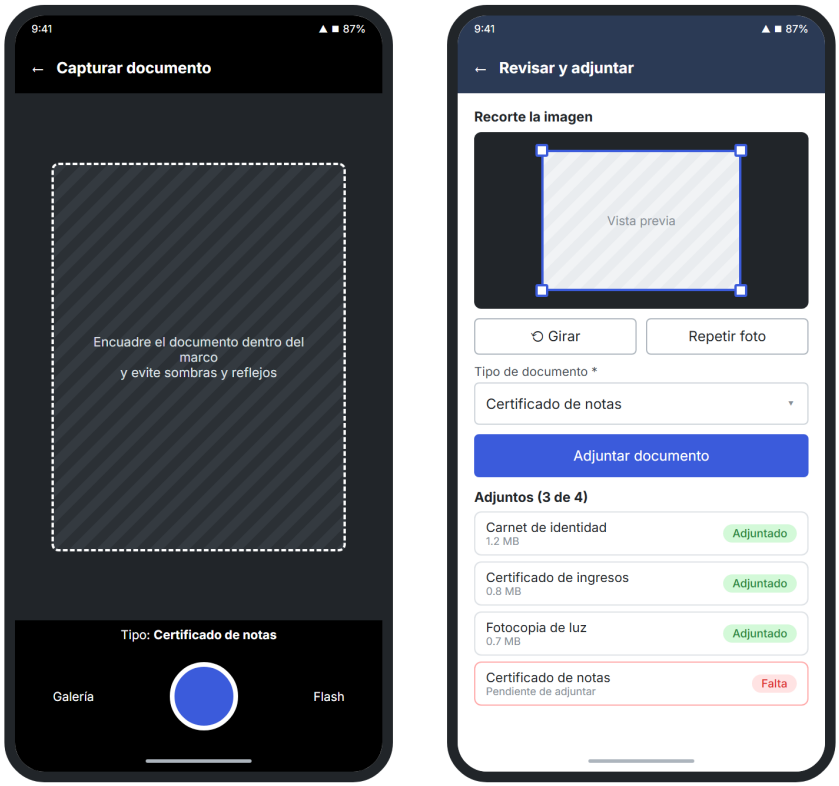

*Nota.* Elaboración propia.

### *7.5.16. Interfaz CU-16 Digitalizar y adjuntar documentos*

La Figura 139 presenta la maqueta de interfaz del CU-16.

**Figura 139**

*Maqueta de interfaz del CU-16 Digitalizar y adjuntar documentos*



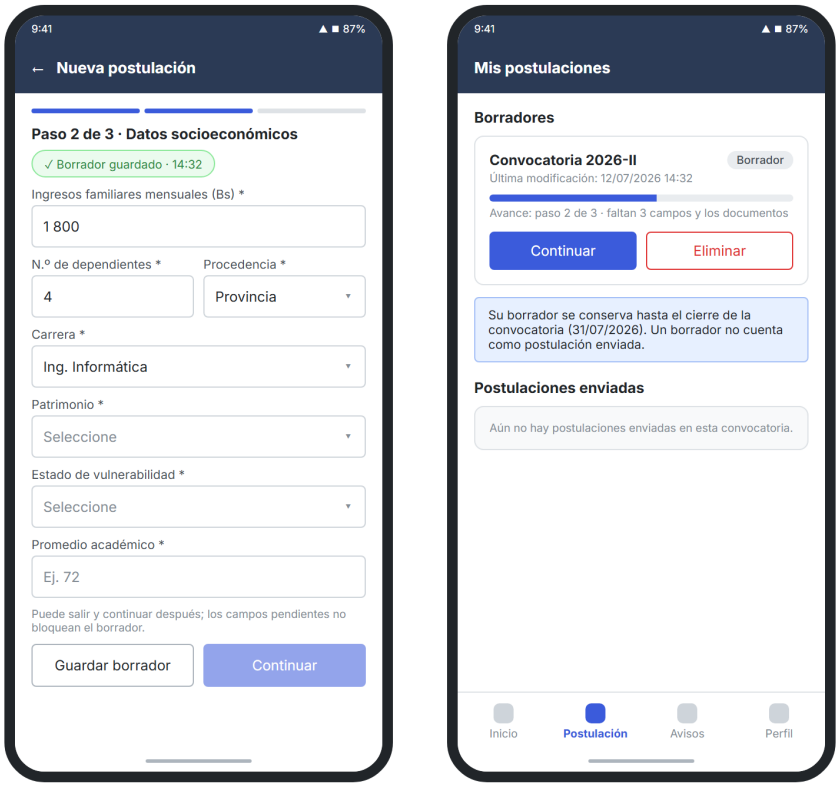

*Nota.* Elaboración propia.

### *7.5.17. Interfaz CU-17 Guardar borrador de postulación*

La Figura 140 presenta la maqueta de interfaz del CU-17.

**Figura 140**

*Maqueta de interfaz del CU-17 Guardar borrador de postulación*



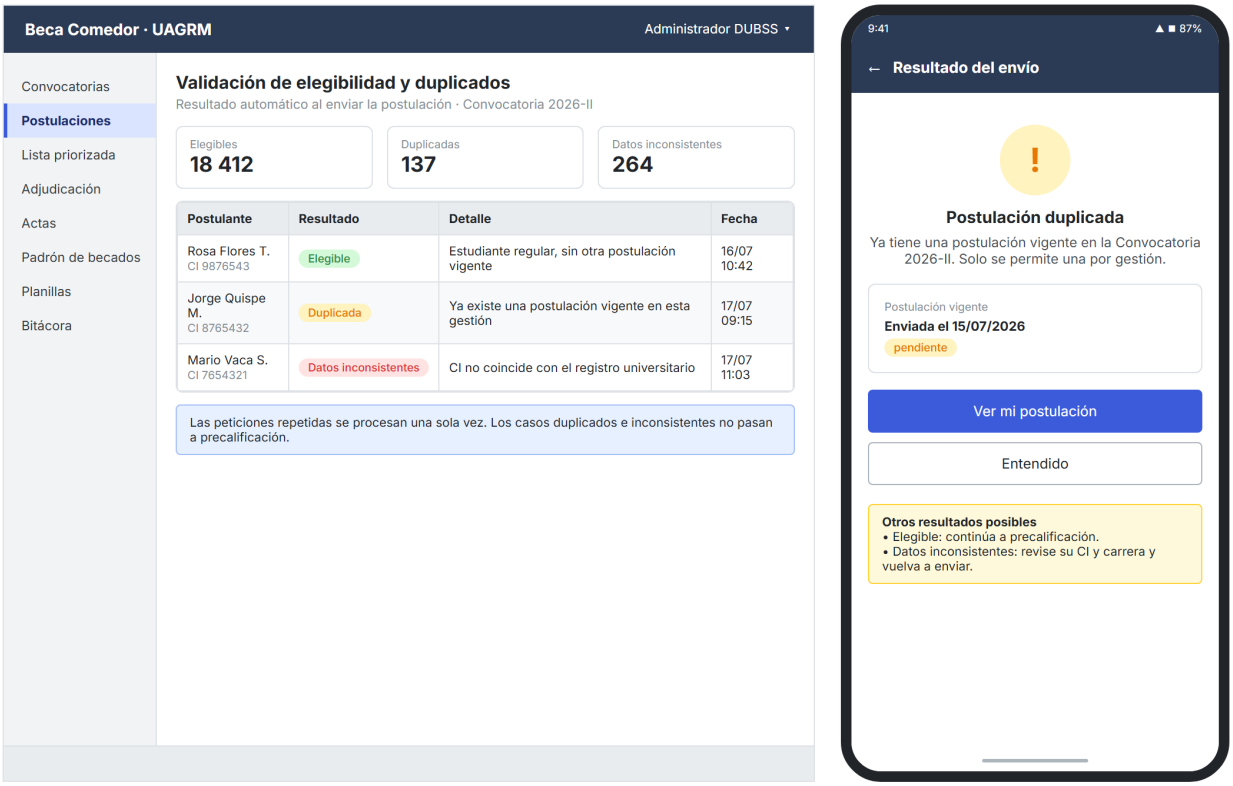

*Nota.* Elaboración propia.

### *7.5.18. Interfaz CU-18 Validar elegibilidad y duplicados*

La Figura 141 presenta la maqueta de interfaz del CU-18.

**Figura 141**

*Maqueta de interfaz del CU-18 Validar elegibilidad y duplicados*




Pantalla 1 · Web: resultados de la validación

Pantalla 2 · Móvil: mensaje al postulante

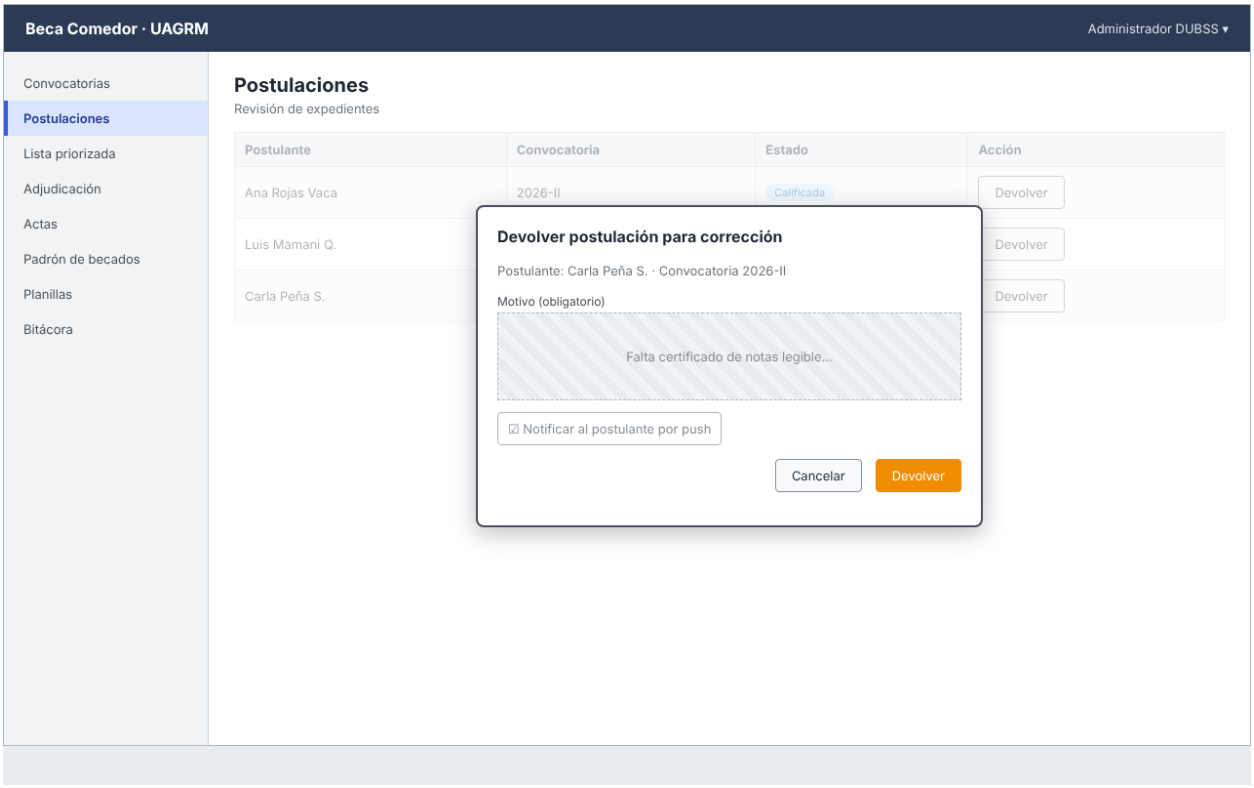
*Nota.* Elaboración propia.

### *7.5.19. Interfaz CU-19 Devolver postulación para corrección*

La Figura 142 presenta la maqueta de interfaz del CU-19.

**Figura 142**

*Maqueta de interfaz del CU-19 Devolver postulación para corrección*



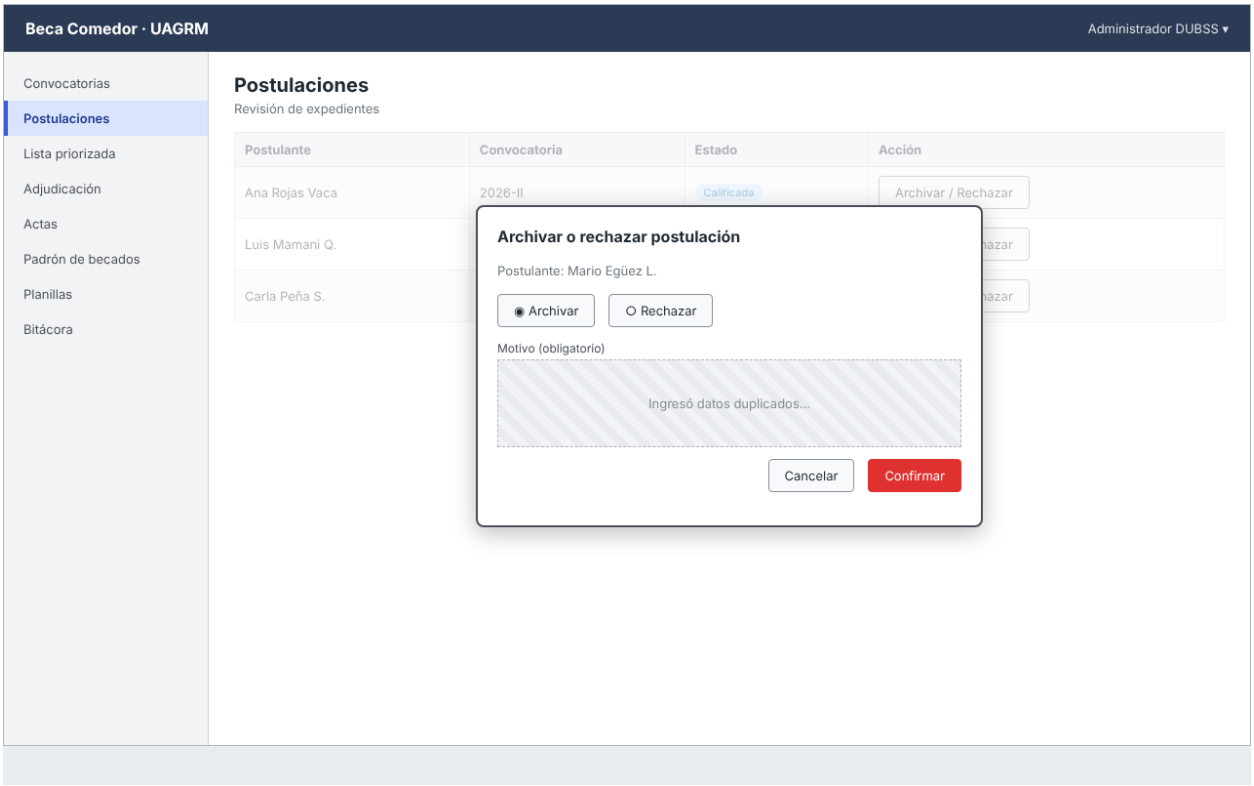

*Nota.* Elaboración propia.

### *7.5.20. Interfaz CU-20 Archivar o rechazar postulación*

La Figura 143 presenta la maqueta de interfaz del CU-20.

**Figura 143**

*Maqueta de interfaz del CU-20 Archivar o rechazar postulación*



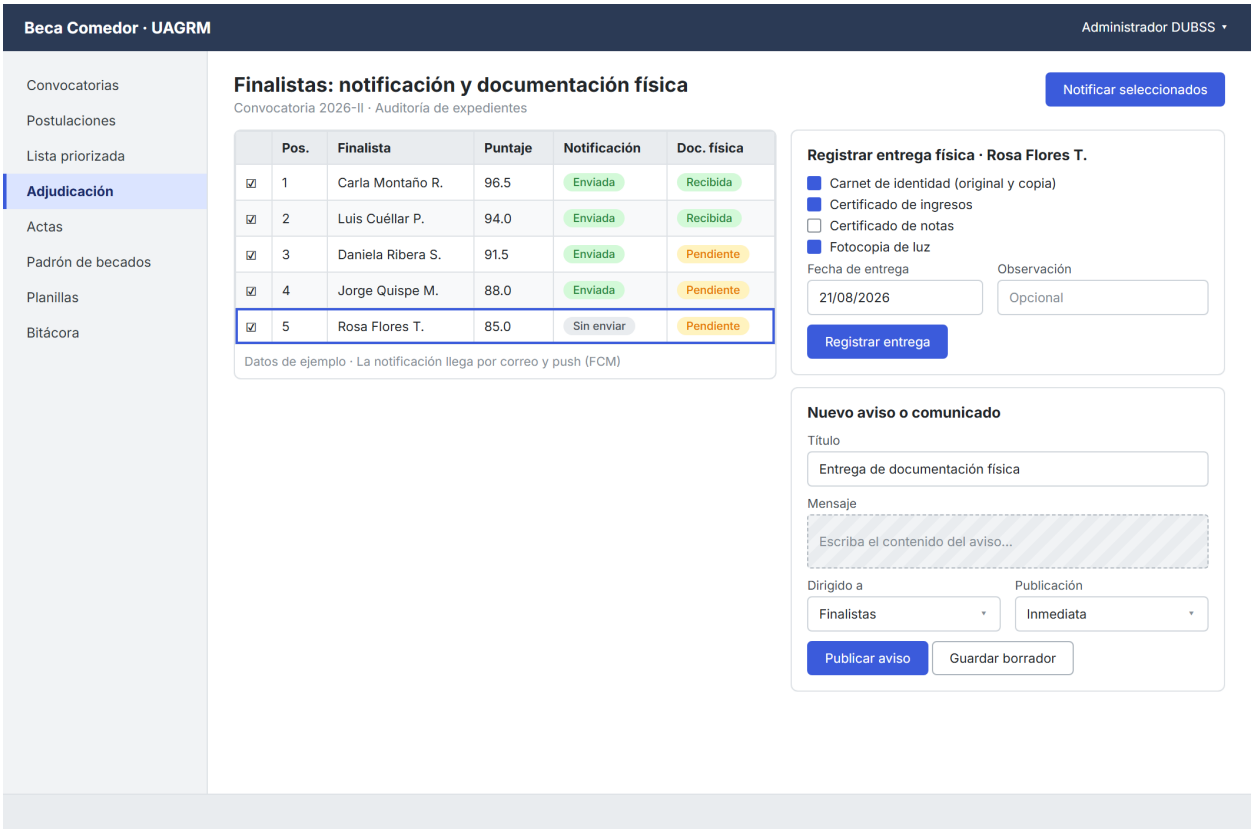

*Nota.* Elaboración propia.

### *7.5.21. Interfaz CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos*

La Figura 144 presenta la maqueta de interfaz del CU-21.

**Figura 144**

*Maqueta de interfaz del CU-21 Notificar finalistas, registrar entrega de documentación física y publicar comunicados/avisos*



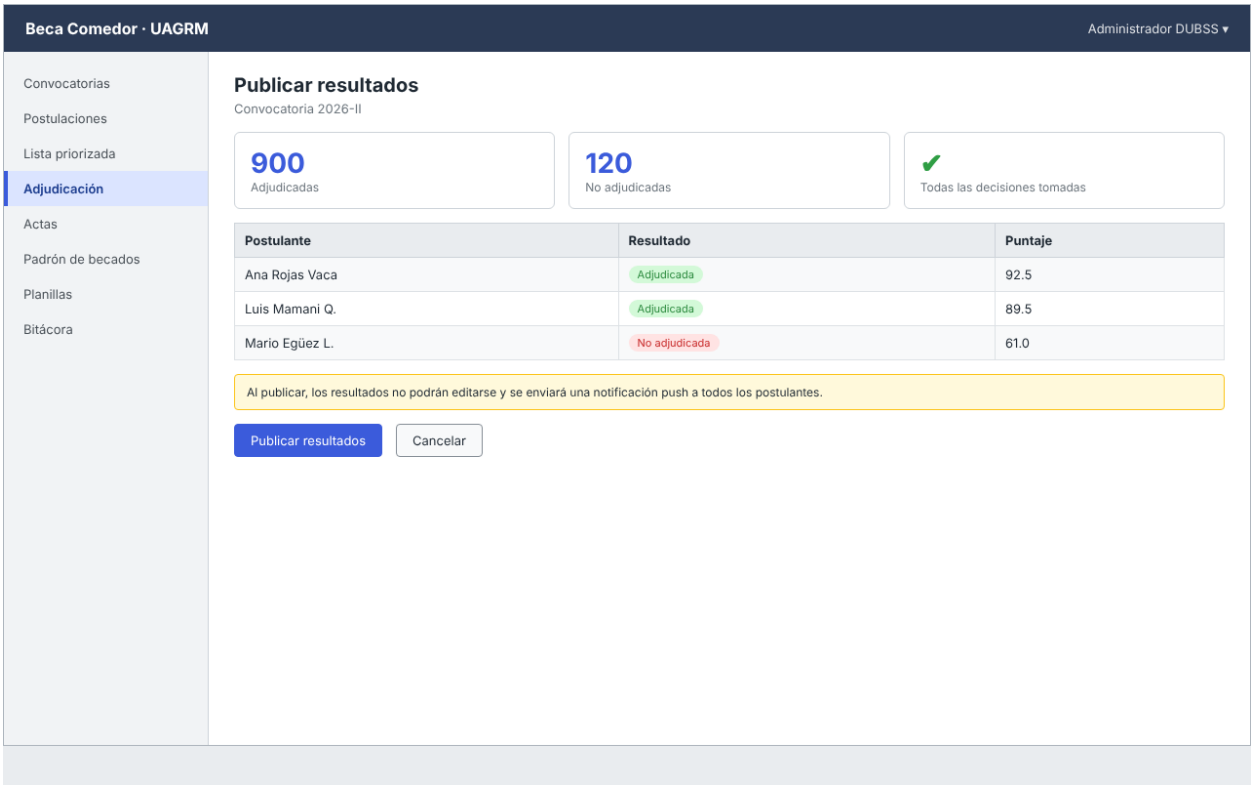

*Nota.* Elaboración propia.

### *7.5.22. Interfaz CU-22 Publicar resultados de adjudicación*

La Figura 145 presenta la maqueta de interfaz del CU-22.

**Figura 145**

*Maqueta de interfaz del CU-22 Publicar resultados de adjudicación*



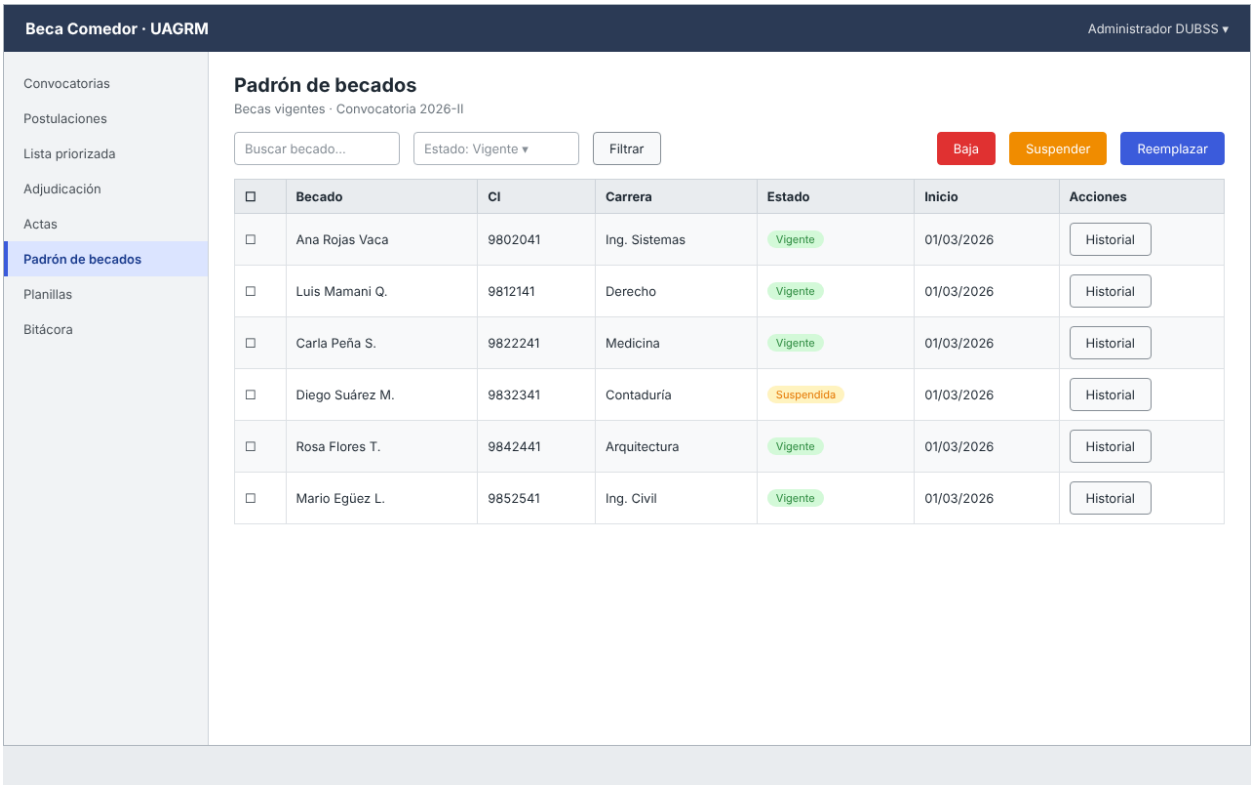

*Nota.* Elaboración propia.

### *7.5.23. Interfaz CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo)*

La Figura 146 presenta la maqueta de interfaz del CU-23.

**Figura 146**

*Maqueta de interfaz del CU-23 Gestionar padrón de becados (baja, suspensión, reemplazo)*



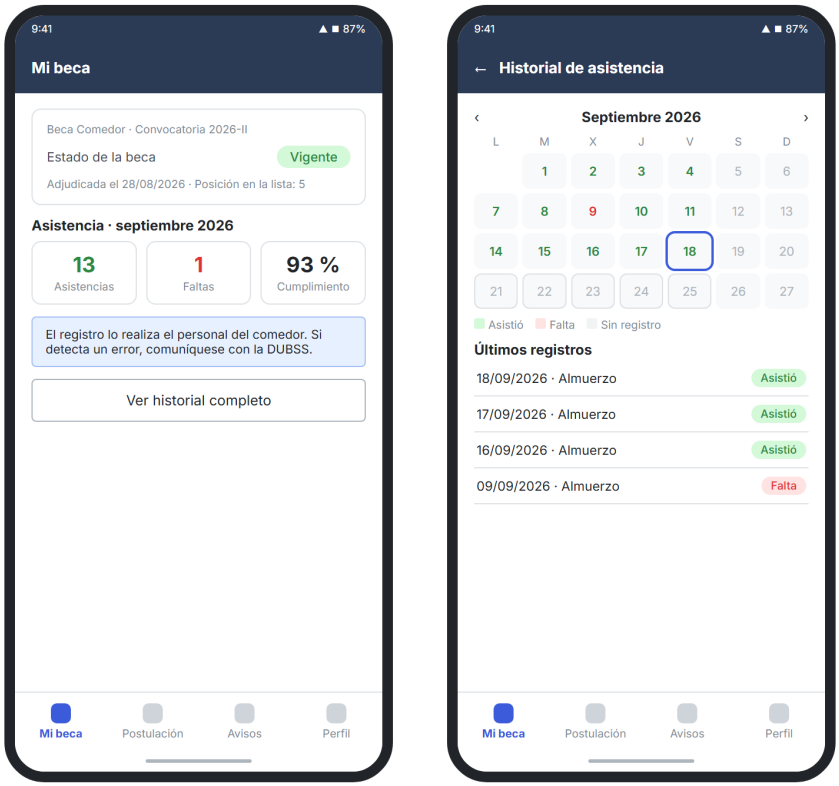

*Nota.* Elaboración propia.

### *7.5.24. Interfaz CU-24 Consultar historial de asistencia y estado de beca*

La Figura 147 presenta la maqueta de interfaz del CU-24.

**Figura 147**

*Maqueta de interfaz del CU-24 Consultar historial de asistencia y estado de beca*



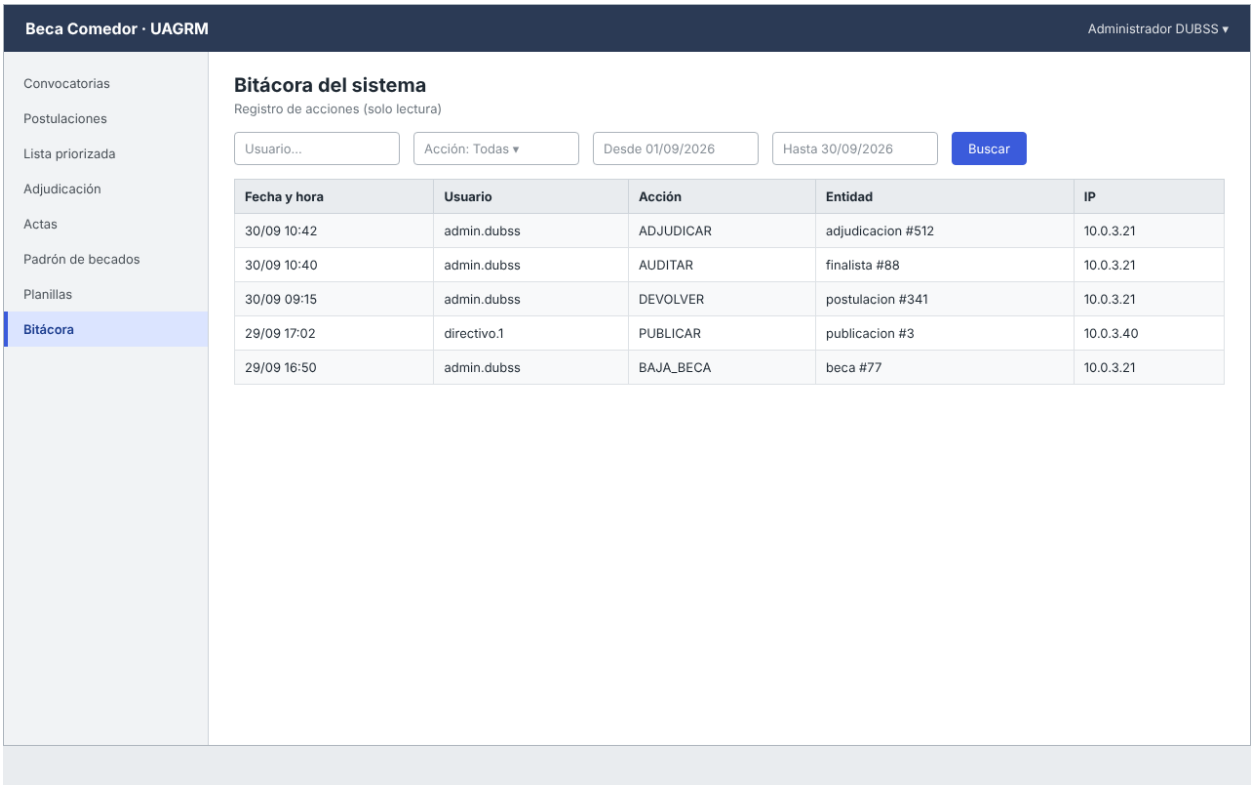

*Nota.* Elaboración propia.

### *7.5.25. Interfaz CU-25 Consultar bitácora de acciones*

La Figura 148 presenta la maqueta de interfaz del CU-25.

**Figura 148**

*Maqueta de interfaz del CU-25 Consultar bitácora de acciones*



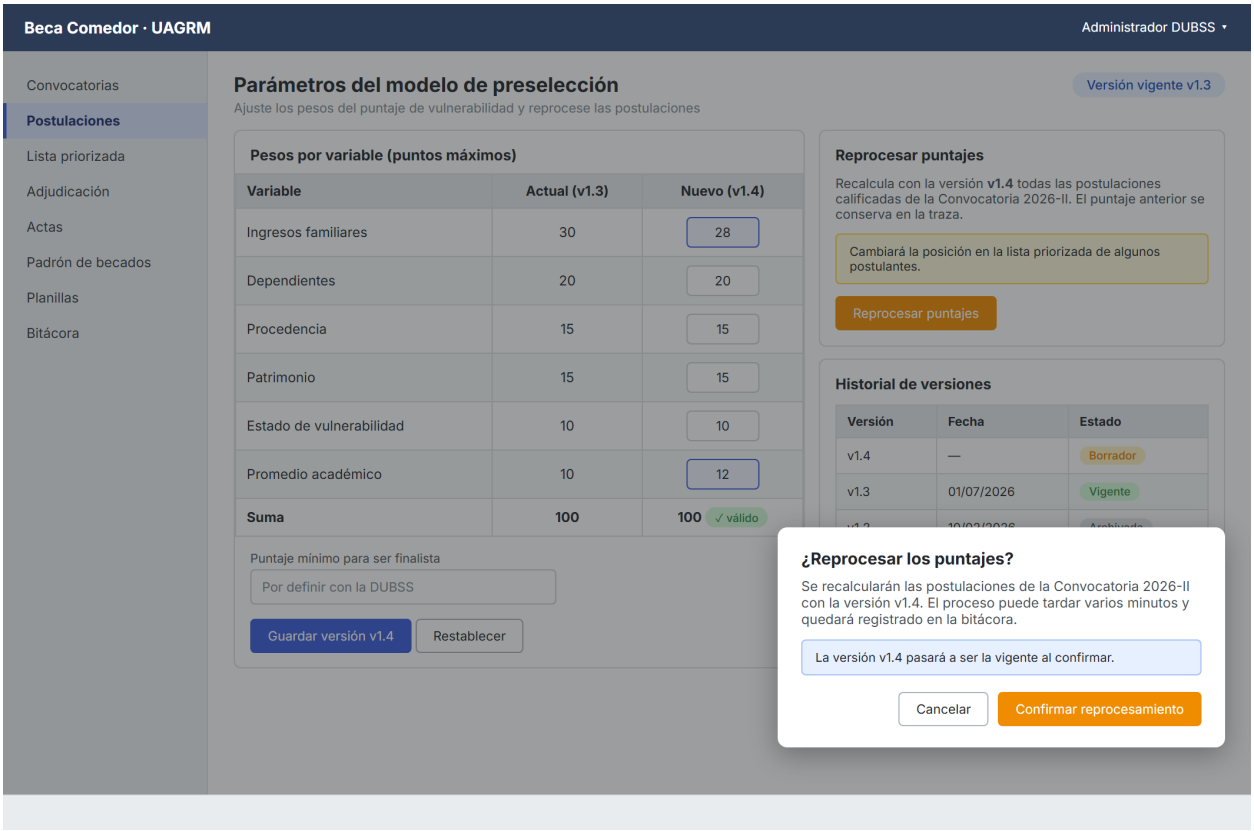

*Nota.* Elaboración propia.

### *7.5.26. Interfaz CU-26 Parametrizar modelo de preselección y reprocesar puntajes*

La Figura 149 presenta la maqueta de interfaz del CU-26.

**Figura 149**

*Maqueta de interfaz del CU-26 Parametrizar modelo de preselección y reprocesar puntajes*




*Nota.* Elaboración propia.

# Conclusiones y Recomendaciones

## Conclusiones

A partir del perfil de proyecto desarrollado, se concluye que el proceso actual de postulación y adjudicación de la Beca Comedor en la Dirección Universitaria de Bienestar Social y Salud (DUBSS) de la Universidad Autónoma Gabriel René Moreno presenta una ineficiencia estructural significativa, derivada de la revisión manual de un volumen de solicitudes que supera ampliamente la capacidad operativa del personal administrativo. El diseño del sistema propuesto, sustentado en el Proceso Unificado de Desarrollo de Software y en un modelo de precalificación basado en Inteligencia Artificial, plantea una solución viable para automatizar la evaluación socioeconómica y garantizar una asignación más objetiva y transparente de los cupos disponibles. El marco teórico, la especificación de requisitos según IEEE 830 y el modelado UML desarrollados hasta este punto constituyen una base sólida para las siguientes fases de diseño detallado, implementación y pruebas del sistema.

## Recomendaciones

Se recomienda continuar con las fases de Elaboración y Construcción del PUDS conforme al cronograma establecido, dando prioridad a la validación del modelo de precalificación con datos históricos reales de la DUBSS antes de su despliegue en producción. Asimismo, se sugiere planificar con anticipación la capacitación del personal administrativo y de auditoría en el uso del nuevo sistema, y establecer desde etapas tempranas los protocolos de seguridad y resguardo de la información socioeconómica de los postulantes, dada la sensibilidad de dichos datos.

## Bibliografía

Banker, R. (2014). *React in Action*. Manning Publications.

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd). Addison-Wesley Professional.

Breiman, L. (2001). Random Forests. *Machine Learning*, *45*(1), 5-32. https://doi.org/10.1023/A:1010933404324

Breiman, L., Friedman, J. H., Stone, C. J., & Olshen, R. A. (1984). *Classification and Regression Trees*. CRC Press.

Cherny, B. (2019). *Programming TypeScript: Making Your JavaScript Applications Scale*. O'Reilly Media.

Chodorow, K. (2013). *MongoDB: The Definitive Guide* (2nd). O'Reilly Media.

Decarli, A. (2021). *Flutter Complete Reference*. CreateSpace Independent Publishing Platform.

Esposito, D. (2021). *Architecting Mobile Solutions for the Enterprise*. Microsoft Press.

Few, S. (2006). *Information Dashboard Design: The Effective Visual Communication of Data*. O'Reilly Media.

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Tesis doctoral, University of California, Irvine].

Golfarelli, M., & Rizzi, S. (2009). *Data Warehouse Design: Modern Principles and Methodologies*. McGraw-Hill Education.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Oliphant, T. E., et al. (2020). Array Programming with NumPy. *Nature*, *585*(7825), 357-362. https://doi.org/10.1038/s41586-020-2649-2

IEEE Computer Society. (1998). *IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998)*. Institute of Electrical y Electronics Engineers. New York, NY. https://doi.org/10.1109/IEEESTD.1998.88286

ISO/IEC/IEEE. (2011). *Systems and Software Engineering: Life Cycle Processes: Requirements Engineering (ISO/IEC/IEEE 29148:2011)*. International Organization for Standardization. Geneva, Switzerland.

Jacobson, I., Booch, G., & Rumbaugh, J. (1999). *The Unified Software Development Process*. Addison-Wesley Professional.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning* (Vol. 112). Springer.

Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit: Practical Techniques for Extracting, Cleaning, Conforming, and Delivering Data*. John Wiley & Sons.

Linsley, T. (2021). TanStack Query Documentation. https://tanstack.com/query

McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, *445*, 51-56.

McKinney, W. (2012). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.

Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.

Mularien, P. (2010). *Spring Security 3*. Packt Publishing.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Duchesnay, É., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, *12*, 2825-2830.

Power, D. J. (2002). *Decision Support Systems: Concepts and Resources for Managers*. Quorum Books.

Pressman, R. S. (2010). *Ingeniería del Software: Un Enfoque Práctico* (7th). McGraw-Hill.

Ramírez, S. (2019). FastAPI Framework, High Performance, Easy to Learn, Fast to Code, Ready for Production. https://fastapi.tiangolo.com/

Riggs, S., Krosing, H., Bartunov, O., & Korotkov, A. (2015). *PostgreSQL 9 Administration Cookbook*. Packt Publishing.

Rouse, M., et al. (2018). *JSON Web Token (JWT) RFC 7519 Standard Implementation Guide* (inf. téc.). IETF.

Rumbaugh, J., Jacobson, I., & Booch, G. (2004). *The Unified Modeling Language Reference Manual* (2nd). Addison-Wesley Professional.

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th). Pearson.

Sommerville, I. (2011). *Ingeniería de Software* (9th). Pearson Educación.

Tidwell, J., Brewer, C., & Valencia, A. (2020). *Designing Interfaces: Patterns for Effective Interaction Design* (3rd). O'Reilly Media.

Walls, C. (2019). *Spring Boot in Action*. Manning Publications.

Windmill, E. (2020). *Flutter in Action*. Manning Publications.

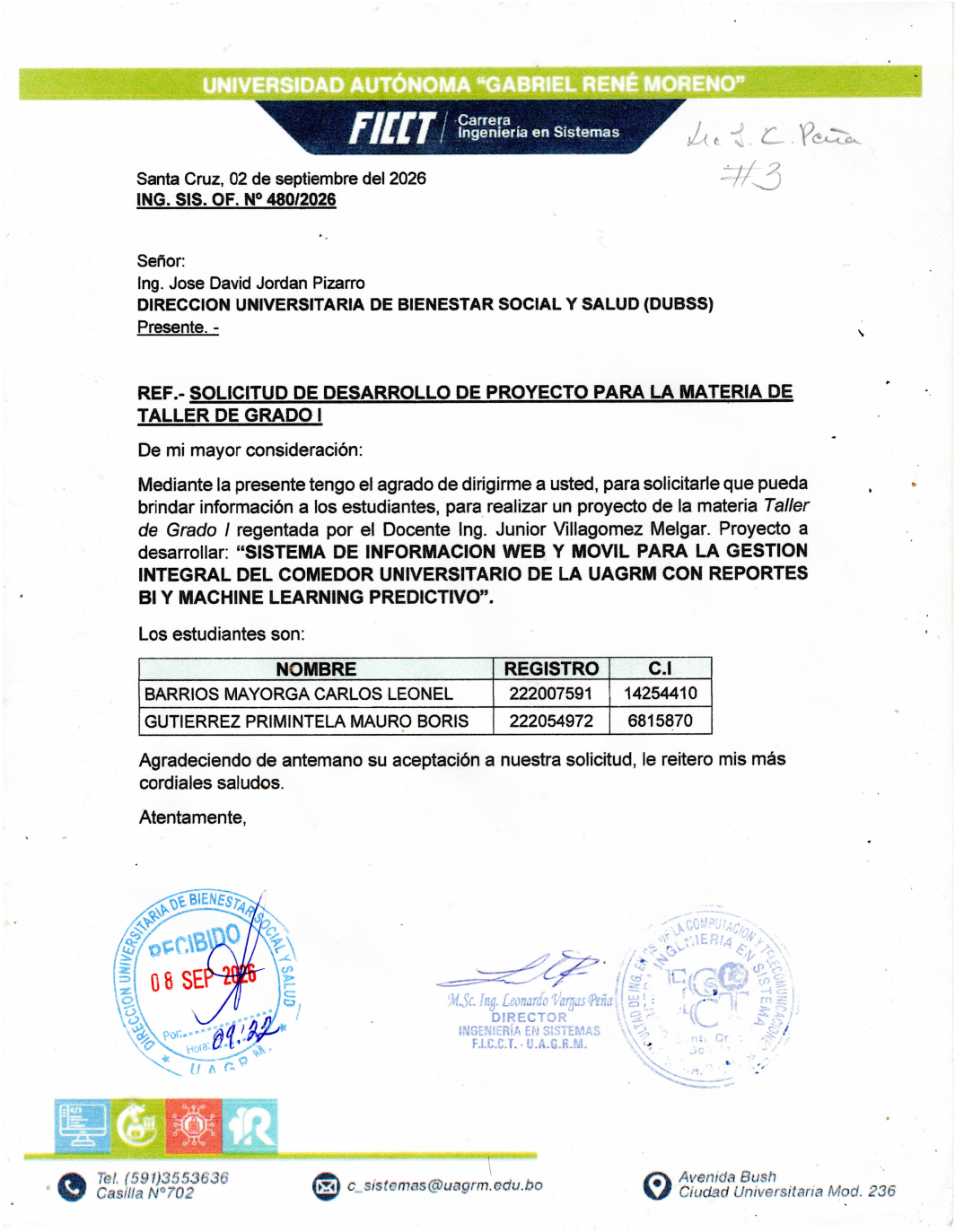
# Anexo

## Carta de Solicitud de Desarrollo del Proyecto

A continuación, se presenta la carta oficial (ING. SIS. OF. N.º 480/2026) mediante la cual la Carrera de Ingeniería en Sistemas de la Facultad de Ingeniería en Ciencias de la Computación y Telecomunicaciones (FICCT - UAGRM) solicitó formalmente a la Dirección Universitaria de Bienestar Social y Salud (DUBSS) la información necesaria para el desarrollo del presente proyecto de Taller de Grado I.

**Figura A1**

*Carta de solicitud de desarrollo de proyecto dirigida a la DUBSS*



UNIVERSIDAD AUTÓNOMA "GABRIEL RENÉ MORENO"

FICCT | Carrera Ingeniería en Sistemas

Santa Cruz, 02 de septiembre del 2026
**ING. SIS. OF. Nº 480/2026**

Señor:
Ing. Jose David Jordan Pizarro
**DIRECCION UNIVERSITARIA DE BIENESTAR SOCIAL Y SALUD (DUBSS)**
Presente. -

**REF.- SOLICITUD DE DESARROLLO DE PROYECTO PARA LA MATERIA DE TALLER DE GRADO I**

De mi mayor consideración:

Mediante la presente tengo el agrado de dirigirme a usted, para solicitarle que pueda brindar información a los estudiantes, para realizar un proyecto de la materia *Taller de Grado I* regentada por el Docente Ing. Junior Villagomez Melgar. Proyecto a desarrollar: **"SISTEMA DE INFORMACION WEB Y MOVIL PARA LA GESTION INTEGRAL DEL COMEDOR UNIVERSITARIO DE LA UAGRM CON REPORTES BI Y MACHINE LEARNING PREDICTIVO".**

Los estudiantes son:

| NOMBRE | REGISTRO | C.I |
|---|---|---|
| BARRIOS MAYORGA CARLOS LEONEL | 222007591 | 14254410 |
| GUTIERREZ PRIMINTELA MAURO BORIS | 222054972 | 6815870 |

Agradeciendo de antemano su aceptación a nuestra solicitud, le reitero mis más cordiales saludos.

Atentamente,

DIRECCION UNIVERSITARIA DE BIENESTAR SOCIAL Y SALUD
RECIBIDO
08 SEP 2026
Por.....
Hora: 09:32
U A G R M.

M.Sc. Ing. Leonardo Vargas Peña
DIRECTOR
INGENIERÍA EN SISTEMAS
F.I.C.C.T. - U.A.G.R.M.

Tel. (591)3553636
Casilla N°702
c_sistemas@uagrm.edu.bo
Avenida Bush
Ciudad Universitaria Mod. 236

*Nota.* Documento oficial emitido por la Carrera de Ingeniería en Sistemas, FICCT - UAGRM, con recepción sellada por la DUBSS el 8 de septiembre de 2026.